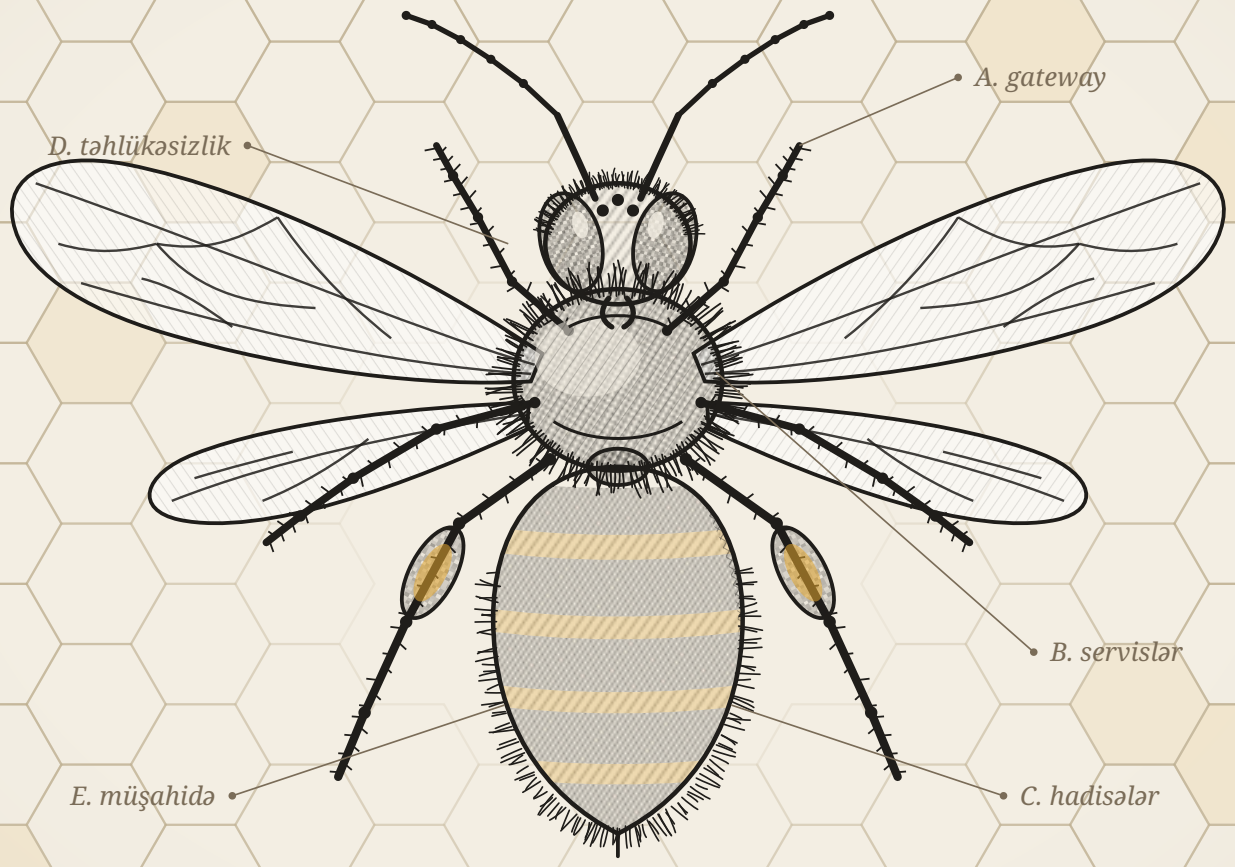
JAVA 25 · SPRING BOOT 4 · KAFKA · gRPC · ELASTICSEARCH · REDIS



Apis mellifera

KOLONİYA · HÜCEYRƏ · HADİSƏ

REAL SİSTEMİN İÇİNDƏN BACKEND MÜHƏNDİSLİYİ

Mikroservislərin Anatomiyası

Java 25 və Spring Boot 4 ilə real backend sistemini daxildən öyrənmək

BİRİNCİ NƏŞR · 2026

Kafka, gRPC, Elasticsearch, Spring Cloud, təhlükəsizlik və observability — real kodla

AAR

REAL SİSTEMİN İÇİNDƏN BACKEND MÜHƏNDİSLİYİ

Mikroservislərin Anatomiyası

Java 25 və Spring Boot 4 ilə real backend sistemini daxildən öyrənmək

A A R

Birinci nəşr · 2026

Mikroservislərin Anatomiyası

Java 25 və Spring Boot 4 ilə real backend sistemini daxildən öyrənmək

Müəllif: **AAR**

Birinci nəşr · 2026

© 2026 AAR. Bütün hüquqlar müəllifə məxsusdur.

Repozitoriya: UserAAR/Insurance-Campaign-Microservices — təhlil olunan commit 1500818 (2026-10-09)

Stek: Java 25 · Spring Boot 4.1.0 · Spring Cloud 2025.1.2 · Hibernate 7.4 · Spring Kafka 4.1 · gRPC 1.80

Dil: Azərbaycan dili · Hədəf oxucu: təcrübəli, lakin 9–12 ay fasilə vermiş Java/Spring backend mühəndisi

Bu kitab repozitoriyanın mənbə kodundan çıxarılıb; layihəyə aid hər iddia fayl:sətir istinadı ilə verilir

Üz qabığı haqqında: üz qabığındakı canlı bal arısıdır (Apis mellifera). Arı koloniyası mikroservis sisteminin metaforasıdır — hər hüceyrə kiçik və müstəqildir, bütöv isə aydın kontraktlar, siqnallar və bölüşdürülmüş məsuliyyət sayəsində işləyir. Anatomik göstəricilər (A–E) kitabın əsas qatlarına uyğundur: gateway, servislər, hadisələr, təhlükəsizlik və müşahidə.

Kitab Markdown mənbələrindən pandoc və Chromium ilə yığılıb. Mətn Noto Serif və Noto Sans, kod JetBrains Mono şrifti ilə (SIL Open Font License) verilib; diaqramlar Mermaid ilə çəkilib.

Kitabdakı kod nümunələri UserAAR/Insurance-Campaign-Microservices repozitoriyasından götürülüb və hər biri fayl:sətir istinadı ilə yoxlanıla bilər. Tövsiyələri öz mühitinizdə tətbiq etməzdən əvvəl yoxlayın.

Mündəricat

Ön söz 9
- Niyə "anatomiya"? 9
- Bu kitabda nəyə söz verirəm 9
- Kim üçün yazmışam 9
- Bu kitab nə deyil 10
- Necə oxumalı 10

I HİSSƏ. SİSTEMİ ANLAMAQ

1. Bu kitabı necə oxumalı 12
- 1.1 Kimin üçün yazmışam və nəyi hədəfləyirəm 12
- 1.2 İki pəncərə ilə oxuyun: PDF və mənbə kodu 12
- 1.3 Etiketlər: fakt, nəticə, naməlum, risk 12
- 1.4 Kitabın quruluşu 13
- 1.5 Tövsiyə olunan oxu yolları 13
- 1.6 Repozitoriyanı lokal işə salmaq (qısa) 13
- 1.7 Terminologiya qeydi 14
- 1.8 Xülasə 14

2. Sistemin biznes və arxitektura xəritəsi 15
- 2.1 Sistem hansı problemi həll edir 15
- 2.2 Aktorlar və imkanlar 15
- 2.3 Əsas biznes obyektləri 16
- 2.4 Komponentlərin inventarı 17
- 2.5 Arxitektura diaqramı (təsdiqlənmiş) 18
- 2.6 Sinxron və asinxron sərhədlər 19
- 2.7 Data sahibliyi 20
- 2.8 Əsas axınların icmalı 20
- 2.9 Komponent düşdükdə nə baş verir 21
- 2.10 Öyrənmə ardıcıllığı 21
- 2.11 Müsahibə sualları 21
- 2.12 Xülasə 22

3. Repozitoriya quruluşu, Maven və build sistemi 23
- 3.1 Niyə build quruluşunu əvvəldən öyrənirik 23
- 3.2 Monorepo və multi-module Maven 23
- 3.3 Versiya idarəetməsi: BOM import və spring-boot-starter-parent olmaması 23
- 3.4 Versiyalar cədvəli 25
- 3.5 Modullar arası asılılıq qrafı 25
- 3.6 Spring Boot 4-ün modullaşdırılmış auto-configuration-u 26
- 3.7 Jackson 3 və Jackson 2 birgə yaşayışı 26
- 3.8 grpc-contract: koddan əvvəl kod generasiyası 27
- 3.9 Test və keyfiyyət plugin-ləri 27
- 3.10 auth-service: annotation processor-lar 28
- 3.11 Tətbiq artefaktları və Dockerfile 29
- 3.12 Build əmrləri və tipik problemlər 29
- 3.13 Yeni layihədə tətbiq 30
- 3.14 Müsahibə sualları 30
- 3.15 Xülasə 31

II HİSSƏ. TƏMƏLLƏRİ TƏZƏLƏMƏK

4. Java 25 və Spring Boot 4 təməlləri — layihə üzərində təzələmə 33
- 4.1 Problem: tətbiq özünü necə yığır? 33
- 4.2 Dependency Injection: konstruktor injeksiyası standartdır 33
- 4.3 Stereotiplər və @Configuration / @Bean 34
- 4.4 Komponent skanı və paylaşılan modullar 35
- 4.5 Auto-configuration və şərtlər 35

4.6 Konfiqurasiya xassələri və mənbələrin prioriteti 36
4.7 Bean həyat dövrü və "fail-fast" başlanğıc 37
4.8 Proxy-lər və AOP: annotasiyalar necə "işləyir" 38
4.9 Java 25 xüsusiyyətləri layihədə 39
4.10 Thread-safety və singleton bean-lər 40
4.11 Yeni layihədə tətbiq: qısa yoxlama siyahısı 41
4.12 Müsahibə sualları 41
4.13 Xülasə 41

5. HTTP, REST və təbəqəli dizayn 42
5.1 Problem: API-ni necə dizayn etməli ki, həm klient, həm də kod sağlam qalsın 42
5.2 HTTP və REST əsasları (təzələmə) 42
5.3 Layihənin REST resurs modeli 43
5.4 Controller: sorğu bağlama və cavab 44
5.5 Cavab zərfi: ApiResponse və ApiError 47
5.6 Üç model: DTO, domen, entity 47
5.7 Səhifələmə (pagination) və sıralama 49
5.8 Təbəqələr və məsuliyyətlər 50
5.9 Yeni layihədə tətbiq 50
5.10 Müsahibə sualları 50
5.11 Xülasə 51

III HİSSƏ. LAYİHƏNİN NÜVƏ İMPLEMENTASİYASI

6. Campaign Service — biznes nüvəsi dərindən 53
6.1 Məsuliyyətlər və sərhədlər 53
6.2 Domen modeli 53
6.3 Kampaniya yaratmaq: create 54
6.4 Oxu əməliyyatları: getById, getAll 56
6.5 Status keçidləri: approve, deactivate, changeStatus 57
6.6 Tarixçə: saveHistory, getHistory 59
6.7 Statistika: getStatistics 59
6.8 Konfiqurasiya 59
6.9 Yeni layihədə tətbiq: zəngin domen modeli ilə status maşını 60
6.10 Uğursuzluq ssenariləri 60
6.11 Müsahibə sualları 61
6.12 Xülasə 61

7. Persistence: PostgreSQL, JPA/Hibernate və Liquibase 63
7.1 Problem: obyektlər, cədvəllər və zaman 63
7.2 Data modeli 63
7.3 Liquibase: nəzəriyyə 65
7.4 campaign-service changelog-u 66
7.5 auth-service changelog-u: sxem təkamülünün real nümunəsi 67
7.6 JPA entity xəritələnməsi 68
7.7 Persistence context, entity vəziyyətləri və dirty checking 70
7.8 Tranzaksiyalar 71
7.9 Sorğu növləri 72
7.10 Sxem validasiyası və testlər arasındakı boşluq 73
7.11 Yeni layihədə tətbiq 73
7.12 Uğursuzluq ssenariləri və diaqnostika 73
7.13 Müsahibə sualları 74
7.14 Xülasə 74

8. Validasiya və istisnaların idarə edilməsi 75
8.1 Problem: səhv məlumat sistemə harada daxil olur, harada dayandırılır? 75
8.2 Bean Validation: qısa nəzəriyyə 75
8.3 Validasiya qaydalarının tam inventarı 75
8.4 İstisna iyerarxiyası və xəta kodları 78
8.5 GlobalExceptionHandler: hansı istisna hansı handler-ə düşür 79
8.6 Təhlükəsizlik qatında xətalar 81

8.7 Gateway, uzaq servislər və asinxron emalda xətalar 82
8.8 Persistence xətaları və rollback 83
8.9 Uğurlu və uğursuz sorğunun izlənməsi 83
8.10 Yeni layihədə tətbiq 83
8.11 Müsahibə sualları 84
8.12 Xülasə 85

9. Auth Service — istifadəçi və token həyat dövrü 86
9.1 Məsuliyyət və sərhəd 86
9.2 Şifrələrin saxlanması: DelegatingPasswordEncoder 86
9.3 Qeydiyyat (register) 87
9.4 Login (login) və token buraxılması 88
9.5 Refresh token rotasiyası (refreshToken) 89
9.6 Şifrə bərpası (forgotPassword, resetPassword) 90
9.7 Logout 91
9.8 Login axını — ardıcıllıq diaqramı 92
9.9 Audit inteqrasiyası 92
9.10 Konfiqurasiya 92
9.11 Uğursuzluq ssenariləri 93
9.12 Yeni layihədə tətbiq və alternativlər 93
9.13 Müsahibə sualları 94
9.14 Xülasə 94

10. Spring Security və JWT — bütün sistem üzrə təhlükəsizlik 95
10.1 Problem: mikroservislərdə "bu sorğunu kim göndərib?" 95
10.2 JWT nəzəriyyəsi 95
10.3 Açarların idarə edilməsi 96
10.4 Servlet servislərində Spring Security arxitekturası 97
10.5 Metod səviyyəli avtorizasiya 99
10.6 Gateway-də JWT yoxlaması (WebFlux) 100
10.7 Servislərarası autentifikasiya 101
10.8 Gateway-dən yan keçmə 102
10.9 Təhdid modeli xülasəsi 102
10.10 Yeni layihədə tətbiq: Spring Security OAuth2 Resource Server 102
10.11 Müsahibə sualları 103
10.12 Xülasə 103

IV HİSSƏ. TEXNOLOGİYALARI MÜSTƏQİL ÖYRƏNMƏK

11. Apache Kafka — hadisə axını və asinxron mesajlaşma 106
11.1 Motivasiya: niyə mesaj brokeri? 106
11.2 Əsas anlayışlar 106
11.3 Qeydin yazılması (producer daxilində) 107
11.4 Qeydin oxunması (consumer daxilində) 108
11.5 Spring Kafka komponentləri 109
11.6 Layihədəki Kafka xəritəsi 109
11.7 Producer tərəfi: KafkaJsonConfiguration 110
11.8 Producer uğursuzluq ssenariləri (F09, F10) 111
11.9 Consumer tərəfi: log-service 113
11.10 Broker konfiqurasiyası (Docker Compose) 115
11.11 Konfiqurasiyanın mənbədən istehlakçıya izlənməsi 116
11.12 Yeni servisdə Kafka inteqrasiyası: tövsiyə olunan şablon 116
11.13 Troubleshooting 117
11.14 Müsahibə sualları 117
11.15 Xülasə 118

12. Elasticsearch — axtarış yönümlü saxlama 119
12.1 Motivasiya: niyə relyasiya bazası deyil? 119
12.2 Əsas anlayışlar 119
12.3 Spring Data Elasticsearch və klient 120
12.4 Repozitoriyadakı implementasiya 121

12.5 Konfiqurasiya 123
12.6 Əsas DB və ES: məsuliyyətlərin bölgüsü 123
12.7 Yeni layihədə tətbiq 124
12.8 Uğursuzluq ssenariləri və troubleshooting 124
12.9 Müsahibə sualları 124
12.10 Xülasə 125

13. gRPC və Protocol Buffers 126
13.1 Motivasiya: servislərarası sinxron çağırışda kontrakt 126
13.2 Protocol Buffers 126
13.3 Generasiya olunan kod 128
13.4 gRPC runtime anlayışları 128
13.5 net.devh grpc-spring-boot-starter 128
13.6 Server: CampaignGrpcEndpoint 129
13.7 Klient: CampaignGrpcClient 130
13.8 İcra axını: dashboard statistikası 132
13.9 Test 132
13.10 Yeni layihədə tətbiq 133
13.11 gRPC və REST müqayisəsi 133
13.12 Müsahibə sualları 134
13.13 Xülasə 134

14. Servislərarası HTTP: HTTP Interface klientləri, Feign müqayisəsi və Resilience4j 135
14.1 Motivasiya: uzaq çağırış heç vaxt lokal çağırış deyil 135
14.2 Spring-də HTTP klient seçimləri 135
14.3 HTTP Interface + HTTP Service registry (Spring Framework 7) 135
14.4 Asinxron icra: @Async + virtual thread-lər 136
14.5 Resilience nümunələri: nəzəriyyə 137
14.6 Layihədəki Resilience4j konfiqurasiyası 138
14.7 AuthAuditService.publishAudit: xətaların yerli emalı 140
14.8 Uğursuzluq ssenariləri 141
14.9 Yeni layihədə tətbiq 141
14.10 Müsahibə sualları 142
14.11 Xülasə 142

15. Redis: paylanmış cache və rate limiting 144
15.1 Redis əsasları 144
15.2 Caching nəzəriyyəsi 144
15.3 Spring Cache abstraksiyası 145
15.4 Layihədə cache 145
15.5 Rate limiting nəzəriyyəsi 146
15.6 Spring Cloud Gateway RedisRateLimiter 147
15.7 İcra axını: rate limit 150
15.8 Uğursuzluq ssenariləri 150
15.9 Yeni layihədə tətbiq və alternativlər 150
15.10 Müsahibə sualları 151
15.11 Xülasə 151

16. Spring Cloud platforması: Gateway, Eureka, LoadBalancer, Config Server və OpenAPI aqreqasiyası 152
16.1 Motivasiya 152
16.2 Spring Cloud Gateway (WebFlux) 152
16.3 Eureka: servis kəşfi 154
16.4 Spring Cloud LoadBalancer 155
16.5 Config Server 155
16.6 OpenAPI aqreqasiyası 156
16.7 Konfiqurasiyanın izlənməsi (gateway) 157
16.8 Uğursuzluq ssenariləri 157
16.9 Alternativlər və yeni layihədə tətbiq 157
16.10 Müsahibə sualları 158
16.11 Xülasə 158

17. Observability: metriklər, trace-lər və loglar 159

17.1 Motivasiya: "nə baş verir?" sualına cavab vermək 159
17.2 Metriklər: Actuator, Micrometer, Prometheus 159
17.3 Distributed tracing 160
17.4 Loglar: fayllar, Promtail, Loki 162
17.5 Grafana: korrelyasiya 163
17.6 Sağlamlıq yoxlamaları 163
17.7 Tətbiq səviyyəli jurnallar və metriklərin üst-üstə düşməsi 164
17.8 Real insident araşdırması: "Dashboard statistikası 500 qaytarır" 164
17.9 Yeni layihədə tətbiq 164
17.10 Müsahibə sualları 164
17.11 Xülasə 165

18. Konteynerlər, Docker Compose və CI/CD 166
18.1 Motivasiya 166
18.2 Dockerfile-lar 166
18.3 docker-compose.yml: tam stek 167
18.4 docker-compose-local.yml: yalnız infrastruktur 168
18.5 CI/CD: .github/workflows/ci-cd.yml 168
18.6 Troubleshooting (README + təhlil) 169
18.7 Yeni layihədə tətbiq: yoxlama siyahısı 169
18.8 Müsahibə sualları 170
18.9 Xülasə 170

19. Test strategiyası və test infrastrukturu 171
19.1 Məqsəd: testlər nəyi sübut etməlidir? 171
19.2 Repozitoriyadakı testlərin inventarı 171
19.3 Test infrastrukturu: baza siniflər 172
19.4 Test texnikaları 174
19.5 Nə test edilir, nə edilmir 174
19.6 Təkmilləşdirmə: tövsiyə olunan testlər (illüstrativ nümunələr, repozitoriyada yoxdur) 175
19.7 Müsahibə sualları 176
19.8 Xülasə 176

V HİSSƏ. UÇDAN-UCA BİZNES AXINLARI

20. Uçdan-uca iş axınları 178
20.1 Bütün axınlar üçün ümumi gateway hissəsi 178
20.2 Axın A: qeydiyyat və login (audit ilə) 179
20.3 Axın B: kampaniyanın yaradılması (tam zəncir) 181
20.4 Axın C: təsdiq və deaktivasiya 183
20.5 Axın D: oxuma sorğuları (kampaniya, siyahı, tarixçə) 183
20.6 Axın E: dashboard statistikası (REST → gRPC → cache → DB) 184
20.7 Axın F: dashboard tarixçəsi və xəta yayılması 185
20.8 Axın G: yavaş sorğu jurnalı (Kafka → Elasticsearch) və onun oxunması 186
20.9 Axın H: refresh, logout, forgot və reset 187
20.10 Axın I: Swagger UI və OpenAPI aqreqasiyası 188
20.11 Axınlar × komponentlər matrisi 188
20.12 Müsahibə sualları 189
20.13 Xülasə 189

VI HİSSƏ. QİYMƏTLƏNDİRMƏ VƏ DİZAYN

21. Mühəndislik qiymətləndirməsi 192
21.1 Metodologiya 192
21.2 Güclü tərəflər 192
21.3 Tapıntılar: təsdiqlənmiş defektlər (D) 193
21.4 Tapıntılar: risklər (R) 194
21.5 Tapıntılar: trade-off-lar (T) 195
21.6 Tapıntılar: saxlanıla bilənlik (M) və hipotezlər (H) 196
21.7 Risk matrisi 197
21.8 Prioritetləşdirilmiş düzəliş yol xəritəsi 197

21.9 SOLID və təbəqələnmə təhlili 198
21.10 Arxitektura səviyyəsində qiymətləndirmə 199
21.11 Komanda üçün açıq suallar (naməlumlar) 199
21.12 Müsahibə sualları 199
21.13 Xülasə 200

22. Oxşar sistemi sıfırdan dizayn etmək 201

22.1 Tələblərdən başlamaq 201
22.2 Birinci qərar: modul-monolit, yoxsa mikroservislər? 201
22.3 Hədəf arxitektura 202
22.4 Addım-addım qurma planı 203
22.5 Əsas dizayn qərarları (ADR formatında) 204
22.6 Kritik komponentlərin nümunə implementasiyası 204
22.7 Paket quruluşu 207
22.8 İstehsala hazırlıq yoxlama siyahısı 207
22.9 Bu repozitoriyadan çıxarılan dərslər 208
22.10 Müsahibə: "Kampaniya idarəetmə sistemi dizayn edin" 208
22.11 Xülasə 208

VII HİSSƏ. PRODUCTİON VƏ MÜSAHİBƏYƏ HAZIRLIQ

23. İstehsal mühəndisliyi: əməliyyat və troubleshooting runbook-u 210

23.1 Sistem xəritəsi: portlar və sağlamlıq yoxlamaları 210
23.2 Nəyi izləmək lazımdır: siqnallar 210
23.3 Diaqnostika alətləri 211
23.4 Hadisə playbook-ları 212
23.5 Yerləşdirmə və geri qaytarma 214
23.6 Tutum və tənzimləmə 215
23.7 Təhlükəsizlik əməliyyatları 215
23.8 Müsahibə sualları 215
23.9 Xülasə 216

24. Sistemi təqdim etmək və müsahibəyə hazırlıq 217

24.1 Yadda saxlanmalı faktlar 217
24.2 Bir dəqiqəlik təqdimat 217
24.3 Beş dəqiqəlik izah (struktur) 218
24.4 Sual bankı 219
24.5 Bu repozitoriya haqqında çətin suallar 222
24.6 Xülasə 223

25. Əlavələr 224

25.1 Terminologiya lüğəti və abbreviaturalar 224
25.2 Endpoint kataloqu 226
25.3 Konfiqurasiya və mühit dəyişənləri arayışı 228
25.4 Fəsillərdə ayrıca müzakirə olunmayan fayllar 229
25.5 Rəsmi sənədlər və əlavə oxu 231
25.6 Audit xülasəsi 231

26. Əhatə matrisi və tapıntı indeksi 233

26.1 Necə oxumalı 233
26.2 Modullar üzrə xülasə 233
26.3 Fayl səviyyəsində matris 233
26.4 Tapıntı indeksi 243

Ön söz

Bu kitabı ilk növbədə özüm üçün yazmağa başladım.

Bir müddət kod yazmadan keçən aylardan sonra yenidən backend mühəndisliyinə qayıdanda gördüm ki, ümumi dərsliklər mənə lazım olanı vermir. `@Transactional`-ın nə etdiyini, Kafka-nın nə olduğunu, JWT-nin necə qurulduğunu bilirdim — amma "bu biliklər real sistemdə necə birləşir, harada sınır və mən onu necə izah edərəm?" sualına sürətli və dəqiq cavab verə bilmirdim. Ona görə başqa yol seçdim: öz layihəmi — **Insurance Campaign Microservices** repozitoriyasını — götürüb onu anatom kimi açdım.

Niyə "anatomiya"?

Anatom orqanizmi təsvir etməklə kifayətlənmir: hər orqanın harada yerləşdiyini, nə iş gördüyünü, digərləri ilə necə bağlandığını və xəstələnəndə nə baş verdiyini göstərir. Bu kitabda da eyni şeyi edirəm. Gateway, auth, campaign, dashboard və log servislərini, onları birləşdirən REST, gRPC və Kafka kanallarını, PostgreSQL, Redis və Elasticsearch qatlarını bir-bir açıram. Hər hissədə əvvəlcə texnologiyanın özünü — problem, nəzəriyyə, daxili mexanizm — izah edirəm, sonra onun bu repozitoriyada necə qurulduğunu kodun konkret sətirləri ilə göstərirəm.

Üz qabığındakı bal arısı da bu fikrin simvoludur. Arı koloniyası mikroservis sisteminin ən yaxşı metaforalarından biridir: hər hüceyrə kiçik və müstəqildir, amma bütöv sistem yalnız aydın kontraktlar, siqnallar və bölüşdürülmüş məsuliyyət sayəsində yaşayır.

Bu kitabda nəyə söz verirəm

- **Hər iddianın arxasında kod var.** Layihəyə aid hər cümlənin yanında fayl yolu və sətir nömrəsi verirəm. Kitabdakı bütün istinadlar və kod çıxarışları avtomatik skriptlə mənbə kodu ilə tutuşdurulub; siz də IDE-ni açıb eyni sətirləri yoxlaya bilərsiniz.
- **Fakt ilə fikri qarışdırmıram.** Koddan birbaşa görünən şeyləri FAKT, məntiqlə çıxardığım nəticələri NƏTİCƏ, repozitoriyadan bilinməyənləri NAMƏLUM, təhlükəli yerləri isə RİSK kimi işarələyirəm.
- **Öz kodumun zəif yerlərini gizlətmirəm.** Kitabın ən dəyərli hissələrindən biri məhz budur. Admin rolunun qeydiyyat zamanı klientdən qəbul edilməsi, parol bərpası token-inin cavabda qaytarılması, Kafka göndərişinin DB tranzaksiyasının içində olması, gRPC çağırışlarında deadline-ın olmaması kimi problemləri açıq yazıram və hər biri üçün necə düzəldəcəyimi göstərirəm. Real mühəndislik mükəmməl kod deyil — öz qərarlarının qiymətini bilməkdir.
- **Öyrəndiyiniz hər şeyi tətbiq edə biləcəksiniz.** Kitabın sonunda eyni sistemi sıfırdan necə dizayn edərdim, istehsalda onu necə işlədərdim və müsahibədə necə izah edərdim — bunların hamısını ayrıca fəsillərdə yazmışam.

Kim üçün yazmışam

Java və Spring Boot ilə işləmiş, mikroservis anlayışlarını bilən, lakin ya fasilədən sonra qayıdan, ya da Kafka, Elasticsearch, gRPC, Spring Cloud, Resilience4j və observability kimi mövzuları real kod üzərində dərindən görmək istəyən mühəndis üçün. Müsahibəyə hazırlaşırsınızsa, hər fəslin sonundakı suallar və 24-cü fəsil birbaşa sizin üçündür.

Bu kitab nə deyil

Bu, Spring-in bütün imkanlarını sadalayan arayış kitabı deyil. Burada yalnız bu sistemdə istifadə olunan və onu başa düşmək üçün lazım olan mövzular var — amma onlar dərinliyi ilə. Kitabda gördüyünüz bütün versiyalar (Java 25, Spring Boot 4.1, Spring Cloud 2025.1 və s.) repozitoriyanın təhlil olunan vəziyyətinə aiddir; tövsiyələri öz mühitinizdə tətbiq etməzdən əvvəl rəsmi sənədlərlə yoxlayın.

Necə oxumalı

Ən yaxşı nəticə üçün kitabı repozitoriya ilə yanaşı, iki pəncərədə oxuyun: bir tərəfdə bu kitab, digər tərəfdə IDE. Oxu yollarını, etiketləri və kitabın quruluşunu 1-ci fəsildə ətraflı izah edirəm.

Uğurlar. Ümid edirəm ki, bu kitab sizə də mənə kömək etdiyi qədər kömək edəcək.

AAR Oktyabr 2026

I hissə. Sistemi anlamaq

Biznes məqsədi, arxitektura xəritəsi, repozitoriya və build

1. Bu kitabı necə oxumalı

1.1 Kimin üçün yazmışam və nəyi hədəfləyirəm

Bu kitabı Java və Spring Boot ilə real backend yazmış, mikroservis anlayışlarını bilən, lakin təxminən 9–12 ay kod yazmamış mühəndis üçün yazmışam — bir vaxtlar mənim özüm də məhz belə oxucu idim. Belə oxucunun iki ehtiyacı olur:

1. **Unudulanı sürətlə bərpa etmək** — DI, bean həyat dövrü, `@Transactional`, JPA, validasiya, Spring Security filtrləri kimi mövzular uzun izahat tələb etmir, lakin "hansı annotasiya nəyi dəyişir və nə vaxt işləmir" suallarına dəqiq cavab tələb edir.
2. **Yeni texnologiyanı dərindən öyrənmək** — Kafka, Elasticsearch, gRPC, Spring Cloud Gateway, Resilience4j, OpenTelemetry kimi mövzuları həm nəzəri əsası ilə, həm də bu repozitoriyadakı konkret implementasiya ilə izah edirəm.

Kitabın son məqsədi budur: siz repozitoriyanı bir sistem kimi başa düşəsiniz, əsas dizayn qərarlarını izah edə biləsiniz, hər əsas axını giriş nöqtəsindən nəticəyə qədər izləyə biləsiniz, texnologiyaları layihədən asılı olmadan anlayasınız və **oxşar mikroservis backend-ini özünüz layihələndirib yaza biləsiniz**.

1.2 İki pəncərə ilə oxuyun: PDF və mənbə kodu

Kitabı repozitoriya ilə yanaşı oxunmaq üçün yazmışam. Layihəyə aid hər iddianın yanında fayl yolunu və sətir nömrəsini verirəm:

> `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:54-77`

Bu istinadı gördükdə IDE-də faylı açın (IntelliJ IDEA-da *Navigate → File* və ya `Ctrl+Shift+N`, sonra `:54`). Kod çıxarışlarının üstündə **"Mənbə:"** sətri olur; çıxarış həmin sətir aralığından götürülüb. Bir hissəni buraxmışamsa, çıxarışda `// ... (buraxılıb)` şərhi görünür. Sətirləri yalnız oxunaqlılıq üçün buraxıram; heç bir çıxarışa repozitoriyada olmayan kod əlavə etmirəm. Repozitoriyada olmayan, yalnız izah üçün yazdığım nümunələri ayrıca **"İllüstrativ nümunə (repozitoriyada yoxdur)"** başlığı ilə verirəm.

Bütün istinadları və kod çıxarışlarını kitabın `build/verify_refs.py` skripti ilə avtomatik yoxlamışam (bax: fəsil 25 və `audit/audit-report.md`).

1.3 Etiketlər: fakt, nəticə, naməlum, risk

Mühəndis üçün ən vacib bacarıqlardan biri "kod nə deyir" ilə "mən nə düşünürəm" arasındakı fərqi saxlamaqdır. Buna görə kitabda dörd etiketdən istifadə edirəm:

Etiket	Mənası	Nümunə
FAKT	Kodla və ya konfiqurasiya ilə birbaşa təsdiqlənir; istinad verilir	`LIFE` kampaniyası `ACTIVE` statusu ilə yaradılır (`CampaignService.java:133-138`)
NƏTİCƏ	Implementasiyadan, framework davranışından və ya bir neçə faktdan əsaslandırılmış çıxarış	Kafka göndərişi tranzaksiya commit-indən əvvəl baş verə bilər
NAMƏLUM	Repozitoriyadan müəyyən edilə bilməz	`campaign-events` mövzusunu repozitoriyadan kənarda kimin oxuduğu

Etiket	Mənası	Nümunə
RİSK	Potensial defekt, təhlükəsizlik və ya etibarlılıq problemi	İstənilən şəxsin `ROLE_ADMIN` ilə qeydiyyatdan keçə bilməsi

Developerin niyyəti koddan oxunmursa, onu fakt kimi təqdim etmirəm: "bu qərarı əsaslandıra biləcək mühəndis səbəbi" kimi **analiz** təqdim edirəm.

1.4 Kitabın quruluşu

Hissə	Məzmun	Nə vaxt oxumalı
I. Sistemi anlamaq	Biznes məqsədi, komponentlər, axınlar, repozitoriya və build	Hər şeydən əvvəl
II. Təməllər	Java 25, Spring Boot 4 mexanizmləri, REST və təbəqəli dizayn	Unudulmuş bilikləri bərpa etmək üçün
III. Nüvə	Campaign Service, persistence, validasiya/istisnalar, Auth Service, security	Layihənin "ürəyi"
IV. Texnologiyalar	Kafka, Elasticsearch, gRPC, HTTP klientləri və Resilience4j, Redis, Spring Cloud, observability, konteynerlər, testlər	Hər biri müstəqil oxuna bilər
V. Axınlar	Çoxservisli uçdan-uca icra yolları	III və IV-dən sonra
VI. Qiymətləndirmə və dizayn	Tapıntılar, kompromislər, oxşar sistemin dizaynı	Tənqidi düşüncə və müsahibə üçün
VII. Production və müsahibə	Runbook, sistemin tam izahı, sual-cavablar, əlavələr	Son mərhələ və təkrar üçün

Hər əsas texniki fəsli təxminən bu ardıcıllıqla qurmuşam: **problem → nəzəriyyə → daxili mexanizm → repozitoriya implementasiyası → icra axını → konfiqurasiya → yeni layihədə tətbiq → alternativlər → xətalar və troubleshooting → müsahibə sualları → xülasə.**

1.5 Tövsiyə olunan oxu yolları

Tam yol (tövsiyə edirəm, 2–3 həftə, gündə 2–3 saat): fəsillərə ardıcıl gedin; hər fəsildən sonra istinad olunan faylları açıb yenidən oxuyun, sonra fəsil sonundakı müsahibə suallarına kitaba baxmadan cavab verin.

Sürətli bərpa (3–4 gün): 2 → 3 → 4 → 6 → 8 → 10 → 20 → 21 → 24.

Yalnız yeni texnologiyalar: 11 (Kafka) → 12 (Elasticsearch) → 13 (gRPC) → 14 (Resilience4j) → 16 (Gateway) → 17 (Observability). Bu fəsillərdə əvvəlcə texnologiyanın özünü öyrədirəm, sonra layihədəki istifadəni göstərirəm.

Müsahibəyə hazırlıq: 24-cü fəsil (sistemin 5 dəqiqəlik izahı və sual-cavablar), sonra hər fəslin sonundakı suallar, sonra 21-ci fəslin tapıntılar cədvəli — müsahibədə "bu sistemdə nəyi dəyişərdiniz?" sualına cavabdır.

1.6 Repozitoriyanı lokal işə salmaq (qısa)

Ətraflı izah 18-ci fəsildədir. Qısa xülasə FAKT (`README.md`, compose faylları):

```
# 1) Yalnız infrastruktur (PostgreSQL, Redis, Kafka, Elasticsearch, observability, SonarQube)
docker compose -f docker-compose-local.yml up -d
```

```
# 2) Servisləri bu ardıcıllıqla IDE-dən və ya Maven ilə başladın
mvn -pl discovery-server spring-boot:run
mvn -pl config-server spring-boot:run
mvn -pl auth-service spring-boot:run
mvn -pl campaign-service spring-boot:run
mvn -pl dashboard-service spring-boot:run
mvn -pl log-service spring-boot:run
mvn -pl api-gateway spring-boot:run

# Alternativ: hər şeyi konteynerdə
docker compose up -d --build
```

Tələblər: JDK 25, Maven 3.9+, Docker. Layihə Java 25 ilə kompilyasiya olunur (`pom.xml:28-29`) və `--enable-preview` ilə işə salınır (Dockerfile-lar və surefire konfiqurasiyası). Bu kitabı hazırladığım mühitdə yalnız Java 21 olduğu üçün layihəni bu nəşr üçün kompilyasiya etməmişəm və testləri işə salmamışam; bütün təhlili statik olaraq mənbə kodu, konfiqurasiya, Maven asılılıq ağacı (`mvn dependency:tree`) və istifadə olunan kitabxanaların mənbə kodu üzərində aparmışam. Ona görə runtime davranışı haqqında iddiaları, kitabxana mənbəyində təsdiqləmədiyim hallarda, **NƏTİCƏ** kimi etiketləyirəm.

Demo istifadəçilər **FAKT** (`auth-service/src/main/resources/db/changelog/db.changelog-master.yml:31-37`, `80-89`, `203-216`): `demo` (`ROLE_USER,ROLE_ADMIN`), `admin` (`ROLE_ADMIN`), `user` (`ROLE_USER`). Şifrələri changelog-da `{noop}` prefiksi ilə açıq mətndə yazılıb (F22); dəyəri bu kitabda təkrar etmirəm — lokal işə salanda onu həmin changelog-dan götürün. Bunlar yalnız lokal demo üçündür.

1.7 Terminologiya qeydi

Texniki terminlərin bir çoxu Azərbaycan dilində sabitləşməyib. Kitabda termini ilk dəfə işlədəndə həm Azərbaycan, həm ingilis dilində verirəm (məsələn, "istehlakçı qrupu (consumer group)"), sonra ən çox işlənən formadan istifadə edirəm. Kod elementlərini (`@Transactional`, `KafkaTemplate`) həmişə orijinal yazılışında saxlayıram. Qısaltmaların tam siyahısı 25-ci fəsildəki lüğətdədir.

1.8 Xülasə

- Kitab real repozitoriyadan çıxarılıb; layihəyə aid hər iddianı fayl:sətir istinadı ilə verirəm.
- Fakt, nəticə, naməlum və risk etiketləri "kod nə deyir" ilə "mən nə düşünürəm" arasındakı sərhədi saxlayır.
- PDF-i IDE ilə yanaşı oxuyun; hər fəsildən sonra sualları kitaba baxmadan cavablandırın.
- Növbəti fəsildə sistemin nə üçün mövcud olduğunu və necə qurulduğunu bütöv şəkildə göstərirəm.

2. Sistemin biznes və arxitektura xəritəsi

Bu fəsil kod siniflərinə girmədən əvvəl sistemin **nə üçün** mövcud olduğunu, **kimin** istifadə etdiyini, **hansı əməliyyatları** dəstəklədiyini və bu əməliyyatların **hansı komponentlər** arasında necə hərəkət etdiyini göstərir. Sonrakı bütün fəsillər bu xəritəyə istinad edir.

2.1 Sistem hansı problemi həll edir

Repozitoriya sığorta şirkəti üçün **marketinq kampaniyalarının idarə edilməsi** platformasını həyata keçirir. Kök `pom.xml`-də layihə belə təsvir olunur **FAKT** (`pom.xml:11-12`): "Java 25 Spring Boot microservices case study for Allianz Sigorta Campaign API". Yəni bu, real biznes ssenarisini modelləşdirən **case study** layihəsidir — istehsal sistemi deyil, lakin istehsal sistemlərində rast gəlinən demək olar ki, bütün infrastruktur komponentlərini ehtiva edir.

Biznes problemi sadədir, lakin bir neçə qaydası var:

1. Sığorta məhsulları dörd kateqoriyaya bölünür: `TSS` (tamamlayıcı sağlıq sığortası), `OSS` (özəl sağlıq sığortası), `LIFE` (həyat sığortası) və `OTHER` **FAKT** (`CampaignCategory.java:3-8`). Kateqoriya adları türk sığorta terminologiyasından gəlir; test məlumatlarında "Tamamlayıcı sağlık sigortası" ifadəsi istifadə olunur (`CampaignServiceTest.java:51`).
2. Kampaniya yaradılarkən onun **ilkin statusu kateqoriyadan asılıdır**: `LIFE` dərhal `ACTIVE`, qalanları `PENDING_APPROVAL` **FAKT** (`CampaignService.java:133-138`).
3. Eyni kateqoriya, başlıq və təsvirə malik ikinci kampaniya yaradıla bilməz — `409 Conflict` **FAKT** (`CampaignService.java:140-150`, `GlobalExceptionHandler.java:36-52`).
4. Status keçidləri məhduddur: yalnız `PENDING_APPROVAL → ACTIVE` (təsdiq) və `ACTIVE|PENDING_APPROVAL → DEACTIVE` (deaktivasiya) **FAKT** (`CampaignService.java:91-111`). `DEACTIVE`-dən geri qayıtmaq yoxdur; README bunu "proof of concept-də qəsdən həyata keçirilməyib" kimi qeyd edir.
5. Hər status dəyişikliyi **tarixçəyə** yazılır və **hadisə (event)** kimi Kafka-ya göndərilir **FAKT** (`CampaignService.java:162-169`, `177-191`).
6. Rəhbərlik üçün **dashboard** statuslar üzrə sayları və kampaniya tarixçəsini göstərir **FAKT** (`DashboardController.java:39-57`).

Bu biznes qaydalarının ətrafında platforma xidmətləri qurulub: autentifikasiya, avtorizasiya, audit, yavaş sorğuların izlənməsi, rate limiting, mərkəzi konfiqurasiya, servis kəşfi və tam observability.

2.2 Aktorlar və imkanlar

Aktor	Necə tanınır	Nə edə bilər	Mənbə
Anonim istifadəçi	Token yoxdur	Qeydiyyat, login, refresh, forgot/reset password, logout	`ServiceSecurityConfiguration.java:96-106`, `AuthController.java`
`ROLE_USER`	JWT `roles` claim	Kampaniyaların siyahısı və detalı	`CampaignController.java:50-64`
`ROLE_ADMIN`	JWT `roles` claim	Yaratmaq, təsdiq, deaktiv, tarixçə, statistika, dashboard, yavaş sorğular	`CampaignController.java:42-92`, `DashboardController.java`, `SlowRequestLogController.java`

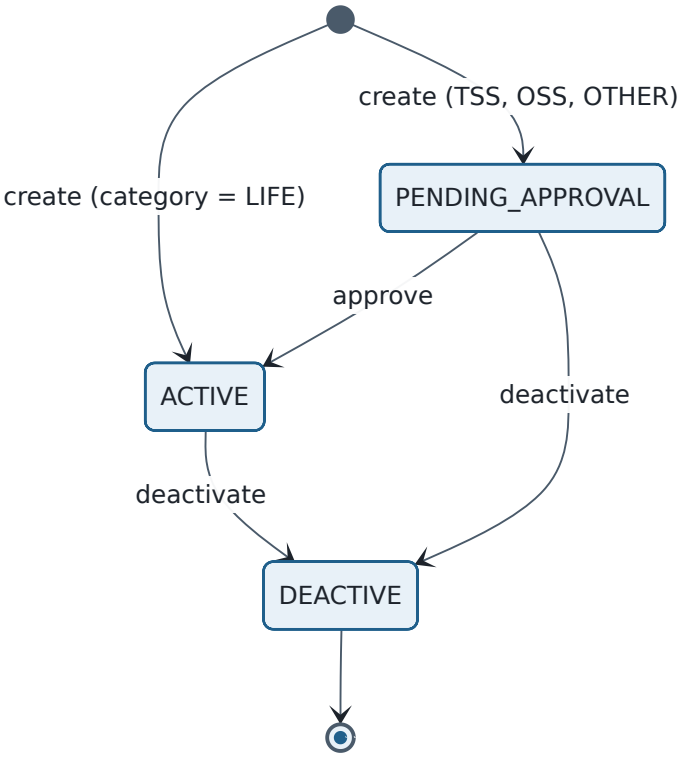
Aktor	Necə tanınır	Nə edə bilər	Mənbə
Daxili servis (`ROLE_INTERNAL_SERVICE`)	auth-service-in imzaladığı JWT	Audit hadisəsini log-service-ə yazmaq	`LogServiceHttpClientConfiguration.java:21-48`, `AuthAuditLogController.java:29-35`
Operator / developer	Brauzer	Swagger UI, Eureka, Kafka UI, Grafana, Prometheus, SonarQube	`docker-compose.yml`, README

RİSK Qeydiyyat sorğusu rolu istifadəçinin özündən qəbul edir (`RegisterRequest.java:18-24`); `UserRole` enum-unda `ROLE_ADMIN` var (`UserRole.java:10-22`). Nəticədə anonim istifadəçi özünü admin kimi qeydiyyatdan keçirə bilər. Bu, 9 və 21-ci fəsillərdə ətraflı təhlil olunur (tapıntı F01).

2.3 Əsas biznes obyektləri

Obyekt	Harada saxlanılır	Sahib servis	Əsas sahələr
Campaign	`campaign_db.campaigns`	campaign-service	`id (uuid)`, `title`, `description`, `category`, `status`, `duplicate_of`, `created_at`, `updated_at`
CampaignStatusHistory	`campaign_db.campaign_status_histories`	campaign-service	`campaign_id`, `previous_status`, `new_status`, `reason`, `changed_at`
AppUser	`auth_db.app_users`	auth-service	`username`, `email`, `password` (hash), `roles` (vergüllə ayrılmış), `status`
RefreshToken	`auth_db.refresh_tokens`	auth-service	`token_hash` (SHA-256), `expires_at`, `revoked_at`, `user_id`
PasswordResetToken	`auth_db.password_reset_tokens`	auth-service	`token_hash`, `expires_at`, `used_at`, `user_id`
AuthAuditLog	Elasticsearch `auth-audit-logs`	log-service	`serviceName`, `eventType`, `usernameOrEmail`, `status`, `message`, `occurredAt`
SlowRequest	Elasticsearch `slow-request-logs`	log-service	`serviceName`, `method`, `path`, `status`, `durationMs`, `traceId`, `headers`, `occurredAt`
CampaignStatusChangedEvent	Kafka `campaign-events`	campaign-service (producer)	`campaignId`, `category`, `previousStatus`, `newStatus`, `reason`, `occurredAt`
SlowRequestLogEvent	Kafka `log-events`	campaign/log-service (producer), log-service (consumer)	yuxarıdakı SlowRequest sahələri

Kampaniyanın status həyat dövrü **FAKT** (`CampaignService.java:56-111`, `CampaignStatus.java`):



Kampaniya status maşını (DUPLICATE statusu enum-da var, lakin heç bir axın onu təyin etmir)

FAKT `CampaignStatus.DUPLICATE` enum-da mövcuddur (`CampaignStatus.java:3-8`) və servis bu statusa malik kampaniyaların yenilənməsini qadağan edir (`CampaignService.java:171-175`), lakin heç bir kod yolu kampaniyanı `DUPLICATE` statusuna keçirmir. Təkrar sorğu sadəcə rədd edilir. **NƏTİCƏ** `DUPLICATE` və `duplicate_of` sütunu əvvəlki dizayn variantının (təkrarların ayrıca sətir kimi saxlanması) qalığıdır; README də "DUPLICATE is intentionally excluded" deyir. Bu, 6-cı fəsildə müzakirə olunur.

2.4 Komponentlərin inventarı

Repozitoriya 10 Maven modulundan ibarətdir: 3 paylaşılan kitabxana və 7 müstəqil yerləşdirilən Spring Boot tətbiqi **FAKT** (`pom.xml:14-25`).

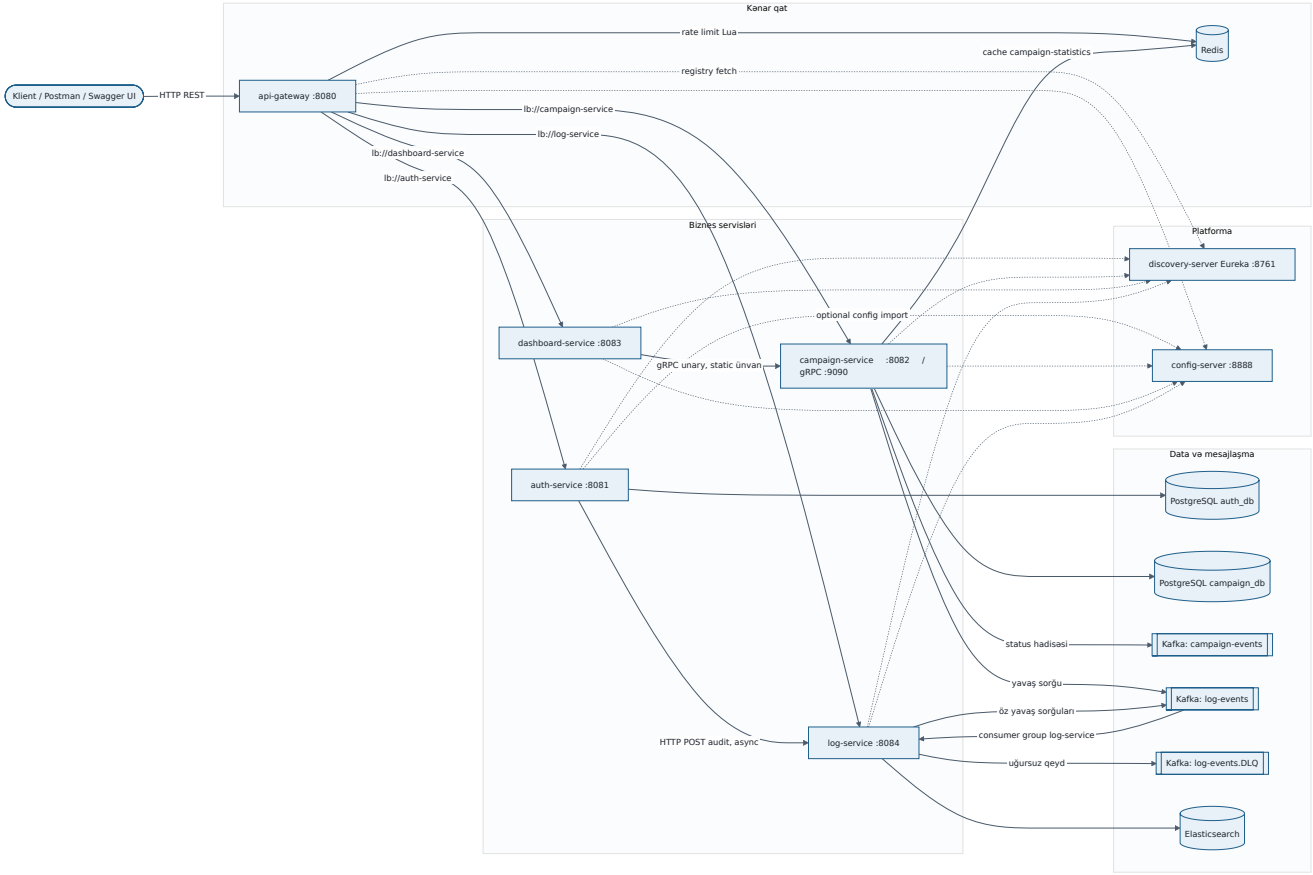
Komponent	Növ	Port	Məsuliyyət
`api-gateway`	Tətbiq (Spring Cloud Gateway, WebFlux)	8080	Vahid giriş nöqtəsi, marşrutlaşdırma, JWT yoxlaması, Redis rate limiting, OpenAPI aqreqasiyası
`auth-service`	Tətbiq (Spring MVC)	8081	İstifadəçilər, rollar, login, refresh/reset token, RSA ilə JWT imzalama, audit hadisələri
`campaign-service`	Tətbiq (MVC + gRPC server)	8082, 9090	Kampaniya həyat dövrü, tarixçə, statistika cache-i, Kafka hadisələri, gRPC API
`dashboard-service`	Tətbiq (MVC + gRPC klient)	8083	Statistika və tarixçəni gRPC ilə alır, REST kimi təqdim edir

Komponent	Növ	Port	Məsuliyyət
`log-service`	Tətbiq (MVC + Kafka consumer)	8084	Yavaş sorğu hadisələrini Kafka-dan oxuyur, auditləri HTTP ilə qəbul edir, Elasticsearch-də saxlayır, DLQ
`discovery-server`	Tətbiq (Eureka Server)	8761	Servis reyestri
`config-server`	Tətbiq (Spring Cloud Config, `native`)	8888	Mərkəzi konfiqurasiya
`common`	Kitabxana	—	`ApiResponse` / `ApiError`, istisnalar, `GlobalExceptionHandler`, Kafka producer, yavaş sorğu filtri, observation
`security`	Kitabxana	—	Servlet servisləri üçün JWT filtri, `SecurityFilterChain`, RSA açar oxuma
`grpc-contract`	Kitabxana	—	`campaign.proto` və ondan generasiya olunan Java kodu

İnfrastruktur **FAKT** (`docker-compose.yml`): iki PostgreSQL instansı (`auth-db` 5433, `campaign-db` 5434), Redis (6379), Apache Kafka KRaft rejimində (9092 host, 19092 daxili), Kafka UI (8090), Elasticsearch 9.4.2 (9200), Prometheus (9091), Grafana (3000), Tempo (3200), Loki (3100), Promtail (9080), OpenTelemetry Collector (4317/4318), SonarQube (9000).

2.5 Arxitektura diaqramı (təsdiqlənmiş)

Aşağıdakı diaqram yalnız kodda və konfiqurasiyada təsdiqlənən əlaqələri göstərir. Bütöv xətlər biznes trafikini, qırıq xətlər platforma əlaqələrini (kəşf, konfiqurasiya, telemetriya) göstərir.



Sistem arxitekturası — yalnız kodda təsdiqlənmiş əlaqələr

Diaqramdan çıxan bir neçə vacib müşahidə:

- **FAKT** `campaign-events` mövzusuna yazılır, lakin repozitoriyada onu oxuyan consumer yoxdur — bu hadisə gələcək və ya xarici istehlakçılar üçün nəşr olunur. **NAMƏLUM** Onu kimin oxuduğu repozitoriyadan müəyyən edilə bilməz.
- **FAKT** Eureka-dan istifadə edən yeganə klient tərəf **api-gateway**-dir (`lb://` URI-ləri və `@LoadBalanced WebClient.Builder`). dashboard-service campaign-service-ə **static** gRPC ünvanı ilə qoşulur (`dashboard-service/src/main/resources/application.yml:12-15`), auth-service log-service-ə **static** HTTP URL ilə (`auth-service/src/main/resources/application.yml:21-24`). Bütün servislər Eureka-da qeydiyyatdan keçir, lakin servislərarası çağırışlar kəşfdən istifadə etmir.
- **FAKT** Yavaş sorğu filtri yalnız classpath-ində `spring-kafka` olan servislərdə işləyir: campaign-service və log-service. `common` modulunda `spring-kafka` `optional` olaraq elan edilib (`common/pom.xml:27-31`), yəni tranzitiv ötürülmür; auth-service və dashboard-service-də bu kitabxana yoxdur (Maven asılılıq ağacı ilə təsdiqlənib). api-gateway `common`-dan ümumiyyətlə asılı deyil.
- **FAKT** Telemetriya (diaqramda göstərilməyib): bütün tətbiqlər trace-ləri OTLP/gRPC ilə OpenTelemetry Collector-a (4317), o da Tempo-ya göndərir; Prometheus biznes servislərini `/actuator/prometheus` üzərindən scrape edir; servislər log fayllarını `./logs` qovluğuna yazır, Promtail onları Loki-yə ötürür (fəsil 17).

2.6 Sinxron və asinxron sərhədlər

Qarşılıqlı əlaqə	Model	Uğursuzluq nəticəsi
Klient → Gateway → servis	Sinxron HTTP	Klient birbaşa xəta görür (401/403/404/409/429/500/503)

Qarşılıqlı əlaqə	Model	Uğursuzluq nəticəsi
Dashboard → Campaign (gRPC)	Sinxron, deadline-sız blocking stub	Campaign düşərsə dashboard 500 qaytarır (fəsil 13)
Auth → Log (audit)	`@Async` + Resilience4j; çağıran üçün asinxron	Audit itir, login davam edir (fəsil 14)
Campaign → Kafka (`campaign-events`)	Fire-and-forget, tranzaksiya daxilində	Hadisə itə və ya commit olunmayan dəyişiklik üçün göndərilə bilər (fəsil 11)
Servis → Kafka (`log-events`)	Fire-and-forget, HTTP filtrinin `finally` blokunda	Yavaş sorğu qeydi itə bilər; Kafka əlçatmazdırsa sorğu gecikə bilər
Kafka → log-service	Consumer group, retry + DLQ	Elasticsearch xətası → 3 təkrar → DLQ
Campaign → Redis cache	Sinxron	Redis xətası `@Cacheable` metodunda xəta yaradır

2.7 Data sahibliyi

Mikroservis arxitekturasının əsas prinsiplərindən biri **"hər servis öz datasının sahibidir"** prinsipidir. Bu repozitoriyada:

- **FAKT** auth-service və campaign-service ayrı PostgreSQL instanslarına malikdir (`docker-compose.yml:93-130`); heç bir servis digərinin bazasına qoşulmur.
- **FAKT** dashboard-service-in öz bazası yoxdur; kampaniya datasını yalnız gRPC kontraktı vasitəsilə alır. Bu, "paylaşılan baza" anti-pattern-indən qaçmağın düzgün nümunəsidir.
- **FAKT** log-service Elasticsearch-in yeganə istifadəçisidir.
- **FAKT** Redis iki fərqli məqsədlə iki servis tərəfindən paylaşılır: gateway rate limit açarları və campaign-service cache-i. Açar ad fəzaları fərqlidir (`request_rate_limiter.*` və `campaign-statistics::*`). **NƏTİCƏ** Bu, lokal case study üçün qəbuledilən sadələşdirmədir; istehsalda adətən ayrı Redis məntiqi bazaları və ya instansları istifadə olunur.

2.8 Əsas axınların icmalı

Detallı izləmə 20-ci fəsildədir. Burada yalnız ümumi mənzərə:

Axın	Giriş nöqtəsi	Toxunduğu komponentlər
Login	`POST /api/v1/auth/login`	Gateway (rate limit) → auth-service → auth_db → (async) log-service → Elasticsearch
Kampaniya yaratmaq	`POST /api/v1/campaigns`	Gateway (rate limit, JWT) → campaign-service (JWT, `@PreAuthorize` , `@Valid`) → campaign_db → Redis (evict) → Kafka `campaign-events` → (filtr) Kafka `log-events` → log-service → Elasticsearch
Təsdiq / deaktivasiya	`PATCH /api/v1/campaigns/{id}/approve|deactivate`	Yuxarıdakı kimi, əlavə status yoxlaması ilə
Dashboard statistikası	`GET /api/v1/dashboard/classifieds/statistics`	Gateway → dashboard-service → gRPC → campaign-service → Redis cache / campaign_db
Kampaniya tarixçəsi	`GET /api/v1/campaigns/{id}/history` və ya dashboard üzərindən	campaign_db
Yavaş sorğuları oxumaq	`GET /api/v1/logs/slow-requests`	Gateway → log-service → Elasticsearch

Axın	Giriş nöqtəsi	Toxunduğu komponentlər
Refresh / reset / logout	`POST /api/v1/auth/...`	auth-service → auth_db → audit

2.9 Komponent düşdükdə nə baş verir

Düşən komponent	Müşahidə olunan nəticə	Etiket
auth_db	Login və qeydiyyat 500 qaytarır; artıq verilmiş access token-lər işləməyə davam edir, çünki digər servislər token-i yalnız public key ilə yoxlayır	NƏTİCƏ
campaign_db	Kampaniya API-ləri 500; statistika cache-dədirsə gRPC cavab verə bilər	NƏTİCƏ
Redis	Gateway rate limiting **açıq qalır** (sorğular buraxılır); campaign statistikası cache xətası verir	FAKT SCG mənbəsi / NƏTİCƏ
Kafka	Hadisələr və yavaş sorğu jurnalları itir; producer metadata gözləyərkən sorğular gecikə bilər	NƏTİCƏ
Elasticsearch	Consumer təkrar cəhd edib DLQ-ya yazır; audit POST xəta verir, auth axını təsirlənmir	NƏTİCƏ
log-service	Auditlər itir (yalnız auth-service logunda xəbərdarlıq)	FAKT `AuthAuditService.java:147-179`
campaign-service	Dashboard 500, gateway campaign marşrutu xəta	NƏTİCƏ
Eureka	Gateway-in `lb://` marşrutları yeni instansları tapa bilmir; keşlənmiş reyestr müəyyən müddət işləyir	NƏTİCƏ
Config Server	`optional:` olduğu üçün servislər lokal konfiqurasiya ilə başlayır	FAKT `spring.config.import`

2.10 Öyrənmə ardıcıllığı

Asılılıqlara görə tövsiyə olunan sıra: build və modul quruluşu (3) → Spring mexanizmləri (4) → REST və təbəqələr (5) → ən sadə biznes servisi olan campaign-service (6) → onun persistence qatı (7) → validasiya və istisnalar (8) → auth-service (9) → bütün sistem üzrə security (10) → asinxron mesajlaşma, axtarış saxlaması, gRPC, HTTP klientləri, Redis, Spring Cloud, observability, konteynerlər, testlər (11–19) → uçdan-uca axınlar (20) → qiymətləndirmə və dizayn (21–22) → production və müsahibə (23–24).

2.11 Müsahibə sualları

S1. Bu sistemi 1 dəqiqədə izah edin. Cavab: Sığorta kampaniyalarını idarə edən mikroservis platformasıdır. Klient sorğuları Spring Cloud Gateway-dən keçir; gateway JWT-ni yoxlayır və Redis əsaslı rate limit tətbiq edir. auth-service istifadəçiləri və token-ləri idarə edir, JWT-ni RSA private key ilə imzalayır; digər servislər public key ilə yoxlayır. campaign-service kampaniya həyat dövrünü (yaratma, təsdiq, deaktivasiya, tarixçə) PostgreSQL-də saxlayır, statistikanı Redis-də cache edir, status dəyişikliklərini Kafka-ya hadisə kimi yazır və gRPC API təqdim edir. dashboard-service statistikanı bu gRPC ilə alır. log-service Kafka-dan yavaş sorğu hadisələrini və HTTP ilə audit hadisələrini qəbul edib Elasticsearch-də saxlayır. Eureka, Config Server və Prometheus/Loki/Tempo/Grafana platforma təbəqəsidir.

S2. Niyə dashboard-service öz bazasından deyil, gRPC-dən istifadə edir? Cavab: Kampaniya datasının sahibi campaign-service-dir. Dashboard-un həmin cədvəllərə birbaşa qoşulması servis sərhədini pozar, sxem dəyişikliyini iki servisə bağlayar. gRPC kontraktı (`campaign.proto`) aydın, tipli və versiyalana bilən interfeys verir. Alternativ — campaign hadisələrindən dashboard-un öz oxu modelini (CQRS read model) qurmaqdır; bu, campaign-service düşdükdə belə dashboard-un işləməsinə imkan verərdi, lakin son ardıcıllıq (eventual consistency) və əlavə infrastruktur gətirərdi.

S3. Hansı əlaqələr asinxrondur və bu nə qazandırır? Cavab: Kafka hadisələri (`campaign-events`, `log-events`) və auth audit (`@Async`). Qazanc: çağıran əməliyyat ikinci dərəcəli yan təsirin (audit, jurnal) uğursuzluğundan təcrid olunur və cavab müddəti artmır. Bədəli: zəmanətli çatdırılma və ardıcıllıq üçün əlavə mexanizmlər (outbox, idempotent consumer, DLQ) lazımdır; bu layihədə onların yalnız bir hissəsi var.

S4. Bu arxitekturada tək uğursuzluq nöqtələri hansılardır? Cavab: Lokal konfiqurasiyada hər infrastruktur komponenti tək instansdır (Kafka replication factor 1, tək Eureka, tək Redis). Biznes baxımından api-gateway (bütün trafik) və campaign-service (həm REST, həm dashboard üçün gRPC) kritikdir. Redis düşərsə rate limiting söndürülmüş kimi davranır — bu, açıq-aşkar seçilmiş fail-open davranışıdır.

2.12 Xülasə

- Sistem kampaniya həyat dövrünü (yaratma → təsdiq → deaktivasiya) və onu dəstəkləyən platforma xidmətlərini həyata keçirir.
- 7 tətbiq, 3 kitabxana modulu; hər biznes servisi öz datasının sahibidir.
- Sinxron əlaqələr: HTTP (gateway, audit) və gRPC (dashboard → campaign). Asinxron: Kafka.
- Eureka yalnız gateway marşrutlaşdırmasında istifadə olunur; servislərarası çağırışlar static ünvanlıdır.
- Növbəti fəsil repozitoriyanın Maven quruluşunu, versiyaları və build prosesini izah edir.

3. Repozitoriya quruluşu, Maven və build sistemi

3.1 Niyə build quruluşunu əvvəldən öyrənirik

Mikroservis repozitoriyasında ilk sual "hansı kod hansı prosesdə işləyir?" sualıdır. Bu sualın cavabı build konfiqurasiyasındadır: hansı modul müstəqil tətbiqdir, hansı kitabxanadır, kim kimdən asılıdır, versiyalar haradan gəlir. Bu fəsildən sonra siz istənilən sinfi gördükdə onun hansı JVM prosesində yaşadığını və hansı asılılıqların onu mümkün etdiyini deyə biləcəksiniz.

3.2 Monorepo və multi-module Maven

Layihə **monorepo**-dur: bütün servislər bir Git repozitoriyasında, bir Maven reaktorunda qurulur. Kök `pom.xml` `packaging=pom` olan **aqreqator və parent** POM-dur FAKT (`pom.xml:5-25`):

Mənbə: `pom.xml:7-25`

```
<groupId>com.casestudy.insurancecampaign</groupId>
<artifactId>campaign-microservices</artifactId>
<version>1.0.0</version>
<packaging>pom</packaging>
<name>Allianz Campaign Microservices</name>
<description>Java 25 Spring Boot microservices case study for Allianz Sigorta Campaign API</description>

<modules>
    <module>common</module>
    <module>security</module>
    <module>grpc-contract</module>
    <module>discovery-server</module>
    <module>config-server</module>
    <module>api-gateway</module>
    <module>auth-service</module>
    <module>campaign-service</module>
    <module>dashboard-service</module>
    <module>log-service</module>
</modules>
```

İki anlayışı ayırmaq lazımdır:

- **Aqreqasiya (`<modules>`)** — `mvn` kökdən işə salındıqda hansı modulların bir reaktorda qurulacağını bildirir. Maven modullar arası asılılıqlara baxıb düzgün sıra müəyyən edir (reaktor sıralaması), siyahının yazılış sırası deyil.
- **Varislik (`<parent>`)** — hər modul `<parent>` olaraq kök POM-u göstərir və onun `properties`, `dependencyManagement`, `pluginManagement` və `build/plugins` bölmələrini miras alır.

Monorepo-nun üstünlüyü: paylaşılan kontraktlar (`grpc-contract`, `common` hadisələri) atomik dəyişdirilir, bir commit-də həm producer, həm consumer yenilənir. Mənfi tərəfi: modullar müstəqil versiyalanmır (hamısı `1.0.0`), bir modulun dəyişməsi bütün image-lərin yenidən qurulmasına səbəb olur (CI-də matrix build hər servisi qurur, fəsil 18).

3.3 Versiya idarəetməsi: BOM import və `spring-boot-starter-parent` olmaması

Diqqətəlayiq qərar: kök POM `spring-boot-starter-parent`-dən miras **almır**. Bunun əvəzinə Spring Boot, Spring Cloud, gRPC və Testcontainers üçün dörd **BOM** (Bill of Materials) idxal edilir FAKT (`pom.xml:69-100`):

Mənbə: pom.xml:69-100

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-dependencies</artifactId>
            <version>${spring.boot.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring.cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
        <dependency>
            <groupId>io.grpc</groupId>
            <artifactId>grpc-bom</artifactId>
            <version>${grpc.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
        <dependency>
            <groupId>org.testcontainers</groupId>
            <artifactId>testcontainers-bom</artifactId>
            <version>${testcontainers.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

BOM nədir? `type=pom, scope=import` olan asılılıq Maven-ə deyir: "həmin POM-un `dependencyManagement` bölməsini bura kopyala". Nəticədə modullar `spring-boot-starter-web` kimi asılılıqları versiyasız yaza bilir; versiya BOM-dan gəlir. Bu, tranzitiv asılılıqların bir-biri ilə uyğun versiyalarda olmasını təmin edir.

`spring-boot-starter-parent` istifadə etməməyin nəticələri NƏTİCƏ:

- Parent POM-un verdiyi plugin konfiqurasiyaları (məsələn, `spring-boot-maven-plugin`-in `repackage` icrası, resursların filtrlənməsi, `-parameters` kompilyator bayrağı) avtomatik gəlmir. Layihə bunları özü edir: `spring-boot-maven-plugin` versiyası `pluginManagement`-də (`pom.xml:111-115`), `repackage` icrası isə hər tətbiq modulunda ayrıca yazılıb (məs. `campaign-service/pom.xml:184-194`); `<parameters>true</parameters>` isə kompilyator konfiqurasiyasındadır (`pom.xml:138-145`).
- `-parameters` bayrağı vacibdir: Spring `@PathVariable UUID id` kimi annotasiyalarda ad verilmədikdə parametr adını bytecode-dan oxuyur. Bu bayraq olmasaydı, `@PathVariable` adları açıq yazılmalı olardı.
- Üstünlük: layihə başqa korporativ parent POM-dan miras ala bilər və bir neçə BOM-u (Cloud, gRPC) bərabər hüquqlu idxal edir. **BOM sırası vacibdir**: eyni artefakt bir neçə BOM-da olarsa, birinci idxal edilən qalib gəlir. Spring Boot BOM-u birincidir.

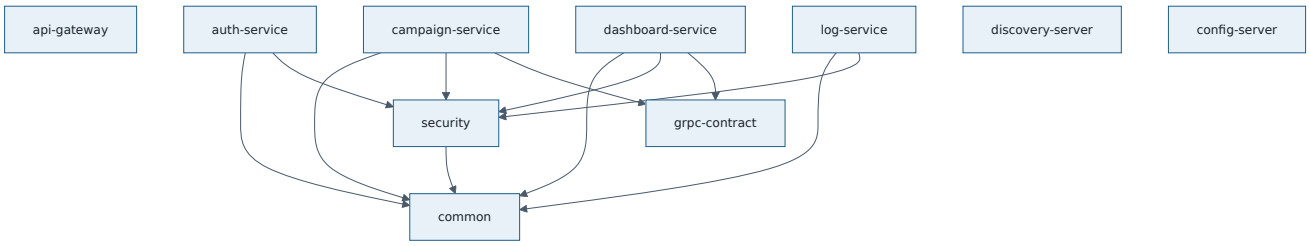
gRPC BOM-un rolu. README-də "gRPC Runtime Version Mismatch" bölməsi `NoClassDefFoundError: io/grpc/InternalGlobalInterceptors` xətasını təsvir edir və həll kimi bütün `grpc-*` modullarının bir BOM ilə uyğunlaşdırılmasını göstərir. NƏTİCƏ Bu xəta adətən `grpc-core` (net.devh starter-dən tranzitiv) və `grpc-api` (birbaşa elan edilmiş) fərqli versiyalarda həll olunduqda yaranır; `grpc-bom` hamısını 1.80.0-a bağlayır. Asılılıq ağacı bunu təsdiqləyir: campaign-service-də `grpc-core`, `grpc-netty-shaded`, `grpc-services` hamısı `1.80.0`-dır.

3.4 Versiyalar cədvəli

Komponent	Versiya	Necə müəyyən edilib
Java	25	`pom.xml:28-29`
Spring Boot	4.1.0	`pom.xml:30`
Spring Cloud	2025.1.2	`pom.xml:31`
Spring Framework / Security	7.0.8 / 7.1.0	`mvn dependency:tree`
Hibernate ORM	7.4.1.Final	`mvn dependency:tree`
Liquibase	5.0.3	`mvn dependency:tree`
Spring Kafka / kafka-clients	4.1.0 / 4.2.1	`mvn dependency:tree`
Spring Data Elasticsearch / Elasticsearch Java client	6.1.0 / 9.4.2	`mvn dependency:tree`
Spring Cloud Gateway	5.0.2	`mvn dependency:tree`
gRPC / Protobuf / net.devh starter	1.80.0 / 4.34.2 / 3.1.0.RELEASE	`pom.xml:35-37`
JJWT	0.12.6	`pom.xml:34`
Resilience4j	2.4.0 (`resilience4j-spring-boot4`)	`auth-service/pom.xml:16`
springdoc-openapi	2.8.13	`pom.xml:32`
Testcontainers	1.21.3	`pom.xml:33`

Bu versiyalar kitabdakı bütün framework davranışı izahlarının əsasıdır. Məsələn, Spring Boot 4-də auto-configuration modullara bölünüb, Jackson 3 standart JSON kitabxanası olub; Spring Kafka 4-də `JsonSerializer` köhnəlmiş (deprecated) elan edilib. Bunlar aşağıda və müvafiq fəsillərdə izah olunur.

3.5 Modullar arası asılılıq qrafı



Maven modul asılılıqları (yalnız layihə daxili modullar)

FAKT `api-gateway`, `discovery-server` və `config-server` heç bir daxili moduldan asılı deyil. Gateway reaktiv (WebFlux) olduğu üçün servlet əsaslı `security` modulunu istifadə edə bilməz; buna görə öz JWT siniflərini (`GatewayJwtTokenService`, `GatewayPemKeyUtils`) saxlayır. **NƏTİCƏ** Bu, kod təkrarına səbəb olsa da, reaktiv və servlet stack-larının qarışmasının qarşısını alır. Eyni səbəbdən observation konfiqurasiyası üç dəfə kopyalanıb (tapıntı F23, fəsil 17).

Paylaşılan modulun "gizli" təsiri

`common` və `security` modullarında `@Component`, `@Configuration`, `@RestControllerAdvice` olan siniflər var. Onlar tətbiqə avtomatik qoşulmur — Spring Boot yalnız `@SpringBootApplication` sinfinin paketini və alt paketlərini skan edir. Bütün tətbiqlər skan bazasını genişləndirir **FAKT** (məs. `CampaignServiceApplication.java:8`):

```
@SpringBootApplication(scanBasePackages = "com.casestudy.insurancecampaign")
```

Bu tək sətir sayəsində `common.exception.GlobalExceptionHandler`, `security.ServiceSecurityConfiguration`, `common.web.SlowRequestLoggingFilter` və s. hər servisdə bean-ə çevrilir. Bu mexanizm 4-cü fəsildə ətraflı izah olunur. Alternativ (və kitabxanalar üçün daha "Spring-vari") yol öz auto-configuration-unuzu `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` faylı ilə qeydiyyatdan keçirməkdir.

3.6 Spring Boot 4-ün modullaşdırılmış auto-configuration-u

Spring Boot 4-də əvvəllər `spring-boot-autoconfigure` jar-ında olan auto-configuration-lar texnologiya üzrə ayrı modullara bölünüb (`spring-boot-kafka`, `spring-boot-liquibase`, `spring-boot-data-redis`, `spring-boot-security` və s.). Starter-lər bu modulları gətirir. Bu repozitoriyada bunun real nəticələri var **FAKT** (POM-lar və asılılıq ağacı):

Modul	POM-da nə var	Nəticə
campaign-service	`spring-kafka` (starter yox)	Boot-un Kafka auto-configuration-u **yoxdur**; `KafkaTemplate` yalnız `common`-dakı `KafkaJsonConfiguration`-dan gəlir
log-service	`spring-kafka` + `spring-boot-kafka`	Boot `ConsumerFactory`-ni `spring.kafka.consumer.*` xassələrindən qurur; `KafkaTemplate` / `ProducerFactory` `@ConditionalOnMissingBean` olduğu üçün `common`-dakı bean-lər qalib gəlir
campaign/auth-service	`liquibase-core` + `spring-boot-liquibase`	Liquibase startup-da changelog-u icra edir
auth-service	`spring-boot-starter-restclient`	`RestClient` və HTTP Service Client (`@ImportHttpServices`) dəstəyi
bütün biznes servisləri	`spring-boot-starter-opentelemetry`	Micrometer Tracing + OTel bridge + OTLP exporter
testlər	`spring-boot-starter-webmvc-test`	`@AutoConfigureMockMvc` yeni paketdə: `org.springframework.boot.webmvc.test.autoconfigure`

NƏTİCƏ Praktiki dərs: Spring Boot 4-də "kitabxananı əlavə etdim, amma auto-configuration işləmir" problemi yarandıqda əvvəlcə müvafiq `spring-boot-*` modulunun və ya starter-in classpath-də olduğunu yoxlayın.

3.7 Jackson 3 və Jackson 2 birgə yaşayışı

Spring Boot 4 standart olaraq **Jackson 3** (`tools.jackson.core:jackson-databind`, paket `tools.jackson.*`) istifadə edir. Asılılıq ağacında campaign-service üçün həm Jackson 3 (`3.1.4`), həm Jackson 2 (`2.21.4`) görünür. Jackson 2 iki yoldan gəlir:

1. `common/pom.xml:32-35` — `jackson-datatype-jsr310` birbaşa elan olunub. **NƏTİCƏ** Bu modul Spring Kafka-nın `JsonSerializer`-i (Jackson 2 əsaslı) `Instant` sahələrini serializə edə bilsin deyə lazımdır.
2. `spring-cloud-netflix-eureka-client → spring-boot-jackson2` — Eureka klienti Boot-un **deprecated** `spring-boot-jackson2` modulunu gətirir. Bu modulun `Jackson2AutoConfiguration` sinfi (`@Deprecated(since = "4.0.0", forRemoval = true)`) `com.fasterxml.jackson.databind.ObjectMapper` bean-i yaradır.

Kodda Jackson 2 `ObjectMapper` birbaşa inject olunur **FAKT**: `ServiceSecurityConfiguration.java:3,49-52` və `GatewayRateLimitErrorResponseFilter.java:4,43-45`. **RİSK** (F20) Bu bean-in mövcudluğu Eureka klientinin tranzitiv asılılığına bağlıdır. Eureka çıxarılsa və ya Spring Cloud gələcək versiyada `spring-boot-jackson2`-ni buraxsa, kontekst `ObjectMapper` bean-i tapmadığı üçün başlamayacaq. Düzgün həll — Jackson 3 `JsonMapper`-i inject etmək və ya Spring-in `HttpMessageConverters`-indən istifadə etmək.

HTTP cavabları (MVC `@RestController`) üçün isə Spring Boot 4-ün standart Jackson 3 konvertoru işləyir. **NƏTİCƏ** Buna görə `Instant` sahələri JSON-da ISO-8601 sətri kimi görünür (testlərdə `"2026-07-27T09:00:00Z"` gözlənilir, `DashboardControllerTest.java:123`).

3.8 `grpc-contract`: koddan əvvəl kod generasiyası

`grpc-contract` modulunda Java kodu yoxdur — yalnız `campaign.proto`. Java sinifləri build zamanı generasiya olunur **FAKT** (`grpc-contract/pom.xml:48-104`):

1. **`os-maven-plugin` extension** (`grpc-contract/pom.xml:49-55`) — `${os.detected.classifier}` dəyişənini (məs. `linux-x86_64`) müəyyən edir, çünki `protoc` native binary-dir.
2. **`protobuf-maven-plugin`** (`grpc-contract/pom.xml:57-74`) — `compile` məqsədi `protoc` ilə mesaj siniflərini (`CampaignStatisticsRequest` və s.), `compile-custom` məqsədi `protoc-gen-grpc-java` plugin-i ilə servis stub-larını (`CampaignGrpcServiceGrpc`) generasiya edir.
3. **`build-helper-maven-plugin`** (`grpc-contract/pom.xml:75-94`) — generasiya olunan qovluqları (`target/generated-sources/protobuf/java` və `.../grpc-java`) mənbə kökü kimi əlavə edir ki, IDE onları tanısın.
4. `javax.annotation-api` (`grpc-contract/pom.xml:40-45`) — generasiya olunan kod hələ də `javax.annotation.Generated` istifadə edir (POM-dakı şərh bunu açıq deyir).

README-dəki "IntelliJ `Cannot resolve symbol 'grpc'`" problemi məhz buradan gəlir: generasiya `mvn install` icra olunmadan baş vermir. Həll: `mvn -pl grpc-contract -am clean install`.

3.9 Test və keyfiyyət plugin-ləri

Kök POM bütün modullara aşağıdakı plugin-ləri tətbiq edir **FAKT** (`pom.xml:137-205`):

Mənbə: `pom.xml:147-171`

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <configuration>
        <argLine>@{argLine} --enable-preview</argLine>
        <useModulePath>false</useModulePath>
    </configuration>
</plugin>

<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-failsafe-plugin</artifactId>
    <configuration>
        <argLine>@{argLine} --enable-preview</argLine>
        <useModulePath>false</useModulePath>
    </configuration>
    <executions>
        <execution>
            <goals>
                <goal>integration-test</goal>
                <goal>verify</goal>
            </goals>
        </execution>
```

```
    </executions>
</plugin>
```

- `@{argLine}` — gecikmiş (late) property qiymətləndirməsidir. JaCoCo-nun `prepare-agent` məqsədi `argLine` xassəsinə `-javaagent:...jacoco.jar` əlavə edir; `@{...}` sintaksisi surefire-ə bu dəyəri test başlayanda oxumağı deyir. Kök POM-da boş `<argLine></argLine>` (`pom.xml:42`) JaCoCo olmadıqda belə dəyişənin təyin olunmasını təmin edir. Bu, JaCoCo + əlavə JVM arqumentlərini birləşdirməyin standart üsuludur.
- `--enable-preview` — Java preview xüsusiyyətlərini aktivləşdirir. **NƏTİCƏ** Kodda preview xüsusiyyəti istifadə olunduğuna dair açıq nümunə tapılmadı (record, switch ifadəsi, pattern matching `instanceof` Java 25-də stabildir). auth-service `--enable-preview` ilə kompilyasiya olunur (`auth-service/pom.xml:181`); preview ilə kompilyasiya olunmuş sinif runtime-da da `--enable-preview` tələb edir — Dockerfile-lardakı bayraq buna görə zəruridir.
- **Surefire və failsafe fərqi**: surefire `*Test` siniflərini `test` fazasında, failsafe `*IT` siniflərini `integration-test` fazasında işlədir. **FAKT** Repozitoriyada `*IT` sinfi yoxdur; Testcontainers istifadə edən inteqrasiya testləri də `*Test` adlanır və surefire ilə işləyir. Failsafe konfiqurasiya olunub, lakin heç bir test icra etmir.
- **JaCoCo** — `prepare-agent` və `verify` fazasında `report` (XML + HTML) (`pom.xml:179-204`). SonarQube əhatəni `**/target/site/jacoco/jacoco.xml`-dən oxuyur; `common`, `security`, `api-gateway`, `config-server`, `discovery-server`, `grpc-contract` əhatə hesablamasından çıxarılıb (`pom.xml:57-64`). **NƏTİCƏ** Bu modulların testi olmadığı üçün çıxarılması əhatə faizini "gözəlləşdirir", lakin həmin kod (məs. `GlobalExceptionHandler`, JWT filtri) bütün servislərin davranışına təsir edir.

3.10 auth-service: annotation processor-lar

auth-service Lombok və MapStruct istifadə edən yeganə moduldur **FAKT** (`auth-service/pom.xml:159-183`):

Mənbə: `auth-service/pom.xml:162-182`

```
<configuration>
    <source>${java.version}</source>
    <target>${java.version}</target>
    <annotationProcessorPaths>
        <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
        </path>
        <path>
            <groupId>org.mapstruct</groupId>
            <artifactId>mapstruct-processor</artifactId>
            <version>${mapstruct.version}</version>
        </path>
        <path>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok-mapstruct-binding</artifactId>
            <version>${lombok-mapstruct-binding.version}</version>
        </path>
    </annotationProcessorPaths>
    <compilerArgs>--enable-preview</compilerArgs>
</configuration>
```

- Lombok `@Getter` getter-ləri, MapStruct isə `AuthMapper` interfeysinin implementasiyasını (`AuthMapperImpl`) kompilyasiya zamanı generasiya edir. `lombok-mapstruct-binding` iki processor-un sırasını uzlaşdırır: MapStruct Lombok-un generasiya etdiyi getter-ləri görməlidir.
- **NƏTİCƏ** Lombok-un versiyası `annotationProcessorPaths`-də göstərilməyib; Maven bu halda versiyanı `dependencyManagement`-dən götürə bilir (Maven 3.9+ compiler plugin 3.12+ bunu dəstəkləyir), kök POM-da compiler plugin `3.14.1`-dir (`pom.xml:105-109`).

- Digər servislər Lombok istifadə etmir: entity-lərdə getter-lər əl ilə yazılıb (məs. `CampaignEntity.java:70-77`). Bu, eyni repozitoriyada iki fərqli üslubun mövcudluğudur — müsahibədə "komanda standartı" sualına nümunədir.

3.11 Tətbiq artefaktları və Dockerfile

Hər tətbiq modulu `spring-boot-maven-plugin:repackage` ilə **icra oluna bilən fat jar** yaradır (jar daxilində `BOOT-INF/lib` asılılıqları və `JarLauncher`). Dockerfile-lar çoxmərhələlidir FAKT (`campaign-service/Dockerfile:1-26`):

Mənbə: campaign-service/Dockerfile:1-26

```
# Stage 1: Build stage
FROM maven:3.9.11-amazoncorretto-25 AS build

# Set working directory
WORKDIR /workspace

# Copy full multi-module project
COPY . .

# Build only campaign-service and required modules
RUN mvn -pl campaign-service -am clean package -DskipTests

# Stage 2: Run stage
FROM amazoncorretto:25

# Set working directory
WORKDIR /app

# Copy the executable Spring Boot JAR file from the build stage
COPY --from=build /workspace/campaign-service/target/campaign-service-*.jar campaign-service.jar

# Expose REST and gRPC ports
EXPOSE 8082 9090

# Set the entrypoint command for running the application
ENTRYPOINT ["java", "--enable-preview", "-jar", "campaign-service.jar"]
```

- `-pl campaign-service -am` — "also make": seçilmiş modulu və onun asılı olduğu modulları (`common`, `security`, `grpc-contract`) qurur.
- `COPY . .` bütün repozitoriyanı build konteksinə köçürür. NƏTİCƏ Bu, Docker layer cache-ini zəiflədir: istənilən faylın dəyişməsi (hətta README) Maven asılılıqlarının yenidən endirilməsinə səbəb olur, çünki `~/.m2` cache-i layer-da saxlanmır. Təkmilləşdirmə: əvvəlcə yalnız POM-ları köçürüb `mvn dependency:go-offline`, sonra mənbələri köçürmək; və ya BuildKit `--mount=type=cache,target=/root/.m2`.
- Run mərhələsi tam JDK image-idir (`amazoncorretto:25`), root istifadəçi ilə işləyir, `HEALTHCHECK` və JVM yaddaş parametrləri yoxdur. Bunlar fəsil 18-də müzakirə olunur.

3.12 Build əmrləri və tipik problemlər

```
mvn clean verify                          # bütün modullar, testlər, JaCoCo hesabatları
mvn -pl campaign-service -am test         # bir servis və asılılıqları
mvn -pl grpc-contract -am clean install   # gRPC siniflərini generasiya edib lokal repoya qoymaq
mvn -pl auth-service spring-boot:run      # servisi işə salmaq (əvvəlcə asılılıqlar install olunmalıdır)
```

Problem	Səbəb	Həll
`Cannot resolve symbol 'grpc'`	Generasiya olunan siniflər yoxdur	`mvn -pl grpc-contract -am clean install`, IDE-də Maven reload

Problem	Səbəb	Həll
`NoClassDefFoundError: io/grpc/InternalGlobalInterceptors`	`grpc-*` versiyaları uyğunsuzdur	`grpc-bom` import (artıq var), `mvn -U clean install`
`release version 25 not supported`	JDK 25 deyil	Project SDK və Maven runner JDK 25
`Preview features are not enabled`	`--enable-preview` olmadan işə salınma	IDE run konfiqurasiyasına VM option əlavə edin
Testlər Docker xətası ilə düşür	Testcontainers Docker tapmır	Docker işləməlidir
Kontekst `No qualifying bean of type ObjectMapper` ilə başlamır (hipotetik)	`spring-boot-jackson2` classpath-dən çıxıb (F20)	Jackson 3 `JsonMapper` istifadə edin

3.13 Yeni layihədə tətbiq

Oxşar monorepo qurarkən:

1. Kök POM-u `packaging=pom` edin; `spring-boot-dependencies`, `spring-cloud-dependencies` və digər BOM-ları `dependencyManagement`-ə idxal edin (Boot BOM-u birinci).
2. Kontraktları (proto, Avro/JSON schema, paylaşılan hadisə record-ları) ayrıca **nazik** modula çıxarın; həmin modul Spring-dən asılı olmasın.
3. Paylaşılan Spring konfiqurasiyasını `scanBasePackages` əvəzinə **öz auto-configuration**-unuz kimi qeydiyyatdan keçirin — bu, `@ConditionalOn...` şərtlərinin düzgün sırada qiymətləndirilməsini və tətbiqin "opt-out" imkanını verir.
4. `spring-boot-maven-plugin`-i `pluginManagement`-də versiyalayın, `repackage`-i yalnız tətbiq modullarında aktiv edin.
5. Test plugin-lərində `@{argLine}` istifadə edin ki, JaCoCo agenti itməsin.

3.14 Müsahibə sualları

S1. BOM import ilə `spring-boot-starter-parent` arasındakı fərq nədir? Cavab: Parent POM həm versiyaları (`dependencyManagement`), həm də plugin konfiqurasiyalarını, resurs filtrlənməsini, kompilyator parametrlərini miras verir. BOM import yalnız versiyaları gətirir. Parent tək ola bilər; BOM-ları istənilən sayda idxal etmək olar. Layihə parent-siz yolu seçib və lazımi plugin konfiqurasiyalarını özü yazıb.

S2. `-am` və `-pl` nə edir? Cavab: `-pl` (projects list) reaktordan yalnız göstərilən modulları seçir; `-am` (also make) onların asılı olduğu modulları da qurur. Dockerfile-larda `mvn -pl campaign-service -am package` `common`, `security`, `grpc-contract` və `campaign-service`-i qurur.

S3. `common`-da `spring-kafka`-nın `optional` olması nə deməkdir və nəyə təsir edir? Cavab: Optional asılılıq tranzitiv ötürülmür. `common` Kafka siniflərinə qarşı kompilyasiya olunur, lakin `common`-dan asılı servis `spring-kafka`-nı özü elan etmədikcə onu almır. `KafkaJsonConfiguration` və `SlowRequestLoggingFilter` `@ConditionalOnClass(name = "org.springframework.kafka.core.KafkaTemplate")` ilə qorunur; nəticədə filtr yalnız campaign və log-service-də aktivdir.

S4. Niyə gateway `security` modulunu istifadə etmir? Cavab: `security` modulu servlet API-yə (`OncePerRequestFilter`, `HttpSecurity`) əsaslanır. Gateway Spring WebFlux (Netty) üzərində işləyir; orada `WebFilter` / `GlobalFilter` və `ServerHttpSecurity` istifadə olunur. Servlet və reaktiv stack-ları bir tətbiqdə qarışdırmaq ya mümkün deyil, ya da gözlənilməz davranış yaradır.

S5. Spring Boot 4-də Jackson ilə bağlı nəyə diqqət etməli? Cavab: Standart Jackson 3-dür (`tools.jackson`), Jackson 2 dəstəyi deprecated `spring-boot-jackson2` modulundadır. Bu layihədə Jackson 2 `ObjectMapper` bean-i Eureka klientinin tranzitiv asılılığı sayəsində mövcuddur; bu kövrək asılılıqdır.

3.15 Xülasə

- 10 modul: 3 kitabxana (`common` , `security` , `grpc-contract`), 7 tətbiq.
- Versiyalar dörd BOM ilə idarə olunur; parent POM `spring-boot-starter-parent` deyil.
- Paylaşılan Spring bean-ləri `scanBasePackages = "com.casestudy.insurancecampaign"` ilə qoşulur.
- Spring Boot 4 modullaşdırılmış auto-configuration və Jackson 3 gətirir; layihədə bunun konkret nəticələri var.
- gRPC kodu build zamanı `protoc` ilə generasiya olunur.
- Növbəti fəsil bu bean-lərin necə yaradıldığını, şərti konfiqurasiyanı və proxy mexanizmlərini təzələyir.

II hissə. Təməlləri təzələmək

Java 25, Spring Boot 4, IoC/DI, auto-configuration, AOP, REST və təbəqəli dizayn

4. Java 25 və Spring Boot 4 təməlləri — layihə üzərində təzələmə

Bu fəsil Spring-in əsas mexanizmlərini abstrakt lüğət kimi deyil, **bu repozitoriyadakı konkret siniflər** üzərində təzələyir. Məqsəd: hər annotasiyanı gördükdə "bu nəyi dəyişir, nə vaxt işləmir, hansı runtime obyekti yaradır" suallarına cavab verə bilmək.

4.1 Problem: tətbiq özünü necə yığır?

`CampaignService` dörd asılılığa ehtiyac duyur: iki repository, bir mapper və bir `KafkaTemplate`. Heç bir yerdə `new CampaignService(...)` yazılmayıb, lakin servis işləyir. Kim yaradır, kim asılılıqları ötürür, nə vaxt? Cavab — **IoC konteyneri** (`ApplicationContext`). Spring Boot startup-da:

1. `@SpringBootApplication` sinfindən başlayaraq **komponent skanı** aparır və bean tərifləri (BeanDefinition) toplayır;
2. **auto-configuration** siniflərini yükləyir və onların **şərtlərini** (`@ConditionalOn...`) qiymətləndirir;
3. bean-ləri asılılıq qrafına görə yaradır, asılılıqları **konstruktor vasitəsilə** ötürür;
4. **BeanPostProcessor**-lar bean-ləri lazım gəldikdə **proxy**-yə bükür (`@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize`, Resilience4j annotasiyaları);
5. web server-i (Tomcat və ya Netty), Kafka listener konteynerlərini, gRPC server-i başladır.

4.2 Dependency Injection: konstruktor injeksiyası standartdır

Layihədə bütün servislər **konstruktor injeksiyası** istifadə edir **FAKT** (`CampaignService.java:40-52`):

Mənbə: `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:40-52`

```
public CampaignService(
        CampaignRepository campaignRepository,
        CampaignStatusHistoryRepository historyRepository,
        CampaignMapper campaignMapper,
        KafkaTemplate<String, Object> kafkaTemplate,
        @Value("${app.kafka.topics.campaign-events:campaign-events}") String campaignEventsTopic
) {
    this.campaignRepository = campaignRepository;
    this.historyRepository = historyRepository;
    this.campaignMapper = campaignMapper;
    this.kafkaTemplate = kafkaTemplate;
    this.campaignEventsTopic = campaignEventsTopic;
}
```

Yada salınacaq qaydalar:

- Sinifdə **tək konstruktor** varsa, `@Autowired` lazım deyil — Spring onu avtomatik istifadə edir.
- Sahələr `final`-dır: obyekt tam qurulmuş vəziyyətdə yaranır, dəyişməzdir, thread-safe-dir.
- Konfiqurasiya dəyəri də konstruktor parametri kimi gəlir (`@Value(... :campaign-events)` — iki nöqtədən sonrakı hissə standart dəyərdir).
- **Testlənə bilmə**: `CampaignServiceTest` Spring konteksti olmadan obyekti birbaşa yaradır (`CampaignServiceTest.java:37`): `new CampaignService(campaignRepository, historyRepository, campaignMapper, kafkaTemplate, "campaign-events")`.

İstisna: `@GrpcClient` sahə injeksiyası

dashboard-service-də gRPC stub sahəyə inject olunur **FAKT** (`CampaignGrpcClient.java:31-32`):

Mənbə: dashboard-service/src/main/java/com/casestudy/insurancecampaign/dashboard/client/CampaignGrpcClient.java:31-32

```
@GrpcClient("campaign-service")
private CampaignGrpcServiceGrpc.CampaignGrpcServiceBlockingStub campaignStub;
```

`@GrpcClient` net.devh kitabxanasının öz `BeanPostProcessor`-u tərəfindən emal olunur; stub Spring bean-i deyil, hər injeksiya nöqtəsi üçün kanal üzərində yaradılır. Nəticə: unit testdə sahəni əl ilə doldurmaq lazım gəlir — test bunu `ReflectionTestUtils.setField` ilə edir (`CampaignGrpcClientTest.java:35-41`). NƏTİCƏ Sahə injeksiyası testlənməni çətinləşdirir; net.devh həm də `@GrpcClient`-in metod/konstruktor parametrində istifadəsini və ya `GrpcChannelFactory` ilə stub-u `@Bean` kimi yaratmağı dəstəkləyir.

4.3 Stereotiplər və `@Configuration` / `@Bean`

Annotasiya	Layihədə harada	Mənası
`@Component`	`CampaignMapper`, `CampaignGrpcClient`, `SlowRequestLoggingFilter`, `JwtAuthenticationFilter`, gateway filtrləri	Ümumi bean
`@Service`	`CampaignService`, `AuthService`, `AuthAuditService`, `JwtTokenService`, `SlowRequestLogConsumer`	Semantik olaraq servis qatı; texniki olaraq `@Component`
`@RestController`	bütün controller-lər	`@Controller` + `@ResponseBody`
`@RestControllerAdvice`	`GlobalExceptionHandler`	Bütün controller-lər üçün mərkəzi `@ExceptionHandler`-lər
`@Repository`	**istifadə olunmayıb**	Spring Data interfeysləri annotasiyasız aşkarlanır; istisna tərcüməsi repository proxy-si tərəfindən edilir
`@Configuration` + `@Bean`	`KafkaJsonConfiguration`, `AuthConfiguration`, `KafkaDlqConfiguration`, gateway konfiqurasiyaları	Üçüncü tərəf obyektlərinin (və ya xüsusi qurulmuş obyektlərin) bean kimi qeydiyyatı
`@GrpcService`	`CampaignGrpcEndpoint`	net.devh: bean + gRPC server-ə servis kimi qeydiyyat
`@Mapper(componentModel = "spring")`	`AuthMapper`	MapStruct generasiya etdiyi implementasiyanı `@Component` edir

`@Configuration` və `proxyBeanMethods`

`@Configuration` sinfi standart olaraq **CGLIB proxy**-yə çevrilir ki, bir `@Bean` metodundan digərini çağırdıqda yeni obyekt deyil, konteynerdəki singleton qaytarılsın. Layihədə `KafkaJsonConfiguration` bu problemi düzgün həll edir: `kafkaTemplate` bean-i `producerFactory()`-ni birbaşa çağırır, **parametr kimi alır** FAKT (`KafkaJsonConfiguration.java:57-60`):

Mənbə: common/src/main/java/com/casestudy/insurancecampaign/common/config/KafkaJsonConfiguration.java:57-60

```
@Bean
public KafkaTemplate<String, Object> kafkaTemplate(ProducerFactory<String, Object> producerFactory) {
    return new KafkaTemplate<>(producerFactory);
}
```

Parametr injeksiyası ilə proxy-yə ehtiyac qalmır. `LogServiceHttpClientConfiguration` isə bunu açıq bildirir: `@Configuration(proxyBeanMethods = false)` (`LogServiceHttpClientConfiguration.java:18`) — bu, startup-u bir az sürətləndirir və GraalVM native image üçün tövsiyə olunan formadır.

4.4 Komponent skanı və paylaşılan modullar

Bütün biznes servisləri və gateway skan bazasını kök paketə genişləndirir **FAKT**:

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/AuthServiceApplication.java:17-20

```
@SpringBootApplication(scanBasePackages = "com.casestudy.insurancecampaign")
@EnableDiscoveryClient
@ImportHttpServices(group = "log-service", types = LogServiceHttpClient.class)
public class AuthServiceApplication {
```

Bu o deməkdir ki, `common` və `security` jar-larındakı bütün `@Component` / `@Configuration` sinifləri istisnasız yüklənir. Nəticələr:

- **FAKT** Hər servis eyni `GlobalExceptionHandler`, `ServiceSecurityConfiguration`, `JwtAuthenticationFilter`-i alır. auth-service də daxil olmaqla — auth-service-də `/api/v1/auth/**` `permitAll` olduğu üçün problem yaranmır.
- **NƏTİCƏ** Servis "bu ortaq bean-i istəmirəm" deyə bilmir (yalnız `@ComponentScan(excludeFilters=...)` ilə). Öz auto-configuration-unuzu yazsaydınız, istifadəçi `@SpringBootApplication(exclude = ...)` və ya `@ConditionalOnMissingBean` ilə əvəzləyə bilərdi.
- `@EnableDiscoveryClient` müasir Spring Cloud-da məcburi deyil — classpath-də Eureka client olduqda qeydiyyat avtomatikdir. Annotasiya zərərsizdir.
- config-server və discovery-server skanı genişləndirmir (`ConfigServerApplication.java`, `DiscoveryServerApplication.java` — sadə `@SpringBootApplication`), onlar `common`-dan asılı da deyil.

4.5 Auto-configuration və şərtlər

Auto-configuration — classpath-ə və mövcud bean-lərə baxıb "ağlabatan standart" bean-lər yaradan `@AutoConfiguration` siniflərdir. Onları idarə edən əsas şərtlər:

Şərt	Nə yoxlayır	Layihədə nümunə
`@ConditionalOnClass`	Sinif classpath-dədirmi	`KafkaJsonConfiguration.java:24-26`, `SlowRequestLoggingFilter.java:31-33`
`@ConditionalOnProperty`	Xassənin dəyəri	`VirtualThreadConfiguration.java:22-25`
`@ConditionalOnMissingBean`	Eyni tipdə bean yoxdursa	Boot-un `KafkaAutoConfiguration`-u (`KafkaTemplate`, `ProducerFactory`)
`@ConditionalOnBean`	Bean varsa	net.devh `GrpcServerSecurityAutoConfiguration` (`AuthenticationManager` tələb edir)

Nümunə: kimin `KafkaTemplate`-i qalib gəlir?

Mənbə: common/src/main/java/com/casestudy/insurancecampaign/common/config/KafkaJsonConfiguration.java:24-49

```
@Configuration
@ConditionalOnClass(name = "org.springframework.kafka.core.KafkaTemplate")
public class KafkaJsonConfiguration {
    // ... (buraxılıb: Javadoc)
    @Bean
    public ProducerFactory<String, Object> producerFactory(@Value("${spring.kafka.bootstrap-servers:local
host:9092}") String bootstrapServers) {
        Map<String, Object> props = new HashMap<>();
        props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
        props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG, StringSerializer.class);
        props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
        props.put(JsonSerializer.ADD_TYPE_INFO_HEADERS, false);
```

```
        return new DefaultKafkaProducerFactory<>(props);
    }
```

1. `@ConditionalOnClass(name = "...")` — sinif adı **sətir** kimi verilib, `KafkaTemplate.class` kimi yox. Səbəb: şərt annotasiyaları ASM ilə, sinif yüklənmədən oxunur; sinif literalı istifadə olunsaydı və classpath-də Kafka olmasaydı, annotasiyanın metadata-sı oxunarkən problem yarana bilərdi. Sətir forması təhlükəsizdir.
2. Bu sinif **istifadəçi konfiqurasiyası** kimi (komponent skanı ilə) yüklənir. Spring Boot auto-configuration-ları istifadəçi bean-lərindən **sonra** emal edir. log-service-də Boot-un `KafkaAutoConfiguration`-u `@ConditionalOnMissingBean(KafkaTemplate.class)` və `@ConditionalOnMissingBean(ProducerFactory.class)` ilə qorunur **FAKT** (spring-boot-kafka 4.1.0 mənbəsi), buna görə `common`-dakı bean-lər qalib gəlir və Boot-unkular yaradılmır.
3. campaign-service-də `spring-boot-kafka` yoxdur, ona görə Boot Kafka auto-configuration-u ümumiyyətlə yoxdur; `spring.kafka.bootstrap-servers` xassəsi yalnız `@Value` ilə oxunur.

NƏTİCƏ Dərs: `@ConditionalOnMissingBean`-i **istifadəçi** `@Configuration`-unda istifadə etmək risklidir, çünki qiymətləndirmə anında digər istifadəçi bean-ləri hələ qeydiyyatdan keçməmiş ola bilər. Bu şərt auto-configuration sinifləri üçün nəzərdə tutulub.

`@ConditionalOnProperty` və `@EnableAsync`

Mənbə: common/src/main/java/com/casestudy/insurancecampaign/common/config/VirtualThreadConfiguration.java:22-38

```
@Configuration
@EnableAsync
@ConditionalOnProperty(name = "spring.threads.virtual.enabled", havingValue = "true")
public class VirtualThreadConfiguration {
    // ... (buraxılıb: Javadoc)
    @Bean(name = "applicationTaskExecutor")
    public AsyncTaskExecutor applicationTaskExecutor() {
        return new TaskExecutorAdapter(Executors.newVirtualThreadPerTaskExecutor());
    }
```

Diqqət çəkən nəticə **NƏTİCƏ**: `@EnableAsync` yalnız bu sinif aktiv olduqda qüvvəyə minir. Bütün servislərdə `spring.threads.virtual.enabled: true` olduğu üçün (məs. `auth-service/src/main/resources/application.yml:18-20`) auth-service-dəki `@Async` audit metodları həqiqətən asinxron işləyir. Kimsə bu xassəni `false` etsə, `@EnableAsync` də yox olar və `AuthAuditService`-in bütün `@Async` metodları **sinxron** icra olunar — login cavabı log-service-in cavabını (və Retry gözləmələrini) gözləyər. Bu, iki müstəqil narahatlığın (virtual thread-lər və asinxronluq) bir şərtə bağlanmasının nümunəsidir.

Bean adı `applicationTaskExecutor` Boot-un standart executor-unun adıdır; eyni adlı bean olduğu üçün Boot-un auto-configuration-u öz executor-unu yaratmır və `@Async` bu bean-i istifadə edir.

4.6 Konfiqurasiya xassələri və mənbələrin prioriteti

`@ConfigurationProperties` record-larla

Mənbə: security/src/main/java/com/casestudy/insurancecampaign/security/JwtProperties.java:35-44

```
@ConfigurationProperties(prefix = "app.jwt")
public record JwtProperties(
        String secret,
        long expirationMinutes,
        String issuer,
        String publicKey,
        String privateKey,
        String publicKeyLocation,
```

```
        String privateKeyLocation
) {
```

- Record konstruktor bağlaması (constructor binding) ilə doldurulur: `app.jwt.public-key-location` → `publicKeyLocation` (relaxed binding: kebab-case, camelCase, `APP_JWT_PUBLIC_KEY_LOCATION` mühit dəyişəni hamısı eyni xassədir).
- Qeydiyyat `@EnableConfigurationProperties(JwtProperties.class)` ilə edilir və bu annotasiya `JwtTokenService` sinfinin üzərindədir (`JwtTokenService.java:36-38`). Bu işləyir, çünki `@EnableConfigurationProperties` istənilən bean sinfində emal olunur, lakin adətən `@Configuration` sinfinə qoyulur.
- Record-un öz metodu ola bilər: `rsaEnabled()` (`JwtProperties.java:56-58`) — konfiqurasiya əsasında rejim seçir.

`@Value` standart dəyərlərlə

`SlowRequestLoggingFilter` üç dəyəri `@Value` ilə standart dəyərlərlə alır (`SlowRequestLoggingFilter.java:48-58`): `${app.slow-request-threshold-ms:5}` , `${app.kafka.topics.log-events:log-events}` , `${spring.application.name:unknown-service}` . Standart dəyər xassə heç bir mənbədə tapılmadıqda istifadə olunur.

Xassə mənbələri və prioritet

Bu layihədə bir xassə bir neçə mənbədən gələ bilər. Prioritet (yuxarıdakı qalib gəlir) [NƏTİCƏ, Spring Boot və Spring Cloud Config-in standart davranışı]:

1. Test xassələri (`@DynamicPropertySource` , `@TestPropertySource`) — yalnız testlərdə;
2. Komanda sətri arqumentləri;
3. Əməliyyat sistemi mühit dəyişənləri (Compose-dakı `environment:` bölməsi);
4. **Config Server-dən gələn xassələr** (`spring.config.import: optional:configserver:...`) — standart olaraq lokal `application.yml` -i üstələyir;
5. Jar daxilindəki `application.yml` ;
6. `@Value` daxilindəki standart dəyərlər.

Konkret nümunə: `spring.kafka.consumer.auto-offset-reset: earliest` yalnız Config Server-in paylandığı faylda var (`config-server/src/main/resources/config/application.yml:7-8`). NƏTİCƏ Config Server əlçatmaz olduqda (`optional:`) log-service bu dəyəri almır və Kafka-nın standart `latest` dəyəri ilə başlayır — yeni consumer group mövcud mesajları atlayar (tapıntı F32). Bu, "mərkəzi konfiqurasiya optional olduqda davranış sakitcə dəyişir" riskinin nümunəsidir.

Profillər. Layihə Spring profillərindən demək olar ki, istifadə etmir: yalnız config-server `native` profili ilə işləyir (`config-server/src/main/resources/application.yml:6-7`) və Compose discovery-server üçün `SPRING_PROFILES_ACTIVE: docker` təyin edir (`docker-compose.yml:11`), lakin `application-docker.yml` faylı yoxdur FAKT. Mühit fərqləri profillərlə deyil, `${ENV_VAR:default}` yer tutanları ilə idarə olunur.

4.7 Bean həyat dövrü və "fail-fast" başlanğıc

Bean həyat dövrü: instansiya (konstruktor) → asılılıqların ötürülməsi → `Aware` callback-ləri → `BeanPostProcessor.postProcessBeforeInitialization` → `@PostConstruct` / `InitializingBean` → `postProcessAfterInitialization` (proxy-lər burada yaranır) → istifadə → `@PreDestroy` .

`JwtTokenService` açarları **konstruktorda** yükləyir FAKT (`JwtTokenService.java:59-73`). Açar faylı yoxdursa və ya PEM formatı səhvdirsə, `IllegalStateException` atılır və **tətbiq başlamır**. Bu, düzgün "fail-fast" davranışıdır: konfiqurasiya xətası ilk sorğuda deyil, deploy zamanı aşkarlanır.

`ObjectProvider`: opsional və tənbəl (lazy) asılılıq

`SlowRequestLoggingFilter` `KafkaTemplate`-i birbaşa deyil, `ObjectProvider` vasitəsilə alır (`SlowRequestLoggingFilter.java:35`, `113-117`):

Mənbə: `common/src/main/java/com/casestudy/insurancecampaign/common/web/SlowRequestLoggingFilter.java:113-117`

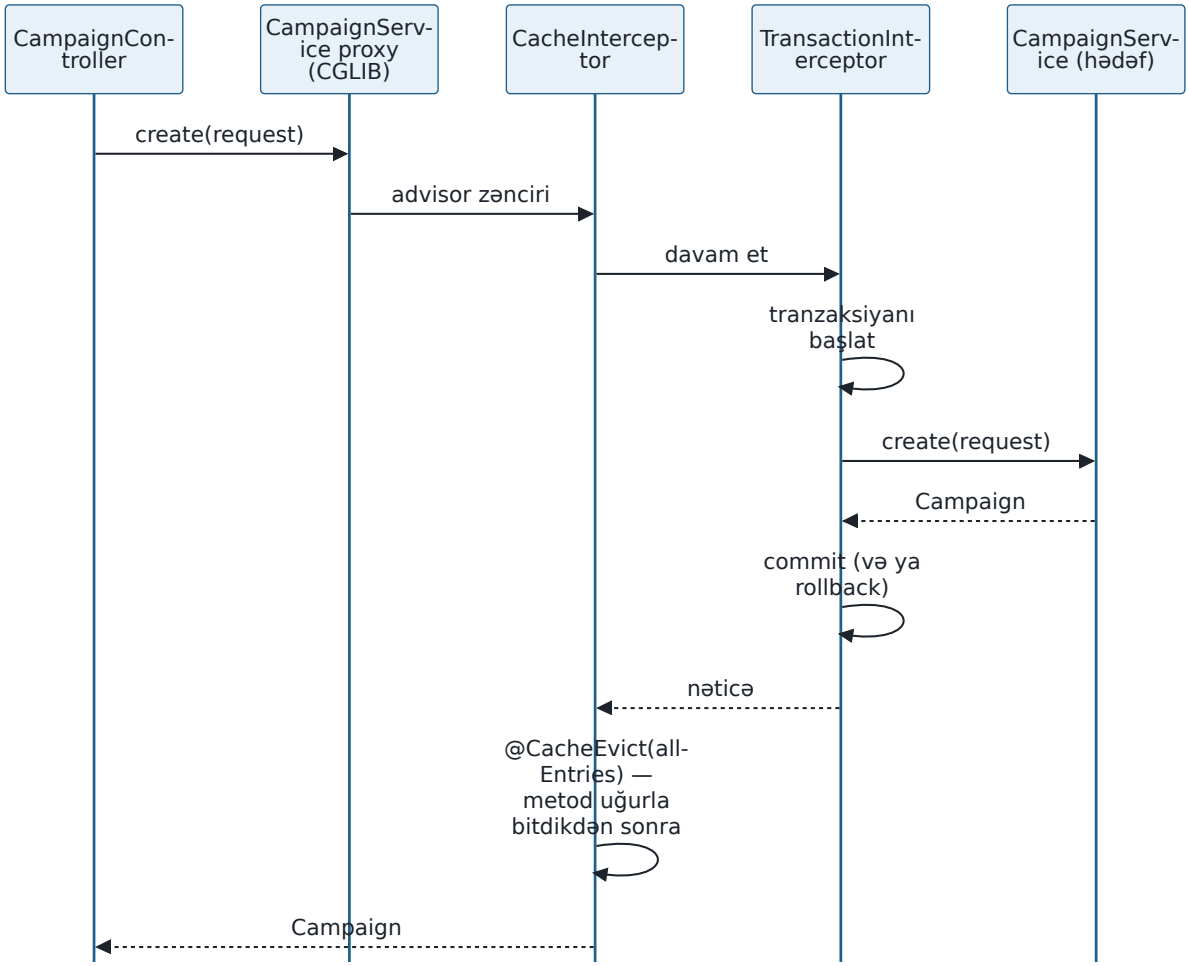
```
KafkaTemplate<String, Object> kafkaTemplate = kafkaTemplateProvider.getIfAvailable();

if (kafkaTemplate ≠ null) {
    kafkaTemplate.send(topicName, request.getRequestURI(), event);
}
```

`ObjectProvider` bean-in olmamasına dözümlüdür (`getIfAvailable()` `null` qaytarır) və onu yalnız lazım olduqda axtarır. Konstruktorun Javadoc-u bunu "allows the filter to work even if Kafka is unavailable" kimi izah edir. **NƏTİCƏ** Qeyd: bu, **bean-in** olmamasına qarşı qorumadır, Kafka **brokerinin** əlçatmaz olmasına qarşı deyil — broker düşdükdə bean var, `send` isə metadata gözləyərək bloklaya bilər (fəsil 11, tapıntı F10).

4.8 Proxy-lər və AOP: annotasiyalar necə "işləyir"

`@Transactional`, `@Cacheable`, `@CacheEvict`, `@Async`, `@PreAuthorize`, `@CircuitBreaker`, `@Retry`, `@Bulkhead` — hamısı eyni mexanizmə əsaslanır: Spring bean-i **proxy** obyekti ilə bükür, proxy metod çağırışını tutub (intercept) əlavə məntiq icra edir, sonra əsl metodu çağırır.



CampaignController-dən CampaignService.create çağırışının proxy zənciri (sadələşdirilmiş)

Diaqramdakı interceptor sırası **illüstrativdir**: NƏTİCƏ `@EnableCaching` və tranzaksiya idarəetməsinin advisor-ları standart olaraq eyni sıra dəyərinə (`Ordered.LOWEST_PRECEDENCE`) malikdir, layihə sıranı açıq təyin etmir; buna görə eviction-un commit-dən əvvəl və ya sonra baş verməsi zəmanətli deyil (fəsil 15, tapıntı F13).

Proxy-lərin əsas tələləri

Tələ	İzah	Layihədə vəziyyət
Öz-özünə çağırış (self-invocation)	`this.metod()` proxy-dən keçmir, annotasiya işləmir	`CampaignService.changeStatus` private-dir və annotasiyasızdır — problem yoxdur. `AuthService` `AuthAuditService` -in `@Async` metodlarını **başqa bean** üzərindən çağırır — düzgündür
`private` / `final` metodlar	CGLIB onları override edə bilmir	Annotasiyalı metodların hamısı `public` -dir
Proxy növü	Interfeys varsa JDK dynamic proxy mümkündür, Spring Boot standart olaraq CGLIB (`proxyTargetClass=true`) istifadə edir	Servislər interfeyssiz siniflərdir — CGLIB
Advisor sırası	Bir neçə aspekt eyni metodda olduqda sıra vacibdir	Resilience4j: Retry(CircuitBreaker(Bulkhead(çağırış))) — fəsil 14
Proxy-ni "görməyən" obyektlər	`new` ilə yaradılan obyektə aspekt tətbiq olunmur	Unit testlər servisləri `new` ilə yaradır — `@Async` , `@Transactional` testdə işləmir; bu qəsdəndir

Layihədəki proxy əsaslı annotasiyalar

Annotasiya	Yer	Kim emal edir	Əlavə qeyd
`@Transactional`	`CampaignService` (7 metod), `AuthService` (6 metod)	`TransactionInterceptor`	Fəsil 7
`@Cacheable` , `@CacheEvict`	`CampaignService`	`CacheInterceptor` (`@EnableCaching` — `CampaignServiceApplication.java:10`)	Fəsil 15
`@Async`	`AuthAuditService` (8 metod)	`AsyncAnnotationBeanPostProcessor` (`@EnableAsync`)	Fəsil 14
`@PreAuthorize`	controller metodları	`AuthorizationManagerBeforeMethodInterceptor` (`@EnableMethodSecurity`)	Fəsil 10
`@CircuitBreaker` , `@Retry` , `@Bulkhead`	`LogServiceHttpClient` interfeys metodu	Resilience4j aspektləri (AspectJ annotasiya üslubu, Spring AOP)	Fəsil 14

4.9 Java 25 xüsusiyyətləri layihədə

Xüsusiyyət	Nümunə	Qeyd
Record	DTO-lar (`CreateCampaignRequest`), domen (`Campaign`), hadisələr (`SlowRequestLogEvent`), konfiqurasiya (`JwtProperties`), zərflər (`ApiResponse<T>`)	Dəyişməz data daşıyıcıları; `equals/hashCode/toString` avtomatik. JPA entity-lər record **deyil** — JPA dəyişkən, no-arg konstruktorlu siniflər tələb edir

Xüsusiyyət	Nümunə	Qeyd
Switch ifadəsi	`CampaignService.java:133-138`, `GlobalExceptionHandler.java:38-43`	Enum üzrə `switch` ifadəsində bütün hallar örtülübsə `default` lazım deyil; yeni enum dəyəri əlavə olunsa kompilyator xəbərdarlıq edir
Pattern matching `instanceof`	`HttpServerObservationConfiguration.java:117-120`	`if (requestUri instanceof String value)` — yoxlama + cast bir addımda
`String.formatted`	`CampaignService.java:152-155`	`"...%s".formatted(...)`
`Stream.toList()`	`JwtAuthenticationFilter.java:76-78`	Dəyişməz siyahı qaytarır
`SequencedCollection.getFirst()`	`CampaignGrpcClientTest.java:166`	Java 21
`HexFormat`	`TokenHashUtils.java:60`	Java 17
Virtual thread-lər	`spring.threads.virtual.enabled: true` + `Executors.newVirtualThreadPerTaskExecutor()`	Aşağıda

Virtual thread-lər — qısa nəzəriyyə

Platform thread-i OS thread-inə 1:1 bağlıdır; bloklayan I/O zamanı OS thread-i boş gözləyir. **Virtual thread** JVM tərəfindən idarə olunan yüngül thread-dir: bloklayan əməliyyatda (socket oxu, `Thread.sleep`, JDBC) virtual thread daşıyıcı (carrier) platform thread-indən "ayrılır" və daşıyıcı başqa virtual thread-i icra edir. Nəticə: hər sorğuya bir thread modeli saxlanılır, lakin minlərlə eyni vaxtlı bloklayan sorğu ucuz olur.

Layihədə:

- `spring.threads.virtual.enabled: true` (bütün servislər) — NƏTİCƏ Spring Boot bu halda Tomcat sorğu emalı və Boot-un öz executor-ları üçün virtual thread-lərdən istifadə edir.
- `VirtualThreadConfiguration` `@Async` üçün hər tapşırığa yeni virtual thread yaradır.

Diqqət ediləcək məqamlar NƏTİCƏ:

- Virtual thread-lər **CPU-intensiv** işi sürətləndirmir.
- `newVirtualThreadPerTaskExecutor()` **məhdudiyyətsizdir** — paralellik başqa vasitə ilə məhdudlaşdırılmalıdır. Bu layihədə auditi Resilience4j `Bulkhead` (maks. 10) məhdudlaşdırır (fəsil 14).
- `ThreadLocal`-a əsaslanan kontekstlər (MDC, `SecurityContextHolder`, tracing) yeni thread-ə avtomatik keçmir. Boot-un öz executor-u `TaskDecorator`/context propagation tətbiq edə bilir, lakin bu xüsusi executor-da decorator yoxdur (tapıntı F34).
- Java 24-dən etibarən `synchronized` bloklarında virtual thread-in daşıyıcıya "yapışması" (pinning) problemi əsasən aradan qaldırılıb (JEP 491); Java 25-də bu daha az narahatlıq yaradır.

4.10 Thread-safety və singleton bean-lər

Spring bean-ləri standart olaraq **singleton**-dur: bütün sorğular eyni obyekti paylaşır. Layihədəki bean-lər vəziyyətsizdir (stateless): sahələr yalnız `final` asılılıqlar və konfiqurasiyadır. Bir neçə maraqlı hal:

- `TokenHashUtils.SECURE_RANDOM` statik `SecureRandom`-dur (`TokenHashUtils.java:21`) — `SecureRandom` thread-safe-dir.
- `JwtAuthenticationFilter` autentifikasiyanı `SecurityContextHolder`-a yazır (`JwtAuthenticationFilter.java:81`). Kontekst standart olaraq `ThreadLocal`-dır; Spring Security sorğu sonunda onu təmizləyir. Virtual thread-lərdə də hər sorğu öz thread-indədir — model eyni qalır.
- `GatewayRateLimitErrorResponseFilter` hər sorğu üçün yeni `ServerHttpResponseDecorator` yaradır — reaktiv mühitdə paylaşılan dəyişkən vəziyyət yoxdur.

4.11 Yeni layihədə tətbiq: qısa yoxlama siyahısı

1. Konstruktor injeksiyası, `final` sahələr; sahə injeksiyasından yalnız framework tələb etdikdə istifadə edin.
2. Konfiqurasiyanı `@ConfigurationProperties` record-ları ilə qruplaşdırın; `@Validated` əlavə edib startup-da doğrulayın (bu layihədə yoxdur — məsələn, `issuer` boş olsa yalnız ilk sorğuda aşkarlanar).
3. Paylaşılan konfiqurasiyanı auto-configuration kimi paketləyin; şərtləri açıq yazın.
4. Proxy əsaslı annotasiyaların self-invocation və sıra tələlərini bilin; sıra vacibdirsə `@Order` / `order` atributları ilə açıq təyin edin (`@EnableTransactionManagement(order = ...)` , `@EnableCaching(order = ...)`).
5. Bir-birindən müstəqil narahatlıqları ayrı şərtlərə bağlayın (`@EnableAsync` virtual thread şərtinə bağlı olmasın).

4.12 Müsahibə sualları

S1. Tək konstruktorlu sinifdə niyə `@Autowired` lazım deyil? Cavab: Spring 4.3-dən bəri yeganə konstruktor avtomatik injeksiya nöqtəsi sayılır. Bir neçə konstruktor varsa, hansının istifadə olunacağını `@Autowired` ilə göstərmək lazımdır.

S2. `@ConditionalOnMissingBean` niyə istifadəçi konfiqurasiyasında təhlükəlidir? Cavab: Şərt bean tərifinin qeydiyyatı anında, o vaxta qədər qeydiyyatdan keçmiş tərifələrə görə qiymətləndirilir. İstifadəçi konfiqurasiyalarının emal sırası zəmanətli olmadığı üçün nəticə təsadüfi ola bilər. Auto-configuration-lar istifadəçi bean-lərindən sonra emal olunduğu üçün orada etibarlıdır.

S3. `@Async` metodunu eyni sinifdən çağırsanız nə olar? Cavab: Çağırış proxy-dən keçmir, metod sinxron icra olunur. Bu layihədə `AuthService` `AuthAuditService`-i ayrıca bean kimi inject etdiyi üçün problem yoxdur.

S4. Bu layihədə `spring.threads.virtual.enabled=false` etsək hansı gözlənilməz dəyişiklik baş verər? Cavab: `VirtualThreadConfiguration` şərti yerinə yetirilmir, onunla birlikdə `@EnableAsync` da itir. `AuthAuditService` metodları sinxron işləyir — login, qeydiyyat və s. log-service çağırışını (Retry gözləmələri ilə birlikdə) gözləyir.

S5. Config Server xassələri ilə lokal `application.yml` toqquşduqda hansı qalib gəlir? Cavab: Standart olaraq Config Server (remote) xassələri lokal fayldakılardan üstündür; mühit dəyişənləri isə hər ikisindən üstündür. Config Server `optional:` idxal olunubsa və əlçatmazdırsa, yalnız lokal dəyərlər qalır.

S6. Virtual thread-lər hansı problemləri həll etmir? Cavab: CPU-intensiv işi sürətləndirmir, aşağı axın sistemlərinin (DB pool, xarici API) tutumunu artırmır və paralelliyi özü məhdudlaşdırmır. Connection pool-un ölçüsü kimi resurslar yenə də darboğaz olaraq qalır; `ThreadLocal` kontekstləri yeni thread-lərə avtomatik ötürülmür.

4.13 Xülasə

- Konteyner bean-ləri skan + auto-configuration + şərtlər əsasında yaradır, asılılıqları konstruktorla ötürür.
- `scanBasePackages` paylaşılan modulları hər servisə qoşur.
- Kafka bean-lərinin seçimi `@ConditionalOnClass` (common) və `@ConditionalOnMissingBean` (Boot) qarşılıqlı təsirindən asılıdır.
- `@EnableAsync` virtual thread xassəsinə bağlıdır — vacib incəlik.
- Proxy əsaslı annotasiyalar (`@Transactional` , `@Cacheable` , `@Async` , `@PreAuthorize` , Resilience4j) eyni AOP mexanizmini paylaşır; sıra və self-invocation tələlərini bilmək lazımdır.
- Növbəti fəsil HTTP/REST qatını və təbəqəli dizaynı təzələyir.

5. HTTP, REST və təbəqəli dizayn

5.1 Problem: API-ni necə dizayn etməli ki, həm klient, həm də kod sağlam qalsın

Kampaniya yaratmaq üçün klientə nə lazımdır? URL, HTTP metodu, gövdə formatı, uğur və xəta cavablarının forması, autentifikasiya. Server tərəfində isə HTTP detallarının biznes məntiqinə sızmaması, biznes qaydalarının bir yerdə olması və persistence modelinin xarici kontrakta çevrilməməsi lazımdır. Bu fəsil həm HTTP/REST əsaslarını, həm də layihədə tətbiq olunan **controller → service → repository** təbəqələrini və **DTO → domen → entity** modellərini izah edir.

5.2 HTTP və REST əsasları (təzələmə)

REST arxitektura üslubudur (protokol deyil): resurslar URI ilə ünvanlanır, standart HTTP metodları ilə manipulyasiya olunur, server tərəfində sessiya vəziyyəti saxlanmır (stateless), cavablar özünü təsvir edir.

Metod	Semantika	Təhlükəsiz (safe)	İdempotent	Layihədə
GET	Oxumaq	Bəli	Bəli	Siyahı, detal, tarixçə, statistika, yavaş sorğular
POST	Yaratmaq / prosesi işə salmaq	Xeyr	Xeyr	Kampaniya yaratmaq, bütün auth əməliyyatları, audit yazmaq
PUT	Resursu tam əvəzləmək	Xeyr	Bəli	İstifadə olunmur
PATCH	Qismən dəyişiklik	Xeyr	Zəmanətli deyil	`approve` , `deactivate`
DELETE	Silmək	Xeyr	Bəli	İstifadə olunmur

İdempotentlik — eyni sorğunun bir neçə dəfə göndərilməsi serverdə bir dəfə göndərilmiş kimi nəticə verir. Şəbəkə xətalarında klient və ya proxy sorğunu təkrarlaya bildiyi üçün vacibdir. Layihədə **NƏTİCƏ**:

- `PATCH /{id}/approve` ikinci dəfə çağırıldıqda kampaniya artıq `ACTIVE` olduğu üçün `409 INVALID_STATUS_TRANSITION` qaytarır (`CampaignService.java:96-98`). Vəziyyət dəyişmir, lakin cavab fərqlidir. Bu "effekt baxımından idempotent, cavab baxımından deyil" haldır.
- `POST /api/v1/campaigns` ikinci dəfə eyni gövdə ilə göndərildikdə `409 DUPLICATE_CAMPAIGN` qaytarır — dublikat qaydası təsadüfən POST-u da effekt baxımından idempotent edir. Bu, idempotency key mexanizminin əvəzi deyil: fərqli məlumatla iki ayrı sorğu yenə iki kampaniya yaradır.

Status kodları layihədə **FAKT** (`GlobalExceptionHandler.java` , `ServiceSecurityConfiguration.java` , `CampaignController.java:45`):

Kod	Nə vaxt
200 OK	Uğurlu oxu və dəyişikliklər (`approve` , `deactivate` , auth əməliyyatları)
201 Created	`POST /api/v1/campaigns` (`@ResponseStatus(HttpStatus.CREATED)`)
400 Bad Request	`@Valid` pozuntusu (`VALIDATION_ERROR`)
401 Unauthorized	Token yoxdur (security entry point), yanlış login (`BusinessException(UNAUTHORIZED)`)

Kod	Nə vaxt
403 Forbidden	Rol çatmır (`@PreAuthorize` → `AccessDeniedException`)
404 Not Found	`ResourceNotFoundException`
409 Conflict	Dublikat, yanlış status keçidi, biznes qaydası pozuntusu (məs. mövcud username)
429 Too Many Requests	Gateway rate limit
500 Internal Server Error	Gözlənilməz xəta **və** (defekt F07) bir sıra əslində 4xx olmalı xətalar

5.3 Layihənin REST resurs modeli

FAKT (`CampaignController.java:30-93` , `DashboardController.java:19-59` , `AuthController.java:25-118` , log-service controller-ləri):

Resurs	Metod və yol	Rol
Kampaniyalar kolleksiyası	`GET /api/v1/campaigns?page&size&sort` , `POST /api/v1/campaigns`	USER/ADMIN; yaratmaq ADMIN
Bir kampaniya	`GET /api/v1/campaigns/{id}`	USER/ADMIN
Status əməliyyatları	`PATCH /api/v1/campaigns/{id}/approve` , `PATCH /api/v1/campaigns/{id}/deactivate`	ADMIN
Alt-resurs: tarixçə	`GET /api/v1/campaigns/{id}/history`	ADMIN
Hesablanmış resurs	`GET /api/v1/campaigns/statistics`	ADMIN
Dashboard	`GET /api/v1/dashboard/classifieds/statistics` , `GET /api/v1/dashboard/campaigns/{campaignId}/history`	ADMIN
Auth əməliyyatları	`POST /api/v1/auth/{register,login,refresh-token,forgot-password,reset-password,logout}`	Hamı
Jurnallar	`POST /api/v1/logs/auth-audit` (INTERNAL_SERVICE), `GET /api/v1/logs/slow-requests` (ADMIN)	—

Dizayn müzakirəsi:

- **Versiya URL-də** (`/api/v1`) — sadə və keşləmə üçün şəffaf; alternativ header və ya media type versiyalamasıdır.
- **Status əməliyyatları "action" endpoint-ləridir** (`/approve` , `/deactivate`). Saf REST yanaşması bunu `PATCH /campaigns/{id}` + `{"status":"ACTIVE"}` kimi modelləşdirərdi. NƏTİCƏ Action endpoint-ləri burada daha yaxşıdır: hər keçidin ayrıca icazəsi, ayrıca biznes qaydası və ayrıca tarixçə səbəbi ("Campaign approved") var; ümumi PATCH bu qaydaları bir metodda `if/else` ilə yoxlamağa məcbur edərdi.
- **`/statistics` və `/{id}` toqquşmur**: Spring MVC literal seqmentli `/statistics` nümunəsini dəyişənli `/{id}` nümunəsindən daha spesifik sayır və ona üstünlük verir. Əks halda `"statistics"` sətri `UUID` -ə çevrilməyə çalışılardı.

- **"classifieds"** sözü (`/api/v1/dashboard/classifieds/statistics`) biznes domeninə (kampaniyalar) uyğun deyil. **NAMƏLUM** Adın mənşəyi koddan müəyyən edilmir; README də eyni yolu istifadə edir. Kontraktı dəyişmək klientləri sındıracağı üçün belə adlar adətən versiya dəyişikliyində düzəldilir.
- **Yaratma cavabında `Location` başlığı yoxdur.** REST konvensiyası `201` ilə yeni resursun URI-sini `Location` başlığında qaytarmağı tövsiyə edir; burada id cavab gövdəsindədir.

5.4 Controller: sorğu bağlama və cavab

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/controller/CampaignController.java:30-64

```
@RestController
@RequestMapping("/api/v1/campaigns")
public class CampaignController {

    private final CampaignService campaignService;
    private final CampaignMapper campaignMapper;

    public CampaignController(CampaignService campaignService, CampaignMapper campaignMapper) {
        this.campaignService = campaignService;
        this.campaignMapper = campaignMapper;
    }

    @Operation(summary = "Create insurance campaign. ROLE_ADMIN only.")
    @PostMapping
    @PreAuthorize("hasRole('ADMIN')")
    @ResponseStatus(HttpStatus.CREATED)
    public ApiResponse<CampaignResponse> create(@Valid @RequestBody CreateCampaignRequest request) {
        return ApiResponse.success(campaignMapper.toResponse(campaignService.create(request)));
    }

    @Operation(summary = "Get campaign by id. ROLE_USER and ROLE_ADMIN are allowed.")
    @GetMapping("/{id}")
    @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
    public ApiResponse<CampaignResponse> getById(@PathVariable UUID id) {
        return ApiResponse.success(campaignMapper.toResponse(campaignService.getById(id)));
    }

    @Operation(summary = "List all campaigns with pagination and sorting. ROLE_USER and ROLE_ADMIN are al
lowed.")
    @GetMapping
    @PreAuthorize("hasAnyRole('USER', 'ADMIN')")
    public ApiResponse<CampaignPageResponse> getAll(
            @PageableDefault(size = 10, sort = "createdAt", direction = Sort.Direction.DESC) Pageable pag
eable
    ) {
        return ApiResponse.success(CampaignPageResponse.from(campaignService.getAll(pageable).map(campaig
nMapper::toResponse)));
    }
```

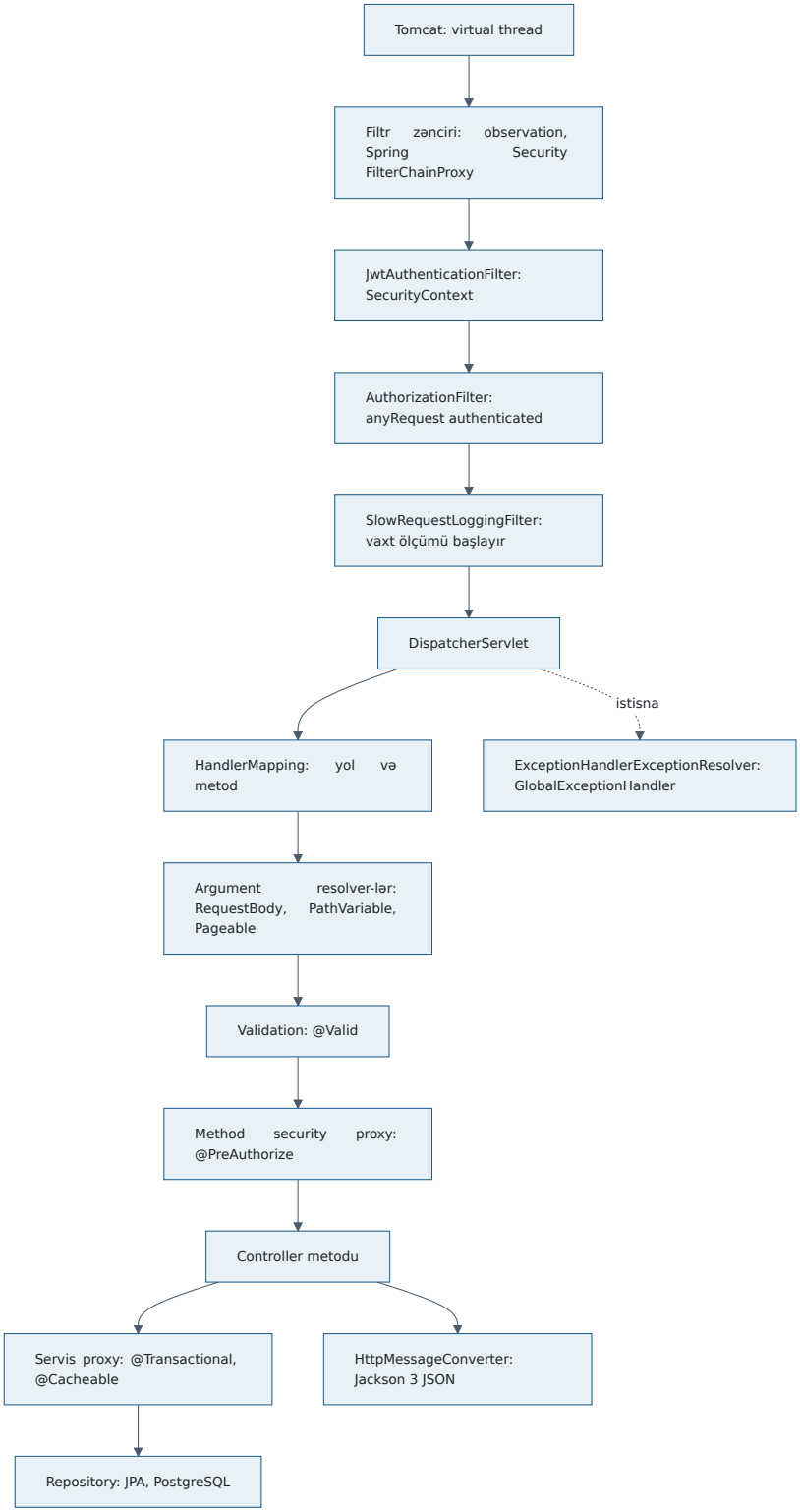
Hər annotasiyanın real təsiri:

Element	Runtime təsiri
`@RestController`	Metodun qaytardığı obyekt `HttpMessageConverter` (Jackson 3) ilə gövdəyə yazılır
`@RequestMapping("/api/v1/campaigns")`	Sinif səviyyəli prefiks; `RequestMappingHandlerMapping` startup-da xəritəni qurur
`@PostMapping` / `@GetMapping("/{id}")`	Metod + yol nümunəsi
`@PreAuthorize("hasRole('ADMIN')")`	Metod çağırılmazdan əvvəl `ROLE_ADMIN` authority-si yoxlanılır; yoxdursa `AccessDeniedException` (fəsil 10)
`@ResponseStatus(CREATED)`	Uğurlu cavab statusu 201

Element	Runtime təsiri
`@Valid @RequestBody`	Gövdə JSON-dan record-a deserializə olunur, sonra Bean Validation işləyir; pozuntu → `MethodArgumentNotValidException` (fəsil 8)
`@PathVariable UUID id`	Yol seqmenti `String → UUID` çevrilir; səhv format → `MethodArgumentTypeMismatchException`
`@PageableDefault(...) Pageable`	`PageableHandlerMethodArgumentResolver` `page`, `size`, `sort` parametrlərini oxuyur; yoxdursa `size=10, sort=createdAt,desc`
`@Operation`	Yalnız OpenAPI sənədi üçün (springdoc)

Controller-in işi yalnız tərcümədir: HTTP → servis çağırışı → domen obyekti → cavab DTO-su. Biznes qaydası controller-də yoxdur. Bu, təbəqəli dizaynın əsas prinsipidir.

Sorğunun Spring MVC daxilində yolu



Servlet əsaslı servisdə HTTP sorğusunun icra yolu (campaign-service)

NƏTİCƏ Filtr sırası: Spring Security-nin `FilterChainProxy`-si servlet konteynerində `-100` sıra ilə qeydiyyatdan keçir; `SlowRequestLoggingFilter` isə adi `@Component` filtr kimi sırasız (ən aşağı prioritet) qeydiyyata düşür. Ona görə yavaş sorğu filtri security-dən **sonra** işləyir: security-nin rədd etdiyi (401) sorğular ölçülmür və ölçülən müddətə JWT doğrulaması daxil deyil. `@Valid` doğrulaması `@PreAuthorize`-dan **əvvəl** baş verir (argument resolver-lər metod proxy-sindən əvvəl işləyir), yəni `ROLE_USER` səhv gövdə göndərərsə 403 əvəzinə 400 alar.

5.5 Cavab zərfi: `ApiResponse` və `ApiError`

Bütün uğurlu cavablar vahid zərfdədir **FAKT** (`ApiResponse.java:16-31`):

Mənbə: common/src/main/java/com/casestudy/insurancecampaign/common/api/ApiResponse.java:16-33

```
public record ApiResponse<T>(
        boolean success,
        T data,
        Instant timestamp
) {

    // ... (buraxılıb: Javadoc)
    public static <T> ApiResponse<T> success(T data) {
        return new ApiResponse<>(true, data, Instant.now());
    }

}
```

Xətalar isə ayrı struktura malikdir **FAKT** (`ApiError.java:20-56`): `code`, `message`, `status`, `path`, `validationErrors`, `timestamp`. Nümunə cavablar:

```
{ "success": true,
  "data": { "id": "…", "title": "TSS Kampanya 2026", "category": "TSS",
            "status": "PENDING_APPROVAL", "duplicateOf": null,
            "createdAt": "2026-07-25T14:00:00Z", "updatedAt": "2026-07-25T14:00:00Z" },
  "timestamp": "2026-07-25T14:00:00.123Z" }
```

```
{ "code": "VALIDATION_ERROR", "message": "Request validation failed", "status": 400,
  "path": "/api/v1/campaigns",
  "validationErrors": { "title": "size must be between 10 and 50" },
  "timestamp": "…" }
```

NƏTİCƏ `success` sahəsi uğurlu cavabda həmişə `true`-dur; xəta cavabında isə ümumiyyətlə yoxdur (fərqli record). Klient uğur/xətanı HTTP statusundan müəyyən etməlidir — bu düzgündür. Zərfin dəyəri: vahid `timestamp` və gələcəkdə `meta`/`traceId` əlavə etmək imkanı. Mənfi tərəfi: hər cavab bir səviyyə dərinləşir (`$.data.id`).

Alternativ: RFC 9457 Problem Details. Spring 6+ `ProblemDetail` tipini və `spring.mvc.problemdetails.enabled` xassəsini dəstəkləyir: standart `type`, `title`, `status`, `detail`, `instance` sahələri, `application/problem+json` media tipi. Xüsusi `ApiError` əvəzinə standartdan istifadə klient kitabxanaları və gateway-lər üçün daha interoperabeldir. Layihə xüsusi formatı seçib; gateway də 429 üçün eyni formatı təkrarlayır (`GatewayRateLimitErrorResponseFilter.java:78-85`), bu da sistem daxilində ardıcıllığı qoruyur.

5.6 Üç model: DTO, domen, entity

Campaign-service-də bir kampaniya üç fərqli tiplə təmsil olunur **FAKT**:

Tip	Sinif	Qat	Məqsəd
Sorğu DTO	`CreateCampaignRequest` (record)	Web	Yalnız klientin göndərə biləcəyi sahələr + validasiya annotasiyaları
Entity	`CampaignEntity` (JPA sinfi)	Persistence	Cədvəl sütunlarının xəritəsi, `@PrePersist` vaxt damğaları, dəyişkən vəziyyət

Tip	Sinif	Qat	Məqsəd
Domen	`Campaign` (record)	Servis	Servisin xaricə qaytardığı dəyişməz təsvir
Cavab DTO	`CampaignResponse` (record)	Web	Klientə gedən kontrakt

Axın: `CreateCampaignRequest → CampaignEntity → Campaign → CampaignResponse` (README də bunu "DTO flow" kimi göstərir). Mapper əl ilə yazılıb **FAKT** (`CampaignMapper.java:10-41`):

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/mapper/CampaignMapper.java:10-24

```
@Component
public class CampaignMapper {
    public Campaign toDomain(CampaignEntity entity) {
        return new Campaign(
                entity.getId(),
                entity.getTitle(),
                entity.getDescription(),
                entity.getCategory(),
                entity.getStatus(),
                entity.getDuplicateOf(),
                entity.getCreatedAt(),
                entity.getUpdatedAt()
        );
    }
```

Niyə entity-ni birbaşa qaytarmırıq?

1. **Kontrakt sabitliyi** — sütun adı dəyişəndə API dəyişməməlidir.
2. **Lazy loading tələsi** — `open-in-view: false` olduğu üçün (`campaign-service/src/main/resources/application.yml:15`) tranzaksiyadan kənarda lazy əlaqəyə toxunmaq `LazyInitializationException` verər. Domen record-u tranzaksiya daxilində tam doldurulur.
3. **Təhlükəsizlik** — entity-də klientə göstərilməməli sahələr ola bilər (auth-service-də `password` hash-i).
4. **Dəyişməzlik** — record-lar thread-safe və proqnozlaşdırıla biləndir.

Layihədəki istisna **FAKT**: tarixçə üçün servis entity qaytarır — `CampaignService.getHistory` `List<CampaignStatusHistoryEntity>` (`CampaignService.java:113-119`), və həm controller (`CampaignController.java:83-85`), həm gRPC endpoint (`CampaignGrpcEndpoint.java:37-44`) entity ilə işləyir. **NƏTİCƏ** Entity-də lazy əlaqə olmadığı üçün texniki problem yoxdur, lakin persistence tipi servis sərhədini keçir — ardıcıllıq üçün domen record-u daha yaxşı olardı.

`duplicateOf` sahəsi kontraktda qalıb, lakin həmişə `null`-dur (heç bir kod onu təyin etmir). Jackson 3 standart olaraq `null` sahələri də yazır, buna görə klient `"duplicateOf": null` görür.

Auth-service: MapStruct və istifadə olunmayan domen modelləri

auth-service-də `AuthMapper` MapStruct interfeysidir (`AuthMapper.java:16-83`); kompilyator `AuthMapperImpl` yaradır. Default metod (`toEntity`) əl ilə yazılıb, qalanları generasiya olunur. `@Mapping(target = "status", expression = "java(entity.getStatus().name())")` enum-u sətrə çevirir.

FAKT `AppUser`, `RefreshToken`, `PasswordResetToken` domen record-ları yalnız mapper-də və onun testində istifadə olunur; `AuthService` entity-lərlə birbaşa işləyir və nəticəni `RegisteredUserResponse` / `TokenResponse` DTO-larına çevirir. **NƏTİCƏ** Bu modellər hazırda ölü koddur (və ya gələcək refaktorinq üçün hazırlıq) — müsahibədə "YAGNI" nümunəsi kimi müzakirə oluna bilər.

Əl ilə mapper və MapStruct müqayisəsi:

	Əl ilə (`CampaignMapper`)	MapStruct (`AuthMapper`)
Şəffaflıq	Tam görünür, debug asandır	Generasiya olunmuş kod `target/generated-sources`-dadır
Sahə əlavə edildikdə	Unutmaq mümkündür	`unmappedTargetPolicy` ilə kompilyasiya xətası/xəbərdarlığı mümkündür
Asılılıq	Yoxdur	Annotation processor, Lombok binding
Performans	Eyni (MapStruct da sadə getter/setter kodu yaradır)	Eyni

5.7 Səhifələmə (pagination) və sıralama

`getAll` `Pageable` qəbul edir, `Page<Campaign>` qaytarır, controller isə öz DTO-suna çevirir **FAKT** (`CampaignPageResponse.java:7-31`):

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/dto/CampaignPageResponse.java:17-31

```
public static CampaignPageResponse from(Page<CampaignResponse> page) {
    List<String> sortOrders = page.getSort().stream()
            .map(order → order.getProperty() + "," + order.getDirection().name().toLowerCase())
            .toList();
    return new CampaignPageResponse(
            page.getContent(),
            page.getNumber(),
            page.getSize(),
            page.getTotalElements(),
            page.getTotalPages(),
            page.isFirst(),
            page.isLast(),
            sortOrders
    );
}
```

Niyə `Page`-i birbaşa qaytarmırıq? `PageImpl`-in JSON forması Spring Data-nın daxili detalıdır və versiyalar arasında sabit deyil; Spring Data 3.3+ bunu birbaşa serializə etdikdə xəbərdarlıq verir və DTO və ya `@EnableSpringDataWebSupport(pageSerializationMode = VIA_DTO)` tövsiyə edir. **FAKT** log-service məhz bunu edir — `ApiResponse<Page<SlowRequestDocument>>` (`SlowRequestLogController.java:38-43`) — və testlər `$.data.content` strukturuna bağlıdır (`SlowRequestLogControllerTest.java:58`). Bu, iki servis arasında uyğunsuzluqdur (tapıntı F26).

Sıralama riskləri **NƏTİCƏ**:

- `sort` parametri istənilən entity xassəsini qəbul edir. Mövcud olmayan xassə (`sort=foo`) Spring Data-da `PropertyReferenceException` yaradır; layihənin catch-all handler-i bunu 500-ə çevirir (F07). Ağ siyahı (`createdAt`, `title`, `status`) daha təhlükəsizdir.
- İndekssiz sütunlar üzrə sıralama (`title`, `createdAt`) böyük cədvəllərdə tam skan + sort deməkdir (fəsil 7).
- Offset səhifələmə (`LIMIT/OFFSET`) dərin səhifələrdə yavaşlayır; böyük datada keyset (cursor) səhifələmə tövsiyə olunur.
- `size` üçün yuxarı hədd Spring Data-nın standart `max-page-size` dəyəridir (2000) — layihə bunu dəyişməyib.

5.8 Təbəqələr və məsuliyyətlər

Qat	Campaign-service	Məsuliyyət	Nə etməməlidir
Web	`CampaignController`, DTO-lar	HTTP ↔ Java tərcüməsi, validasiya tetiklənməsi, icazə annotasiyaları	Biznes qaydaları, SQL
Servis	`CampaignService`	Biznes qaydaları, tranzaksiya sərhədləri, yan təsirlər (tarixçə, hadisə, cache)	HTTP tipləri (`HttpServletRequest`)
Persistence	`CampaignRepository`, entity-lər	Saxlama və sorğular	Biznes qərarları
İnteqrasiya	`CampaignGrpcEndpoint`, `KafkaTemplate`	Digər protokollar	—

`CampaignGrpcEndpoint` də eyni `CampaignService`-i çağırır — biznes məntiqi iki giriş protokolu (REST və gRPC) arasında təkrarlanmır. Bu, servis qatının ayrılmasının birbaşa faydasıdır.

5.9 Yeni layihədə tətbiq

İllüstrativ nümunə (repozitoriyada yoxdur) — `201` + `Location` və ağ siyahılı sıralama:

```
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public ResponseEntity<ApiResponse<CampaignResponse>> create(@Valid @RequestBody CreateCampaignRequest req
uest,
                                                            UriComponentsBuilder uri) {
    CampaignResponse body = mapper.toResponse(service.create(request));
    URI location = uri.path("/api/v1/campaigns/{id}").buildAndExpand(body.id()).toUri();
    return ResponseEntity.created(location).body(ApiResponse.success(body));
}

private static final Set<String> SORTABLE = Set.of("createdAt", "title", "status");

private Pageable sanitize(Pageable p) {
    p.getSort().forEach(o -> {
        if (!SORTABLE.contains(o.getProperty())) {
            throw new BusinessException(ErrorCode.VALIDATION_ERROR, "Unsupported sort: " + o.getProperty
());
        }
    });
    return p;
}
```

Yoxlama siyahısı: resurs adları cəm halda; versiya; hər endpoint üçün rol; uğur və xəta cavablarının sənədləşdirilməsi (OpenAPI); səhifələmə DTO-su; sıralama ağ siyahısı; `Location`; standart xəta formatı (Problem Details və ya öz formatınız, lakin hər yerdə eyni).

5.10 Müsahibə sualları

S1. PUT və PATCH fərqi nədir, `approve` üçün hansı daha uyğundur? Cavab: PUT resursu tam əvəzləyir və idempotentdir; PATCH qismən dəyişiklikdir. `approve` resursun vəziyyətini qayda əsasında dəyişən əməliyyatdır, tam əvəzləmə deyil; PATCH və ya POST (`/approvals` alt-resursu) uyğundur. Layihə PATCH action endpoint-i seçib.

S2. Entity-ni controller-dən birbaşa qaytarmağın riskləri nələrdir? Cavab: Kontrakt persistence sxeminə bağlanır, həssas sahələr sızır, lazy əlaqələr `open-in-view=false` olduqda xəta verir və ya `true` olduqda gizli N+1 sorğuları yaradır, iki istiqamətli əlaqələr sonsuz rekursiya yarada bilər.

S3. Niyə `@Valid` səhv sorğuda `@PreAuthorize`-dan əvvəl işləyir və bu nə vaxt problemdir? Cavab: Metod arqumentləri (deserializasiya + validasiya) handler metodunu çağırmazdan əvvəl hazırlanır; `@PreAuthorize` isə controller bean-inin proxy-sində, metod çağırışı zamanı yoxlanılır. Nəticədə icazəsiz istifadəçi validasiya mesajlarını görə bilir — bu, kiçik informasiya sızmasıdır. URL səviyyəli (`authorizeHttpRequests`) qaydalar filtrdə, validasiyadan əvvəl işləyir.

S4. Offset və keyset səhifələmə fərqi? Cavab: Offset `LIMIT n OFFSET m` istifadə edir — sadədir, istənilən səhifəyə keçid mümkündür, lakin böyük `m` dəyərlərində DB bütün əvvəlki sətirləri keçir və paralel insertlər səhifələrdə sürüşmə yaradır. Keyset son görülən açardan (`WHERE (created_at, id) < (?, ?)`) davam edir — sabit performans, lakin ixtiyari səhifəyə keçid yoxdur.

S5. Cavab zərfinin (envelope) üstünlük və çatışmazlıqları? Cavab: Üstünlük — vahid struktur, metadata üçün yer. Çatışmazlıq — HTTP semantikasını təkrarlayır (`success` statusdan məlumdur), klient kodunu dərinləşdirir, standart alətlərlə (Problem Details) uyğunluğu azaldır.

5.11 Xülasə

- REST resursları: kampaniya kolleksiyası, alt-resurs tarixçə, hesablanmış statistika, status action endpoint-ləri.
- Controller yalnız tərcümə edir; qaydalar servisdədir; entity-lər xaricə çıxmır (tarixçə istisnası ilə).
- `ApiResponse` / `ApiError` vahid formatı təmin edir; Problem Details standart alternativdir.
- Səhifələmə DTO ilə edilir (campaign), log-service isə `Page`-i birbaşa qaytarır.
- Növbəti fəsil campaign-service-in biznes məntiqinə dərindən baxır.

III hissə. Layihənin nüvə implementasiyası

Campaign Service, persistence, validasiya, istisnalar, Auth Service və təhlükəsizlik

6. Campaign Service — biznes nüvəsi dərindən

campaign-service sistemin biznes mərkəzidir: kampaniya qaydaları burada yaşayır, digər servislər (dashboard, log-service) yalnız onun nəticələrindən istifadə edir. Bu fəsil servisi sinif-sinif deyil, **əməliyyat-əməliyyat** izah edir; persistence detalları 7-ci, Kafka 11-ci, gRPC 13-cü, cache 15-ci fəsildə dərinləşdirilir.

6.1 Məsuliyyətlər və sərhədlər

Məsuliyyət	Harada
Kampaniya yaratmaq, ilkin statusu təyin etmək, dublikatı rədd etmək	`CampaignService.create`
Oxumaq (id üzrə, səhifələnmiş siyahı)	`getById`, `getAll`
Status keçidləri (təsdiq, deaktivasiya)	`approve`, `deactivate`, `changeStatus`
Status tarixçəsi	`saveHistory`, `getHistory`
Statusa görə statistika (cache ilə)	`getStatistics`
Status hadisələrini nəşr etmək	`publishStatusChange` → Kafka `campaign-events`
REST API	`CampaignController`
gRPC API (dashboard üçün)	`CampaignGrpcEndpoint`

Data sahibliyi: `campaign_db` PostgreSQL bazası (`campaigns`, `campaign_status_histories`) və Redis-dəki `campaign-statistics` cache-i.

Paket quruluşu **FAKT** (`campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/`):

```
campaign/
├── CampaignServiceApplication.java   @SpringBootApplication, @EnableCaching
├── controller/CampaignController.java
├── domain/        Campaign (record), CampaignCategory, CampaignStatus
├── dto/           CreateCampaignRequest, CampaignResponse, CampaignPageResponse,
│                  CampaignHistoryResponse, CampaignStatisticsResponse
├── entity/        CampaignEntity, CampaignStatusHistoryEntity
├── grpc/          CampaignGrpcEndpoint
├── mapper/        CampaignMapper
├── repository/    CampaignRepository, CampaignStatusHistoryRepository
└── service/       CampaignService
```

Bu, **texniki qatlara görə** (layered) paketləşdirmədir. Alternativ — **feature-a görə** paketləşdirmə (`campaign/create`, `campaign/approval` ...) böyük servislərdə kohesiyanı artırır; bu ölçüdə servis üçün qat paketləri tamamilə qəbuledilir.

6.2 Domen modeli

Mənbə: `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/domain/Campaign.java:6-15`

```java
public record Campaign(
        UUID id,
        String title,
        String description,
        CampaignCategory category,
        CampaignStatus status,
        UUID duplicateOf,
        Instant createdAt,
        Instant updatedAt
) { }
```

`Campaign` dəyişməz record-dur; biznes qaydaları burada deyil, servisdədir. Status dəyişikliyi entity-nin `changeStatus` metodu ilə edilir, o da heç bir yoxlama aparmır **FAKT** (`CampaignEntity.java:79-81`):

```
public void changeStatus(CampaignStatus status) {
    this.status = status;
}
```

NƏTİCƏ Bu, **anemik domen modeli** (anemic domain model) yanaşmasıdır: entity yalnız data daşıyır, qaydalar servis metodlarındadır. Kiçik servis üçün sadə və oxunaqlıdır. Mənfi tərəfi: qayda yalnız servis vasitəsilə qorunur — başqa kod `entity.changeStatus(ACTIVE)` çağırsa, keçid yoxlanmadan dəyişər. "Zəngin domen modeli" alternativi aşağıda (6.9) göstərilir.

`DUPLICATE` statusu və `duplicate_of` sütunu: köhnə dizaynın izi

FAKT

- `CampaignStatus.DUPLICATE` mövcuddur (`CampaignStatus.java:3-8`);
- `campaigns.duplicate_of` sütunu yaradılıb (`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:35-37`);
- dublikat yoxlaması `StatusNot(DUPLICATE)` şərti ilə aparılır (`CampaignService.java:140-150`);
- partial unique indeks `WHERE status <> 'DUPLICATE'` şərtinə malikdir (`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:101-113`);
- `ensureNotDuplicate` `DUPLICATE` statuslu kampaniyanın yenilənməsini qadağan edir (`CampaignService.java:171-175`);
- lakin **heç bir kod yolu** kampaniyanı `DUPLICATE` statusu ilə yaratmır və ya ona keçirmir; `duplicateOf` həmişə `null` ötürülür (`CampaignService.java:60-67`).

NƏTİCƏ İlkin dizaynda təkrar sorğu ayrıca sətir kimi `DUPLICATE` statusu və orijinala istinadla (`duplicate_of`) saxlanılırdı; sonra qayda "dublikatı rədd et (409)" kimi dəyişdirilib (README: "no duplicate campaign row is inserted"). Partial indeksin `WHERE status <> 'DUPLICATE'` şərti də məhz köhnə modeli dəstəkləyirdi: dublikat sətirlər unikallıq qaydasından kənarda qalmalı idi. Hazırda bu elementlər zərərsiz, lakin çaşdırıcı ölü koddur. Statistika endpoint-i `DUPLICATE: 0` qaytarır (aşağıda), dashboard isə onu süzgəcdən keçirir.

6.3 Kampaniya yaratmaq: `create`

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:54-77

```
@Transactional
@CacheEvict(value = "campaign-statistics", allEntries = true)
public Campaign create(CreateCampaignRequest request) {
    rejectIfDuplicateExists(request);

    CampaignStatus status = resolveInitialStatus(request.category());
    CampaignEntity entity = new CampaignEntity(
            UUID.randomUUID(),
            request.title(),
            request.description(),
            request.category(),
            status,
            null
    );

    try {
        CampaignEntity saved = campaignRepository.saveAndFlush(entity);
        saveHistory(saved.getId(), null, saved.getStatus(), "Campaign created");
        publishStatusChange(saved.getId(), saved.getCategory(), null, saved.getStatus(), "CREATED");
        return campaignMapper.toDomain(saved);
    } catch (DataIntegrityViolationException exception) {
        throw new DuplicateCampaignException(duplicateMessage(request));
```

```
    }
}
```

Addım-addım:

1. `@Transactional` — metod bir DB tranzaksiyasında icra olunur; runtime istisna atılarsa rollback.
2. `@CacheEvict(allEntries = true)` — metod uğurla bitdikdən sonra `campaign-statistics` cache-i tam təmizlənir, çünki yeni kampaniya sayları dəyişdirir.
3. **Dublikat yoxlaması (tətbiq səviyyəsi)** — `existsByCategoryAndTitleAndDescriptionAndStatusNot(...)` Spring Data **törəmə sorğusu** (derived query): metod adından `select ... where category=? and title=? and description=? and status<>? limit 1` qurulur FAKT (`CampaignRepository.java:13-18`, `CampaignService.java:140-150`). Varsa — `DuplicateCampaignException` → `409 DUPLICATE_CAMPAIGN`.
4. **İlkin status** — Java switch ifadəsi FAKT (`CampaignService.java:133-138`):

   Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:133-138

   ```
   private CampaignStatus resolveInitialStatus(CampaignCategory category) {
       return switch (category) {
           case LIFE → CampaignStatus.ACTIVE;
           case TSS, OSS, OTHER → CampaignStatus.PENDING_APPROVAL;
       };
   }
   ```

   `default` yoxdur — bütün enum dəyərləri əhatə olunub. NƏTİCƏ Yeni kateqoriya əlavə edilsə, bu `switch` kompilyasiya xətası verəcək (enum üzrə switch ifadəsi tam olmalıdır) — bu, qaydanın unudulmamasını təmin edən faydalı xüsusiyyətdir.
5. **Entity yaratmaq** — id tətbiqdə `UUID.randomUUID()` ilə yaradılır, `duplicateOf = null`.
6. `saveAndFlush` — INSERT dərhal DB-yə göndərilir (flush). Məqsəd: unikal indeks pozuntusu tranzaksiyanın sonunda deyil, **burada, `try` blokunun içində** baş versin ki, `catch` onu tuta bilsin.
7. **İkinci müdafiə xətti (DB səviyyəsi)** — iki paralel sorğu eyni anda 3-cü addımı keçə bilər (hər ikisi "yoxdur" görür). Belə halda ikinci INSERT partial unique indeks `uk_campaign_unique_non_duplicate`-i pozur (changelog `003-prevent-duplicate-campaigns`), Spring bunu `DataIntegrityViolationException`-a tərcümə edir və kod onu `DuplicateCampaignException`-a çevirir. Bu, **"check-then-act" yarışının (race condition) DB constraint-i ilə bağlanmasının** klassik nümunəsidir — tətbiq yoxlaması yalnız gözəl xəta mesajı üçündür, həqiqi zəmanəti DB verir.
8. **Tarixçə** — `previousStatus = null`, səbəb "Campaign created".
9. **Hadisə** — `CampaignStatusChangedEvent` Kafka-ya, açar `campaignId` (fəsil 11).
10. **Nəticə** — `saved` (merge-in qaytardığı **idarə olunan** nüsxə) domen record-una çevrilir.

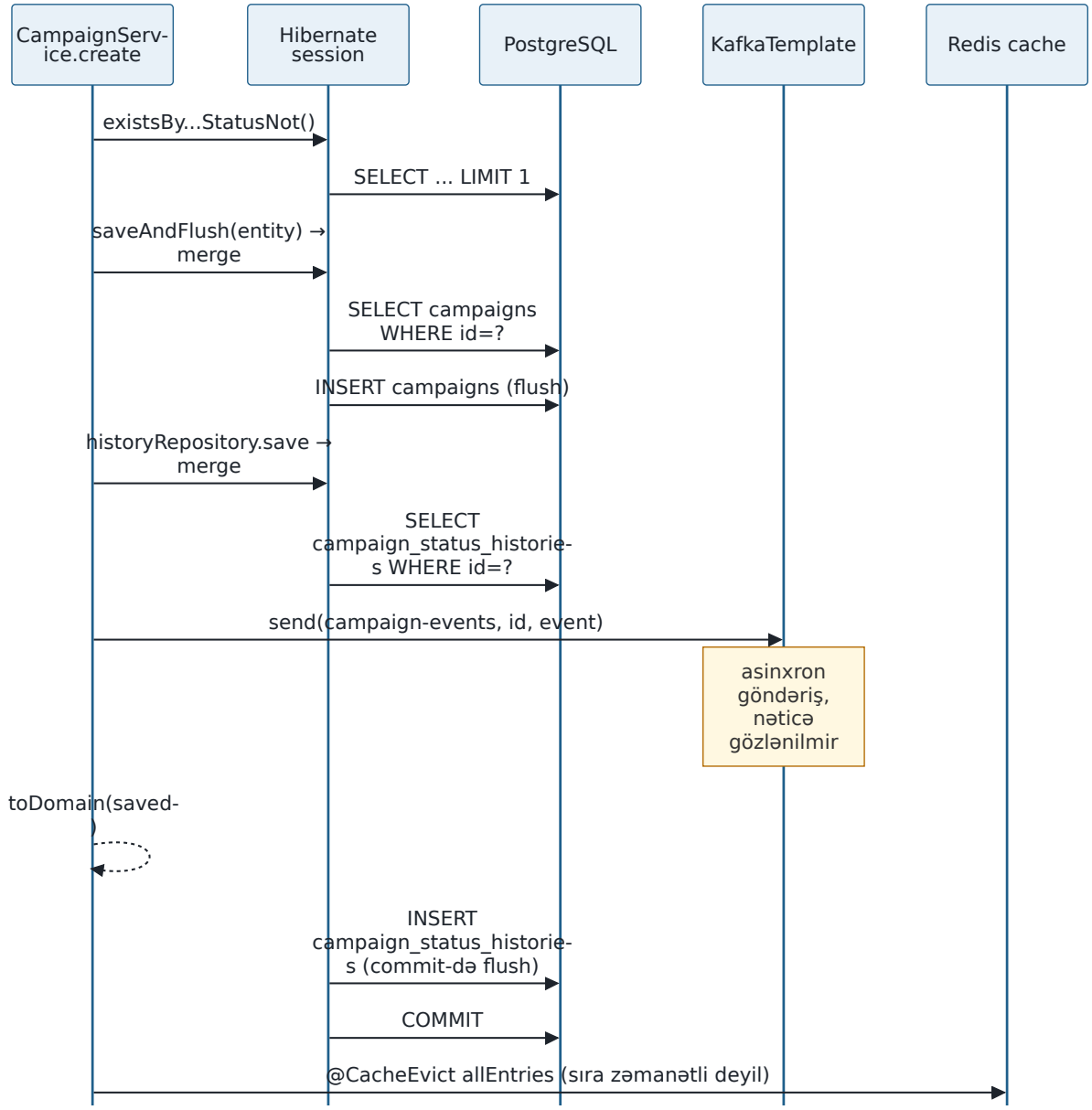
Gizli detal: niyə `saved`, niyə `entity` deyil?

NƏTİCƏ `CampaignEntity`-nin id-si əl ilə təyin olunur və `@Version` sahəsi yoxdur. Spring Data JPA `SimpleJpaRepository.save()` entity-nin "yeni" olub-olmadığını id-nin `null` olmasına görə müəyyən edir; id artıq təyin olunduğu üçün entity "köhnə" sayılır və `persist` əvəzinə `merge` çağırılır. `merge`:

- əvvəlcə `SELECT ... FROM campaigns WHERE id = ?` icra edir (sətir yoxdur),
- sonra ötürülən obyektin vəziyyətini **yeni idarə olunan nüsxəyə** kopyalayır və onu persist edir (`@PrePersist` bu nüsxədə işləyir, `createdAt/updatedAt` orada təyin olunur),
- **həmin nüsxəni qaytarır**; ötürülən `entity` obyekti idarə olunmayan (detached) qalır və onun `createdAt`-ı `null`-dur.

Kod `saved.getCreatedAt()`-dan istifadə etdiyi üçün düzgün işləyir. `entity`-dən istifadə etsəydi, cavabda `createdAt: null` olardı. Dərs: **`save()`-in qaytardığı dəyəri həmişə istifadə edin.** Əlavə SELECT-dən qaçmaq üçün entity `Persistable<UUID>` implementasiya edib `isNew()`-i özü idarə edə bilər, və ya `@Version` sahəsi əlavə oluna bilər (fəsil 7, tapıntı F14).

`create` zamanı DB və Kafka əməliyyatlarının sırası



create() daxilində SQL və Kafka əməliyyatlarının sırası (Hibernate davranışına əsaslanan NƏTİCƏ)

Bu diaqramdan çıxan vacib nəticə NƏTİCƏ: **Kafka hadisəsi tarixçə sətri DB-yə yazılmazdan və tranzaksiya commit olunmazdan əvvəl göndərilir.** Commit uğursuz olsa (məsələn, DB bağlantısı kəsilsə), hadisə artıq yolda ola bilər və mövcud olmayan kampaniyadan xəbər verər. Bu, **dual-write** problemidir (tapıntı F09, fəsil 11-də Transactional Outbox ilə həlli).

6.4 Oxu əməliyyatları: `getById`, `getAll`

Mənbə: `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:79-89`

```
@Transactional(readOnly = true)
public Campaign getById(UUID id) {
```

```
    return campaignRepository.findById(id)
            .map(campaignMapper::toDomain)
            .orElseThrow(() -> new ResourceNotFoundException("Campaign not found: " + id));
}

@Transactional(readOnly = true)
public Page<Campaign> getAll(Pageable pageable) {
    return campaignRepository.findAll(pageable).map(campaignMapper::toDomain);
}
```

- `readOnly = true` — **NƏTİCƏ** Spring Hibernate session-un flush rejimini `MANUAL` edir (dirty checking snapshot-ları üçün iş azalır) və JDBC bağlantısına read-only işarəsi ötürür; PostgreSQL drayveri bunu `SET TRANSACTION READ ONLY` kimi tətbiq edə bilər. Replika marşrutlaşdırması (read replica routing) qurulubsa, bu işarə ondan istifadə üçün də əsasdır. Bu layihədə replika yoxdur.
- `findAll(pageable)` iki sorğu icra edir: məlumat (`ORDER BY ... LIMIT ? OFFSET ?`) və cəmi say (`SELECT count(*)`).
- Mapping tranzaksiya daxilində edilir — `open-in-view: false` şəraitində doğru yerdir.

6.5 Status keçidləri: `approve`, `deactivate`, `changeStatus`

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:91-111

```
@Transactional
@CacheEvict(value = "campaign-statistics", allEntries = true)
public Campaign approve(UUID id) {
    CampaignEntity entity = findEntity(id);
    ensureNotDuplicate(entity);
    if (entity.getStatus() ≠ CampaignStatus.PENDING_APPROVAL) {
        throw new InvalidStatusTransitionException("Only PENDING_APPROVAL campaigns can be approved");
    }
    return changeStatus(entity, CampaignStatus.ACTIVE, "Campaign approved");
}

@Transactional
@CacheEvict(value = "campaign-statistics", allEntries = true)
public Campaign deactivate(UUID id) {
    CampaignEntity entity = findEntity(id);
    ensureNotDuplicate(entity);
    if (entity.getStatus() ≠ CampaignStatus.ACTIVE && entity.getStatus() ≠ CampaignStatus.PENDING_APPRO
VAL) {
        throw new InvalidStatusTransitionException("Only ACTIVE or PENDING_APPROVAL campaigns can be deac
tivated");
    }
    return changeStatus(entity, CampaignStatus.DEACTIVE, "Campaign deactivated");
}
```

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:162-169

```
private Campaign changeStatus(CampaignEntity entity, CampaignStatus targetStatus, String reason) {
    CampaignStatus previousStatus = entity.getStatus();
    entity.changeStatus(targetStatus);
    CampaignEntity saved = campaignRepository.save(entity);
    saveHistory(saved.getId(), previousStatus, targetStatus, reason);
    publishStatusChange(saved.getId(), saved.getCategory(), previousStatus, targetStatus, reason);
    return campaignMapper.toDomain(saved);
}
```

Qaydalar cədvəli **FAKT**:

Cari status	`approve`	`deactivate`
`PENDING_APPROVAL`	→ `ACTIVE`	→ `DEACTIVE`
`ACTIVE`	409 `INVALID_STATUS_TRANSITION`	→ `DEACTIVE`

Cari status	`approve`	`deactivate`
`DEACTIVE`	409	409
`DUPLICATE` (praktikada yoxdur)	409 `DUPLICATE_CAMPAIGN`	409 `DUPLICATE_CAMPAIGN`
kampaniya yoxdur	404 `RESOURCE_NOT_FOUND`	404

Dirty checking və `save()`-in rolu

`findEntity` entity-ni cari tranzaksiyanın **persistence context**-inə yükləyir; entity "idarə olunan" (managed) vəziyyətdədir. `entity.changeStatus(...)` yalnız Java sahəsini dəyişir; Hibernate commit zamanı yüklənmə anındakı snapshot ilə müqayisə edib `UPDATE` yaradır (**dirty checking**). Buna görə `campaignRepository.save(entity)` burada texniki olaraq lazımsızdır — idarə olunan entity üçün `merge` eyni instansı qaytarır. **NƏTİCƏ** Zərərsizdir və bəzən "niyyəti açıq göstərmək" üçün yazılır, lakin JPA-nın işləmə modelini gizlədir.

Defekt: cavabdakı `updatedAt` köhnədir (F36)

[NƏTİCƏ, yüksək ehtimal] `CampaignEntity.preUpdate()` `@PreUpdate` ilə işarələnib (`CampaignEntity.java:65-68`). JPA spesifikasiyasına görə `@PreUpdate` callback-i **flush zamanı**, UPDATE SQL-i yaradılarkən çağırılır. `changeStatus` daxilində flush baş vermir (`save` idarə olunan entity üçün flush etmir, tarixçə entity-sinin `merge`-i id üzrə yükləmədir və auto-flush yaratmır). Nəticədə `campaignMapper.toDomain(saved)` hələ köhnə `updatedAt` dəyərini oxuyur; yeni dəyər yalnız commit zamanı entity-yə və DB-yə yazılır. Klient `approve` cavabında `status: ACTIVE`, lakin **dəyişməmiş** `updatedAt` görür; sonrakı `GET` isə yeni vaxtı göstərir. Servis testləri `approve/deactivate`-i əhatə etmədiyi üçün bu aşkarlanmayıb. Həll variantları: `saveAndFlush` istifadə etmək, `updatedAt`-ı `changeStatus` daxilində açıq təyin etmək, və ya Hibernate `@UpdateTimestamp` (yenə flush-a bağlıdır).

Paralellik: lost update riski (F37)

NƏTİCƏ PostgreSQL-in standart izolyasiya səviyyəsi **READ COMMITTED**-dir. İki admin eyni `PENDING_APPROVAL` kampaniyasını eyni anda biri təsdiq, digəri deaktiv etsə:

Zaman	Tranzaksiya A (approve)	Tranzaksiya B (deactivate)
t1	SELECT → `PENDING_APPROVAL`	
t2		SELECT → `PENDING_APPROVAL`
t3	yoxlama keçir, status=ACTIVE, tarixçə, hadisə	
t4		yoxlama keçir, status=DEACTIVE, tarixçə, hadisə
t5	COMMIT	
t6		UPDATE A-nın sətir kilidini gözləyir, sonra COMMIT (son yazan qalib)

Nəticə: son status `DEACTIVE`, tarixçədə isə həm `PENDING_APPROVAL→ACTIVE`, həm `PENDING_APPROVAL→DEACTIVE` var; Kafka-ya iki ziddiyyətli hadisə gedib. Həllər:

- **Optimistik kilid** — `@Version private long version;` əlavə etmək: B-nin UPDATE-i `WHERE id=? AND version=?` ilə 0 sətir yeniləyər və `OptimisticLockException` (Spring: `ObjectOptimisticLockingFailureException`) atılar → 409.
- **Pessimistik kilid** — `@Lock(LockModeType.PESSIMISTIC_WRITE)` ilə `SELECT ... FOR UPDATE`.
- **Şərti UPDATE** — `UPDATE campaigns SET status='ACTIVE' WHERE id=? AND status='PENDING_APPROVAL'` və təsirlənən sətir sayını yoxlamaq.

6.6 Tarixçə: `saveHistory`, `getHistory`

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:113-119

```
@Transactional(readOnly = true)
public List<CampaignStatusHistoryEntity> getHistory(UUID id) {
    if (!campaignRepository.existsById(id)) {
        throw new ResourceNotFoundException("Campaign not found: " + id);
    }
    return historyRepository.findByCampaignIdOrderByChangedAtAsc(id);
}
```

- Mövcud olmayan kampaniya üçün boş siyahı deyil, 404 qaytarılır — "kampaniya var, tarixçəsi boşdur" ilə "kampaniya yoxdur" fərqləndirilir. Bu, düzgün API dizaynıdır.
- `findByCampaignIdOrderByChangedAtAsc` törəmə sorğudur (`CampaignStatusHistoryRepository.java:9-11`). **NƏTİCƏ** `campaign_id` üzrə indeks yoxdur (FK constraint PostgreSQL-də avtomatik indeks yaratmır) — tarixçə cədvəli böyüdükcə bu sorğu tam skana çevrilir (tapıntı F15, fəsil 7).
- `changedAt` tətbiqdə `Instant.now()` ilə təyin olunur (`CampaignStatusHistoryEntity.java:40-47`). Eyni millisaniyədə iki qeyd olarsa sıra qeyri-müəyyəndir; `id` ikinci sıralama açarı kimi əlavə oluna bilərdi.

6.7 Statistika: `getStatistics`

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:121-131

```
@Transactional(readOnly = true)
@Cacheable("campaign-statistics")
public Map<String, Long> getStatistics() {
    Map<CampaignStatus, Long> result = new EnumMap<>(CampaignStatus.class);
    for (CampaignStatus status : CampaignStatus.values()) {
        result.put(status, 0L);
    }
    campaignRepository.countByStatus().forEach(item → result.put(item.getStatus(), item.getCount()));
    return result.entrySet().stream()
            .collect(java.util.stream.Collectors.toMap(entry → entry.getKey().name(), Map.Entry::getValu
e));
}
```

- **Sıfırla əvvəlcədən doldurma** — `GROUP BY` yalnız mövcud statusları qaytarır; əvvəlcədən 0 yazmaq bütün statusların cavabda olmasını təmin edir (o cümlədən `DUPLICATE: 0`).
- **JPQL proyeksiyası** **FAKT** (`CampaignRepository.java:20-26`): `select c.status as status, count(c.id) as count from CampaignEntity c group by c.status` — nəticə `CampaignStatusCountProjection` interfeys proyeksiyasına (alias-lar getter adlarına uyğun gəlir) xəritələnir.
- **Cache** — `@Cacheable("campaign-statistics")` parametrsiz metod üçün açar `SimpleKey.EMPTY` -dir; Redis-də açar `campaign-statistics::SimpleKey []` formasındadır. Dəyər `HashMap` (Collectors.toMap) — JDK serializasiyası ilə saxlanılır (fəsil 15).
- `EnumMap` -dan `String` açarlı xəritəyə çevirmə həm JSON (REST), həm gRPC (`setStatus(String)`) üçün rahatdır.

6.8 Konfiqurasiya

FAKT (`campaign-service/src/main/resources/application.yml`):

Xassə	Dəyər	Təsiri
`server.port`	8082	REST
`grpc.server.port`	`${CAMPAIGN_GRPC_PORT:9090}`	gRPC (sətir 29-31)

Xassə	Dəyər	Təsiri
`spring.datasource.*`	`${CAMPAIGN_DB_URL:jdbc:postgresql://localhost:5434/campaign_db}`	DB (sətir 8-11)
`spring.jpa.hibernate.ddl-auto`	`validate`	Sxemi Liquibase yaradır, Hibernate yalnız yoxlayır (sətir 12-15)
`spring.jpa.open-in-view`	`false`	Lazy yükləmə controller/view-da işləmir (sətir 15)
`spring.liquibase.change-log`	`classpath:/db/changelog/db.changelog-master.yml`	Miqrasiyalar (sətir 16-17)
`spring.cache.type`	`redis`	`RedisCacheManager` (sətir 22-23)
`spring.kafka.bootstrap-servers`	`${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}`	`KafkaJsonConfiguration` tərəfindən oxunur (sətir 24-25)
`app.slow-request-threshold-ms`	5	Yavaş sorğu filtri (sətir 78)
`app.kafka.topics.*`	`log-events`, `campaign-events`	Mövzu adları (sətir 79-82)
`app.jwt.public-key-location`	`classpath:keys/public.pem`	Token doğrulaması (sətir 83-86)

6.9 Yeni layihədə tətbiq: zəngin domen modeli ilə status maşını

İllüstrativ nümunə (repozitoriyada yoxdur) — keçid qaydalarını entity-yə və enum-a köçürmək:

```
public enum CampaignStatus {
    PENDING_APPROVAL, ACTIVE, DEACTIVE;

    private static final Map<CampaignStatus, Set<CampaignStatus>> ALLOWED = Map.of(
            PENDING_APPROVAL, EnumSet.of(ACTIVE, DEACTIVE),
            ACTIVE, EnumSet.of(DEACTIVE),
            DEACTIVE, EnumSet.noneOf(CampaignStatus.class));

    public boolean canMoveTo(CampaignStatus target) {
        return ALLOWED.get(this).contains(target);
    }
}

@Entity
public class CampaignEntity {
    @Version private long version;              // lost update qorunması
    // ...
    public CampaignStatus transitionTo(CampaignStatus target) {
        if (!status.canMoveTo(target)) {
            throw new InvalidStatusTransitionException(status + " → " + target + " is not allowed");
        }
        CampaignStatus previous = status;
        status = target;
        updatedAt = Instant.now();              // flush-dan asılı olmayan vaxt damğası
        return previous;
    }
}
```

Bu variantda servis yalnız orkestrasiya edir (yüklə → `transitionTo` → tarixçə → outbox), qayda isə bir yerdə və unit test ilə asanlıqla yoxlanılan formadadır.

6.10 Uğursuzluq ssenariləri

Ssenari	Mərhələ	Nəticə	Təkrar etmək təhlükəsizdirmi?
Səhv gövdə (qısa başlıq)	Validasiya	400 `VALIDATION_ERROR`	Bəli, düzəldib

Ssenari	Mərhələ	Nəticə	Təkrar etmək təhlükəsizdirmi?
Yanlış `category` dəyəri (`"XYZ"`)	JSON deserializasiyası	`HttpMessageNotReadableException` → **500** (F07)	Bəli
Səhv UUID formatı (`/campaigns/abc`)	Path variable çevrilməsi	**500** (F07)	—
Dublikat	Servis / DB unique indeks	409 `DUPLICATE_CAMPAIGN`	Effekt yoxdur
Yanlış status keçidi	Servis	409 `INVALID_STATUS_TRANSITION`	Effekt yoxdur
DB əlçatmaz	Repository	500 (mesajda daxili detal ola bilər)	Bəli (yaratma dublikat qaydası sayəsində təhlükəsizdir)
Kafka əlçatmaz	`kafkaTemplate.send`	Sorğu `max.block.ms` (60 s) qədər gecikə bilər, sonra hadisə itir (log) — F10	—
Redis əlçatmaz	`@CacheEvict` / `@Cacheable`	**NƏTİCƏ** Standart `CacheErrorHandler` xətanı yenidən atır → 500; `create` -də commit artıq baş vermiş ola bilər	Dublikat qaydası sayəsində təkrar 409 verə bilər
Paralel approve/deactivate	Servis	Hər ikisi uğurlu (F37)	—

6.11 Müsahibə sualları

S1. Dublikat yoxlamasını niyə həm tətbiqdə, həm DB-də edirsiniz? Cavab: Tətbiq yoxlaması aydın xəta mesajı və DB-yə yazmadan rədd etmək üçündür, lakin iki paralel sorğu arasında yarış (check-then-act) var. Yalnız DB constraint-i atomik zəmanət verir. `saveAndFlush` constraint pozuntusunun `try` daxilində baş verməsini təmin edir ki, `DataIntegrityViolationException` 409-a çevrilsin.

S2. `save()` -in qaytardığı obyektdən istifadə etməmək nə vaxt səhvə səbəb olur? Cavab: Entity yeni olmadıqda (`isNew()==false` , məs. əl ilə təyin olunmuş id) `merge` çağırılır və o, **başqa** idarə olunan nüsxə qaytarır. Callback-lər (`@PrePersist`) və generasiya olunan dəyərlər həmin nüsxədə olur. Ötürülən obyekt detached qalır.

S3. `approve` cavabında `updatedAt` niyə dəyişməyə bilər? Cavab: `@PreUpdate` flush zamanı çağırılır. Mapping commit-dən (flush-dan) əvvəl edildiyi üçün köhnə dəyər oxunur. Həll: `saveAndFlush` , açıq vaxt təyini və ya mapping-i flush-dan sonra etmək.

S4. İki admin eyni anda təsdiq və deaktivasiya etsə nə olar, necə qarşısını alarsınız? Cavab: Optimistik kilid olmadığı üçün hər ikisi uğurlu olar, son commit qalib gəlir, tarixçə və hadisələr ziddiyyətli olar. `@Version` ilə optimistik kilid (konflikt nadirdirsə ən yaxşı seçim), pessimistik `SELECT FOR UPDATE` və ya şərti UPDATE.

S5. Status qaydalarını servisdə və ya entity-də saxlamağın fərqi nədir? Cavab: Servisdə — sadə, lakin qayda yalnız həmin servis metodları ilə qorunur (anemik model). Entity/enum-da — qayda invariant kimi obyektin özündədir, hər çağırış yolu onu keçmək məcburiyyətindədir və asan unit test olunur.

6.12 Xülasə

- campaign-service kampaniya qaydalarının yeganə sahibidir; REST və gRPC eyni servis qatından istifadə edir.
- Yaratma: tətbiq səviyyəli dublikat yoxlaması + DB partial unique indeks; `saveAndFlush` + `catch` .

- Status keçidləri servis metodlarında yoxlanılır; tarixçə və Kafka hadisəsi hər keçiddə yazılır.
- Gizli incəliklər: `merge` səbəbindən əlavə SELECT, `@PreUpdate` səbəbindən köhnə `updatedAt`, optimistik kilidin olmaması, hadisənin commit-dən əvvəl göndərilməsi.
- Növbəti fəsil bu davranışların altındakı persistence mexanizmlərini (JPA, Hibernate, Liquibase) izah edir.

7. Persistence: PostgreSQL, JPA/Hibernate və Liquibase

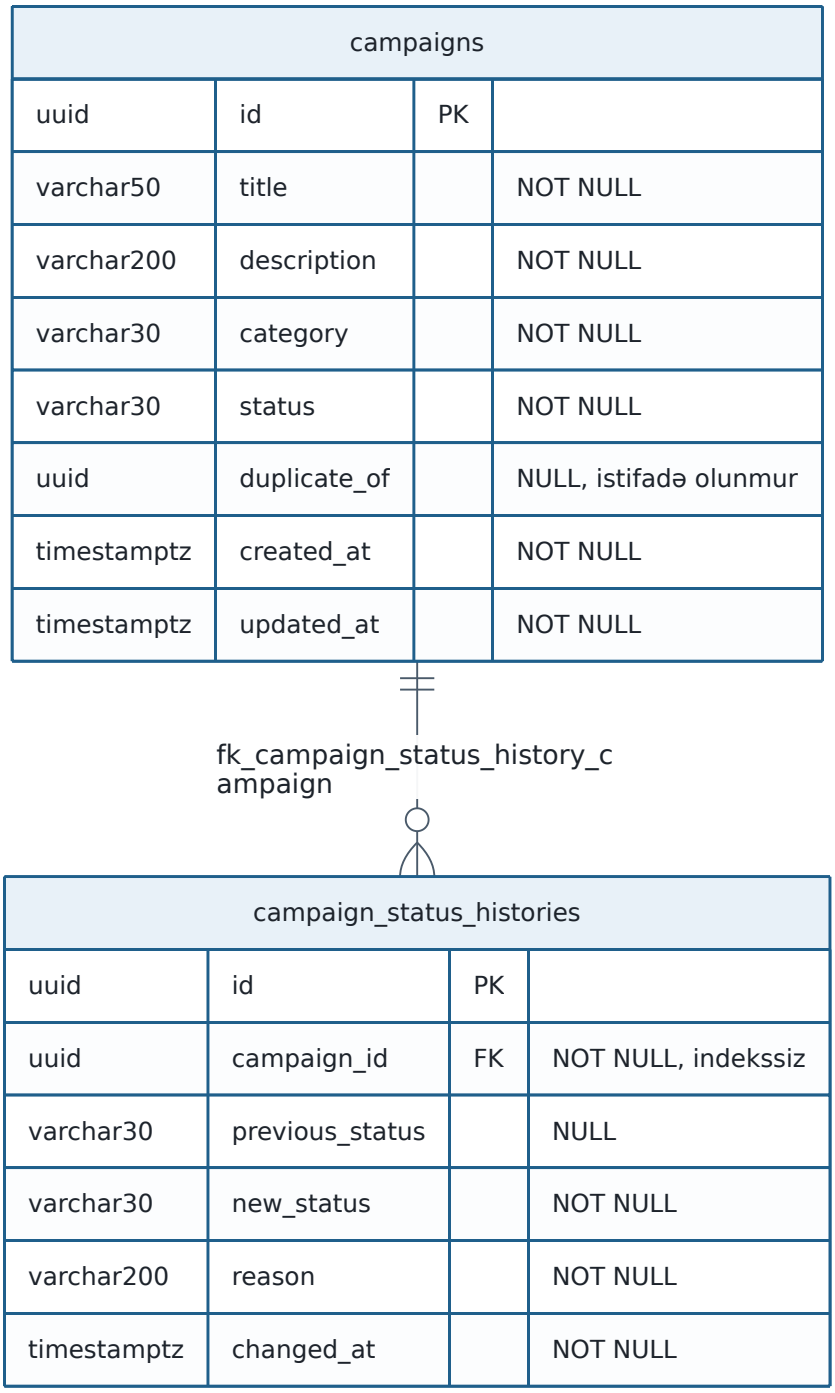
7.1 Problem: obyektlər, cədvəllər və zaman

Persistence qatı üç fərqli problemi həll edir: (1) **sxemin** zamanla idarə olunan şəkildə dəyişməsi (miqrasiya),

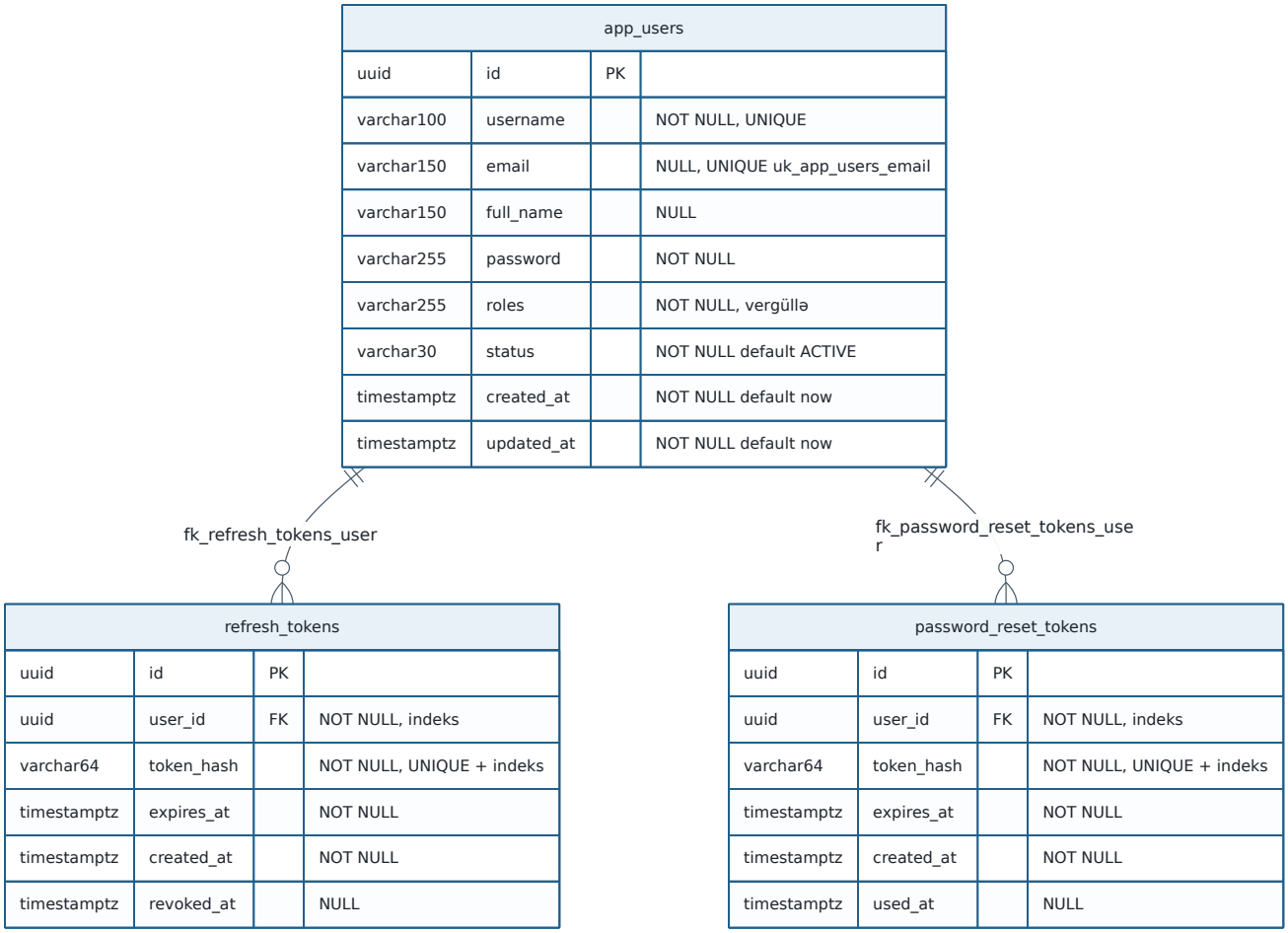
2. **obyekt ↔ sətir** xəritələnməsi (ORM), (3) **tranzaksiya** və paralellik altında datanın düzgünlüyü. Layihədə bunlar uyğun olaraq Liquibase, JPA/Hibernate 7.4 və Spring `@Transactional` ilə həll olunur. Bu fəsil hər birini əvvəl nəzəri, sonra repozitoriyadakı real nümunələrlə izah edir.

7.2 Data modeli

İki müstəqil PostgreSQL bazası var; aralarında FK yoxdur və ola bilməz (fərqli serverlər).



campaign_db sxemi (Liquibase changelog-undan bərpa edilib)



auth_db sxemi (Liquibase changelog-undan bərpa edilib)

Diqqət çəkən dizayn qərarları:

- **UUID primary key-lər tətbiqdə yaradılır** (`UUID.randomUUID()`, versiya 4). Üstünlük: id DB-yə getmədən məlumdur (hadisələrdə, tarixçədə dərhal istifadə olunur), servislər arası unikaldır, ardıcıl id-lərin sızması yoxdur. Mənfi: təsadüfi UUID-lər B-tree indeksdə təsadüfi yerlərə yazılır (səhifə bölünmələri, cache lokallığının azalması); zamanla sıralanan **UUIDv7** bu problemi azaldır.
- **Enum-lar `varchar` kimi saxlanılır** (`@Enumerated(EnumType.STRING)`) — enum sırası dəyişəndə data pozulmur (`ORDINAL`-ın əsas təhlükəsi). DB tərəfində `CHECK` constraint yoxdur; yalnız Java enum-u dəyərləri məhdudlaşdırır.
- **Rollar vergüllə ayrılmış sətirdir** (`app_users.roles = 'ROLE_USER,ROLE_ADMIN'`). Sadədir, lakin normallaşdırılmamış: "bütün adminləri tap" sorğusu `LIKE '%ROLE_ADMIN%'` tələb edir, rol adının dəyişdirilməsi hər sətri yeniləmək deməkdir. Alternativ: `user_roles(user_id, role)` cədvəli və ya `@ElementCollection`.
- **Tarixçə kampaniyaya id ilə istinad edir, `@ManyToOne` ilə yox** — `CampaignStatusHistoryEntity.campaignId` sadə `UUID` sahəsidir (`CampaignStatusHistoryEntity.java:20-21`), DB-də isə FK var. Bu, DDD-dəki "aqreqatlara id ilə istinad" prinsipinə uyğundur: tarixçəni yükləmək kampaniyanı yükləmir, lazy proxy problemləri yoxdur.

7.3 Liquibase: nəzəriyyə

Liquibase sxem dəyişikliklərini versiya nəzarəti altında saxlayan miqrasiya alətidir. Əsas anlayışlar:

Anlayış	İzah
Changelog	Dəyişikliklərin siyahısı (YAML/XML/JSON/SQL). Layihədə hər servisdə bir `db.changelog-master.yml`
ChangeSet	Atomik dəyişiklik vahidi; identifikasiyası `id + author + changelog yolu` üçlüyüdür
`DATABASECHANGELOG`	Liquibase-in yaratdığı cədvəl: hansı changeSet nə vaxt icra olunub, MD5 checksum
`DATABASECHANGELOGLOCK`	Eyni anda iki prosesin miqrasiya etməsinin qarşısını alan kilid
Checksum	ChangeSet məzmununun hash-i; icra olunmuş changeSet sonradan dəyişdirilərsə növbəti startup `ValidationFailedException` ilə dayanır
Rollback	`createTable`, `addColumn`, `createIndex` kimi əməliyyatlar üçün avtomatik; `sql`, `insert`, `update` üçün əl ilə yazılmalıdır

Spring Boot inteqrasiyası: `spring-boot-liquibase` (və `liquibase-core`) classpath-də olduqda `SpringLiquibase` bean-i tətbiq başlayarkən — JPA `EntityManagerFactory`-dən **əvvəl** — changelog-u icra edir FAKT (`campaign-service/pom.xml:91-99`, `campaign-service/src/main/resources/application.yml:16-17`). Bu sıra vacibdir: Hibernate `ddl-auto: validate` ilə sxemi yoxlayanda cədvəllər artıq mövcud olmalıdır.

7.4 campaign-service changelog-u

ChangeSet	Nə edir	Mənbə
`001-create-campaigns`	`campaigns` cədvəli; `idx_campaign_duplicate_lookup (category, title, description)` adi indeks	`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:2-57`
`002-create-campaign-status-histories`	Tarixçə cədvəli; FK `campaign_id → campaigns.id`	`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:58-99`
`003-prevent-duplicate-campaigns`	Raw SQL ilə **partial unique index**; açıq rollback	`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:101-113`

Mənbə: `campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:101-113`

```yaml
- changeSet:
    id: 003-prevent-duplicate-campaigns
    author: casestudy
    changes:
      - sql:
          splitStatements: false
          sql: |
            CREATE UNIQUE INDEX IF NOT EXISTS uk_campaign_unique_non_duplicate
            ON campaigns (category, title, description)
            WHERE status <> 'DUPLICATE';
    rollback:
      - sql:
          sql: DROP INDEX IF EXISTS uk_campaign_unique_non_duplicate;
```

- **Partial (şərti) indeks** PostgreSQL xüsusiyyətidir: indeks yalnız `WHERE` şərtini ödəyən sətirləri əhatə edir. Liquibase-in ümumi `createIndex` əməliyyatı bunu ifadə etmədiyi üçün raw SQL istifadə olunub — DB-yə xas xüsusiyyət lazım olanda bu, qəbul olunmuş yanaşmadır.
- `IF NOT EXISTS` — changeSet-i təkrar icraya qarşı dözümlü edir (məsələn, indeks əl ilə yaradılıbsa).

- Açıq `rollback` bloku — raw SQL üçün Liquibase əks əməliyyatı bilmir.

RİSK İndeks dizaynı (tapıntı F15):

- `idx_campaign_duplicate_lookup` (001) və `uk_campaign_unique_non_duplicate` (003) eyni üç sütunu əhatə edir. Statuslar heç vaxt `DUPLICATE` olmadığı üçün partial indeks praktiki olaraq bütün sətirləri əhatə edir və dublikat yoxlama sorğusu üçün də istifadə oluna bilər. Adi indeks artıqdır: hər INSERT/UPDATE iki indeksi yeniləyir.
- `campaign_status_histories.campaign_id` üzrə indeks yoxdur. PostgreSQL FK üçün avtomatik indeks yaratmır. Nəticə: `findByCampaignIdOrderByChangedAtAsc` tam skan edir; həmçinin `campaigns`-dan sətir silmək (gələcəkdə) FK yoxlaması üçün bütün tarixçəni skan edər. Tövsiyə: `CREATE INDEX ON campaign_status_histories (campaign_id, changed_at)`.
- `getAll` standart olaraq `created_at DESC` sıralayır; bu sütunda indeks yoxdur.

7.5 auth-service changelog-u: sxem təkamülünün real nümunəsi

auth-service changelog-u sxemin **zamanla necə dəyişdiyini** göstərir — tarixçəyə hörmət edən miqrasiyanın dərsliyi kimi oxunur:

ChangeSet	Nə edir	Mənbə
`001-create-app-users`	`app_users (id, username UNIQUE, password, roles)`; `demo` istifadəçisini `{noop}<redakte>` formatlı açıq şifrə ilə əlavə edir	`auth-service/src/main/resources/db/changelog/db.changelog-master.yml:2-37`
`002-extend-app-users-for-self-contained-auth`	`email`, `full_name`, `status (default ACTIVE)`, `created_at`, `updated_at (default CURRENT_TIMESTAMP)` əlavə edir; `demo`-nu yeniləyir; `uk_app_users_email`; `admin` istifadəçisini əlavə edir	`auth-service/src/main/resources/db/changelog/db.changelog-master.yml:39-89`
`003-create-refresh-tokens`	Cədvəl, FK, `user_id` və `token_hash` indeksləri	`auth-service/src/main/resources/db/changelog/db.changelog-master.yml:91-145`
`004-create-password-reset-tokens`	Eyni struktur	`auth-service/src/main/resources/db/changelog/db.changelog-master.yml:147-201`
`005-insert-default-role-user`	`user` istifadəçisi	`auth-service/src/main/resources/db/changelog/db.changelog-master.yml:203-216`

Öyrənmə nöqtələri **NƏTİCƏ**:

1. **NOT NULL sütunun mövcud cədvələ əlavə edilməsi.** 002-də `status` `nullable: false` ilə əlavə olunur. Cədvəldə artıq `demo` sətri var; `defaultValue: ACTIVE` olmasaydı, PostgreSQL mövcud sətirlər üçün NULL ilə NOT NULL constraint-ini pozduğu üçün əməliyyatı rədd edərdi. Default dəyər bu problemi həll edir.
2. **Data miqrasiyası sxem miqrasiyası ilə birlikdə.** `update ... where username = 'demo'` mövcud sətri yeni sütunlarla doldurur. Bu, "expand → migrate data" mərhələsidir.
3. **Seed data miqrasiyada.** Demo istifadəçilər changeSet-lərə yazılıb; bu, **hər mühitdə** (istehsal da daxil) `password` şifrəli admin yaradır. **RİSK** İstehsal üçün seed-lər `context: dev` kimi kontekstlərə bağlanmalı və ya ayrıca idarə olunmalıdır.
4. **`{noop}` prefiksi** — `DelegatingPasswordEncoder` hash alqoritmini prefiksdən müəyyən edir; `{noop}` "şifrə açıq mətndir" deməkdir. Seed istifadəçilərinin şifrəsi DB-də açıq saxlanılır; qeydiyyatdan keçən

istifadəçilərin şifrəsi isə `{bcrypt}...` olur (fəsil 9).

5. **Artıq indekslər** (F16): `token_hash` sütunu `unique: true` ilə yaradılır — PostgreSQL unique constraint üçün avtomatik unikal indeks yaradır. Sonra ayrıca `idx_refresh_tokens_token_hash` də yaradılır. İki indeks eyni sütunda; ikincisi lazımsızdır.

Liquibase ilə bağlı əməliyyat riskləri

Risk	Mexanizm	Simptom	Həll
İcra olunmuş changeSet-in redaktəsi	Checksum uyğunsuzluğu	Startup `ValidationFailedException`	Yeni changeSet yazın; zəruri halda `validCheckSum`
Yarımçıq miqrasiya və ya çökmüş pod	`DATABASECHANGELOGLOCK LOCKED=true` qalır	Yeni instans "Waiting for changelog lock"	Kilidi əl ilə buraxmaq (`liquibase release-locks`)
Çox instansın eyni anda başlaması	Kilid sayəsində ardıcıllaşır	Startup gecikir	Miqrasiyanı ayrıca job kimi icra etmək
Uzun sürən DDL (böyük cədvəldə indeks)	Cədvəl kilidlənir	Downtime	PostgreSQL-də `CREATE INDEX CONCURRENTLY` (tranzaksiyadan kənarda, `runInTransaction: false`)
Geri dönüş	`sql` / `insert` üçün avtomatik rollback yoxdur	Rollback mümkün olmur	Hər riskli changeSet-ə açıq `rollback`; daha yaxşısı — geriyə uyğun (backward-compatible) "expand/contract" miqrasiyaları

7.6 JPA entity xəritələnməsi

Mənbə: `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/entity/CampaignEntity.java:17-68`

```java
@Entity
@Table(name = "campaigns")
public class CampaignEntity {
    @Id
    private UUID id;

    @Column(nullable = false, length = 50)
    private String title;

    @Column(nullable = false, length = 200)
    private String description;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private CampaignCategory category;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 30)
    private CampaignStatus status;

    @Column(name = "duplicate_of")
    private UUID duplicateOf;

    @Column(name = "created_at", nullable = false)
    private Instant createdAt;

    @Column(name = "updated_at", nullable = false)
    private Instant updatedAt;

    protected CampaignEntity() {
    }
```

```
    // ... (buraxılıb: public konstruktor)
    @PrePersist
    void prePersist() {
        Instant now = Instant.now();
        createdAt = now;
        updatedAt = now;
    }

    @PreUpdate
    void preUpdate() {
        updatedAt = Instant.now();
    }
```

Element	Mənası və praktik nəticəsi
`@Id` (generator yoxdur)	Id tətbiq tərəfindən təyin olunur → `save` = `merge` → əlavə SELECT (F14)
`@Column(nullable=false, length=50)`	Yalnız DDL generasiyası (`create`) üçün metadata. `ddl-auto: validate` **NƏTİCƏ** sütunun mövcudluğunu və tip uyğunluğunu yoxlayır, uzunluq və nullability uyğunsuzluğunu adətən yoxlamır. Həqiqi qayda DB-dədir
`@Enumerated(STRING)`	`varchar` sütunda enum adı
`Instant ↔ timestamp with time zone`	Hibernate 6+ `Instant` -i UTC ilə `timestamptz` -yə xəritələyir
`protected` no-arg konstruktor	JPA tələbi (proxy və refleksiya ilə yaratma); `protected` biznes kodunda istifadəni əngəlləyir
`@PrePersist` / `@PreUpdate`	Persist/flush zamanı callback-lər; `@PreUpdate` -in flush zamanı işləməsi F36-ya səbəb olur
Setter yoxdur, `changeStatus` metodu var	İnkapsulyasiya; Hibernate sahələrə birbaşa çatır (field access, çünki `@Id` sahədədir)

auth-service entity-ləri Lombok istifadə edir **FAKT** (`AppUserEntity.java:22-26`):

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/model/entity/AppUserEntity.java:22-26

```
@Getter
@Entity
@Table(name = "app_users")
@NoArgsConstructor(access = AccessLevel.PROTECTED)
public class AppUserEntity {
```

`getRoles()` Lombok getter-ini **əl ilə əvəzləyir** və sətri siyahıya çevirir (`AppUserEntity.java:134-139`) — sahə `String` , getter `List<String>` qaytarır. Bu, MapStruct-un `AppUser.roles` (List) sahəsinə xəritələməsini də mümkün edir.

Əlaqələr (association) və lazy yükləmə

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/model/entity/RefreshTokenEntity.java:34-36

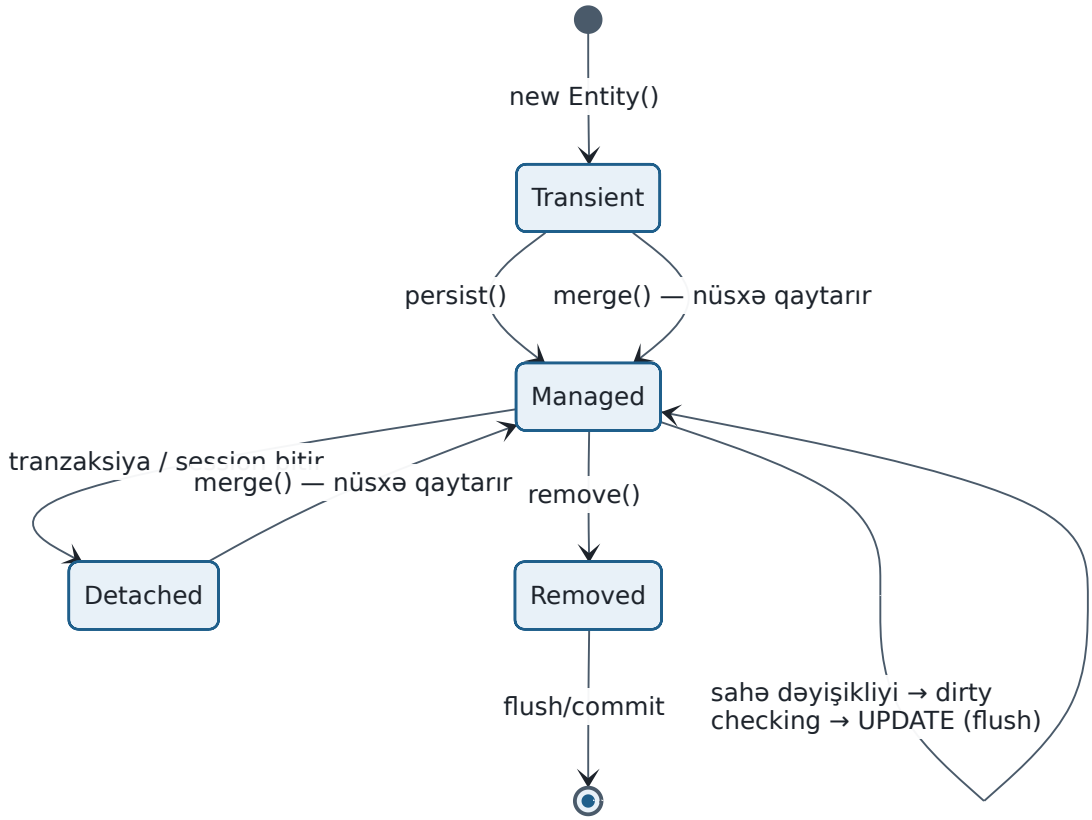
```
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(name = "user_id", nullable = false)
private AppUserEntity user;
```

- `@ManyToOne` -un JPA standartı **EAGER**-dir; açıq `LAZY` yazmaq vacib praktikadır. LAZY olduqda `user` sahəsi Hibernate proxy-si olur və ilk metod çağırışında (`getUser().isActive()`) ayrıca `SELECT` icra olunur.
- `optional = false` Hibernate-ə əlaqənin həmişə mövcud olduğunu deyir (inner join, null yoxlaması lazım deyil).
- `AuthService.refreshToken` daxilində (`AuthService.java:175-190`) `refreshToken.getUser()` tranzaksiya daxilində çağırılır — proxy initsializasiyası uğurludur. Eyni çağırış tranzaksiyadan kənarda olsaydı və `open-in-view=false` olduğu üçün `LazyInitializationException` atılardı.

N+1 problemi

N+1 — N elementli siyahı üçün 1 sorğu + hər element üçün lazy əlaqəni yükləmək üçün N sorğu. Bu layihədə siyahı qaytaran yerlərdə lazy əlaqəyə toxunulmur: kampaniya siyahısı əlaqəsizdir, tarixçə id ilə istinad edir, token-lər tək-tək işlənir. **NƏTİCƏ** Hazırda N+1 riski yoxdur. Gələcəkdə "istifadəçinin bütün aktiv sessiyaları + istifadəçi adı" kimi siyahı lazım olsa, həll yolları: `JOIN FETCH` JPQL, `@EntityGraph(attributePaths = "user")`, Hibernate `@BatchSize` və ya DTO proyeksiyası. Aşkarlama: `spring.jpa.properties.hibernate.generate_statistics=true`, SQL log-u, testdə sorğu sayını yoxlamaq.

7.7 Persistence context, entity vəziyyətləri və dirty checking



JPA entity vəziyyətləri

Persistence context (Hibernate `Session`) tranzaksiya boyunca yüklənmiş entity-lərin "birinci səviyyə cache"-i və dəyişiklik izləyicisidir. Layihədə dirty checking-in ən aydın nümunəsi `resetPassword`-dur **FAKT** (`AuthService.java:244-260`):

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:244-260

```
@Transactional
public LogoutResponse resetPassword(ResetPasswordRequest request) {
    PasswordResetTokenEntity resetToken = passwordResetTokenRepository.findByTokenHash(TokenHashUtils.sha
256(request.resetToken()))
            .orElseThrow(() -> new BusinessException(ErrorCode.UNAUTHORIZED, "Invalid password reset toke
n"));

    if (resetToken.isUsed() || resetToken.isExpired()) {
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "Password reset token is expired or already u
sed");
    }

    AppUserEntity user = resetToken.getUser();
    user.changePassword(passwordEncoder.encode(request.newPassword()));
    resetToken.markUsed();
```

```
        refreshTokenRepository.revokeAllActiveTokensForUser(user.getId(), Instant.now());
        authAuditService.publishResetPasswordAuditAsync(user.getUsername());

        return new LogoutResponse("Password was reset successfully. Active refresh tokens were revoked.");
    }
```

Burada **heç bir `save()` çağırışı yoxdur**, lakin iki UPDATE baş verir:

1. `user.changePassword(...)` — `app_users.password` yenilənir (lazy proxy initsializasiya olunur, sonra dəyişir);
2. `resetToken.markUsed()` — `password_reset_tokens.used_at` yenilənir.

Hər ikisi idarə olunan entity olduğu üçün commit zamanı Hibernate snapshot-larla müqayisə edib UPDATE yaradır.

Bulk update və persistence context. `revokeAllActiveTokensForUser` `@Modifying` JPQL UPDATE-dir FAKT (`RefreshTokenRepository.java:40-42`):

```
@Modifying
@Query("update RefreshTokenEntity token set token.revokedAt = :revokedAt where token.user.id = :userId an
d token.revokedAt is null")
void revokeAllActiveTokensForUser(@Param("userId") UUID userId, @Param("revokedAt") Instant revokedAt);
```

NƏTİCƏ Bulk əməliyyatlar birbaşa DB-də işləyir, persistence context-dəki entity-ləri yeniləmir. Bu metoddan əvvəl `RefreshTokenEntity` yüklənsəydi, onun yaddaşdakı `revokedAt`-ı köhnə qalardı. `resetPassword`-da belə entity yüklənmədiyi üçün problem yoxdur. Ümumi qorunma: `@Modifying(clearAutomatically = true, flushAutomatically = true)`. Həmçinin Hibernate bulk UPDATE-dən əvvəl müvafiq cədvələ təsir edən gözləyən dəyişiklikləri auto-flush edir.

`refreshToken` metodunda rotasiya da dirty checking-dir: `refreshToken.revoke()` (`AuthService.java:187`) — save yoxdur, UPDATE commit-də.

7.8 Tranzaksiyalar

`@Transactional` standart parametrləri: `propagation = REQUIRED` (mövcud tranzaksiyaya qoşul, yoxdursa yarat), izolyasiya — DB standartı (PostgreSQL: READ COMMITTED), **rollback yalnız `RuntimeException` və `Error` üçün**.

Metod	Tranzaksiya	Yazılar	Rollback halları
`CampaignService.create`	RW	campaigns, history	`DuplicateCampaignException`, DB xətaları
`approve` / `deactivate`	RW	UPDATE campaigns, INSERT history	`ResourceNotFound`, `InvalidStatusTransition` (heç nə yazılmamış olur)
`getById` / `getAll` / `getHistory` / `getStatistics`	RO	—	—
`AuthService.register`	RW	app_users	`BusinessException` (mövcud username/email)
`AuthService.login`	RW	refresh_tokens	`BusinessException(UNAUTHORIZED)`
`refreshToken`	RW	UPDATE köhnə token, INSERT yeni	—
`forgotPassword`	RW	password_reset_tokens	`ResourceNotFound`, `BusinessException`

Metod	Tranzaksiya	Yazılar	Rollback halları
`resetPassword`	RW	app_users, reset token, bulk refresh_tokens	`BusinessException`
`logout`	RW	UPDATE refresh_tokens	`BusinessException`

`BusinessException` `RuntimeException`-dır (`BusinessException.java:9`), buna görə biznes xətası tranzaksiyanı geri qaytarır. Checked exception-lar standart olaraq rollback etmir — Spring-in ən çox unudulan qaydalarından biri.

`create` daxilində tutulan istisna. `saveAndFlush` `DataIntegrityViolationException` atdıqda repository-nin öz tranzaksiya proxy-si (`SimpleJpaRepository` metodları `@Transactional`-dır) mövcud tranzaksiyaya qoşulub və istisna onun sərhədindən keçdiyi üçün tranzaksiyanı **rollback-only** kimi işarələyir **NƏTİCƏ**. Kod istisnanı tutub yeni `DuplicateCampaignException` atır; xarici tranzaksiya da runtime istisna ilə bitdiyi üçün rollback olunur. Kod istisnanı tutub uğurlu nəticə qaytarsaydı, commit cəhdi `UnexpectedRollbackException` verərdi.

Tranzaksiya və asinxron yan təsirlər. `register`, `login` və digər auth metodları `@Async` audit metodunu tranzaksiya **daxilində** çağırır (məs. `AuthService.java:123-125`). Audit virtual thread-də dərhal başlaya bilər — commit-dən əvvəl. **NƏTİCƏ** Commit uğursuz olsa, log-service "User registered successfully" qeydini saxlayacaq, istifadəçi isə yaranmayacaq. Düzgün yanaşma: hadisəni `@TransactionalEventListener(phase = AFTER_COMMIT)` ilə commit-dən sonra nəşr etmək.

`open-in-view: false` (hər iki servis) — Spring Boot standart olaraq OSIV-i aktiv saxlayır və xəbərdarlıq verir. Söndürülməsi düzgündür: DB bağlantısı HTTP cavabı yazılana qədər tutulmur, lazy yükləmə səhvləri gizlənmir.

Bağlantı hovuzu (connection pool) və virtual thread-lər

NƏTİCƏ Spring Boot HikariCP-ni standart `maximumPoolSize = 10` ilə konfiqurasiya edir; layihə bunu dəyişmir. Virtual thread-lər minlərlə eyni vaxtlı sorğunu ucuz edir, lakin DB bağlantısı hələ də 10-dur — 11-ci tranzaksiya `connectionTimeout` (standart 30 s) qədər gözləyir. Xüsusilə təhlükəli kombinasiya: `create` / `approve` tranzaksiya daxilində `kafkaTemplate.send` çağırır; Kafka əlçatmaz olduqda send `max.block.ms`-ə (60 s) qədər bloklaya bilər (F10) və bu müddətdə **DB bağlantısı tutulur**. 10 belə sorğu hovuzu tükəndirir və bütün DB əməliyyatları (oxular daxil) dayanır. Bu, "bir asılılığın uğursuzluğunun başqasına yayılması" (cascading failure) nümunəsidir.

7.9 Sorğu növləri

Növ	Nümunə	Qeyd
Törəmə (derived)	`existsByCategoryAndTitleAndDescriptionAndStatusNot`, `findByUsername`, `findByTokenHash`, `findByCampaignIdOrderByChangedAtAsc`	Metod adından JPQL; startup-da yoxlanılır (səhv xassə adı → başlanğıc xətası)
JPQL `@Query` + proyeksiya	`countByStatus` (`CampaignRepository.java:20-26`)	Alias-lar interfeys getter-lərinə xəritələnir
`@Modifying` bulk	`revokeAllActiveTokensForUser`	Persistence context-dən yan keçir
CRUD	`findById`, `existsById`, `save`, `saveAndFlush`, `findAll(Pageable)`	`JpaRepository`

7.10 Sxem validasiyası və testlər arasındakı boşluq

İstehsal konfiqurasiyası: Liquibase sxemi yaradır, Hibernate `validate` edir. Testlərdə isə **FAKT** (`campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractTestContainerConfiguration.java:32-33`):

```
registry.add("spring.jpa.hibernate.ddl-auto", () → "create-drop");
registry.add("spring.liquibase.enabled", () → false);
```

Nəticə **RİSK** (F27): testlərdəki sxem Hibernate-in entity annotasiyalarından yaratdığı sxemdir. Partial unique indeks, `idx_*` indeksləri, `defaultValue`-lar, seed istifadəçilər testdə **yoxdur**. Changelog-dakı səhv (məsələn, YAML sintaksis xətası və ya entity ilə uyğunsuz tip) testlərdə aşkarlanmaz, yalnız deploy zamanı aşkarlanar. Tövsiyə: ən azı bir inteqrasiya testi Liquibase aktiv və `ddl-auto=validate` ilə işləsin; dublikat yaradılması DB constraint-i səviyyəsində sınansın.

7.11 Yeni layihədə tətbiq

1. Miqrasiya alətini (Liquibase və ya Flyway) birinci gündən istifadə edin, `ddl-auto=validate`.
2. Hər biznes invariantını mümkün qədər DB constraint-i ilə də qoruyun (unique, FK, CHECK).
3. Entity-lərdə `@Version`, LAZY əlaqələr, setter-siz metodlar; `save()` nəticəsini istifadə edin və ya `Persistable`.
4. FK sütunlarına indeks əlavə edin; sorğu planlarını `EXPLAIN ANALYZE` ilə yoxlayın.
5. Testlərdə real DB (Testcontainers) **və real miqrasiyalar**; Testcontainers image versiyasını istehsalla eyniləşdirin.
6. Yan təsirləri (hadisə, audit) commit-dən sonra və ya outbox ilə nəşr edin.

Liquibase və Flyway müqayisəsi:

	Liquibase	Flyway
Format	YAML/XML/JSON/SQL, DB-dən asılı olmayan abstraksiya	Əsasən SQL (və Java miqrasiyaları)
Rollback	Bir çox əməliyyat üçün avtomatik	Pulsuz versiyada "undo" yoxdur; irəli miqrasiya yanaşması
Öyrənmə əyrisi	Daha çox anlayış	Sadə
DB-yə xas xüsusiyyət	`sql` changeSet-i ilə	Təbii

7.12 Uğursuzluq ssenariləri və diaqnostika

Ssenari	Simptom	Diaqnostika	Həll
DB əlçatmaz startup-da	Liquibase bağlantı xətası, tətbiq başlamır	Log, `pg_isready`	DB-ni qaldırmaq; Compose-da `condition: service_healthy` (campaign-service-də yoxdur, F29)
Sxem entity ilə uyğun deyil	`SchemaManagementException` (validate)	Startup log	Yeni changeSet
Checksum uyğunsuzluğu	`ValidationFailedException`	`DATABASECHANGELOG.MD5SUM`	Redaktəni geri qaytarmaq, yeni changeSet
Unique pozuntu	`DataIntegrityViolationException`	Log, constraint adı	campaign-də 409; auth qeydiyyatında tutulmur → 500 (F18)

Ssenari	Simptom	Diaqnostika	Həll
Hovuz tükənməsi	`SQLTransientConnectionException: Connection is not available`	Hikari metrikləri (`hikaricp_connections_pending`)	Uzun tranzaksiyaları qısaltmaq, xarici I/O-nu tranzaksiyadan çıxarmaq
Lost update	Ziddiyyətli tarixçə	Tarixçə cədvəli	`@Version`

7.13 Müsahibə sualları

S1. `persist` və `merge` fərqi nədir və Spring Data hansını nə vaxt çağırır? Cavab: `persist` transient obyekti idarə olunan edir (eyni instans); `merge` obyektin vəziyyətini idarə olunan nüsxəyə kopyalayır və nüsxəni qaytarır, yoxdursa əvvəlcə SELECT edir. `SimpleJpaRepository.save` `isNew()` true olduqda persist, əks halda merge çağırır; `isNew()` id-nin null olmasına (və ya `@Version`-a) baxır.

S2. Dirty checking nədir? `resetPassword`-da niyə `save` yoxdur? Cavab: İdarə olunan entity-lərin vəziyyəti flush zamanı yüklənmə snapshot-u ilə müqayisə olunur və dəyişən sahələr üçün UPDATE yaradılır. `resetPassword`-dakı entity-lər tranzaksiya daxilində yüklənib, ona görə dəyişikliklər avtomatik yazılır.

S3. Niyə checked exception tranzaksiyanı rollback etmir? Cavab: Spring EJB konvensiyasını izləyir: checked exception "gözlənilən biznes nəticəsi" sayılır. Rollback üçün `@Transactional(rollbackFor = Exception.class)` lazımdır. Bu layihədə bütün biznes istisnaları runtime-dır.

S4. Partial unique index nədir və burada nə üçündür? Cavab: Yalnız `WHERE` şərtini ödəyən sətirlərə tətbiq olunan unikal indeks. Burada `status <> 'DUPLICATE'` sətirləri üçün `(category, title, description)` unikallığını təmin edir — köhnə dizaynda dublikat sətirlərin saxlanmasına icazə vermək üçün idi; indi isə paralel yaratma yarışını bağlayan son müdafiədir.

S5. Testlərdə `ddl-auto=create-drop` istifadə etməyin riski? Cavab: Test sxemi istehsal sxemindən fərqlidir: miqrasiyalar, partial indekslər, default-lar, trigger-lər yoxdur. DB constraint-lərinə əsaslanan davranış test edilmir, changelog xətaları gec aşkarlanır.

S6. Virtual thread-lər DB-yə nə qədər eyni vaxtlı sorğu göndərə bilər? Cavab: Hovuzun ölçüsü qədər (standart 10). Qalanları bağlantı gözləyir. Virtual thread-lər bloklanmanı ucuzlaşdırır, lakin DB tutumunu artırmır; tranzaksiya daxilində xarici I/O hovuzu tez tükəndirir.

7.14 Xülasə

- İki baza, hər servis öz sxeminə sahibdir; sxem Liquibase ilə, xəritələmə JPA ilə.
- campaign changelog-u partial unique indeks ilə dublikat yarışını bağlayır; indeks dizaynında artıq və çatışmayan indekslər var.
- auth changelog-u sxem təkamülünü (NOT NULL + default, data update, seed) göstərir; seed-lər istehsala da düşür.
- Dirty checking, merge/persist, bulk update, lazy yükləmə — layihədə real nümunələri var.
- Tranzaksiya daxilində xarici çağırışlar (Kafka, async audit) həm ardıcıllıq, həm hovuz tükənməsi riski yaradır.
- Növbəti fəsil validasiya və istisna emalını bütün servislər üzrə izləyir.

8. Validasiya və istisnaların idarə edilməsi

8.1 Problem: səhv məlumat sistemə harada daxil olur, harada dayandırılır?

Hər sorğu bir neçə "süzgəcdən" keçir: JSON formatı, tip çevrilməsi, sahə qaydaları, icazələr, biznes qaydaları, DB constraint-ləri. Hər süzgəcin öz xəta tipi və öz HTTP statusu var. Yaxşı dizaynda: (1) hər qayda **bir yerdə** və **ən uyğun qatda** yoxlanılır, (2) hər xəta klientə **proqnozlaşdırıla bilən formatda** qaytarılır, (3) heç bir daxili detal sızmır, (4) gözlənilməz xətalar **loglanır**. Bu fəsil repozitoriyadakı bütün validasiya qaydalarını və xəta yollarını inventarlaşdırır və bu dörd meyara görə qiymətləndirir.

8.2 Bean Validation: qısa nəzəriyyə

Jakarta Bean Validation 3 spesifikasiya, **Hibernate Validator** onun implementasiyasıdır (`spring-boot-starter-validation`). Constraint annotasiyaları (`@NotBlank`, `@Size`, `@Pattern`, `@Email`, `@NotNull`) sahələrə, metod parametrlərinə və ya sinfə qoyulur; `Validator` obyektləri yoxlayıb `ConstraintViolation` dəstini qaytarır.

Validasiyanı **kim və nə vaxt** işə salır — ən çox səhv başa düşülən hissə:

Tetikləyici	Mexanizm	Xəta tipi
`@Valid @RequestBody` (MVC)	`RequestResponseBodyMethodProcessor` deserializasiyadan sonra	`MethodArgumentNotValidException`
`@Validated` sinif + constraint-li metod parametrləri	`MethodValidationPostProcessor` proxy-si	`ConstraintViolationException`
Spring 6.1+ MVC daxili metod validasiyası	Controller parametrlərində birbaşa constraint olduqda	`HandlerMethodValidationException`
JPA `@PrePersist` (Hibernate Validator classpath-dədirsə)	Bean Validation inteqrasiyası	`ConstraintViolationException`

Annotasiya varsa, işləyir — yanlış təsəvvürdür. Constraint yalnız yuxarıdakı tetikləyicilərdən biri olduqda yoxlanılır. Layihədə **yeganə tetikləyici `@Valid @RequestBody`-dir** FAKT — `@Validated` heç yerdə istifadə olunmur (grep nəticəsi: `@Valid` yalnız `CampaignController.java:46` və `AuthController.java`-dakı 6 metodda).

Record-larda constraint-lər. Record komponentinə qoyulan annotasiya, `@Target`-inə uyğun olaraq, sahəyə, accessor metoduna və kanonik konstruktor parametrinə yayılır. NƏTİCƏ Hibernate Validator getter constraint-lərini yalnız JavaBeans adlandırmasına (`getX` / `isX`) uyğun metodlarda axtarır; record accessor-ları (`title()`) bu formada deyil, ona görə qaydalar sahə səviyyəsində bir dəfə yoxlanılır — dublikat pozuntu yaranmır.

8.3 Validasiya qaydalarının tam inventarı

campaign-service

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/dto/CreateCampaignRequest.java:9-21

```
public record CreateCampaignRequest(
        @NotBlank
        @Size(min = 10, max = 50)
        @Pattern(regexp = "^[\\p{L}\\d].*", message = "title must start with a Turkish letter, another le
tter, or a digit")
```

```
        String title,

        @NotBlank
        @Size(min = 20, max = 200)
        String description,

        @NotNull
        CampaignCategory category
) { }
```

#	Qayda	Harada təyin olunub	Kim icra edir	Mərhələ	Xəta / status	Təkrarlanırmı	Yerləşmə uyğundurmu
C1	`title` boş deyil	`CreateCampaignRequest.java:10`	Hibernate Validator	`@Valid`	400 `VALIDATION_ERROR`	DB `NOT NULL`	Bəli
C2	`title` 10–50 simvol	`:11`	HV	`@Valid`	400	DB `varchar(50)` (yalnız yuxarı həd)	Bəli
C3	`title` hərf və ya rəqəmlə başlayır	`:12`	HV (`Pattern`)	`@Valid`	400, xüsusi mesaj	Yox	Bəli
C4	`description` boş deyil, 20–200	`:15-16`	HV	`@Valid`	400	DB `varchar(200)`	Bəli
C5	`category` null deyil	`:19`	HV	`@Valid`	400	DB `NOT NULL`	Bəli
C6	`category` mövcud enum dəyəridir	Jackson enum deserializasiyası	Jackson 3	Deserializasiya	`HttpMessageNotReadableException` → **500** (F07)	Yox	Mərhələ düzgün, status səhv
C7	`id` UUID formatıdır	`@PathVariable UUID`	`ConversionService`	Arqument həlli	`MethodArgumentTypeMismatchException` → **500** (F07)	Yox	Mərhələ düzgün, status səhv
C8	Dublikat yoxdur	`CampaignService.java:140-150` + partial unique indeks	Servis + PostgreSQL	Servis / flush	409 `DUPLICATE_CAMPAIGN`	**Bəli (qəsdən)**: tətbiq + DB	Bəli — DB son zəmanətdir
C9	Kampaniya mövcuddur	`CampaignService.java:157-160`, `115-117`	Servis	Servis	404	Yox	Bəli
C10	Status keçidinə icazə var	`CampaignService.java:96-98`, `107-109`	Servis	Servis	409 `INVALID_STATUS_TRANSITION`	Yox	Servis — bəli; domen obyektində daha yaxşı olardı
C11	`DUPLICATE` statuslu yenilənmir	`CampaignService.java:171-175`	Servis	Servis	409 `DUPLICATE_CAMPAIGN`	Yox	Ölü qayda (status heç vaxt təyin olunmur)

#	Qayda	Harada təyin olunub	Kim icra edir	Mərhələ	Xəta / status	Təkrarlanırmı	Yerləşmə uyğundurmu
C12	`sort` xassəsi mövcuddur	Spring Data	Repository	Sorğu qurulması	`PropertyReferenceException` → **500**	Yox	Controller-də ağ siyahı daha yaxşıdır
C13	Rol: ADMIN / USER	`@PreAuthorize`	Method security	Controller proxy	403 `FORBIDDEN`	Gateway rolları yoxlamır	Bəli

NƏTİCƏ C3 — Java regex-də `.` sətir sonu simvollarını (`\n`) tutmur və Bean Validation `Matcher.matches()` (bütün sətrə uyğunluq) istifadə edir. Ona görə daxilində yeni sətir olan başlıq (`"Kampaniya\n2026 yay"`) `@Pattern`-dən keçmir. `\d` yalnız ASCII rəqəmləri tutur. `\p{L}` isə bütün Unicode hərflərini (ç, ğ, ı, İ, ö, ş, ü, ə) tutur — README-dəki "Turkish letter" tələbi ödənilir.

auth-service

Mənbə: `auth-service/src/main/java/com/casestudy/insurancecampaign/auth/model/dto/request/RegisterRequest.java:18-24`

```
public record RegisterRequest(
        @NotBlank @Size(min = 3, max = 100) String username,
        @NotBlank @Email @Size(max = 150) String email,
        @NotBlank @Size(min = 8, max = 100) String password,
        @NotBlank @Size(min = 2, max = 150) String fullName,
        @NotNull UserRole role
) { }
```

#	Qayda	Harada	Kim / mərhələ	Xəta	Qeyd
A1	`username` 3–100, boş deyil	`RegisterRequest.java:19`	HV / `@Valid`	400	DB `varchar(100) UNIQUE`
A2	`email` formatı, ≤150	`:20`	HV	400	DB `varchar(150)`, `uk_app_users_email`
A3	`password` 8–100	`:21`	HV	400	Mürəkkəblik qaydası yoxdur
A4	`fullName` 2–150	`:22`	HV	400	—
A5	`role` null deyil, enum	`:23`	HV + Jackson	400 / yanlış dəyər → 500	**RİSK** F01: `ROLE_ADMIN` qəbul olunur
A6	`username` / `email` unikaldır	`AuthService.java:108-113` + DB unique	Servis + DB	409 `BUSINESS_RULE_VIOLATION`; yarışda 500 (F18)	Normallaşdırmadan sonra yoxlanılır
A7	Login sahələri boş deyil	`LoginRequest.java:11-14`	HV	400	—
A8	İstifadəçi mövcud, aktiv, şifrə uyğundur	`AuthService.java:142-157`	Servis	401 `UNAUTHORIZED`	Mesajlar fərqlidir → enumerasiya (F03)
A9	Refresh token mövcud, ləğv olunmayıb, vaxtı keçməyib, istifadəçi aktivdir	`AuthService.java:177-185`	Servis	401	—

#	Qayda	Harada	Kim / mərhələ	Xəta	Qeyd
A10	Reset token mövcud, istifadə olunmayıb, vaxtı keçməyib	`AuthService.java:246-251`	Servis	401	—
A11	`newPassword` 8–100	`ResetPasswordRequest.java:12-15`	HV	400	Qeydiyyat qaydası ilə eynidir (təkrar, lakin uyğun)
A12	Forgot: istifadəçi mövcud və aktivdir	`AuthService.java:207-212`	Servis	404 / 401	404 enumerasiyanı asanlaşdırır (F03)

Normallaşdırma validasiyadan sonra baş verir. `username` və `email` servisdə `trim().toLowerCase(Locale.ROOT)` olunur (`StringUtils.java:36-38`, `AuthService.java:105-106`). `@Size(min = 3)` trim-dən əvvəl yoxlanılır: `" ab "` (5 simvol) keçir, normallaşdırmadan sonra `"ab"` (2 simvol) saxlanılır. **NƏTİCƏ** Kiçik uyğunsuzluq; həll — normallaşdırmanı deserializasiyada (custom deserializer) və ya kanonik konstruktorda etmək, ya da validasiyanı normallaşdırılmış dəyər üzərində təkrarlamaq.

`Locale.ROOT` ilə kiçik hərfə çevirmə vacib detaldır: türk lokalında `"I".toLowerCase()` nöqtəsiz `ı` verir; `ROOT` lokalı bu "Turkish I problem"-ini aradan qaldırır. Layihənin türk dilli kontekstini nəzərə alsaq, bu, düzgün seçimdir.

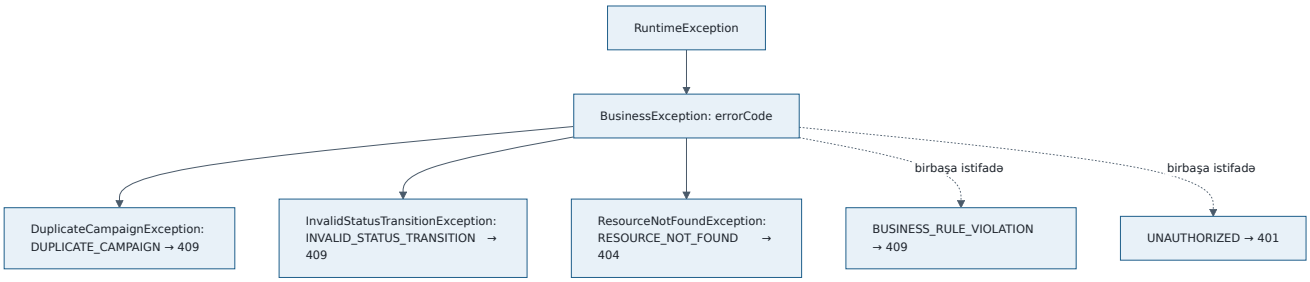
log-service

FAKT (`AuthAuditLogController.java:30-35`, `AuthAuditLogRequest.java:15-22`): audit sorğusunda **heç bir validasiya yoxdur** — nə `@Valid`, nə constraint. `null` sahələr birbaşa Elasticsearch-ə yazılır. Endpoint yalnız `ROLE_INTERNAL_SERVICE` üçün açıq olduğu üçün risk aşağıdır, lakin "daxili klientə etibar" müdafiə qatını azaldır (F39). `GET /slow-requests` `page` və `size` üçün mənfi/sıfır dəyərləri yoxlamır: `PageRequest.of(-1, 20)` `IllegalArgumentException` atır → catch-all → 500.

Təhlükəsizlik və DB qatı

Qayda	Harada	Nəticə
JWT imzası, issuer, vaxtı	`JwtTokenService.parseClaims` (`JwtTokenService.java:128-133`)	Filtrdə istisna (F08)
Token mövcudluğu (qorunan yollar)	`ServiceSecurityConfiguration`	401 JSON
NOT NULL, uzunluq, unique, FK	Liquibase changelog-ları	`DataIntegrityViolationException`

8.4 İstisna iyerarxiyası və xəta kodları



Layihənin istisna iyerarxiyası və HTTP xəritələnməsi

Mənbə: `common/src/main/java/com/casestudy/insurancecampaign/common/exception/GlobalExceptionHandler.java:36-52`

```
@ExceptionHandler(BusinessException.class)
public ResponseEntity<ApiError> handleBusinessException(BusinessException exception, HttpServletRequest r
```

```
equest) {
    HttpStatus status = switch (exception.getErrorCode()) {
        case RESOURCE_NOT_FOUND -> HttpStatus.NOT_FOUND;
        case DUPLICATE_CAMPAIGN, INVALID_STATUS_TRANSITION, BUSINESS_RULE_VIOLATION -> HttpStatus.CONFLIC
T;
        case UNAUTHORIZED -> HttpStatus.UNAUTHORIZED;
        default -> HttpStatus.BAD_REQUEST;
    };

    return ResponseEntity.status(status)
            .body(ApiError.of(
                    exception.getErrorCode().name(),
                    exception.getMessage(),
                    status.value(),
                    request.getRequestURI()
            ));
}
```

`ErrorCode` enum-unda 8 dəyər var (`ErrorCode.java:10-52`). Xəritə **FAKT**:

`ErrorCode`	HTTP	Kim atır
`RESOURCE_NOT_FOUND`	404	`ResourceNotFoundException` (campaign, auth forgot-password)
`DUPLICATE_CAMPAIGN`	409	`DuplicateCampaignException`
`INVALID_STATUS_TRANSITION`	409	`InvalidStatusTransitionException`
`BUSINESS_RULE_VIOLATION`	409	`AuthService.register` (username/email mövcuddur)
`UNAUTHORIZED`	401	`AuthService` (login, token yoxlamaları)
`VALIDATION_ERROR`	400 (`default`)	Birbaşa atılmır; `ApiError.validation(...)` sabit kodu
`FORBIDDEN`	400 (`default`)	Biznes istisnası kimi atılmır; yalnız security handler-lərində kod kimi
`INTERNAL_SERVER_ERROR`	400 (`default`)	Biznes istisnası kimi atılmır; catch-all handler-də kod

NƏTİCƏ `default → BAD_REQUEST` hələ istifadə olunmayan kodlar üçün səhv ehtiyat dəyərdir: gələcəkdə kimsə `new BusinessException(ErrorCode.FORBIDDEN, ...)` atsa, 403 deyil, 400 qaytarılacaq. Switch-dən `default` -u çıxarıb hər halı açıq yazmaq kompilyatoru yeni enum dəyərlərində xəbərdarlıq etməyə məcbur edərdi.

8.5 `GlobalExceptionHandler` : hansı istisna hansı handler-ə düşür

`@RestControllerAdvice` (`GlobalExceptionHandler.java:23`) bütün controller-lər üçün `@ExceptionHandler` metodlarını qeydiyyatdan keçirir. Spring `ExceptionHandlerExceptionResolver` atılan istisna üçün **ən yaxın** (iyerarxiyada ən spesifik) tipi olan handler-i seçir (`ExceptionDepthComparator`).

Handler	Tutur	Status	Mənbə
`handleBusinessException`	`BusinessException` və alt sinifləri	`ErrorCode` -a görə	`:36-52`
`handleAccessDenied`	`AccessDeniedException` (o cümlədən `@PreAuthorize` -ın `AuthorizationDeniedException` -ı)	403	`:64-73`
`handleAuthentication`	`AuthenticationException`	401	`:84-93`

Handler	Tutur	Status	Mənbə
`handleValidation`	`MethodArgumentNotValidException`	400 + sahə xətaları	`:105-119`
`handleUnexpected`	`Exception` — **qalan hər şey**	500	`:130-139`

Mənbə: `common/src/main/java/com/casestudy/insurancecampaign/common/exception/GlobalExceptionHandler.java:105-139`

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException exception, HttpServletRe
quest request) {
    Map<String, String> errors = new LinkedHashMap<>();

    exception.getBindingResult().getFieldErrors()
            .forEach(error -> errors.put(error.getField(), error.getDefaultMessage()));

    return ResponseEntity.badRequest()
            .body(ApiError.validation(
                    "Request validation failed",
                    HttpStatus.BAD_REQUEST.value(),
                    request.getRequestURI(),
                    errors
            ));
}
// ... (buraxılıb: Javadoc)
@ExceptionHandler(Exception.class)
public ResponseEntity<ApiError> handleUnexpected(Exception exception, HttpServletRequest request) {
    return ResponseEntity.internalServerError()
            .body(ApiError.of(
                    ErrorCode.INTERNAL_SERVER_ERROR.name(),
                    exception.getMessage(),
                    HttpStatus.INTERNAL_SERVER_ERROR.value(),
                    request.getRequestURI()
            ));
}
```

Detallar:

- `validationErrors` `Map<String,String>`-dir: bir sahədə bir neçə pozuntu olarsa (`title` həm qısa, həm `Pattern`-ə uyğun deyil), `put` sonuncunu saxlayır — klient yalnız bir mesaj görür. `Map<String, List<String>>` daha məlumatlıdır. Sinif səviyyəli (global) xətalar (`getGlobalErrors()`) ümumiyyətlə daxil edilmir.
- Mesajlar Hibernate Validator-un standart ingilis mesajlarıdır (`"size must be between 10 and 50"`), yalnız `@Pattern` xüsusi mesaja malikdir.

Catch-all handler-in yan təsirləri (F07, F38)

NƏTİCƏ Spring MVC-də istisna resolver-lərinin sırası: `ExceptionHandlerExceptionResolver` (sizin `@ExceptionHandler`-lər) → `ResponseStatusExceptionResolver` → `DefaultHandlerExceptionResolver`. `@ExceptionHandler(Exception.class)` olduqda birinci resolver **hər** istisnanı tutur və Spring-in standart 4xx xəritələnməsi heç vaxt işə düşmür:

İstisna	Standart Spring statusu	Layihədə	Tipik səbəb
`HttpMessageNotReadableException`	400	**500**	Səhv JSON, yanlış enum (`"category":"XYZ"`)
`MethodArgumentTypeMismatchException`	400	**500**	`/campaigns/not-a-uuid`
`MissingServletRequestParameterException`	400	**500**	Məcburi parametr yoxdur
`HttpRequestMethodNotSupportedException`	405	**500**	`DELETE /api/v1/campaigns/{id}`

İstisna	Standart Spring statusu	Layihədə	Tipik səbəb
`HttpMediaTypeNotSupportedException`	415	**500**	`Content-Type: text/plain`
`NoResourceFoundException`	404	**500**	Mövcud olmayan yol (autentifikasiyalı istifadəçi üçün)
`PropertyReferenceException`	— (500)	500	`sort=foo` — əslində 400 olmalıdır
`DataIntegrityViolationException`	— (500)	500	auth qeydiyyat yarışı (F18)
`IllegalArgumentException`	— (500)	500	`page=-1` (log-service)

Əlavə problemlər:

1. **Məlumat sızması** — `exception.getMessage()` klientə qaytarılır. Jackson xətası JSON strukturunu və sinif adlarını, `DataIntegrityViolationException` SQL mətnini və constraint adını ehtiva edə bilər.
2. **Loglama yoxdur** — sinifdə logger yoxdur. İstisna `@ExceptionHandler` tərəfindən emal olunduqda `DispatcherServlet` onu error səviyyəsində loglamır. Nəticə: istehsalda 500 cavabları **stack trace-siz** olur; yeganə iz klientin gördüyü mesaj və trace-dəki xəta statusudur (F38).

İllüstrativ nümunə (repozitoriyada yoxdur) — düzgün variant:

```
@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler { // Spring-in 4xx xəritəsini saxlayır
    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(DataIntegrityViolationException.class)
    ResponseEntity<ApiError> conflict(DataIntegrityViolationException ex, HttpServletRequest req) {
        log.warn("Constraint violation at {}", req.getRequestURI(), ex);
        return ResponseEntity.status(409).body(ApiError.of("CONFLICT", "Resource state conflict", 409, req.getRequestURI()));
    }

    @ExceptionHandler(Exception.class)
    ResponseEntity<ApiError> unexpected(Exception ex, HttpServletRequest req) {
        log.error("Unhandled exception at {}", req.getRequestURI(), ex);           // stack trace log-da
        return ResponseEntity.internalServerError()
                .body(ApiError.of("INTERNAL_SERVER_ERROR", "Unexpected error", 500, req.getRequestURI())); // detal yoxdur
    }
    // handleMethodArgumentNotValid(...) override edilərək ApiError formatı saxlanılır
}
```

8.6 Təhlükəsizlik qatında xətalar

Security xətaları MVC-dən **əvvəl**, filtrlərdə baş verir və `@RestControllerAdvice` onları görmür. Buna görə `ServiceSecurityConfiguration` öz handler-lərini təyin edir FAKT (`ServiceSecurityConfiguration.java:80-95`, `121-134`):

Mənbə: `security/src/main/java/com/casestudy/insurancecampaign/security/ServiceSecurityConfiguration.java:80-95`

```
.exceptionHandling(exception → exception
        .authenticationEntryPoint((request, response, authException) → writeError(
                request,
                response,
                HttpStatus.UNAUTHORIZED,
                ErrorCode.UNAUTHORIZED.name(),
                "Authentication is required"
        ))
```

```
        .accessDeniedHandler((request, response, accessDeniedException) -> writeError(
                request,
                response,
                HttpStatus.FORBIDDEN,
                ErrorCode.FORBIDDEN.name(),
                "Access denied"
        ))
)
```

İki fərqli 403 yolu var və hər ikisi eyni formatı verir:

1. URL səviyyəli qayda filtrdə rədd edirsə → `accessDeniedHandler` (bu layihədə URL qaydaları yalnız `authenticated()` -dir, ona görə praktikada 401 olur).
2. `@PreAuthorize` metod səviyyəsində rədd edirsə → istisna `DispatcherServlet` daxilində atılır → `GlobalExceptionHandler.handleAccessDenied` → 403 `FORBIDDEN`. Testlər bunu yoxlayır (məs. `CampaignControllerTest.java:97-120`).

Etibarsız JWT (F08)

`JwtAuthenticationFilter` `parseClaims` -in atdığı `JwtException` -ı tutmur (`JwtAuthenticationFilter.java:70-83`). **NƏTİCƏ** İstisna `FilterChainProxy` -dən çıxır; `ExceptionTranslationFilter` yalnız `AuthenticationException` və `AccessDeniedException` -ı tanıdığı üçün (və JWT filtri ondan əvvəl yerləşdiyi üçün) emal etmir. İstisna servlet konteynerinə çatır, Spring Boot onu `/error` -a ERROR dispatch kimi yönləndirir. Bu dispatch də Spring Security-dən keçir; JWT filtri (OncePerRequestFilter) ERROR dispatch-də standart olaraq işləmir, kontekst anonimdir və `/error` `permitAll` siyahısında deyil — nəticə çox güman ki, entry point-in 401 cavabıdır, lakin bu, framework-un bir neçə qatlı davranışına bağlıdır və repozitoriyada test edilməyib. MockMvc testlərində (ERROR dispatch yoxdur) istisna testə qədər yayılardı. Düzgün dizayn: filtrdə `JwtException` -ı tutmaq, kontekst qurmadan zənciri davam etdirmək və ya `AuthenticationEntryPoint` -ə açıq ötürmək; ən yaxşısı Spring Security-nin `oauth2ResourceServer().jwt()` mexanizmindən istifadə etmək (fəsil 10).

8.7 Gateway, uzaq servislər və asinxron emalda xətalar

Mənbə	Xəta	Klientin gördüyü
Gateway JWT filtri	Token yoxdur / etibarsızdır	**401, boş gövdə** (`GatewayJwtAuthenticationFilter.java:82-94`) — `ApiError` formatı deyil
Gateway rate limit	Limit aşılıb	429 + `ApiError` -a bənzər JSON + `Retry-After: 5`
Gateway, servis instansı yoxdur	`lb://` həll olunmur	**NƏTİCƏ** 503 Spring WebFlux-un standart xəta formatında
Dashboard → campaign gRPC	`StatusRuntimeException` (`UNAVAILABLE`, `UNKNOWN`, `DEADLINE_EXCEEDED`)	Catch-all → 500, mesaj məs. `"UNAVAILABLE: io exception"` (F19)
campaign gRPC endpoint daxilində istisna	`ResourceNotFoundException`, `IllegalArgumentException` (səhv UUID)	gRPC `UNKNOWN` (net.devh-də `@GrpcAdvice` yoxdur) → dashboard 500
Auth → log-service	HTTP xətası, timeout, CB açıq, bulkhead dolu	Tutulur və `WARN` loglanır (`AuthAuditService.java:155-179`); klient təsirlənmir
Kafka producer	Göndərmə uğursuz	`KafkaTemplate` -in standart `LoggingProducerListener` -i **NƏTİCƏ** xətanı loglayır; HTTP cavabı uğurlu qalır

Mənbə	Xəta	Klientin gördüyü
Kafka consumer	Listener istisnası	`DefaultErrorHandler` : 3 təkrar (1 s), sonra DLQ (fəsil 11)
Kafka deserializasiya	Səhv JSON	DLQ-ya getmir; consumer eyni offseti təkrar oxuyur (F11)
`@Async` metod	İstisna	`publishAudit` hər şeyi tutur; tutulmasaydı `AsyncUncaughtExceptionHandler` (standart: log)

Xəta formatlarının sistem üzrə vahidliyi. Servislər `ApiError`, gateway 429 üçün `ApiError`-a bənzər format, gateway 401 üçün boş gövdə, WebFlux standart xətaları üçün isə fərqli JSON qaytarır. NƏTİCƏ Klient üçün dörd fərqli forma var. Təkmilləşdirmə: gateway-də `ErrorWebExceptionHandler` ilə vahid format, JWT filtrində JSON gövdə.

8.8 Persistence xətaları və rollback

- Biznes istisnaları `RuntimeException` olduğu üçün tranzaksiya rollback olur (fəsil 7).
- `DataIntegrityViolationException` campaign-də 409-a çevrilir, auth-da 500 qalır (F18).
- DB bağlantı xətaları (`CannotCreateTransactionException` , `DataAccessResourceFailureException`) catch-all ilə 500.
- Optimistik kilid olmadığı üçün `ObjectOptimisticLockingFailureException` heç vaxt yaranmır — paralel yeniləmələr sakitcə bir-birini əvəzləyir (F37).

8.9 Uğurlu və uğursuz sorğunun izlənməsi

Uğurlu: `POST /api/v1/campaigns {"title":"TSS Kampanya 2026","description":"Tamamlayıcı sağlık sigortası için özel kampanya açıklaması","category":"TSS"}` , admin token:

1. Security: JWT etibarlıdır, `ROLE_ADMIN` authority-si qurulur.
2. Jackson 3: JSON → `CreateCampaignRequest` ; `"TSS"` → `CampaignCategory.TSS` .
3. `@Valid` : başlıq 17 simvol, hərflə başlayır; təsvir 54 simvol; kateqoriya var — keçir.
4. `@PreAuthorize("hasRole('ADMIN')")` keçir.
5. `CampaignService.create` → 201 + `ApiResponse` .

Uğursuz — qısa başlıq (`"title":"TSS"`): 1–2. Eyni.

3. `@Size(min = 10)` pozulur → `MethodArgumentNotValidException` .
4. `GlobalExceptionHandler.handleValidation` → 400, `{"code":"VALIDATION_ERROR","validationErrors": {"title":"size must be between 10 and 50"}}` .
5. `@PreAuthorize` heç vaxt yoxlanılmır, servis çağırılmır.

Uğursuz — yanlış kateqoriya (`"category":"HEALTH"`):

2. Jackson `HttpMessageNotReadableException` (enum dəyəri tapılmadı) atır.
3. `handleUnexpected` → **500** `INTERNAL_SERVER_ERROR` , mesajda Jackson-un daxili izahı; heç nə loglanmır.

8.10 Yeni layihədə tətbiq

Xüsusi constraint (illüstrativ nümunə, repozitoriyada yoxdur) — trim-dən sonra uzunluğu yoxlayan annotasiya:

```java
@Target({FIELD, PARAMETER, RECORD_COMPONENT})
@Retention(RUNTIME)
@Constraint(validatedBy = TrimmedSizeValidator.class)
public @interface TrimmedSize {
    int min() default 0;
    int max() default Integer.MAX_VALUE;
    String message() default "length after trimming must be between {min} and {max}";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}

public class TrimmedSizeValidator implements ConstraintValidator<TrimmedSize, String> {
    private int min, max;
    @Override public void initialize(TrimmedSize a) { min = a.min(); max = a.max(); }
    @Override public boolean isValid(String v, ConstraintValidatorContext ctx) {
        if (v == null) return true;                // null-u @NotNull/@NotBlank yoxlasın
        int len = v.trim().length();
        return len ⩾ min && len ⩽ max;
    }
}
```

Validasiya qrupları — eyni DTO müxtəlif əməliyyatlarda fərqli qaydalara tabe olduqda (`OnCreate`, `OnUpdate` interfeysləri, `@Validated(OnCreate.class)`). Bu layihədə hər əməliyyatın öz DTO-su olduğu üçün qruplara ehtiyac yoxdur — bu, qruplardan daha sadə və tövsiyə olunan yanaşmadır.

Qaydaların yerləşdirilməsi prinsipi:

- Format və sintaksis (uzunluq, regex, null) → DTO, Bean Validation.
- Biznes qaydası (status keçidi, unikallıq məntiqi) → domen/servis.
- Atomik invariant (unikallıq, referensial bütövlük) → **həm də** DB constraint-i.
- İcazə → security (URL və ya metod səviyyəsi).

8.11 Müsahibə sualları

S1. `@Valid` və `@Validated` fərqi nədir? Cavab: `@Valid` Jakarta standartıdır, kaskad (iç-içə obyekt) validasiyasını və MVC arqument validasiyasını işə salır. `@Validated` Spring annotasiyasıdır: validasiya qruplarını dəstəkləyir və sinif üzərində qoyulduqda metod səviyyəli validasiya üçün proxy yaradır.

S2. Niyə bu layihədə yanlış enum dəyəri 500 qaytarır? Cavab: Jackson deserializasiyası `HttpMessageNotReadableException` atır. Spring bunu standart olaraq 400-ə çevirərdi, lakin `@ExceptionHandler(Exception.class)` hər şeyi tutduğu üçün `DefaultHandlerExceptionResolver`-ə növbə çatmır.

S3. Security filtrlərində yaranan xətalar niyə `@RestControllerAdvice`-ə çatmır? Cavab: Advice `DispatcherServlet` daxilində işləyir; filtrlər isə ondan əvvəl, servlet konteyneri səviyyəsində işləyir. Filtr xətaları `AuthenticationEntryPoint` / `AccessDeniedHandler` və ya `HandlerExceptionResolver`-i filtrə inject etməklə emal olunmalıdır.

S4. Validasiya nə vaxt servis qatında olmalıdır? Cavab: Qayda başqa dataya (DB-dəki mövcud vəziyyət) və ya bir neçə sahənin biznes mənasına bağlıdırsa — məsələn, status keçidi, unikallıq. Format qaydaları DTO-da, kontekst tələb edən qaydalar servisdə/domendə.

S5. 500 cavabında `exception.getMessage()` qaytarmağın riski? Cavab: Daxili detalların (SQL, sinif adları, fayl yolları, bəzən şəxsi məlumat) sızması və hücumçu üçün kəşfiyyat imkanı. Klientə ümumi mesaj və korrelyasiya id-si (traceId), detallar isə yalnız log-a.

S6. Eyni qayda həm tətbiqdə, həm DB-də olarsa, bu təkrar (duplication) pisdirmi? Cavab: Yarış şəraitində yalnız DB atomik zəmanət verir; tətbiq yoxlaması isə yaxşı xəta mesajı və erkən rədd üçündür. Bu "müdafiə dərinliyi"dir, mənasız təkrar deyil — şərt odur ki, DB pozuntusu da düzgün xətaya çevrilsin.

8.12 Xülasə

- Validasiyanın yeganə tetikləyicisi `@Valid @RequestBody`-dir; `@Validated` istifadə olunmur.
- campaign və auth DTO-larında qaydalar DB sütun ölçüləri ilə uyğundur; log-service audit sorğusu validasiyasızdır.
- Biznes istisnaları `ErrorCode` ilə HTTP-yə xəritələnir; catch-all handler Spring-in standart 4xx-lərini 500-ə çevirir, daxili mesajları sızdırır və loglamır.
- Security xətaları ayrıca handler-lərlə eyni formatda qaytarılır; etibarsız JWT yolu test edilməyib.
- Növbəti fəsil auth-service-in istifadəçi və token həyat dövrünü izah edir.

9. Auth Service — istifadəçi və token həyat dövrü

9.1 Məsuliyyət və sərhəd

auth-service sistemin **identifikasiya mərkəzidir** (identity provider rolu): istifadəçiləri saxlayır, şifrələri yoxlayır, **access token** (JWT) və **refresh token** buraxır, şifrə bərpasını idarə edir. README-yə görə servis "fully self-contained"-dir: ayrıca user-service yoxdur **FAKT** (`AuthServiceApplication.java`, repozitoriyada başqa istifadəçi servisi yoxdur).

Digər servislər auth-service-ə **runtime-da müraciət etmir**: token-i public key ilə özləri yoxlayırlar (fəsil 10). Bu, mühüm arxitektura xüsusiyyətidir — auth-service düşsə belə, artıq verilmiş access token-lər vaxtı bitənə qədər işləməyə davam edir.

Endpoint	Metod	Servis metodu	Tranzaksiya	Audit hadisəsi
`/api/v1/auth/register`	POST	`register`	RW	`REGISTER`
`/api/v1/auth/login`	POST	`login`	RW	`LOGIN_SUCCESS` / `LOGIN_FAILURE`
`/api/v1/auth/refresh-token`	POST	`refreshToken`	RW	`REFRESH_TOKEN`
`/api/v1/auth/forgot-password`	POST	`forgotPassword`	RW	`FORGOT_PASSWORD`
`/api/v1/auth/reset-password`	POST	`resetPassword`	RW	`RESET_PASSWORD`
`/api/v1/auth/logout`	POST	`logout`	RW	`LOGOUT`

Bütün endpoint-lər `permitAll`-dur (`ServiceSecurityConfiguration.java:96-106`) və gateway-də JWT yoxlamasından azaddır (`GatewayJwtAuthenticationFilter.java:39-50`), lakin gateway rate limit-inə tabedir.

9.2 Şifrələrin saxlanması: `DelegatingPasswordEncoder`

Mənbə: `auth-service/src/main/java/com/casestudy/insurancecampaign/auth/config/AuthConfiguration.java:30-33`

```java
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}
```

Nəzəriyyə. Şifrələr heç vaxt açıq və ya geri qaytarıla bilən formada saxlanmamalıdır. Bunun üçün **yavaş, duzlu (salted) hash** funksiyaları (BCrypt, Argon2, scrypt, PBKDF2) istifadə olunur: yavaşlıq brute-force-u bahalaşdırır, təsadüfi duz eyni şifrələrin eyni hash verməsinin və rainbow table-ların qarşısını alır.

`DelegatingPasswordEncoder` hash-in əvvəlinə alqoritm id-si yazır: `{bcrypt}$2a$10$...`. `matches()` prefiksi oxuyub uyğun encoder-i seçir. Standart yeni hash alqoritmi **bcrypt**-dir (cost 10). Bu dizayn alqoritmi gələcəkdə dəyişməyə imkan verir: köhnə hash-lər köhnə alqoritmlə yoxlanılır, yeniləri yeni alqoritmlə yaradılır.

Layihədə **FAKT**:

- Seed istifadəçilərin şifrəsi `{noop}<redaktə>` formatında saxlanılır (changelog) — `NoOpPasswordEncoder`, açıq mətn.
- Qeydiyyat `passwordEncoder.encode(...)` istifadə edir (`AuthService.java:120`) → `{bcrypt}...`.

- Reset `passwordEncoder.encode(...)` istifadə edir (`AuthService.java:254`).

NƏTİCƏ `PasswordEncoder.upgradeEncoding()` istifadə olunmur. Spring Security-nin standart `DaoAuthenticationProvider`-i login zamanı köhnə formatlı hash-i avtomatik yeniləyə bilər (`UserDetailsPasswordService`); bu layihədə autentifikasiya əl ilə yazıldığı üçün `{noop}` seed şifrələri heç vaxt avtomatik hash-ə çevrilmir.

NƏTİCƏ BCrypt cost 10 təxminən onlarla millisaniyə CPU vaxtı aparır. Virtual thread-lər bunu sürətləndirmir; çox sayda paralel login CPU-nu doldura bilər. Gateway rate limit-i (5 saniyədə 1 sorğu / IP) bu baxımdan da qoruyucudur (fəsil 15).

9.3 Qeydiyyat (`register`)

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:103-126

```java
@Transactional
public RegisteredUserResponse register(RegisterRequest request) {
    String username = StringUtils.normalize(request.username());
    String email = StringUtils.normalize(request.email());

    if (appUserRepository.existsByUsername(username)) {
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "Username already exists");
    }
    if (appUserRepository.existsByEmail(email)) {
        throw new BusinessException(ErrorCode.BUSINESS_RULE_VIOLATION, "Email already exists");
    }

    AppUserEntity user = authMapper.toEntity(
            UUID.randomUUID(),
            username,
            email,
            request.fullName().trim(),
            passwordEncoder.encode(request.password()),
            request.role().name()
    );
    AppUserEntity savedUser = appUserRepository.save(user);
    authAuditService.publishRegisterAuditAsync(username, email, request.role().name());
    return authMapper.toRegisteredUserResponse(savedUser);
}
```

Addımlar: normallaşdırma (trim + kiçik hərf, `Locale.ROOT`) → unikallıq yoxlamaları (409) → bcrypt hash → entity (rol enum adı sətir kimi) → `save` (merge, F14) → asinxron audit → cavab DTO-su (şifrə daxil deyil).

Təhlükəsizlik defekti: özünü admin təyin etmə (F01)

FAKT `RegisterRequest.role` `UserRole` tipindədir və `UserRole` `ROLE_USER` və `ROLE_ADMIN` dəyərlərinə malikdir (`UserRole.java:10-22`). Endpoint anonimdir. README də bunu xüsusiyyət kimi təqdim edir: "Register `ROLE_USER` or `ROLE_ADMIN`". **RİSK** Nəticə: istənilən şəxs `{"role":"ROLE_ADMIN"}` ilə qeydiyyatdan keçib dərhal kampaniya yarada, təsdiqləyə, statistika və yavaş sorğu jurnallarını oxuya bilər. Bu, **imtiyaz eskalasiyası** (privilege escalation) zəifliyidir; case study üçün test rahatlığı ola bilər, istehsalda isə qəbuledilməzdir.

Həll variantları:

1. İctimai qeydiyyat həmişə `ROLE_USER` versin; rol sahəsini DTO-dan çıxarmaq.
2. Admin yaratma ayrıca, `ROLE_ADMIN` tələb edən endpoint-də (`POST /api/v1/admin/users`).
3. İlk admin-i miqrasiya/konfiqurasiya ilə və ya bootstrap əmri ilə yaratmaq.

Paralel qeydiyyat yarışı (F18)

İki sorğu eyni username ilə eyni anda gəlsə, hər ikisi `existsByUsername = false` görə bilər. İkinci INSERT `app_users.username` unikallığını pozur. **NƏTİCƏ** `save()` merge olduğu üçün INSERT commit zamanı flush olunur; xəta `DataIntegrityViolationException` kimi tranzaksiya commit-ində yaranır, servis onu tutmur, `GlobalExceptionHandler` catch-all ilə 500 və SQL mesajı qaytarır. campaign-service-in `saveAndFlush` + `catch` nümunəsi burada da tətbiq oluna bilərdi.

9.4 Login (`login`) və token buraxılması

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:140-161

```
@Transactional
public TokenResponse login(LoginRequest request) {
    Optional<AppUserEntity> optionalUser = findByUsernameOrEmail(request.usernameOrEmail());
    if (optionalUser.isEmpty()) {
        authAuditService.publishLoginFailureAuditAsync(request.usernameOrEmail(), "Invalid username/email
or password");
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "Invalid username/email or password");
    }

    AppUserEntity user = optionalUser.get();

    if (!user.isActive()) {
        authAuditService.publishLoginFailureAuditAsync(request.usernameOrEmail(), "User is not active");
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "User is not active");
    }
    if (!passwordEncoder.matches(request.password(), user.getPassword())) {
        authAuditService.publishLoginFailureAuditAsync(request.usernameOrEmail(), "Invalid username/email
or password");
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "Invalid username/email or password");
    }

    authAuditService.publishLoginSuccessAuditAsync(request.usernameOrEmail());
    return createTokenResponse(user);
}
```

`findByUsernameOrEmail` dəyərdə `@` olub-olmamasına görə email və ya username üzrə axtarır (`AuthService.java:328-334`).

`createTokenResponse`: iki fərqli token

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:292-315

```
private TokenResponse createTokenResponse(AppUserEntity user) {
    Instant issuedAt = Instant.now();
    Instant accessTokenExpiresAt = issuedAt.plusSeconds(jwtProperties.expirationMinutes() * 60);
    Instant refreshTokenExpiresAt = issuedAt.plusSeconds(refreshTokenExpirationDays * 24 * 60 * 60);

    String accessToken = jwtTokenService.generateToken(user.getUsername(), user.getRoles());
    String refreshToken = TokenHashUtils.randomToken();
    refreshTokenRepository.save(new RefreshTokenEntity(
            UUID.randomUUID(),
            user,
            TokenHashUtils.sha256(refreshToken),
            refreshTokenExpiresAt
    ));

    return new TokenResponse(
            accessToken,
            refreshToken,
            "Bearer",
            issuedAt,
            accessTokenExpiresAt,
            refreshTokenExpiresAt,
```

```
            user.getRoles()
    );
}
```

	Access token	Refresh token
Format	JWT (RS256 imzalı), özünü təsvir edən	Təsadüfi 48 bayt, Base64URL (`TokenHashUtils.java:39-43`)
Ömür	120 dəqiqə (`app.jwt.expiration-minutes`)	7 gün (`app.auth.refresh-token-expiration-days`)
Harada yoxlanılır	Hər servisdə, lokal olaraq (public key)	Yalnız auth-service-də, DB üzrə
Server tərəfdə saxlanılırmı	Xeyr (stateless)	Bəli, yalnız SHA-256 hash-i
Ləğv edilə bilərmi	Xeyr (vaxtı bitənə qədər)	Bəli (`revoked_at`)

Kiçik incəlik NƏTİCƏ: `accessTokenExpiresAt` burada ayrıca `Instant.now()` ilə hesablanır, JWT-nin daxilindəki `exp` isə `JwtTokenService.generateToken`-də ayrıca `Instant.now()` ilə (`JwtTokenService.java:95-103`). Ölçülər eynidir, lakin bir neçə millisaniyəlik fərq ola bilər — klient üçün əhəmiyyətsizdir.

Niyə refresh token SHA-256 ilə, bcrypt ilə yox?

NƏTİCƏ Refresh token 384 bit təsadüfilik daşıyır — onu brute-force etmək mümkün deyil, ona görə yavaş hash lazım deyil. Sürətli deterministik hash iki üstünlük verir: (1) token-i **hash üzrə indeksdə axtarmaq** olur (`findByTokenHash`) — bcrypt-in duzu buna imkan verməzdi; (2) DB sızsa belə, hash-lərdən istifadə oluna bilən token əldə edilmir. Şifrələr isə aşağı entropiyalıdır və yavaş hash tələb edir. Bu fərq müsahibədə tez-tez soruşulur.

İstifadəçi enumerasiyası və timing (F03)

RİSK

- "User is not active" mesajı yalnız mövcud istifadəçi üçün qaytarılır → hesabın mövcudluğu sızır.
- İstifadəçi tapılmadıqda bcrypt `matches` çağırılmır → cavab nəzərəçarpacaq dərəcədə **sürətli** olur; tapıldıqda bcrypt hesablanır. Vaxt fərqi ilə mövcud istifadəçi adlarını aşkar etmək mümkündür.
- Həll: bütün hallarda eyni mesaj; istifadəçi tapılmadıqda da saxta hash ilə `matches` çağırmaq (sabit vaxt).

9.5 Refresh token rotasiyası (`refreshToken`)

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:175-190

```
@Transactional
public TokenResponse refreshToken(RefreshTokenRequest request) {
    RefreshTokenEntity refreshToken = refreshTokenRepository.findByTokenHash(TokenHashUtils.sha256(reques
t.refreshToken()))
            .orElseThrow(() -> new BusinessException(ErrorCode.UNAUTHORIZED, "Invalid refresh token"));

    if (refreshToken.isRevoked() || refreshToken.isExpired()) {
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "Refresh token is expired or revoked");
    }
    if (!refreshToken.getUser().isActive()) {
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "User is not active");
    }

    refreshToken.revoke();
    authAuditService.publishRefreshTokenAuditAsync(refreshToken.getUser().getUsername());
    return createTokenResponse(refreshToken.getUser());
}
```

Rotasiya — hər istifadədə köhnə refresh token ləğv olunur və yenisi verilir. Faydası: oğurlanmış refresh token-in istifadə pəncərəsi daralır.

RİSK İki çatışmazlıq:

1. **Təkrar istifadə aşkarlanması yoxdur** (reuse detection). OAuth 2.0 Security BCP tövsiyəsinə görə, artıq ləğv olunmuş refresh token yenidən təqdim olunarsa, bu oğurluq əlamətidir və həmin "token ailəsinin" bütün token-ləri ləğv edilməlidir. Burada sadəcə 401 qaytarılır, hücumçunun aldığı yeni token işləməyə davam edir.
2. **Yarış (F17)** — `RefreshTokenEntity`-də `@Version` yoxdur. Eyni token ilə iki paralel sorğu hər ikisi `isRevoked() == false` görə bilər və hər ikisi yeni token cütü alar. Optimistik kilid və ya şərti UPDATE (`UPDATE refresh_tokens SET revoked_at = now() WHERE id = ? AND revoked_at IS NULL` + təsirlənən sətir sayı) həll edir.

9.6 Şifrə bərpası (`forgotPassword`, `resetPassword`)

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:205-230

```
@Transactional
public ForgotPasswordResponse forgotPassword(ForgotPasswordRequest request) {
    AppUserEntity user = findByUsernameOrEmail(request.usernameOrEmail())
            .orElseThrow(() → new ResourceNotFoundException("User not found"));

    if (!user.isActive()) {
        throw new BusinessException(ErrorCode.UNAUTHORIZED, "User is not active");
    }

    String resetToken = TokenHashUtils.randomToken();
    Instant expiresAt = Instant.now().plusSeconds(passwordResetTokenExpirationMinutes * 60);
    PasswordResetTokenEntity entity = new PasswordResetTokenEntity(
            UUID.randomUUID(),
            user,
            TokenHashUtils.sha256(resetToken),
            expiresAt
    );
    passwordResetTokenRepository.save(entity);
    authAuditService.publishForgotPasswordAuditAsync(request.usernameOrEmail());

    return new ForgotPasswordResponse(
            "Password reset token was generated. In a production system this token should be sent by emai
l.",
            resetToken,
            expiresAt
    );
}
```

Kritik defekt: reset token-in cavabda qaytarılması (F02)

FAKT Endpoint anonimdir və cavab xam `resetToken`-i ehtiva edir. Kodun öz mesajı bunu etiraf edir: "In a production system this token should be sent by email." RİSK Nəticə: hədəfin istifadəçi adını və ya emailini bilən hər kəs `forgot-password` → cavabdakı token → `reset-password` ilə **istənilən hesabın şifrəsini dəyişə bilər** (hesabın ələ keçirilməsi, account takeover). Seed `admin` istifadəçisi də daxil olmaqla. Demo üçün qəbuledilə bilən, istehsalda mütləq aradan qaldırılmalı davranışdır. Düzgün axın:

1. `forgot-password` istifadəçinin mövcudluğundan asılı olmayaraq həmişə eyni cavabı qaytarır (`202 Accepted`, "Əgər hesab varsa, email göndərildi").
2. Token yalnız istifadəçinin email-inə (yan kanal) göndərilir.
3. Token tək istifadəlik, qısa ömürlü (burada 15 dəq — düzgündür), hash-lənmiş saxlanılır (düzgündür).
4. Yeni token yarandıqda köhnə aktiv reset token-ləri ləğv edilir (burada edilmir: eyni istifadəçi üçün bir neçə aktiv reset token ola bilər).

`resetPassword` (`AuthService.java:244-260`) özü düzgün qurulub: token tək istifadəlik (`markUsed`), vaxt yoxlanılır, şifrə bcrypt ilə yenilənir, istifadəçinin **bütün aktiv refresh token-ləri bulk UPDATE ilə ləğv edilir**. NƏTİCƏ Lakin artıq verilmiş access token-lər (120 dəq) işləməyə davam edir (F35) — şifrəni hücumçudan qorumaq üçün dəyişdirən istifadəçi bu müddətdə qorunmur. Həll: qısa ömürlü access token (5–15 dəq), token-də `pwd_changed_at` / versiya claim-i və ya revocation siyahısı.

9.7 Logout

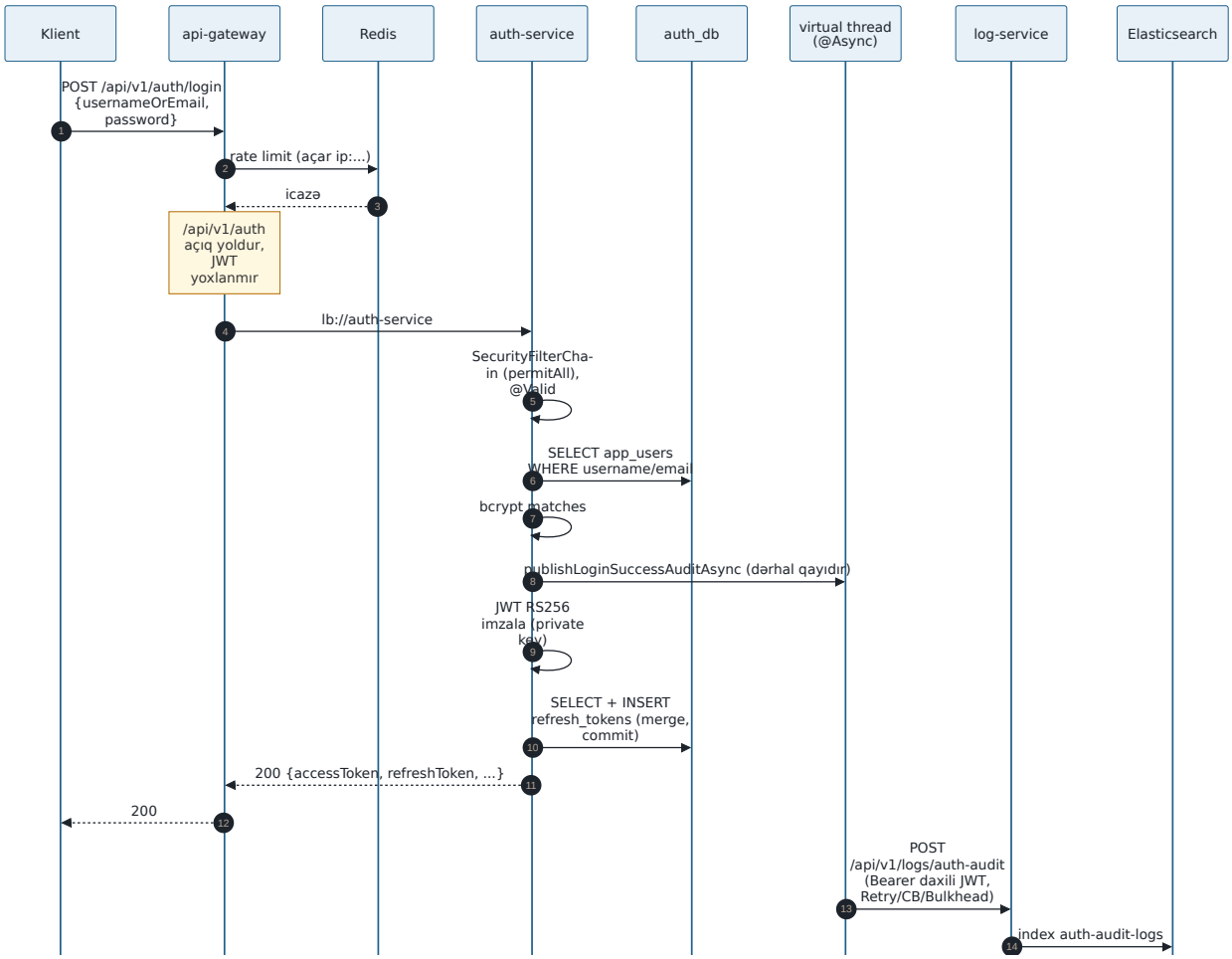
Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthService.java:269-279

```
@Transactional
public LogoutResponse logout(LogoutRequest request) {
    RefreshTokenEntity refreshToken = refreshTokenRepository.findByTokenHash(TokenHashUtils.sha256(reques
t.refreshToken()))
            .orElseThrow(() → new BusinessException(ErrorCode.UNAUTHORIZED, "Invalid refresh token"));

    if (!refreshToken.isRevoked()) {
        refreshToken.revoke();
    }
    authAuditService.publishLogoutAuditAsync(refreshToken.getUser().getUsername());
    return new LogoutResponse("Logout completed successfully");
}
```

- Artıq ləğv olunmuş token ilə təkrar logout uğurludur — **idempotent** davranış (test: `AuthServiceTest.shouldLogoutWhenRefreshTokenIsAlreadyRevoked`, `AuthServiceTest.java:689-719`).
- Access token ləğv olunmur (stateless JWT-nin təbii məhdudiyyəti).
- Logout autentifikasiya tələb etmir: refresh token-ə sahib olan hər kəs onu ləğv edə bilər — refresh token özü sahiblik sübutudur, bu qəbuledilir.

9.8 Login axını — ardıcıllıq diaqramı



Login axını (gateway → auth-service → auth_db, asinxron audit log-service-ə)

9.9 Audit inteqrasiyası

`AuthAuditService` hər hadisə üçün `@Async` metod təqdim edir (`AuthAuditService.java:51-135`) və hamısı `publishAudit`-ə (`:137-180`) yönəlir. `publishAudit` `LogServiceHttpClient.sendAuthAuditLog`-u çağırır və bütün istisnaları (circuit breaker açıq, bulkhead dolu, digər) tutub `WARN` loglayır. Mexanizmin detalları (HTTP Interface client, daxili JWT, Resilience4j) 14-cü fəsildədir.

Müşahidələr [FAKT/NƏTİCƏ]:

- `publishLoginAuditAsync` "backward compatibility" üçün saxlanılıb, `AuthService`-də istifadə olunmur (`AuthAuditService.java:71-74`).
- Audit mesajlarında email və username açıq yazılır (`"User registered successfully. email=..."`) və həm auth-service log faylına, həm Elasticsearch-ə düşür. **RİSK** Şəxsi məlumatın (PII) loglanması məlumatların qorunması qaydaları (GDPR və s.) baxımından məhdudlaşdırılmalı, saxlanma müddəti təyin edilməlidir.
- Uğursuz login cəhdi ilə istifadəçinin daxil etdiyi xam `usernameOrEmail` auditə yazılır — səhvən şifrəni username sahəsinə yazan istifadəçinin şifrəsi log-a düşə bilər. Bu, praktikada rast gəlinən bir riskdir.

9.10 Konfiqurasiya

FAKT (`auth-service/src/main/resources/application.yml`):

Xassə	Dəyər	İstifadə yeri
`app.jwt.issuer`	`allianz-campaign-api`	JWT `iss` , bütün servislərdə yoxlanılır
`app.jwt.expiration-minutes`	120	Access token ömrü
`app.jwt.private-key-location`	`${JWT_PRIVATE_KEY_LOCATION:classpath:keys/private.pem}`	Yalnız auth-service-də — imzalama
`app.jwt.public-key-location`	`${JWT_PUBLIC_KEY_LOCATION:classpath:keys/public.pem}`	RSA rejimini aktivləşdirir
`app.auth.refresh-token-expiration-days`	7	`AuthService` konstruktoru `@Value`
`app.auth.password-reset-token-expiration-minutes`	15	Eyni
`spring.http.serviceclient.log-service.base-url`	`${LOG_SERVICE_URL:http://localhost:8084}`	`@ImportHttpServices(group = "log-service")`
`resilience4j.*.logServiceHttpClient`	CB, Retry, Bulkhead	Fəsil 14

RİSK (F22) Private key `auth-service/src/main/resources/keys/private.pem` repozitoriyada və jar daxilindədir; Compose da `classpath:keys/private.pem` istifadə edir. Repozitoriyaya çıxışı olan hər kəs etibarlı `ROLE_ADMIN` token-i imzalaya bilər. README bunu "demo keys" kimi qeyd edir və istehsal üçün `file:/run/secrets/...` təklif edir — düzgün yanaşma: açar secret manager-dən (Vault, Kubernetes Secret) gəlməli, repozitoriyada heç vaxt olmamalıdır.

9.11 Uğursuzluq ssenariləri

Ssenari	Nəticə	Qeyd
auth_db düşüb	Bütün auth əməliyyatları 500	Digər servislərin token yoxlaması işləyir
log-service düşüb	Audit itir, WARN log; login işləyir	Retry 3 × 500 ms virtual thread-də — cavabı gecikdirmir
Private key yoxdur / səhvdir	Tətbiq başlamır (`IllegalStateException`)	Fail-fast (`JwtTokenService.java:59-73`)
Public/private key cütü uyğun deyil	Token-lər imzalanır, digər servislər imzanı rədd edir	Startup-da aşkarlanmır — smoke test lazımdır
Saat sürüşməsi (clock skew) servislər arasında	`exp` / `iat` yoxlamaları səhv nəticə verə bilər	JJWT `clockSkewSeconds` təyin edilməyib
Paralel refresh	İki token cütü (F17)	—

9.12 Yeni layihədə tətbiq və alternativlər

Öz auth-service-ini yazmaq layihədəki kimi tam nəzarət verir, lakin məsuliyyət də böyükdür: şifrə siyasəti, hesab kilidlənməsi (brute-force), MFA, email doğrulaması, token ailələri, açar rotasiyası (JWKS), audit, PII. Alternativlər:

Seçim	Nə vaxt
Keycloak / Auth0 / Okta / Entra ID	İstehsal, SSO, MFA, sosial login lazımdırsa; servislər yalnız resource server olur
Spring Authorization Server	OAuth 2.1/OIDC uyğun öz serverinizi Spring ilə qurmaq
Öz servisi (bu layihə)	Öyrənmə, sadə daxili sistemlər, xüsusi tələblər

Öz servisinizi yazırsınızsa, minimum yoxlama siyahısı: ictimai qeydiyyat yalnız minimal rol; reset token yalnız email-ə; vahid xəta mesajları və sabit vaxt; uğursuz cəhdlərin sayılması və kilid; qısa access token (5–15 dəq); refresh token rotasiyası + reuse detection; JWKS endpoint ilə açar rotasiyası; private key secret store-da.

9.13 Müsahibə sualları

S1. Access token və refresh token-i niyə ayırırıq? Cavab: Access token tez-tez göndərilir və hər servisdə lokal yoxlanılır — stateless və qısa ömürlü olmalıdır, çünki ləğv etmək çətindir. Refresh token nadir göndərilir, yalnız auth-service-ə gedir, server tərəfdə saxlanılır və ləğv edilə bilər. Bu ayrılıq performansı (lokal yoxlama) və nəzarəti (ləğv imkanı) birləşdirir.

S2. Refresh token-ləri niyə hash-lənmiş saxlayırsınız və niyə bcrypt deyil? Cavab: DB sızsa token-lər istifadə oluna bilməsin deyə. Token yüksək entropiyalıdır, brute-force qeyri-mümkündür, ona görə sürətli SHA-256 kifayətdir və indeks üzrə axtarışa imkan verir; bcrypt-in təsadüfi duzu axtarışı mümkünsüz edərdi.

S3. Bu auth-service-də ən ciddi iki problem nədir? Cavab: (1) Anonim qeydiyyatda `ROLE_ADMIN` seçmək mümkündür — imtiyaz eskalasiyası. (2) `forgot-password` reset token-i cavabda qaytarır — istifadəçi adını bilən hər kəs hesabı ələ keçirə bilər. Hər ikisi demo rahatlığı üçündür, istehsalda kritik zəiflikdir.

S4. Refresh token reuse detection nədir? Cavab: Rotasiyada hər token bir "ailəyə" aiddir. Artıq istifadə olunmuş (ləğv olunmuş) token yenidən gəlsə, bu, token-in oğurlandığını göstərir: server bütün ailəni ləğv edir və istifadəçini yenidən login etməyə məcbur edir.

S5. Şifrə dəyişdikdən sonra köhnə access token-lər niyə hələ işləyir və nə etmək olar? Cavab: JWT stateless-dir, resource server-lər yalnız imza və `exp`-i yoxlayır. Variantlar: qısa ömür; token-də istifadəçi "security version" claim-i və servislərin onu yoxlaması (state tələb edir); revocation siyahısı/cache; introspection.

S6. Login zamanı istifadəçi tapılmadıqda bcrypt-i niyə yenə də çağırmaq lazımdır? Cavab: Cavab vaxtının mövcud və mövcud olmayan istifadəçi üçün eyni olması üçün; əks halda timing hücumu ilə mövcud hesablar aşkarlanır.

9.14 Xülasə

- auth-service istifadəçiləri, şifrələri (bcrypt), access (JWT RS256) və refresh (təsadüfi, SHA-256 hash-li) token-ləri idarə edir; digər servislər ondan runtime-da asılı deyil.
- Rotasiya, tək istifadəlik reset token, logout idempotentliyi, PII-dən başqa hash-lənmiş saxlanma — düzgün qurulub.
- Kritik təhlükəsizlik problemləri: özünü admin qeydiyyatı (F01), reset token-in cavabda qaytarılması (F02), enumerasiya (F03), repozitoriyadakı private key (F22); etibarlılıq: refresh yarışı (F17), qeydiyyat yarışı (F18).
- Növbəti fəsil token-in bütün sistem üzrə necə yoxlandığını (Spring Security, gateway) izah edir.

10. Spring Security və JWT — bütün sistem üzrə təhlükəsizlik

10.1 Problem: mikroservislərdə "bu sorğunu kim göndərib?"

Monolitdə istifadəçi sessiyası server yaddaşında saxlanılır. Mikroservislərdə sorğu bir neçə prosesdən keçir və hər biri müstəqil qərar verməlidir: istifadəçi kimdir, hansı rolları var, token etibarlıdırmı. Sessiyanı paylaşmaq (sticky session, mərkəzi sessiya bazası) miqyaslanmanı çətinləşdirir. **Stateless token** yanaşması bunu həll edir: identifikasiya mərkəzi (auth-service) imzalı token verir, hər servis imzanı **lokal olaraq** yoxlayır.

10.2 JWT nəzəriyyəsi

JWT (JSON Web Token, RFC 7519) üç Base64URL hissədən ibarətdir: `header.payload.signature`.

```
header  = {"alg":"RS256"}                                    (JJWT əlavə sahələr də yaza bilər)
payload = {"iss":"allianz-campaign-api","sub":"admin","roles":["ROLE_ADMIN"],"iat":1767000000,"exp":17670
07200}
signature = RSASSA-PKCS1-v1_5-SHA256(base64url(header) + "." + base64url(payload), privateKey)
```

- Payload **şifrələnmir**, yalnız imzalanır — həssas məlumat qoyulmamalıdır. Layihədə yalnız username və rollar var.
- **Standart claim-lər**: `iss` (kim buraxıb), `sub` (kimin üçün), `iat` (nə vaxt), `exp` (nə vaxt bitir), `aud` (kim üçün nəzərdə tutulub — layihədə yoxdur), `jti` (unikal id — yoxdur).
- **Xüsusi claim**: `roles` — siyahı.

İmza alqoritmləri:

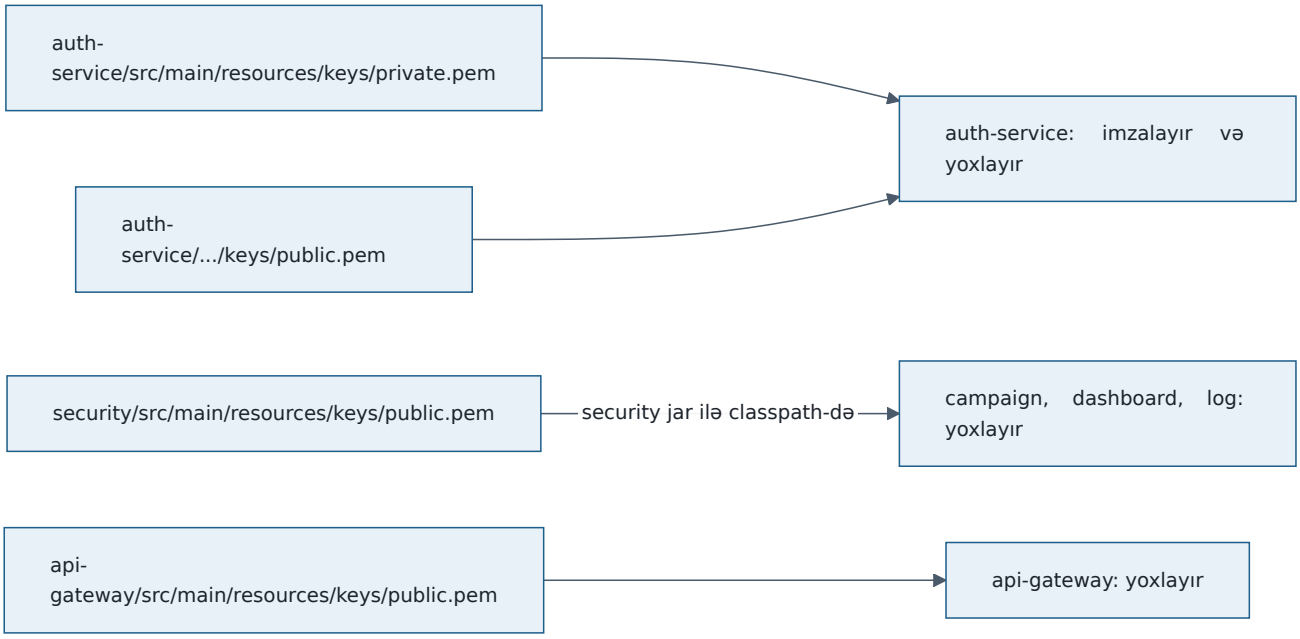
	HMAC (HS256)	RSA (RS256)
Açar	Bir paylaşılan sirr	Private (imzalama) + public (yoxlama)
Kim token yarada bilər	Sirri bilən **hər** servis	Yalnız private key sahibi
Mikroservislər üçün	Bütün servislər sirri bilməlidir — bir servisin sızması hamısını təhlükəyə atır	Servislər yalnız public key bilir — sızma token yaratmağa imkan vermir

Layihə hər iki rejimi dəstəkləyir, lakin RSA istifadə edir (README: "This replaces the old shared JWT_SECRET approach"). FAKT `JwtProperties.rsaEnabled()` hər hansı açar xassəsi təyin olunubsa RSA rejimini seçir (`JwtProperties.java:56-58`); bütün servislərin `application.yml`-ində `app.jwt.public-key-location` var.

Doğrulama addımları (JJWT `parseSignedClaims`): (1) formatı ayır; (2) header-dəki alqoritmin açar tipinə uyğunluğunu yoxla; (3) imzanı public key ilə yoxla; (4) `exp`-i yoxla (`ExpiredJwtException`), `nbf` varsa onu da; (5) tələb olunan claim-ləri yoxla — burada `requireIssuer` (`IncorrectClaimException` / `MissingClaimException`).

NƏTİCƏ **Alqoritm qarışdırma hücumu** (alg confusion: RS256 token-i public key-i HMAC sirri kimi istifadə edən HS256 token ilə əvəzləmək) JJWT 0.12-də `verifyWith(PublicKey)` istifadə edildikdə mümkün deyil: açar tipi gözlənilən alqoritm ailəsini müəyyən edir. `alg: none` da qəbul olunmur, çünki `parseSignedClaims` imza tələb edir.

10.3 Açarların idarə edilməsi



RSA açarlarının paylanması (repozitoriyadakı faktiki fayllar)

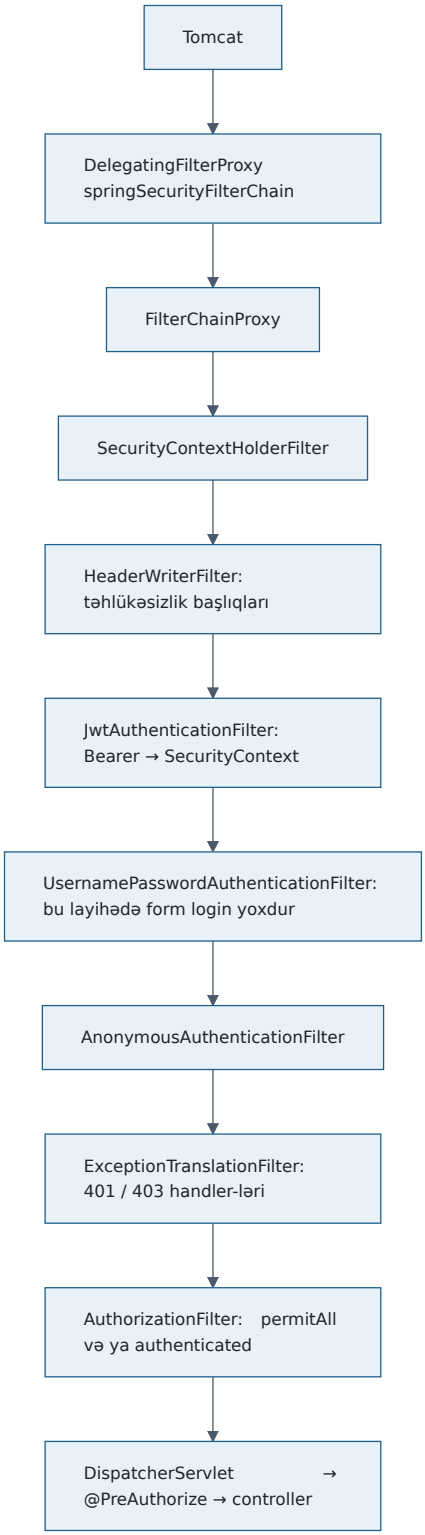
FAKT Public key-in üç nüsxəsi var. auth-service `security` modulundan da asılı olduğu üçün classpath-də iki `keys/public.pem` olur; **NƏTİCƏ** `classpath:keys/public.pem` birinci tapılanı qaytarır (adətən tətbiqin öz resursları asılılıq jar-larından əvvəl gəlir). Nüsxələr eyni olduqca problem yoxdur; açar rotasiyasında hamısının eyni anda yenilənməsi lazımdır — bu, əl ilə idarə olunan kövrək prosesdir.

`PemKeyUtils` açarı belə oxuyur **FAKT** (`PemKeyUtils.java:108-124`):

1. Inline PEM (`app.jwt.public-key`) varsa — onu istifadə et (mühit dəyişəni ilə ötürmək üçün rahatdır);
2. Yoxdursa `location`-u Spring `DefaultResourceLoader` ilə aç (`classpath:`, `file:`, URL);
3. Resurs tapılmasa — dəyəri adi fayl yolu kimi oxu;
4. PEM başlıqlarını və boşluqları sil, Base64 decode et (`:138-144`);
5. Public key — `X509EncodedKeySpec`; private key — **PKCS#8** (`-----BEGIN PRIVATE KEY-----`). PKCS#1 formatı (`BEGIN RSA PRIVATE KEY`) dəstəklənmir, xəta mesajı bunu açıq deyir (`:83-91`).

Daha yaxşı alternativ — JWKS. Auth server public açarlarını `/.well-known/jwks.json` endpoint-ində `kid` (key id) ilə nəşr edir; resource server-lər açarı oradan yükləyib cache edir. Açar rotasiyası: yeni açar əlavə olunur, köhnə token-lər köhnə `kid` ilə yoxlanmağa davam edir. Bu layihədə `kid` və JWKS yoxdur.

10.4 Servlet servislərində Spring Security arxitekturası



Servlet servisində Spring Security filtr zənciri (layihə konfiqurasiyasına görə sadələşdirilmiş)

Mənbə: security/src/main/java/com/casestudy/insurancecampaign/security/ServiceSecurityConfiguration.java:75-109

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(exception -> exception
```

```
                .authenticationEntryPoint((request, response, authException) -> writeError(
                        request,
                        response,
                        HttpStatus.UNAUTHORIZED,
                        ErrorCode.UNAUTHORIZED.name(),
                        "Authentication is required"
                ))
                .accessDeniedHandler((request, response, accessDeniedException) -> writeError(
                        request,
                        response,
                        HttpStatus.FORBIDDEN,
                        ErrorCode.FORBIDDEN.name(),
                        "Access denied"
                ))
        )
        .authorizeHttpRequests(auth -> auth
                .requestMatchers(
                        "/api/v1/auth/**",
                        "/actuator/**",
                        "/v3/api-docs/**",
                        "/swagger-ui/**",
                        "/swagger-ui.html",
                        "/grpc.health.v1.Health/**"
                ).permitAll()
                .anyRequest().authenticated()
        )
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

Hər qərarın izahı:

Konfiqurasiya	Niyə	Qeyd
`csrf.disable()`	CSRF hücumu brauzerin cookie-ləri avtomatik göndərməsinə əsaslanır. Token `Authorization` başlığında göndərildikdə brauzer onu avtomatik əlavə etmir	Token cookie-də saxlanılsaydı, CSRF qorunması lazım olardı
`STATELESS`	`HttpSession` yaradılmır; hər sorğu öz token-i ilə autentifikasiya olunur	Miqyaslanma, sticky session yoxdur
`authenticationEntryPoint`	Autentifikasiya olmadıqda `ApiError` JSON (401)	Standart cavab `WWW-Authenticate` başlığı ilə boş gövdə olardı
`accessDeniedHandler`	Filtr səviyyəsində 403 üçün JSON	`@PreAuthorize` 403-ü isə `GlobalExceptionHandler` verir
`permitAll` siyahısı	Auth endpoint-ləri, actuator, OpenAPI, Swagger	`/grpc.health.v1.Health/**` mənasızdır: gRPC ayrı Netty server-də (9090) işləyir, HTTP filtrindən keçmir
`anyRequest().authenticated()`	Qalan hər şey token tələb edir	Rol yoxlaması metod səviyyəsindədir
`addFilterBefore(jwt, UsernamePasswordAuthenticationFilter)`	JWT filtri anonim autentifikasiya və avtorizasiyadan əvvəl işləməlidir	—
`@EnableMethodSecurity`	`@PreAuthorize` aktivdir	`ServiceSecurityConfiguration.java:35-38`

RİSK (F40) `/actuator/**` `permitAll`-dur və `health,info,prometheus,metrics` açıqdır. Compose servis portlarını host-a açdığı üçün metriklər autentifikasiyasız oxunur. Metriklər daxili endpoint adlarını, versiyaları, yük nümunələrini göstərir. Tövsiyə: actuator-u ayrı idarə portuna (`management.server.port`) çıxarmaq və şəbəkə səviyyəsində məhdudlaşdırmaq və ya yalnız `health`-i açıq saxlamaq.

`JwtAuthenticationFilter`

Mənbə: security/src/main/java/com/casestudy/insurancecampaign/security/JwtAuthenticationFilter.java:67-84

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain fil
terChain)
        throws ServletException, IOException {
    String authorization = request.getHeader("Authorization");
    if (authorization ≠ null && authorization.startsWith("Bearer ")) {
        String token = authorization.substring(7);
        Claims claims = jwtTokenService.parseClaims(token);
        @SuppressWarnings("unchecked")
        List<String> roles = claims.get("roles", List.class);
        Collection<SimpleGrantedAuthority> authorities = roles.stream()
                .map(SimpleGrantedAuthority::new)
                .toList();
        UsernamePasswordAuthenticationToken authentication = new UsernamePasswordAuthenticationToken(
                claims.getSubject(), null, authorities);
        SecurityContextHolder.getContext().setAuthentication(authentication);
    }
    filterChain.doFilter(request, response);
}
```

- Başlıq yoxdursa filtr heç nə etmir; sorğu anonim davam edir və `AuthorizationFilter` qorunan yol üçün 401 verir.
- `roles` claim-indən `SimpleGrantedAuthority` yaradılır. Token-də `ROLE_ADMIN` yazıldığı üçün `hasRole('ADMIN')` (avtomatik `ROLE_` prefiksi əlavə edir) işləyir.
- Üç arqumentli `UsernamePasswordAuthenticationToken` konstruktoru obyekti **autentifikasiya olunmuş** kimi işarələyir. Principal sadə `String` username-dir.

Problemlər və incəliklər:

1. **İstisnalar tutulmur** (F08) — fəsil 8.6-ya baxın.
2. **`roles` claim-i yoxdursa** — `claims.get("roles", List.class)` `null` qaytarır və `roles.stream()` `NullPointerException` atır. Yalnız auth-service token verdiyi və hər token-də `roles` olduğu üçün praktikada baş vermir, lakin müdafiə kodu yoxdur.
3. **İkiqat qeydiyyat** NƏTİCƏ — filtr `@Component`-dir və `OncePerRequestFilter`-dən törəyir. Spring Boot kontekstdəki hər `Filter` bean-ini **servlet konteynerinə də** avtomatik qeydiyyatdan keçirir. Beləliklə filtr həm security zəncirində, həm də adi servlet filtri kimi mövcuddur. `OncePerRequestFilter` sorğu atributu ilə ikinci icranı ötürür, ona görə funksional problem yoxdur, lakin bu, gizli asılılıqdır. Təmiz variant: filtri `@Bean` kimi yaradıb `FilterRegistrationBean.setEnabled(false)` ilə servlet qeydiyyatını söndürmək və ya `@Component`-i çıxarıb yalnız security konfiqurasiyasında `new` ilə yaratmaq.
4. `"Bearer "` prefiksi böyük-kiçik hərfə həssasdır (RFC 6750 sxem adını case-insensitive sayır).

10.5 Metod səviyyəli avtorizasiya

`@EnableMethodSecurity` `@PreAuthorize`-ı aktivləşdirir. İfadələr SpEL-dir; `hasRole('ADMIN')` → `ROLE_ADMIN` authority-si. Yoxlama controller bean-inin proxy-sində, metod çağırılmazdan əvvəl edilir; rədd edildikdə Spring Security 6.3+ `AuthorizationDeniedException` (`AccessDeniedException` alt sinfi) atır.

Rol matrisi FAKT:

Endpoint	ADMIN	USER	INTERNAL_SERVICE	Anonim	Mənbə
`POST /api/v1/campaigns`	✓	—	—	—	`CampaignController.java:43-44`

Endpoint	ADMIN	USER	INTERNAL_SERVICE	Anonim	Mənbə
`GET /api/v1/campaigns`, `/{id}`	✓	✓	—	—	`:51-52`, `:58-59`
`PATCH /{id}/approve`, `/{id}/deactivate`	✓	—	—	—	`:67-68`, `:74-75`
`GET /{id}/history`, `/statistics`	✓	—	—	—	`:81-82`, `:88-89`
`GET /api/v1/dashboard/**`	✓	—	—	—	`DashboardController.java:40-41`, `:53-54`
`GET /api/v1/logs/slow-requests`	✓	—	—	—	`SlowRequestLogController.java:36-37`
`POST /api/v1/logs/auth-audit`	—	—	✓	—	`AuthAuditLogController.java:30-31`
`POST /api/v1/auth/**`	✓	✓	✓	✓	`ServiceSecurityConfiguration.java:97-104`
gRPC `CampaignGrpcService/*` (9090)	açıq	açıq	açıq	**açıq**	F04

NƏTİCƏ Avtorizasiya **son servisdə** edilir — gateway yalnız token-in etibarlılığını yoxlayır, rolu yox. Bu, düzgün prinsipdir: servis öz qaydalarının sahibidir və gateway-dən yan keçən sorğu da (birbaşa port) eyni qaydalara tabedir. README də bunu vurğulayır: "the final authorization decision is made inside campaign-service".

10.6 Gateway-də JWT yoxlaması (WebFlux)

Mənbə: `api-gateway/src/main/java/com/casestudy/insurancecampaign/gateway/filter/GatewayJwtAuthenticationFilter.java:75-95`

```
@Override
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    String path = exchange.getRequest().getURI().getPath();
    if (OPEN_PATHS.stream().anyMatch(path::startsWith)) {
        return chain.filter(exchange);
    }

    String authorization = exchange.getRequest().getHeaders().getFirst(HttpHeaders.AUTHORIZATION);
    if (authorization == null || !authorization.startsWith("Bearer ")) {
        exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
        return exchange.getResponse().setComplete();
    }

    try {
        jwtTokenService.parseClaims(authorization.substring(7));
        return chain.filter(exchange);
    } catch (RuntimeException exception) {
        exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
        return exchange.getResponse().setComplete();
    }
}
```

- `GlobalFilter` — bütün marşrutlara tətbiq olunur. Sıra `100` (`:108-111`): marşrutun `RequestRateLimiter` filtri (default filter, sıra 1) ondan əvvəl işləyir, ona görə token-siz sorğular da əvvəlcə rate limit-ə düşür.
- Açıq yollar prefiks uyğunluğu ilə yoxlanılır (`OPEN_PATHS`, `:39-50`). `startsWith` ilə `/api/v1/authXYZ` də açıq sayılar — praktiki riski yoxdur, çünki belə marşrut yoxdur.
- Token yoxlanılır, lakin claim-lər **istifadə olunmur** və aşağı axına əlavə başlıq (məs. `X-User`) ötürülmür; `Authorization` başlığı olduğu kimi servisə gedir və servis onu yenidən yoxlayır (**müdafiə dərinliyi**). Bədəli: hər sorğuda imza iki dəfə (gateway + servis), rate limit açarı üçün isə üçüncü dəfə yoxlanılır (`GatewayRateLimiterConfiguration.java:46-58`). RS256 yoxlaması ucuzdur, lakin bunu bilmək lazımdır.
- Uğursuzluqda **boş gövdəli 401** — servislərin `ApiError` formatından fərqlidir.
- **NƏTİCƏ** Reaktiv kodda `parseClaims` CPU əməliyyatıdır və Netty event-loop thread-ində icra olunur; qısa olduğu üçün qəbuledilir. Bloklayan I/O isə event-loop-da qadağandır.

Gateway Spring Security istifadə etmir (`spring-boot-starter-security` asılılığı yoxdur — `api-gateway/pom.xml`), ona görə təhlükəsizlik başlıqları (`X-Content-Type-Options` və s.) gateway cavablarına əlavə olunmur; servislərdən gələn cavablarda isə Spring Security-nin standart başlıqları var.

10.7 Servislərarası autentifikasiya

Kanal	Mexanizm	Qiymətləndirmə
Gateway → servislər	İstifadəçinin token-i ötürülür	Düzgün: servis istifadəçi kontekstində qərar verir
auth-service → log-service	Hər sorğu üçün yeni daxili JWT: `sub=auth-service`, `roles=[ROLE_INTERNAL_SERVICE]`, private key ilə imzalı	Yaxşı nümunə: log-service yalnız bu rolu qəbul edir
dashboard → campaign (gRPC)	**Autentifikasiya yoxdur**	**RİSK** F04

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/config/LogServiceHttpClientConfiguration.java:32-48

```
@Bean
RestClientHttpServiceGroupConfigurer logServiceHttpServiceGroupConfigurer(
        JwtTokenService jwtTokenService
) {
    return groups -> groups.forEachClient((group, clientBuilder) -> {
        if (!LOG_SERVICE_GROUP.equals(group.name())) {
            return;
        }

        clientBuilder.defaultRequest(request -> request.headers(headers ->
                headers.setBearerAuth(jwtTokenService.generateToken(
                        SERVICE_SUBJECT,
                        List.of(INTERNAL_SERVICE_ROLE)
                ))
        ));
    });
}
```

NƏTİCƏ `defaultRequest` callback-i hər sorğu qurulanda çağırılır, buna görə hər audit çağırışı yeni token (RSA imza əməliyyatı) yaradır. Token 120 dəqiqə etibarlı olduğu üçün cache-lənə bilərdi; audit yükü aşağı olduğundan bu, kiçik optimallaşdırma imkanıdır. Daha vacib məqam: `ROLE_INTERNAL_SERVICE` token-ini yalnız private key sahibi (auth-service) yarada bilər — `UserRole` enum-unda bu rol yoxdur, ona görə istifadəçi qeydiyyatı ilə əldə edilə bilməz.

gRPC kanalı (F04). **FAKT** campaign-service gRPC server-i 9090-da işləyir, `ServiceSecurityConfiguration` yalnız HTTP-ni qoruyur. net.devh-in gRPC security auto-configuration-u (`GrpcServerSecurityAutoConfiguration`) `@ConditionalOnBean(AuthenticationManager.class)` ilə qorunur

(kitabxana mənbəsi), layihədə isə `AuthenticationManager` bean-i yoxdur. dashboard-service stub-a token əlavə etmir. Compose 9090-ı host-a açır (`docker-compose.yml:182-184`). **RİSK** Nəticə: istənilən şəbəkə klienti (`grpcurl -plaintext localhost:9090 ...`) bütün statistikanı və istənilən kampaniyanın tarixçəsini autentifikasiyasız oxuya bilər; həmçinin net.devh standart olaraq reflection və health servislərini aktiv edir, bu da API-nin kəşfini asanlaşdırır. Həllər: portu yalnız daxili şəbəkəyə açmaq, mTLS, və ya JWT-ni gRPC metadata-da (`authorization` başlığı) ötürüb server tərəfdə interceptor ilə yoxlamaq.

10.8 Gateway-dən yan keçmə

FAKT Compose 8081–8084 portlarını da host-a açır. Birbaşa servisə gedən sorğu:

- **rate limit**-dən yan keçir (rate limit yalnız gateway-dədir);
- autentifikasiya və avtorizasiyadan yan keçə **bilmir** — servislər token-i özləri yoxlayır;
- auth endpoint-ləri birbaşa portda da açıqdır → brute-force login gateway limitsiz mümkündür.

NƏTİCƏ İstehsalda servislər yalnız daxili şəbəkədən əlçatan olmalı, xarici trafik yalnız gateway (və ya ingress) vasitəsilə gəlməlidir. Lokal Compose-da portların açıq olması developer rahatlığıdır.

10.9 Təhdid modeli xülasəsi

Təhdid	Mövcud müdafiə	Boşluq	Tapıntı
Saxta token	RS256, public key ilə yoxlama	Private key repozitoriyadadır	F22
Oğurlanmış access token	120 dəq ömür	Ləğv mexanizmi yoxdur, ömür uzundur	F35
Oğurlanmış refresh token	Rotasiya, hash-lənmiş saxlama	Reuse detection yoxdur, yarış	F17
İmtiyaz eskalasiyası	`@PreAuthorize`	Qeydiyyatda `ROLE_ADMIN` seçilir	F01
Hesabın ələ keçirilməsi	Reset token tək istifadəlik, 15 dəq	Token cavabda qaytarılır	F02
Brute-force login	Gateway rate limit (IP)	XFF saxtalaşdırma, birbaşa port, Redis düşəndə fail-open	F05, F06
Enumerasiya	—	Fərqli mesajlar, timing, 404	F03
Daxili API-yə icazəsiz giriş	log-service rol yoxlaması	gRPC tamamilə açıq	F04
Məlumat sızması	—	500-də istisna mesajı, açıq actuator	F07, F40

10.10 Yeni layihədə tətbiq: Spring Security OAuth2 Resource Server

İllüstrativ nümunə (repozitoriyada yoxdur) — xüsusi filtr yazmadan eyni funksionallıq. Spring Security-nin `oauth2ResourceServer` dəstəyi token çıxarmanı, xəta emalını (`WWW-Authenticate`), istisnaları və authority çevrilməsini standart olaraq edir:

```
@Bean
SecurityFilterChain api(HttpSecurity http, JwtDecoder decoder) throws Exception {
    return http
        .csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(a -> a
            .requestMatchers("/api/v1/auth/**", "/actuator/health").permitAll()
```

```
            .anyRequest().authenticated())
        .oauth2ResourceServer(o -> o.jwt(j -> j
            .decoder(decoder)
            .jwtAuthenticationConverter(rolesConverter())))
        .build();
}

@Bean
JwtDecoder jwtDecoder(@Value("${app.jwt.public-key-location}") RSAPublicKey key) {
    NimbusJwtDecoder decoder = NimbusJwtDecoder.withPublicKey(key).build();
    decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer("allianz-campaign-api"));
    return decoder;                                    // və ya NimbusJwtDecoder.withJwkSetUri(".../jwks.
json")
}

private JwtAuthenticationConverter rolesConverter() {
    JwtGrantedAuthoritiesConverter roles = new JwtGrantedAuthoritiesConverter();
    roles.setAuthoritiesClaimName("roles");
    roles.setAuthorityPrefix("");                      // token-də artıq ROLE_ prefiksi var
    JwtAuthenticationConverter c = new JwtAuthenticationConverter();
    c.setJwtGrantedAuthoritiesConverter(roles);
    return c;
}
```

Gateway üçün analoji: `ServerHttpSecurity.oauth2ResourceServer(o -> o.jwt(...))` və ya `TokenRelay` filtri.

10.11 Müsahibə sualları

S1. Niyə mikroservislərdə RS256 HS256-dan üstündür? Cavab: HS256-da yoxlayan hər servis imzalaya da bilər; sirr bütün servislərdə olmalıdır. RS256-da yalnız auth-service private key-ə malikdir; digərləri public key ilə yalnız yoxlayır. Bir servisin kompromisi token saxtalaşdırmağa imkan vermir.

S2. Stateless JWT ilə CSRF-i söndürmək nə vaxt təhlükəsizdir? Cavab: Token brauzerin avtomatik göndərmədiyi yerdə (Authorization başlığı) saxlanıldıqda. Token cookie-də olarsa, brauzer onu cross-site sorğularda da göndərir və CSRF qorunması (SameSite, CSRF token) lazımdır.

S3. Gateway token-i yoxlayırsa, servislər niyə yenidən yoxlayır? Cavab: Müdafiə dərinliyi (gateway-dən yan keçən daxili trafik, səhv konfiqurasiya), servisin öz avtorizasiya qərarı üçün claim-lərə ehtiyacı, və "zero trust" prinsipi — şəbəkə daxilində olmaq etibar sayılmır.

S4. `hasRole('ADMIN')` və `hasAuthority('ROLE_ADMIN')` fərqi? Cavab: `hasRole` avtomatik `ROLE_` prefiksi əlavə edir; nəticədə eyni yoxlamadır. Authority-lər prefikssiz saxlanılırsa (`ADMIN`), `hasRole` işləməz, `hasAuthority('ADMIN')` lazımdır.

S5. Filtr daxilində atılan `JwtException` niyə `@ControllerAdvice` ilə tutulmur və necə düzəltmək olar? Cavab: Filtr `DispatcherServlet`-dən əvvəl işləyir. Həll: filtrdə tutub `AuthenticationEntryPoint`-ə ötürmək, `HandlerExceptionResolver`-i filtrə inject edib istifadə etmək və ya standart `BearerTokenAuthenticationFilter`-dən (oauth2ResourceServer) istifadə etmək.

S6. gRPC endpoint-ini necə qorumaq olar? Cavab: Şəbəkə izolyasiyası, mTLS (kanal səviyyəsində servis identifikasiyası), metadata-da JWT və server interceptor (net.devh-də `GrpcAuthenticationReader` + `AuthenticationManager` və ya xüsusi `ServerInterceptor`), metod səviyyəli icazələr.

10.12 Xülasə

- RS256 JWT: auth-service imzalayır, digərləri public key ilə lokal yoxlayır.
- Servlet servisləri ortaq `security` modulundan `SecurityFilterChain` + `JwtAuthenticationFilter` + method security alır; gateway WebFlux `GlobalFilter` ilə öz yoxlamasını edir; avtorizasiya son servisdə.
- Servislərarası: auth → log daxili rol ilə qorunur, dashboard → campaign gRPC qorunmur.

- Əsas risklər: özünü admin qeydiyyatı, reset token sızması, açıq gRPC və actuator, repozitoriyadakı private key, filtr istisnaları.
- IV hissə texnologiyaları müstəqil öyrədir; ilk olaraq Apache Kafka.

IV hissə. Texnologiyaları müstəqil öyrənmək

Kafka, Elasticsearch, gRPC, HTTP klientləri və Resilience4j, Redis, Spring Cloud, observability, konteynerlər, testlər

11. Apache Kafka — hadisə axını və asinxron mesajlaşma

Bu fəsil əvvəlcə Kafka-nı layihədən asılı olmadan öyrədir, sonra repozitoriyadakı hər Kafka inteqrasiyasını tam izləyir, sonda yeni servisdə düzgün Kafka inteqrasiyasının necə qurulacağını göstərir.

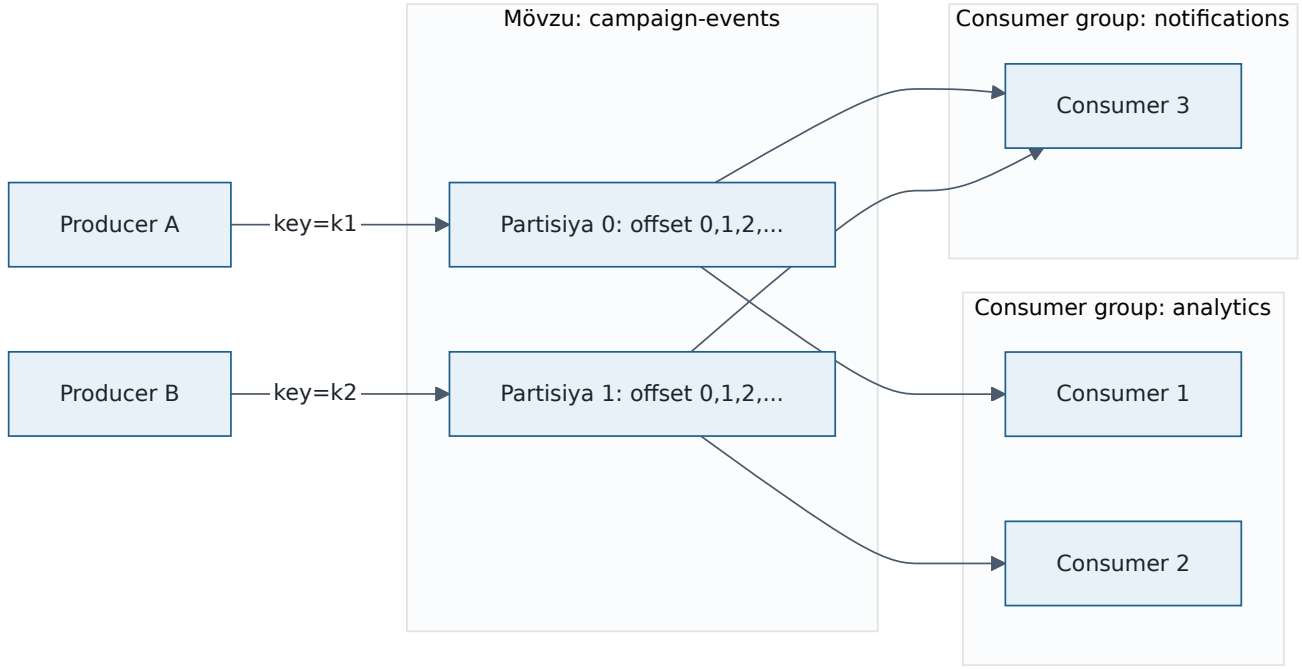
11.1 Motivasiya: niyə mesaj brokeri?

campaign-service kampaniyanı yaradanda iki növ iş var: **əsas iş** (kampaniyanı saxlamaq) və **yan təsirlər** (dashboard-un, analitikanın, bildiriş sisteminin bundan xəbər tutması; yavaş sorğuların jurnallaşdırılması). Yan təsirləri sinxron HTTP ilə etsək:

- hər yeni maraqlanan servis campaign-service-də kod dəyişikliyi tələb edir (sıx bağlılıq);
- istehlakçı servis yavaşdırsa, kampaniya yaratma da yavaşlayır; düşürsə, ya xəta verir, ya da məlumat itir;
- yük pikləri birbaşa istehlakçıya ötürülür.

Mesaj brokeri producer-i consumer-dən ayırır: producer hadisəni davamlı (durable) jurnala yazır, consumer-lər öz sürətləri ilə oxuyur. Kafka bunu **paylanmış, replikasiya olunan, yalnız sona yazılan jurnal** (distributed append-only log) modeli ilə edir.

11.2 Əsas anlayışlar



Kafka-nın əsas anlayışları: mövzu, partisiyalar, consumer group-lar

Anlayış	İzah
Broker	Kafka server prosesi; partisiyaları diskdə saxlayır və xidmət edir
Cluster	Brokerlər qrupu. Metadata-nı **KRaft** (Kafka Raft) controller-ləri idarə edir; Kafka 4.x-də ZooKeeper yoxdur

Anlayış	İzah
Topic (mövzu)	Məntiqi hadisə kateqoriyası, məs. `log-events`
Partition	Mövzunun sıralı, dəyişməz jurnal hissəsi. Paralelliyin və sıranın vahidi
Offset	Partisiya daxilində qeydin ardıcıl nömrəsi
Replica	Partisiyanın başqa brokerdəki nüsxəsi. Bir **leader** yazı/oxuya xidmət edir, **follower**-lər kopyalayır; sinxron olanlar **ISR** (in-sync replicas)
Producer	Qeyd (record) yazan klient
Consumer	Qeyd oxuyan klient
Consumer group	Eyni `group.id`-li consumer-lər; hər partisiya qrup daxilində **yalnız bir** consumer-ə təyin olunur. Fərqli qruplar eyni datanı müstəqil oxuyur
Committed offset	Qrupun partisiya üzrə "burada qaldım" işarəsi; `__consumer_offsets` daxili mövzusunda saxlanılır
Retention	Qeydlər oxunandan sonra silinmir; vaxt/ölçü siyasətinə görə saxlanılır — replay mümkündür

Sıra zəmanəti yalnız partisiya daxilindədir. Eyni açarlı qeydlər (standart partitioner `murmur2(key) % partitions` istifadə edir) eyni partisiyaya düşür və yazıldığı sırada oxunur. Mövzu səviyyəsində qlobal sıra yoxdur. Partisiya sayı artırılarsa, açar → partisiya xəritəsi dəyişir və köhnə/yeni qeydlər üçün sıra pozula bilər.

11.3 Qeydin yazılması (producer daxilində)

Qeyd (record): **key**, **value**, **headers**, **timestamp**, hədəf mövzu (və istəyə görə partisiya).

Producer daxili axını: `send()` → serializer-lər → partitioner → yaddaşdakı partisiya üzrə **batch** (`batch.size`, `linger.ms`) → fon `Sender` thread-i batch-ları brokerə göndərir → broker cavabı (`acks`) → callback/future.

Parametr	Mənası	Kafka 4.x klient standartı NƏTİCƏ
`acks`	`0` gözləmə yox, `1` yalnız leader, `all` bütün ISR	`all`
`enable.idempotence`	Təkrar göndərmələrdə broker dublikatı aradan qaldırır (producer id + ardıcıllıq nömrəsi)	`true`
`retries`, `delivery.timeout.ms`	Müvəqqəti xətalarda təkrar; ümumi çatdırılma vaxt limiti	`Integer.MAX_VALUE`, 120 s
`linger.ms`	Batch dolması üçün gözləmə	5 ms (Kafka 4.0-dan)
`max.block.ms`	`send()`-in metadata/bufer üçün **çağıran thread-də** bloklana biləcəyi maksimum vaxt	60 s
`buffer.memory`	Göndərilməmiş qeydlər üçün yaddaş	32 MB

`max.block.ms` bu layihə üçün xüsusilə vacibdir (11.8-ə baxın).

İdempotent producer yalnız **bir producer sessiyası daxilində, şəbəkə təkrarlarından** yaranan dublikatları aradan qaldırır. Tətbiq `send()`-i iki dəfə çağırarsa (məs. HTTP sorğusu təkrarlanarsa), iki ayrı qeyd yaranır.

Tranzaksional producer (`transactional.id`) bir neçə mövzuya yazmanı və consumer offset commit-ini atomik edir (read-process-write). Consumer tərəfdə `isolation.level=read_committed` lazımdır. Bu, **Kafka daxilində** exactly-once verir; xarici sistemə (PostgreSQL, Elasticsearch) yazma bu zəmanətə daxil deyil.

11.4 Qeydin oxunması (consumer daxilində)

Consumer `poll()` döngüsündə işləyir: brokerdən qeyd partiyası alır, emal edir, offset-i commit edir.

- **Qrup koordinasiyası və rebalans**: consumer qoşulanda/çıxanda və ya `max.poll.interval.ms` (standart 5 dəq) ərzində `poll()` çağırmadıqda qrup koordinatoru partisiyaları yenidən paylayır. Rebalans zamanı emal dayanır; commit olunmamış qeydlər yeni sahibinə yenidən verilir.
- **Offset commit**: avtomatik (`enable.auto.commit=true`, periodik) və ya manual. Spring Kafka konteyneri `enable.auto.commit`-i açıq təyin olunmayıbsa `false` edir və offset-i özü commit edir **FAKT** (spring-kafka 4.1.0 `KafkaMessageListenerContainer`). Standart `AckMode.BATCH`: `poll()`-dan gələn bütün qeydlər listener tərəfindən emal olunduqdan sonra commit.
- **`auto.offset.reset`**: qrupun commit olunmuş offset-i yoxdursa haradan başlamalı — `latest` (standart, yalnız yeni qeydlər) və ya `earliest` (əvvəldən).

Çatdırılma semantikası

Semantika	Necə alınır	Nəticə
At-most-once	Emaldan **əvvəl** commit	Çökmədə qeyd itə bilər, dublikat yoxdur
At-least-once	Emaldan **sonra** commit (Spring standartı)	İtki yoxdur, çökmə/rebalans zamanı **dublikat** mümkündür
Exactly-once (Kafka daxilində)	Tranzaksional producer + `read_committed`	Yalnız Kafka→Kafka axınları üçün
Effektiv exactly-once (xarici sistemlə)	At-least-once + **idempotent consumer**	Tətbiq dizaynı ilə

Praktiki qayda: **xarici yan təsiri olan consumer-lər idempotent olmalıdır** — eyni hadisəni iki dəfə emal etmək bir dəfə emal etmək kimi nəticə verməlidir (təbii idempotent əməliyyat, deterministik id ilə upsert, emal olunmuş hadisə id-lərinin cədvəli).

Xətalar, təkrarlar, DLQ, "zəhərli" mesajlar

- **Müvəqqəti xəta** (DB düşüb): bir neçə dəfə backoff ilə təkrar.
- **Daimi xəta** (səhv data): təkrar mənasızdır — qeydi **dead-letter** mövzusuna köçürüb davam etmək.
- **Zəhərli mesaj (poison pill)**: deserializasiya oluna bilməyən qeyd. Əgər consumer onu keçə bilmirsə, partisiya tamamilə dayanır — çünki offset irəliləmir.
- **Sıra və təkrarlar**: bloklayan təkrar (eyni thread-də gözləmək) sıranı qoruyur, lakin partisiyanı saxlayır; bloklamayan təkrar (retry mövzuları) partisiyanı azad edir, sıranı pozur.

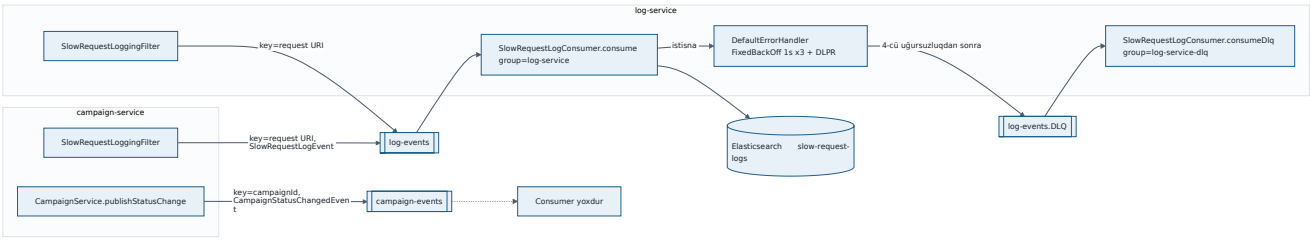
Sxem uyğunluğu

Hadisə formatı producer-consumer kontraktıdır. Sahə əlavə etmək (consumer tanımadığı sahəni nəzərə almırsa) geriyə uyğundur; sahənin adını dəyişmək və ya silmək deyil. Böyük sistemlərdə **Schema Registry** (Avro/Protobuf/JSON Schema) uyğunluğu yazma anında yoxlayır. Bu layihədə sxem yalnız paylaşılan Java record-larıdır (`common/event`).

11.5 Spring Kafka komponentləri

Komponent	Rolu
`ProducerFactory` / `DefaultKafkaProducerFactory`	Producer konfiqurasiyası, paylaşılan thread-safe `KafkaProducer` instansı
`KafkaTemplate`	`send(topic, key, value)` → `CompletableFuture<SendResult>`
`ConsumerFactory`	Consumer konfiqurasiyası
`@KafkaListener`	Metodu listener kimi qeydiyyatdan keçirir; `@EnableKafka` emal edir
`ConcurrentKafkaListenerContainerFactory`	Listener konteynerlərini (poll döngüsü, thread-lər, commit, xəta emalı) qurur
`DefaultErrorHandler`	Listener istisnasında qeydi yenidən emal üçün "seek" edir, backoff, sonra recoverer
`DeadLetterPublishingRecoverer`	Uğursuz qeydi DLT/DLQ mövzusuna istisna başlıqları ilə göndərir
`ErrorHandlingDeserializer`	Deserializasiya xətasını tutub qeydə əlavə edir ki, xəta handler-i onu emal edə bilsin
`JsonSerializer` / `JsonDeserializer`	Jackson 2 əsaslı JSON; spring-kafka 4.0-dan `@Deprecated(forRemoval = true)` — yerinə `JacksonJsonSerializer` / `JacksonJsonDeserializer` (Jackson 3)

11.6 Layihədəki Kafka xəritəsi



Repozitoriyadakı bütün Kafka axınları

Mövzu	Producer	Açar	Dəyər	Consumer	Sübut
`campaign-events`	campaign-service	`campaignId.toString()`	`CampaignStatusChangedEvent`	**yoxdur**	`CampaignService.java:181-191`
`log-events`	campaign-service, log-service (filtr)	`request.getRequestURI()`	`SlowRequestLogEvent`	log-service, group `log-service`	`SlowRequestLoggingFilter.java:113-117`, `SlowRequestLogConsumer.java:35-47`
`log-events.DLQ`	`DeadLetterPublishingRecoverer`	orijinal açar	orijinal dəyər + istisna başlıqları	log-service, group `log-service-dlq`	`KafkaDlqConfiguration.java:26-33`, `SlowRequestLogConsumer.java:54-65`

Mövzular kodda yaradılmır (`NewTopic` bean-i yoxdur); broker `KAFKA_AUTO_CREATE_TOPICS_ENABLE: 'true'` və `KAFKA_NUM_PARTITIONS: 1` ilə avtomatik yaradır **FAKT** (`docker-compose.yml:322-323`). Deməli hər mövzunun 1 partisiyası və replication factor 1 var.

11.7 Producer tərəfi: `KafkaJsonConfiguration`

Fəsil 4-də şərti konfiqurasiya baxımından baxdıq; indi Kafka baxımından **FAKT** (`KafkaJsonConfiguration.java:41-49`):

- Yalnız 4 parametr təyin olunur: `bootstrap.servers`, `StringSerializer` açar, `JsonSerializer` dəyər, `spring.json.add.type.headers=false`.
- Qalan hər şey Kafka klientinin standartlarıdır: `acks=all`, idempotentlik açıq, `max.block.ms=60000` və s.
- **Tip başlıqları söndürülüb**: `JsonSerializer` standart olaraq `__TypeId__` başlığına Java sinif adını yazır; consumer onu deserializasiya üçün istifadə edir. Söndürmək producer və consumer-i Java sinif adlarından ayırır (dil müstəqilliyi, paket adının dəyişməsinə dözümlülük), lakin consumer **standart tipi** bilməlidir — log-service bunu `spring.json.value.default.type` ilə edir.
- **NƏTİCƏ** `JsonSerializer` öz `ObjectMapper`-ini (`JacksonUtils.enhancedObjectMapper()`) yaradır: Java Time modulu qeydiyyatdan keçir, lakin `WRITE_DATES_AS_TIMESTAMPS` söndürülmür. Ona görə `Instant occurredAt` JSON-da ədədi epoch dəyəri kimi (`1767000000.123456789`) yazılır, HTTP API-lərdəki ISO sətrindən fərqli. Hər iki tərəf eyni kitabxanadan istifadə etdiyi üçün uyğundur; başqa dildə consumer üçün bu sənədləşdirilməlidir. `common/pom.xml:32-35`-dəki `jackson-datatype-jsr310` asılılığı məhz bu modulun classpath-də olması üçündür.
- Spring Boot-un Kafka auto-configuration-u campaign-service-də yoxdur, log-service-də isə `KafkaTemplate` / `ProducerFactory` üçün geri çəkilir. **NƏTİCƏ** Nəticədə producer-də Micrometer metrikləri (`MicrometerProducerListener`) və observation (trace başlıqları) aktiv deyil; trace əlaqəsi əvəzinə `SlowRequestLogEvent.traceId` sahəsi payload-da ötürülür.

`campaign-events` — biznes hadisəsi

Mənbə: campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:181-191

```
private void publishStatusChange(UUID campaignId, CampaignCategory category, CampaignStatus previousStatu
s, CampaignStatus newStatus, String reason) {
    CampaignStatusChangedEvent event = new CampaignStatusChangedEvent(
            campaignId,
            category.name(),
            previousStatus == null ? null : previousStatus.name(),
            newStatus.name(),
            reason,
            Instant.now()
    );
    kafkaTemplate.send(campaignEventsTopic, campaignId.toString(), event);
}
```

- **Açar = kampaniya id-si** — doğru seçimdir: bir kampaniyanın bütün hadisələri eyni partisiyaya düşür və sıra ilə oxunur (`CREATED → Campaign approved → Campaign deactivated`).
- **Hadisədə `eventId` yoxdur** — istehlakçı dublikatı aşkar edə bilmir (idempotentlik üçün `campaignId + newStatus` istifadə oluna bilər, çünki keçidlər təkrarlanmır, lakin bu, kontraktda açıq yazılmayıb).
- **`reason` sahəsi qeyri-ardıcıldır** **FAKT**: yaratmada `"CREATED"`, keçidlərdə `"Campaign approved"` kimi insan oxuyacağı mətn (`CampaignService.java:72` və `:99`, `:110`). Hadisə kontraktında maşın tərəfindən oxunan `eventType` (`CAMPAIGN_CREATED`, `CAMPAIGN_APPROVED`) daha yaxşı olardı.
- **Nəticə yoxlanmır** — `send()` `CompletableFuture` qaytarır, kod onu nəzərə almır (fire-and-forget). Xəta yalnız `KafkaTemplate`-in standart `LoggingProducerListener`-i ilə loglanır.

- **Consumer yoxdur** (F12) — NAMƏLUM gələcək və ya repozitoriyadan kənar istehlakçı nəzərdə tutula bilər.

`log-events` — yavaş sorğu jurnalı

Mənbə: common/src/main/java/com/casestudy/insurancecampaign/common/web/SlowRequestLoggingFilter.java:88-120

```
@Override
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain fil
terChain)
        throws ServletException, IOException {
    long start = System.nanoTime();

    try {
        filterChain.doFilter(request, response);
    } finally {
        long durationMs = (System.nanoTime() - start) / 1_000_000;

        if (durationMs > thresholdMs) {
            SlowRequestLogEvent event = new SlowRequestLogEvent(
                    applicationName,
                    request.getMethod(),
                    request.getRequestURI(),
                    response.getStatus(),
                    durationMs,
                    Optional.ofNullable(MDC.get("traceId")).orElse("N/A"),
                    Map.of(
                            "user-agent", Optional.ofNullable(request.getHeader("User-Agent")).orElse("N/
A"),
                            "x-forwarded-for", Optional.ofNullable(request.getHeader("X-Forwarded-For")).
orElse("N/A")
                    ),
                    Instant.now()
            );

            KafkaTemplate<String, Object> kafkaTemplate = kafkaTemplateProvider.getIfAvailable();

            if (kafkaTemplate != null) {
                kafkaTemplate.send(topicName, request.getRequestURI(), event);
            }
        }
    }
}
```

- `System.nanoTime()` monoton saatdır — müddət ölçmək üçün düzgün seçim (`currentTimeMillis` saat düzəlişlərindən təsirlənir).
- `finally` — istisna olsa belə ölçülür.
- `shouldNotFilter` actuator, Swagger və OpenAPI yollarını istisna edir (`:67-74`) — Prometheus scrape-ləri Kafka-ya yazılmır.
- Həd 5 ms-dir (F25) — praktikada demək olar ki, hər sorğu "yavaş" sayılır və Kafka-ya gedir. Bu, case study-də Kafka axınını göstərmək üçündür; istehsalda həd p95/p99 gecikmə əsasında (məs. 500 ms–1 s) seçilməlidir.
- Açar sorğu yoludur. 1 partisiyada fərq etmir; çox partisiyada populyar yollar (`/api/v1/campaigns`) "isti partisiya" (hot partition) yarada bilər.
- `traceId` MDC-dən oxunur — Micrometer Tracing bu açarı MDC-yə yazır; log faylı, Tempo trace-i və Elasticsearch qeydi bu id ilə əlaqələndirilə bilər.

11.8 Producer uğursuzluq ssenariləri (F09, F10)

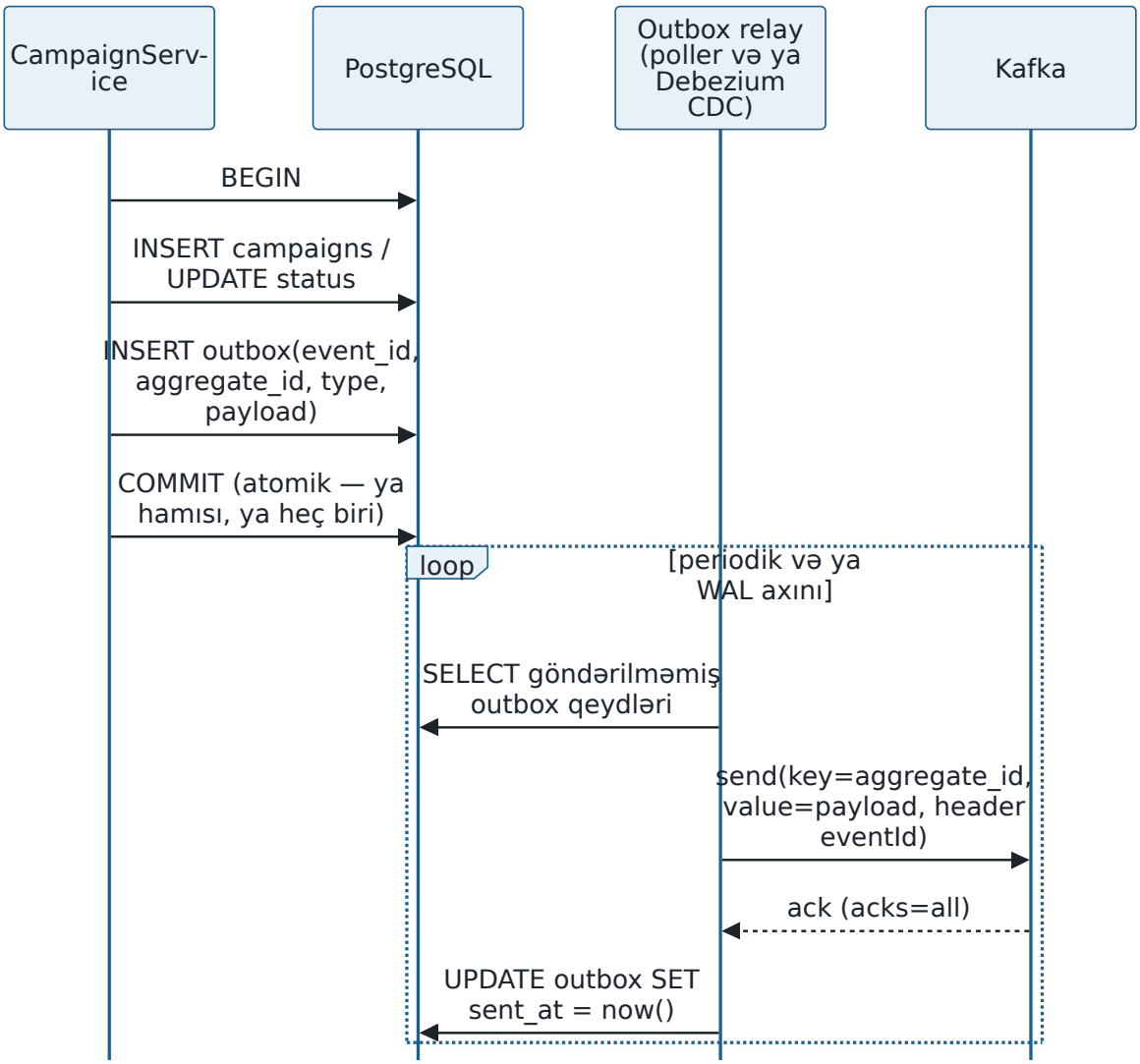
Kafka brokeri əlçatmazdır [NƏTİCƏ, Kafka producer semantikasına əsasən]:

1. `send()` mövzu metadata-sı olmadan partisiya seçə bilmir və metadata üçün çağıran thread-də `max.block.ms` (60 s) qədər gözləyir.
2. `SlowRequestLoggingFilter` bunu `finally` blokunda, cavab klientə tam ötürülməmiş ola bilən anda edir — klient gecikmə görür.
3. `CampaignService.create/approve/deactivate` bunu **açıq DB tranzaksiyası daxilində** edir: DB bağlantısı və sətir kilidləri 60 s tutulur; HikariCP hovuzu (10) tez tükənir (fəsil 7.8).
4. 60 s sonra göndəriş uğursuz future ilə bitir, hadisə **itir**, yalnız log qalır; DB tranzaksiyası isə commit olunur.

Commit uğursuz olur, hadisə artıq göndərilib — fəsil 6.3: hadisə tarixçə INSERT-indən və commit-dən əvvəl göndərilir. Consumer mövcud olmayan vəziyyət haqqında hadisə alar.

Bu iki ssenari birlikdə **dual-write problemi**dir: iki müstəqil sistemə (DB və Kafka) atomik yazmaq mümkün deyil.

Həll: Transactional Outbox



Transactional Outbox nümunəsi (illüstrativ, repozitoriyada yoxdur)

Outbox at-least-once verir (relay göndərib commit etməzdən əvvəl çöksə, təkrar göndərər), ona görə consumer-lər `eventId` ilə idempotent olmalıdır. Daha yüngül alternativ: `@TransactionalEventListener(phase = AFTER_COMMIT)` ilə hadisəni commit-dən sonra göndərmək — commit olunmamış hadisə problemini həll edir, lakin commit-dən sonra proses çökərsə hadisə itir.

11.9 Consumer tərəfi: log-service

Konfiqurasiya

Mənbə: log-service/src/main/resources/application.yml:8-16

```yaml
kafka:
  bootstrap-servers: ${KAFKA_BOOTSTRAP_SERVERS:localhost:9092}
  consumer:
    group-id: log-service
    key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
    value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
    properties:
      spring.json.trusted.packages: com.casestudy.insurancecampaign.*
      spring.json.value.default.type: com.casestudy.insurancecampaign.common.event.SlowRequestLogEvent
```

- `spring.json.trusted.packages` — `JsonDeserializer` tip başlığından gələn sinfi yalnız etibarlı paketlərdən yaradır (ixtiyari sinfin deserializasiyası təhlükəsizlik riskidir). Burada tip başlığı yoxdur, standart tip istifadə olunur.
- `spring.json.value.default.type` — bütün qeydlər `SlowRequestLogEvent` kimi oxunur. **NƏTİCƏ** Bu o deməkdir ki, eyni consumer factory ilə başqa tipli mövzu oxumaq olmaz; DLQ listener-i də eyni tipi alır.
- `auto-offset-reset` lokal faylda yoxdur; Config Server-in paylaşılan faylında `earliest`-dir (F32).

Listener-lər

Mənbə: log-service/src/main/java/com/casestudy/insurancecampaign/log/service/SlowRequestLogConsumer.java:35-65

```java
@KafkaListener(topics = "${app.kafka.topics.log-events:log-events}", groupId = "log-service")
public void consume(SlowRequestLogEvent event) {
    repository.save(new SlowRequestDocument(
            event.serviceName(),
            event.method(),
            event.path(),
            event.status(),
            event.durationMs(),
            event.traceId(),
            event.headers(),
            event.occurredAt()
    ));
}

// ... (buraxılıb: Javadoc)
@KafkaListener(topics = "${app.kafka.topics.log-events:log-events}.DLQ", groupId = "log-service-dlq")
public void consumeDlq(ConsumerRecord<String, Object> record) {
    log.error(
            "DLQ message received. topic={}, partition={}, offset={}, key={}, value={}, headers={}",
            record.topic(),
            record.partition(),
            record.offset(),
            record.key(),
            record.value(),
            headersAsString(record)
    );
}
```

- Mövzu adı property placeholder ilə həll olunur; DLQ mövzusu `${...}.DLQ` — sətir birləşməsi ilə.
- `groupId` annotasiyada açıq verilib, yml-dəki `group-id`-ni üstələyir. Testlərdə `spring.kafka.consumer.group-id=log-service-test` təyin edilir, lakin annotasiya bunu əvəzləyir — test listener-ləri onsuz da `auto-startup=false`-dur.
- DLQ consumer-i **ayrı qrupdadır** — DLQ oxunması əsas axının offset-lərinə təsir etmir.
- **Consumer idempotent deyil** [FAKT/NƏTİCƏ]: `SlowRequestDocument` hər dəfə yeni `UUID.randomUUID()` id alır (`SlowRequestDocument.java:31-41`). Eyni Kafka qeydi təkrar çatdırılarsa (rebalans, çökmə, retry),

Elasticsearch-də dublikat sənəd yaranır. Düzəliş: deterministik id, məs. `topic-partition-offset` və ya hadisədə `eventId`.

Xəta emalı və DLQ

Mənbə: log-service/src/main/java/com/casestudy/insurancecampaign/log/config/KafkaDlqConfiguration.java:26-51

```
@Bean
public DefaultErrorHandler defaultErrorHandler(KafkaTemplate<String, Object> kafkaTemplate) {
    DeadLetterPublishingRecoverer recoverer = new DeadLetterPublishingRecoverer(
            kafkaTemplate,
            (ConsumerRecord<?, ?> record, Exception exception) -> new TopicPartition(record.topic() + ".D
LQ", record.partition())
    );
    return new DefaultErrorHandler(recoverer, new FixedBackOff(1_000L, 3));
}

// ... (buraxılıb: Javadoc)
@Bean
public ConcurrentKafkaListenerContainerFactory<String, Object> kafkaListenerContainerFactory(
        ConsumerFactory<String, Object> consumerFactory,
        DefaultErrorHandler defaultErrorHandler
) {
    ConcurrentKafkaListenerContainerFactory<String, Object> factory = new ConcurrentKafkaListenerContaine
rFactory<>();
    factory.setConsumerFactory(consumerFactory);
    factory.setCommonErrorHandler(defaultErrorHandler);
    return factory;
}
```

Davranış addım-addım [NƏTİCƏ, spring-kafka semantikası]:

1. Listener istisna atır (məs. Elasticsearch əlçatmazdır → `DataAccessResourceFailureException`).
2. `DefaultErrorHandler` uğursuz qeydin offsetinə (və qalan qeydlərin offsetlərinə) **seek** edir və backoff qədər (1 s) consumer thread-ini gözlədir; növbəti `poll()` eyni qeydi yenidən verir.
3. `FixedBackOff(1000, 3)` : **3 təkrar** — cəmi 4 cəhd (ilkin + 3), təxminən 3 s gözləmə.
4. Son cəhd də uğursuzdursa, `DeadLetterPublishingRecoverer` qeydi `log-events.DLQ` mövzusunun **eyni nömrəli partisiyasına** göndərir və başlıqlara orijinal mövzu/partisiya/offset, istisna sinfi, mesaj və stack trace əlavə edir; sonra offset irəliləyir.
5. DLQ consumer-i qeydi ERROR səviyyəsində loglayır (yenidən emal etmir).

Kompromislər:

- **Bloklayan retry** — 4 cəhd ərzində partisiya dayanır, sıra qorunur. 3 s qısa olduğu üçün `max.poll.interval.ms` problemi yoxdur.
- **DLQ partisiya sayı** — destination resolver orijinal partisiya nömrəsini istifadə edir; DLQ mövzusunun partisiyası azdırsa, göndəriş uğursuz olar. Avtomatik yaradılmış hər iki mövzunun 1 partisiyası var — işləyir, lakin mövzular açıq yaradılsa bu asılılıq yadda saxlanmalıdır.
- **Bütün istisnalar təkrar olunur** — `IllegalArgumentException` kimi daimi xətalar da 3 dəfə təkrarlanır. `defaultErrorHandler.addNotRetryableExceptions(...)` ilə ayırmaq olar.
- **Xüsusi konteyner factory-si Boot-un konfiqurasiyasını əvəzləyir** — `spring.kafka.listener.*` xassələri (concurrency, ack-mode, observation) bu factory-yə tətbiq olunmur, çünki Boot-un `ConcurrentKafkaListenerContainerFactoryConfigurer`-i istifadə edilmir. NƏTİCƏ `spring.kafka.listener.auto-startup=false` (testlərdə) da buna görə tətbiq olunmaya bilər; testlərin heç biri listener-in işləməsindən asılı deyil.

Deserializasiya xətası — ən təhlükəli boşluq (F11)

`JsonDeserializer` birbaşa value deserializer kimi konfiqurasiya olunub, `ErrorHandlingDeserializer` ilə bükülməyib. [FAKT, spring-kafka 4.1.0 mənbəsi] Səhv JSON gəldikdə:

- istisna listener-də deyil, `KafkaConsumer.poll()` daxilində yaranır (`RecordDeserializationException` , `SerializationException` -ın alt sinfi);
- konteyner onu `CommonErrorHandler.handleOtherException` -a ötürür;
- `DefaultErrorHandler.handleOtherException` `SerializationException` üçün `IllegalStateException("This error handler cannot process 'SerializationException's directly; please consider configuring an 'ErrorHandlingDeserializer'...")` atır;
- `KafkaMessageListenerContainer.handleConsumerException` bunu "Consumer exception" kimi loglayır və döngü davam edir;
- növbəti `poll()` **eyni offsetdən** başlayır → eyni xəta → sonsuz döngü. Partisiya bloklanır, log daşır, DLQ boş qalır.

Düzəliş:

```
spring.kafka.consumer:
  value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer
  properties:
    spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JacksonJsonDes
erializer
```

Bu halda xəta qeydə "əlavə edilir", listener çağırılmadan `DefaultErrorHandler` onu dərhal (deserializasiya xətası standart olaraq təkrarlanmayan sayılır) DLQ-ya göndərir.

11.10 Broker konfiqurasiyası (Docker Compose)

FAKT (`docker-compose.yml:300-335`):

Parametr	Dəyər	Mənası
`KAFKA_PROCESS_ROLES`	`broker,controller`	KRaft "combined" rejimi — bir proses həm broker, həm controller (yalnız dev üçün)
`KAFKA_LISTENERS`	`CONTROLLER://:9093, PLAINTEXT_HOST://:9092, PLAINTEXT://:19092`	Üç dinləyici
`KAFKA_ADVERTISED_LISTENERS`	`PLAINTEXT_HOST://localhost:9092, PLAINTEXT://kafka:19092`	Klientlərə qaytarılan ünvanlar
`KAFKA_INTER_BROKER_LISTENER_NAME`	`PLAINTEXT`	Brokerlər arası trafik
`*_REPLICATION_FACTOR` , `*_MIN_ISR`	1	Tək broker
`KAFKA_AUTO_CREATE_TOPICS_ENABLE`	true	İlk istifadədə mövzu yaranır
`KAFKA_NUM_PARTITIONS`	1	Avtomatik mövzuların partisiya sayı

Niyə iki dinləyici? Klient əvvəlcə `bootstrap.servers` ilə qoşulur, sonra broker ona **advertised** ünvanı qaytarır və klient sonrakı bütün bağlantıları həmin ünvana açır. Host-dakı IDE üçün bu `localhost:9092` , konteynerlər üçün `kafka:19092` olmalıdır (konteyner daxilində `localhost` konteynerin özüdür). Buna görə Compose servisləri `KAFKA_BOOTSTRAP_SERVERS: kafka:19092` istifadə edir. Bu, Docker-də Kafka ilə ən çox rast gəlinən səhvin həllidir.

Kafka UI (`kafbat/kafka-ui` , 8090) mövzuları, mesajları, consumer group-ları və lag-ı göstərir.

11.11 Konfiqurasiyanın mənbədən istehlakçıya izlənməsi

Dəyər	Mənbə	İstehlakçı
`KAFKA_BOOTSTRAP_SERVERS=kafka:19092`	Compose env	`spring.kafka.bootstrap-servers` → campaign: `KafkaJsonConfiguration.producerFactory` (`@Value`); log: Boot `KafkaProperties` (consumer) + `KafkaJsonConfiguration` (producer)
`app.kafka.topics.campaign-events`	`campaign-service/src/main/resources/application.yml:82`	`CampaignService` konstruktoru (`@Value`)
`app.kafka.topics.log-events`	`application.yml` (campaign, log, config-server)	`SlowRequestLoggingFilter` (`@Value`), `@KafkaListener(topics = "${...}")`
`spring.kafka.consumer.*`	`log-service/src/main/resources/application.yml:10-16`	Boot `DefaultKafkaConsumerFactory`
`spring.kafka.consumer.auto-offset-reset=earliest`	Config Server paylaşılan fayl	log-service consumer (yalnız Config Server əlçatan olduqda)

11.12 Yeni servisdə Kafka inteqrasiyası: tövsiyə olunan şablon

İllüstrativ nümunə (repozitoriyada yoxdur):

```
// 1) Mövzuları açıq elan edin (partisiya, replika, saxlama)
@Bean NewTopic campaignEvents() {
    return TopicBuilder.name("campaign-events").partitions(6).replicas(3)
            .config(TopicConfig.MIN_IN_SYNC_REPLICAS_CONFIG, "2").build();
}

// 2) Hadisə zərfi: eventId, type, version, occurredAt, payload
public record EventEnvelope<T>(UUID eventId, String type, int version, Instant occurredAt, T payload) {}

// 3) Producer: idempotent, acks=all (standart), qısa max.block.ms, outbox relay-dən göndəriş
spring.kafka.producer.properties.max.block.ms: 5000

// 4) Consumer: ErrorHandlingDeserializer + JacksonJsonDeserializer, idempotent emal
@KafkaListener(topics = "campaign-events", groupId = "dashboard-read-model")
@Transactional
public void on(EventEnvelope<CampaignStatusChanged> e) {
    if (processedEvents.existsById(e.eventId())) return;          // dublikat
    readModel.apply(e.payload());
    processedEvents.save(new ProcessedEvent(e.eventId()));         // eyni DB tranzaksiyası
}

// 5) Xəta emalı: eksponensial backoff, təkrarlanmayan istisnalar, DLT
@Bean DefaultErrorHandler errorHandler(KafkaOperations<Object, Object> template) {
    var handler = new DefaultErrorHandler(new DeadLetterPublishingRecoverer(template),
            new ExponentialBackOffWithMaxRetries(5));
    handler.addNotRetryableExceptions(IllegalArgumentException.class, ValidationException.class);
    return handler;
}
```

Test: Testcontainers Kafka + Awaitility ilə "hadisə göndər → consumer nəticəsini gözlə"; DLQ ssenarisi üçün səhv mesaj göndərib DLQ mövzusunu oxumaq. Bu layihədə belə test yoxdur (fəsil 19).

11.13 Troubleshooting

Simptom	Ehtimal olunan səbəb	Diaqnostika	Həll
Servis sorğuları ~60 s gecikir	Kafka əlçatmaz, `send()` metadata gözləyir	Log: `TimeoutException ... not present in metadata after 60000 ms`	Brokeri qaldırmaq; `max.block.ms`-i azaltmaq; göndərişi request thread-indən çıxarmaq
log-service eyni xətanı sonsuz loglayır	Poison pill + `ErrorHandlingDeserializer` yoxdur	Log: "cannot process 'SerializationException's"	`ErrorHandlingDeserializer`; offset-i əl ilə keçmək (`kafka-consumer-groups.sh --reset-offsets --shift-by 1`)
Elasticsearch-də dublikat sənədlər	Təkrar çatdırılma + random id	Eyni `traceId` / `occurredAt` ilə iki sənəd	Deterministik id
Consumer lag böyüyür	Yavaş emal, ES gecikməsi	`kafka-consumer-groups.sh --describe --group log-service`, Kafka UI	Partisiya + concurrency artırmaq, batch listener, bulk indexing
Yeni consumer group köhnə mesajları oxumur	`auto-offset-reset=latest` (Config Server əlçatmaz)	Startup log-da effektiv `auto.offset.reset`	Lokal konfiqurasiyada açıq təyin etmək
IDE-dən qoşulanda `kafka:19092` tapılmır	Səhv advertised listener	`kafka-broker-api-versions.sh`	`localhost:9092` istifadə
DLQ-ya göndəriş uğursuz	DLQ-da partisiya çatmır	Producer xətası	DLQ mövzusunu eyni partisiya sayı ilə yaratmaq

11.14 Müsahibə sualları

S1. Kafka-da sıra zəmanəti haradadır və açar seçimi niyə vacibdir? Cavab: Yalnız partisiya daxilində. Eyni açarlı qeydlər eyni partisiyaya düşür, ona görə bir obyektin hadisələrini sıralı saxlamaq üçün açar həmin obyektin id-si olmalıdır — layihədə `campaignId`. Partisiya sayının dəyişməsi açar xəritəsini dəyişir.

S2. At-least-once nə deməkdir və consumer-dən nə tələb edir? Cavab: Hər qeyd ən azı bir dəfə emal olunur; çökmə və rebalans zamanı təkrar ola bilər, çünki offset emaldan sonra commit olunur. Consumer idempotent olmalıdır. log-service-in consumer-i idempotent deyil — dublikat ES sənədləri yaranır.

S3. Bu layihədə `kafkaTemplate.send()`-in tranzaksiya daxilində çağırılmasının iki riskini deyin. Cavab: (1) Hadisə commit-dən əvvəl göndərilir — commit uğursuz olsa, mövcud olmayan dəyişiklik haqqında hadisə yayılır; send uğursuz olsa, DB dəyişikliyi qalır, hadisə itir (dual-write). (2) Kafka əlçatmaz olduqda `send()` `max.block.ms` qədər bloklayır, DB bağlantısını və kilidləri tutur, bağlantı hovuzunu tükəndirir.

S4. Transactional Outbox necə işləyir? Cavab: Biznes dəyişikliyi və hadisə qeydi eyni DB tranzaksiyasında `outbox` cədvəlinə yazılır. Ayrı relay (poller və ya CDC, məs. Debezium) commit olunmuş qeydləri Kafka-ya göndərir və işarələyir. Atomiklik DB tranzaksiyası ilə təmin olunur; çatdırılma at-least-once-dir, consumer-lər `eventId` ilə idempotent olur.

S5. Poison pill nədir və bu layihədə nə baş verər? Cavab: Consumer-in heç vaxt emal edə bilmədiyi mesaj. Burada deserializasiya xətası `ErrorHandlingDeserializer` olmadan poll səviyyəsində baş verir; `DefaultErrorHandler` onu emal edə bilmir, konteyner eyni offseti təkrar oxuyur və partisiya dayanır. DLQ işə düşmür.

S6. `FixedBackOff(1000, 3)` neçə cəhd deməkdir? Cavab: İlkin cəhd + 3 təkrar = 4 cəhd, aralarında 1 s. Sonra recoverer (DLQ).

S7. Consumer group-un iki fərqli istifadəsini nümunə ilə izah edin. Cavab: Eyni qrupda bir neçə instans — yükün bölüşdürülməsi (hər partisiya bir instansa). Fərqli qruplar — eyni hadisələrin müstəqil istehlakı (log-service əsas mövzunu `log-service` qrupu ilə, DLQ-nu `log-service-dlq` qrupu ilə oxuyur; gələcəkdə `campaign-events`-i analitika və dashboard ayrı qruplarla oxuya bilər).

S8. Exactly-once Kafka-da nə vaxt həqiqətən mümkündür? Cavab: Kafka-dan oxuyub Kafka-ya yazan axınlarda, tranzaksional producer və `read_committed` consumer ilə. Xarici sistemə yazma (DB, ES, HTTP) üçün exactly-once zəmanəti yoxdur — at-least-once + idempotentlik və ya outbox/inbox nümunələri ilə "effektiv bir dəfə" əldə edilir.

S9. `ADD_TYPE_INFO_HEADERS=false`-ın üstünlüyü və bədəli? Cavab: Producer Java sinif adlarını mesaja yazmır — kontrakt dil və paketdən asılı olmur, refaktorinq consumer-i sındırmır. Bədəli: consumer hədəf tipi özü bilməlidir (`value.default.type`), bir mövzuda bir neçə hadisə tipi saxlamaq çətinləşir.

11.15 Xülasə

- Kafka paylanmış jurnaldır: mövzu → partisiya → offset; sıra partisiya daxilindədir; consumer group-lar yükü bölür və müstəqil istehlak edir.
- Layihədə üç mövzu: `campaign-events` (consumer-siz biznes hadisəsi), `log-events` (yavaş sorğular → Elasticsearch), `log-events.DLQ` .
- Producer: JSON, tip başlıqsız, fire-and-forget, tranzaksiya daxilində — dual-write və bloklanma riskləri.
- Consumer: at-least-once, 4 cəhd + DLQ; `ErrorHandlingDeserializer` yoxdur (poison pill), idempotent deyil.
- Növbəti fəsil yavaş sorğu və audit qeydlərinin saxlandığı Elasticsearch-i öyrədir.

12. Elasticsearch — axtarış yönümlü saxlama

12.1 Motivasiya: niyə relyasiya bazası deyil?

Yavaş sorğu və audit qeydləri **çox yazılan, nadir dəyişən, zamanla sıralanan, sərbəst mətnlə və müxtəlif sahələrlə axtarılan** datadır: "son bir saatda campaign-service-də 500 ms-dən uzun POST sorğuları", "demo istifadəçisinin uğursuz login cəhdləri", "mesajında 'expired' sözü olan audit qeydləri". Relyasiya bazası bunu edə bilər, lakin:

- sərbəst mətn axtarışı (`LIKE '%...%'`) indeks istifadə etmir və relevansa görə sıralamır;
- hər yeni axtarış ölçüsü üçün yeni indeks lazımdır;
- milyonlarla jurnal qeydinin üfüqi miqyaslanması çətindir.

Elasticsearch (Apache Lucene üzərində qurulmuş paylanmış axtarış və analitika mühərriki) sənəd (JSON) saxlayır, hər sahəni avtomatik indeksləyir, tam mətn axtarışı, filtrləmə, aqreqasiya və üfüqi miqyaslanma verir. Bədəli: tranzaksiyalar, join-lər və güclü ardıcıllıq yoxdur; yeni yazılan sənəd dərhal deyil, **refresh**-dən sonra görünür.

12.2 Əsas anlayışlar

Anlayış	İzah	Relyasiya analoqu (təqribi)
Cluster / node	Serverlər qrupu / bir server	DB serveri
Index	Sənədlər kolleksiyası + mapping + parametrlər	Cədvəl
Document	JSON obyekti, `_id` ilə	Sətir
Mapping	Sahələrin tipləri və analiz qaydaları	Sxem
Shard	İndeksin hissəsi (Lucene indeksi); primary + replica	Partisiya
Refresh	Yazılanların axtarışa açıq olması (standart ~1 s)	—
`_source`	Orijinal JSON-un saxlanılan nüsxəsi	—

İnverted index və analiz

Lucene-in əsası **inverted index**-dir: hər **term** (söz) üçün onu ehtiva edən sənədlərin siyahısı.

```
Sənəd 1: "Login completed successfully"
Sənəd 2: "Invalid username/email or password"

Analiz (standard analyzer): tokenizer sözlərə bölür, lowercase filtri kiçik hərfə çevirir
  1 → [login, completed, successfully]
  2 → [invalid, username, email, or, password]

Inverted index:
  completed → [1]     email → [2]     invalid → [2]     login → [1]
  or → [2]            password → [2]  successfully → [1] username → [2]
```

`"password"` axtarışı lüğətdə term-i tapır və sənəd siyahısını dərhal qaytarır — cədvəli skan etmir.

Sahə tipləri:

Tip	Analiz	Nə üçün	Sıralama/aqreqasiya
`text`	Bəli (tokenize, lowercase)	Tam mətn axtarışı	Xeyr (fielddata olmadan)
`keyword`	Xeyr, dəyər olduğu kimi	Dəqiq uyğunluq, filtr, sıralama, aqreqasiya	Bəli (doc_values)
`date`	—	Vaxt aralığı sorğuları	Bəli
`long`, `integer`	—	Rəqəm, aralıq	Bəli
`object`	—	İç-içə sahələr (hər alt sahə ayrı sahədir)	—

Dinamik mapping: mapping-də olmayan sahə ilk dəfə gəldikdə ES tipini təxmin edir. Sətir üçün standart nəticə: `text` + `keyword` alt sahəsi (`field.keyword`, `ignore_above: 256`); tam ədəd → `long`; JSON obyekti → `object` və onun hər açarı ayrıca sahə. Rahatdır, lakin səhv tip təxmini və **mapping partlaması** (çoxlu fərqli açar → çoxlu sahə) riski var.

Sorğular

- **Query context** — "nə qədər uyğundur?" — relevans balı (**BM25**: term tezliyi, tərs sənəd tezliyi, sahə uzunluğu).
- **Filter context** — "uyğundurmu, bəli/xeyr?" — bal hesablanmır, nəticə cache-lənir. `bool.filter`, `term`, `range`.
- Tipik jurnal axtarışı: `bool { filter: [term serviceName, range occurredAt, range durationMs] }` + `sort: occurredAt desc`.

Səhifələmə: `from/size` sadədir, lakin `from + size ≤ index.max_result_window` (standart 10 000). Dərin səhifələmə üçün `search_after` (+ Point-in-Time).

İndeks həyat dövrü: jurnal datası üçün vaxt əsaslı indekslər və ya **data stream**-lər, **ILM** (hot → warm → delete), **rollover**. Mapping dəyişikliyi (sahə tipini dəyişmək) mövcud indeksdə mümkün deyil — yeni indeks yaradılıb **reindex** edilir, **alias** ilə klientlər yeni indeksə keçirilir.

12.3 Spring Data Elasticsearch və klient

FAKT Asılılıq ağacı: `spring-boot-starter-data-elasticsearch` → Spring Data Elasticsearch **6.1.0** → rəsmi **Elasticsearch Java Client 9.4.2** (`co.elastic.clients:elasticsearch-java`). Bu klient HTTP/JSON üzərindən ES REST API-si ilə danışır (köhnə `RestHighLevelClient` artıq istifadə olunmur). Server image-i də 9.4.2-dir (`docker-compose.yml:353`).

Spring Data qatları:

- `@Document(indexName = ...)` — sinfi indeksə bağlayır. Repository işə düşəndə indeks yoxdursa, Spring Data onu annotasiyalardan qurulan mapping ilə **yaradır** (`createIndex` standart olaraq `true`) NƏTİCƏ.
- `@Field(type = ...)` — açıq mapping. `@Field` olmayan sahələr yaradılan mapping-ə daxil edilmir və ES-in dinamik mapping-inə qalır NƏTİCƏ.
- `ElasticsearchRepository<T, ID>` — CRUD, səhifələmə, törəmə sorğular (`findByServiceName...`).
- `ElasticsearchOperations` — aşağı səviyyəli API (`NativeQuery`, `CriteriaQuery`).

12.4 Repozitoriyadakı implementasiya

Sənəd modelləri

Mənbə: log-service/src/main/java/com/casestudy/insurancecampaign/log/model/document/AuthAuditLogDocument.java:15-37

```java
@Document(indexName = "auth-audit-logs")
public class AuthAuditLogDocument {

    @Id
    private String id;

    @Field(type = FieldType.Keyword)
    private String serviceName;

    @Field(type = FieldType.Keyword)
    private String eventType;

    @Field(type = FieldType.Keyword)
    private String usernameOrEmail;

    @Field(type = FieldType.Keyword)
    private String status;

    @Field(type = FieldType.Text)
    private String message;

    @Field(type = FieldType.Date, format = DateFormat.date_time)
    private Instant occurredAt;
```

Bu, **yaxşı mapping dizaynı**dır: filtrlənən/aqreqasiya olunan sahələr `keyword` (`eventType = LOGIN_FAILURE`, `status = FAILED`), sərbəst mətn `text`, vaxt `date`. "Son bir gündə istifadəçi üzrə uğursuz login sayı" kimi aqreqasiya birbaşa mümkündür.

Mənbə: log-service/src/main/java/com/casestudy/insurancecampaign/log/model/document/SlowRequestDocument.java:13-26

```java
@Document(indexName = "slow-request-logs")
public class SlowRequestDocument {
    @Id
    private String id;
    private String serviceName;
    private String method;
    private String path;
    private int status;
    private long durationMs;
    private String traceId;
    private Map<String, String> headers;

    @Field(type = FieldType.Date, format = DateFormat.date_time)
    private Instant occurredAt;
```

NƏTİCƏ Burada yalnız `occurredAt` açıq mapping-ə malikdir. Qalan sahələr ilk sənəd indekslənəndə dinamik mapping ilə yaranır: `serviceName`, `method`, `path`, `traceId` → `text` + `.keyword`; `status`, `durationMs` → `long`; `headers` → `object` (`headers.user-agent`, `headers.x-forwarded-for` ayrıca sahələr). Nəticələr:

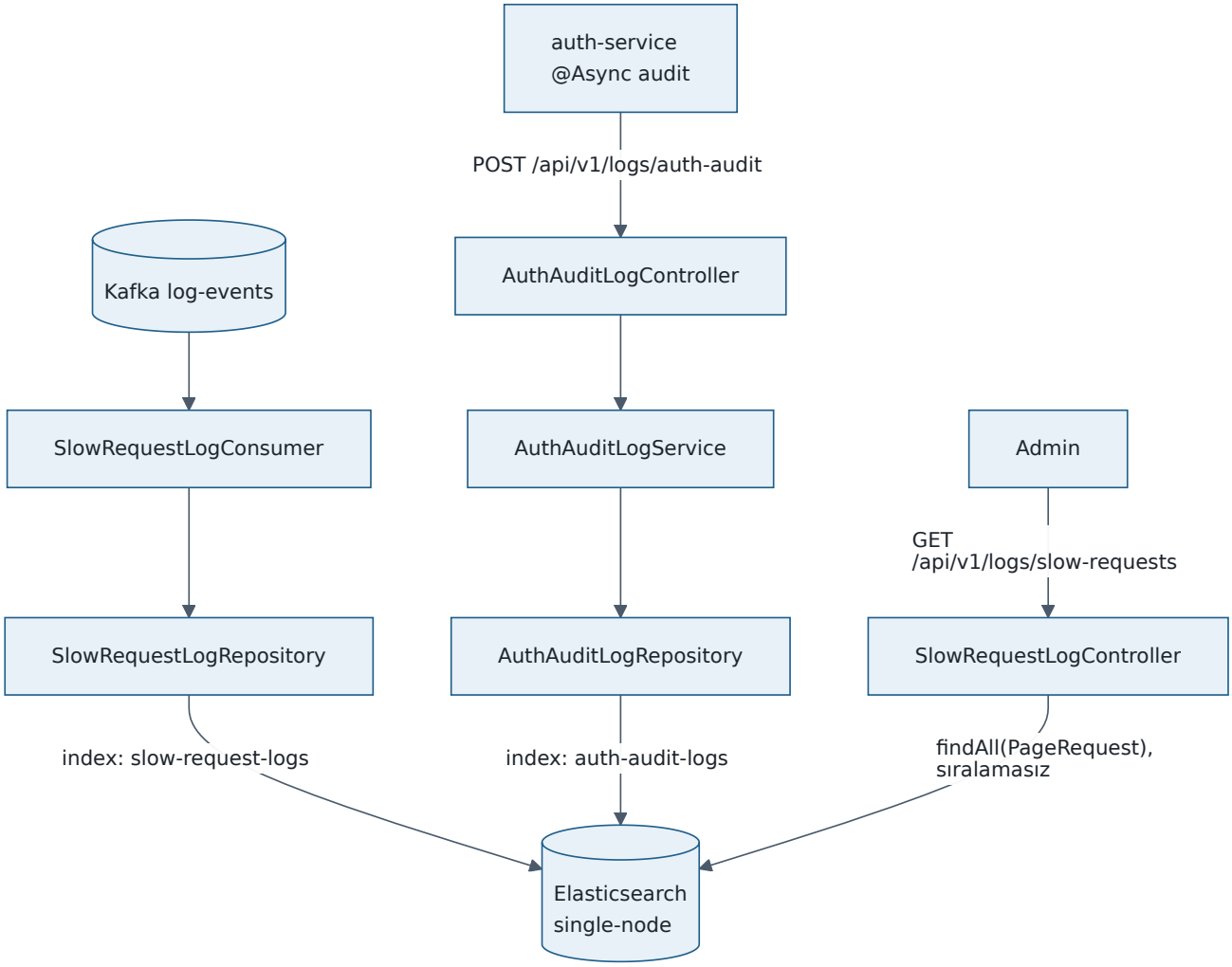
- `path` üzrə dəqiq filtr üçün `path.keyword` istifadə edilməlidir; `path` (`text`) `/api/v1/campaigns`-ı `api`, `v1`, `campaigns` tokenlərinə bölür.
- `headers` hər yeni başlıq açarı üçün yeni sahə yaradır. Filtr yalnız iki sabit açar yazdığı üçün hazırda təhlükəsizdir; ixtiyari başlıqlar yazılsaydı, mapping partlaması riski yaranardı (ES-də `index.mapping.total_fields.limit` = 1000). Belə map-lər üçün `flattened` tipi daha uyğundur.
- İki sənəd sinfi arasında üslub fərqi var — biri açıq, digəri dinamik mapping (F42).

Id strategiyası. Hər iki sənəd konstruktorda `UUID.randomUUID().toString()` id alır (`AuthAuditLogDocument.java:42-57`, `SlowRequestDocument.java:31-41`). Eyni hadisə iki dəfə indekslənərsə (Kafka təkrar çatdırılması, HTTP retry), iki sənəd yaranır. Deterministik id (`eventId`, `topic-partition-offset`) yazmanı **idempotent upsert** edərdi: eyni `_id` ilə ikinci yazı birincini əvəzləyir.

Yazma yolları

1. **Kafka → ES**: `SlowRequestLogConsumer.consume` → `SlowRequestLogRepository.save` (fəsil 11).
2. **HTTP → ES**: `AuthAuditLogController.writeAuthAudit` → `AuthAuditLogService.save` → `AuthAuditLogRepository.save` FAKT (`AuthAuditLogService.java:25-36`).

`save()` ES-in index API-sini (`PUT /<index>/_doc/<id>`) çağırır; sənəd primary shard-a yazılır, replikalara (burada yoxdur) kopyalanır. NƏTİCƏ Axtarışa görünmə refresh intervalından (standart 1 s) asılıdır; yazıdan dərhal sonra `GET /slow-requests` yeni qeydi göstərməyə bilər — bu, ES-in **near real-time** təbiətidir.



log-service-də Elasticsearch-ə iki yazma yolu və bir oxuma yolu

Oxuma yolu

Mənbə: `log-service/src/main/java/com/casestudy/insurancecampaign/log/controller/SlowRequestLogController.java:35-43`

```
@Operation(summary = "List slow requests logged because they exceeded 5 ms")
@GetMapping("/slow-requests")
@PreAuthorize("hasRole('ADMIN')")
public ApiResponse<Page<SlowRequestDocument>> getSlowRequests(
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "20") int size
) {
```

```
    return ApiResponse.success(repository.findAll(PageRequest.of(page, size)));
}
```

- Servis qatı yoxdur — controller repository-ni birbaşa çağırır. Sadə oxuma üçün qəbuledilir, lakin qatların ardıcıllığı pozulur.
- **NƏTİCƏ** `findAll(PageRequest.of(page, size))` sıralamasız `match_all` axtarışıdır; nəticələrin sırası `occurredAt`-ə görə deyil, ES-in daxili sırasıdır — istifadəçi ən son yavaş sorğuları görmür (F42). Düzəliş: `PageRequest.of(page, size, Sort.by(DESC, "occurredAt"))`.
- `page * size + size > 10 000` olduqda ES `max_result_window` xətası verir → catch-all → 500.
- Yalnız siyahı var; servis, yol, müddət, vaxt aralığı üzrə filtr yoxdur. **NƏTİCƏ** ES burada əsasən **sənəd anbarı** kimi istifadə olunur; onun əsas gücü (axtarış və aqreqasiya) API-də istifadə edilmir. Kibana (və ya ES API) ilə birbaşa sorğu mümkündür.
- Audit qeydləri üçün **oxuma endpoint-i yoxdur** — onlar yalnız ES-də birbaşa sorğu ilə görünür.

12.5 Konfiqurasiya

FAKT (`log-service/src/main/resources/application.yml:17-20`):

```
elasticsearch:
  uris: ${ELASTICSEARCH_URIS:http://localhost:9200}
  connection-timeout: ${SPRING_ELASTICSEARCH_CONNECTION_TIMEOUT:10s}
  socket-timeout: ${SPRING_ELASTICSEARCH_SOCKET_TIMEOUT:30s}
```

- `connection-timeout` — TCP bağlantısının qurulması; `socket-timeout` — cavab gözləmə. 30 s jurnal yazısı üçün uzundur: ES "donduqda" consumer thread-i hər cəhddə 30 s gözləyə bilər; 4 cəhd ~2 dəqiqəyə yaxın partisiyanı saxlayar, bu da `max.poll.interval.ms` (5 dəq) həddinə yaxınlaşır.

Compose **FAKT** (`docker-compose.yml:352-373`): `discovery.type: single-node`, `xpack.security.enabled: 'false'` (autentifikasiya və TLS yoxdur — yalnız lokal), `ES_JAVA_OPTS: -Xms512m -Xmx512m`, healthcheck klaster statusunun ən azı `yellow` olmasını gözləyir. log-service `depends_on: elasticsearch: condition: service_healthy` və `restart: on-failure:5` ilə qurulub (`docker-compose.yml:271-280`). **NƏTİCƏ** Bu birləşmə məntiqlidir: Spring Data ES repository işə düşərkən indeks yaratmağa çalışır və ES əlçatmazdırsa kontekst başlamaya bilər — restart siyasəti bunu yumşaldır.

12.6 Əsas DB və ES: məsuliyyətlərin bölgüsü

Bu layihədə ES **yeganə** saxlama yeridir (jurnal datası üçün) — sinxronizasiya problemi yoxdur. Əgər gələcəkdə kampaniyalar üçün tam mətn axtarışı lazım olsa (ES ikinci dərəcəli indeks olaraq), klassik problemlər yaranar:

Yanaşma	Necə	Ardıcıllıq	Risk
Dual write (servis həm DB-yə, həm ES-ə yazır)	Sinxron	Zəif	Biri uğursuz olsa — uyğunsuzluq
Hadisə əsaslı (campaign-events → indeksləyici consumer)	Asinxron	Son ardıcıllıq	Gecikmə, sıra, dublikat — açar və idempotent upsert ilə həll
CDC (Debezium → Kafka → ES)	Asinxron	Son ardıcıllıq	Əlavə infrastruktur
Periodik tam reindex	Batch	Gecikmiş	Böyük yük

NƏTİCƏ `campaign-events` mövzusu (açar = `campaignId`, consumer-siz) məhz ikinci variant üçün hazır mənbədir.

12.7 Yeni layihədə tətbiq

İllüstrativ nümunə (repozitoriyada yoxdur):

```
@Document(indexName = "slow-request-logs")
@Setting(refreshInterval = "5s")                                    // jurnal üçün daha az tez refresh
public class SlowRequestDocument {
    @Id private String id;                                          // deterministik: topic-partition-offs
et
    @Field(type = FieldType.Keyword) private String serviceName;
    @Field(type = FieldType.Keyword) private String method;
    @Field(type = FieldType.Keyword) private String path;
    @Field(type = FieldType.Integer) private int status;
    @Field(type = FieldType.Long) private long durationMs;
    @Field(type = FieldType.Keyword) private String traceId;
    @Field(type = FieldType.Flattened) private Map<String, String> headers;
    @Field(type = FieldType.Date, format = DateFormat.date_time) private Instant occurredAt;
}

public interface SlowRequestLogRepository extends ElasticsearchRepository<SlowRequestDocument, String> {
    Page<SlowRequestDocument> findByServiceNameAndDurationMsGreaterThanEqual(
            String serviceName, long minDuration, Pageable pageable);   // törəmə sorğu → bool filter
}
```

Əlavə praktikalar: index template və ya data stream + ILM (məs. 7 gün saxla, sonra sil) (F43); bulk indexing (`saveAll` və ya `BulkIngester`) yüksək həcmdə; Testcontainers ES ilə mapping və sorğu testləri; təhlükəsizlik (TLS + API key) istehsalda.

12.8 Uğursuzluq ssenariləri və troubleshooting

Ssenari	Mərhələ	Nəticə	Diaqnostika
ES startup-da əlçatmaz	Repository initsializasiyası (indeks yaratma)	log-service başlamaya bilər; Compose restart edir	log-service log-u, `curl :9200/_cluster/health`
ES runtime-da əlçatmaz	`repository.save`	Consumer: 4 cəhd → DLQ; audit POST: 500 → auth-service Retry/CB → WARN	DLQ mesajları, auth-service WARN log-ları
Mapping konflikti (məs. sahə əvvəl `long` , indi sətir)	Indexing	`mapper_parsing_exception` → consumer DLQ	DLQ başlığı `kafka_dlt-exception-message`
Disk dolur	ES flood-stage watermark	İndekslər `read_only_allow_delete` olur, yazılar rədd edilir	`_cat/allocation` , `_cluster/settings`
Dərin səhifə (`page=600&size=20`)	Axtarış	`max_result_window` xətası → 500	ES xəta mesajı
Yeni yazılan qeyd görünmür	Refresh	Bir saniyəyə qədər gecikmə	`?refresh=wait_for` və ya gözləmə

12.9 Müsahibə sualları

S1. `text` və `keyword` fərqi nədir? `eventType` hansı olmalıdır? Cavab: `text` analiz olunur (tokenlərə bölünür, kiçik hərfə çevrilir) — tam mətn axtarışı üçün; `keyword` olduğu kimi saxlanılır — dəqiq uyğunluq, filtr, sıralama və aqreqasiya üçün. `eventType` sabit dəyərlər toplusudur → `keyword` .

S2. Inverted index nədir? Cavab: Term → sənəd siyahısı xəritəsi. Axtarış term-i lüğətdə tapır və uyğun sənədləri dərhal alır; cədvəl skanı olmur. Analiz prosesi indeksləmə və axtarış zamanı eyni olmalıdır ki, termlər uyğun gəlsin.

S3. ES-də yazdığınız sənəd niyə dərhal axtarışda görünmür? Cavab: Yazılar əvvəlcə yaddaş buferinə və translog-a düşür; refresh (standart 1 s) yeni Lucene seqmenti açır və sənədi axtarışa görünən edir. Bu, near real-time modelidir. Lazım olduqda `refresh=wait_for`.

S4. Dinamik mapping-in riskləri? Cavab: Səhv tip təxmini (rəqəm sətir kimi gələrsə), lazımsız `text` analizləri, map-lərdən mapping partlaması, və ilk gələn sənədin tipi "dondurması" — sonrakı fərqli tiplər rədd edilir.

S5. 10 000-dən çox nəticəni necə səhifələmək olar? Cavab: `search_after` (son sənədin sıralama dəyərləri ilə) + Point-in-Time; ixrac üçün scroll (köhnəlmiş). `from/size` `max_result_window` ilə məhdudlaşır.

S6. Kafka consumer-i ES-ə yazırsa, dublikatları necə önləyərsiniz? Cavab: Sənəd id-sini deterministik edərək (hadisə id-si və ya `topic-partition-offset`) — ikinci yazı birincini əvəzləyir (idempotent upsert). Random UUID dublikat yaradır.

12.10 Xülasə

- ES tam mətn axtarışı və analitika üçün inverted index əsaslı paylanmış mühərrikdir; near real-time, tranzaksiyasız.
- log-service iki indeksə yazır: `auth-audit-logs` (açıq, düzgün mapping) və `slow-request-logs` (əsasən dinamik mapping).
- Yazma: Kafka consumer və HTTP audit; oxuma: yalnız sıralamasız səhifələnmiş siyahı.
- Təkmilləşdirmələr: deterministik id, açıq mapping, sıralama və filtrlər, ILM/saxlama siyasəti.
- Növbəti fəsil dashboard ↔ campaign arasındakı gRPC kanalını öyrədir.

13. gRPC və Protocol Buffers

13.1 Motivasiya: servislərarası sinxron çağırışda kontrakt

dashboard-service-ə campaign-service-dən iki şey lazımdır: statuslar üzrə saylar və bir kampaniyanın tarixçəsi. REST ilə də etmək olardı; layihə bunun üçün **gRPC** seçib. gRPC-nin təklif etdikləri:

- **Kontrakt-öncəlikli (contract-first) dizayn** — servis və mesajlar `.proto` faylında təsvir olunur, klient və server kodu ondan generasiya olunur; tip uyğunsuzluğu kompilyasiya zamanı aşkarlanır.
- **Binary serializasiya (Protobuf)** — JSON-dan kiçik və sürətli.
- **HTTP/2** — bir TCP bağlantısında çoxlu paralel axın (multiplexing), başlıq sıxılması, axın (streaming).
- **Zəngin semantika** — deadline, ləğv (cancellation), standart status kodları, metadata.

Bədəli: brauzerdən birbaşa istifadə çətindir (gRPC-Web proxy lazımdır), insan oxuya bilən deyil (curl əvəzinə `grpcurl`), HTTP cache/CDN infrastrukturu tətbiq olunmur, debugging üçün əlavə alətlər tələb edir.

13.2 Protocol Buffers

Layihənin kontraktı **FAKT** (`grpc-contract/src/main/proto/campaign.proto:1-36`):

Mənbə: `grpc-contract/src/main/proto/campaign.proto:1-36`

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "com.casestudy.insurancecampaign.grpc";
option java_outer_classname = "CampaignProto";

service CampaignGrpcService {
  rpc GetCampaignStatistics(CampaignStatisticsRequest) returns (CampaignStatisticsResponse);
  rpc GetCampaignHistory(CampaignHistoryRequest) returns (CampaignHistoryResponse);
}

message CampaignStatisticsRequest {}

message CampaignStatisticsResponse {
  repeated CampaignStatusCount counts = 1;
}

message CampaignStatusCount {
  string status = 1;
  int64 count = 2;
}

message CampaignHistoryRequest {
  string campaignId = 1;
}

message CampaignHistoryResponse {
  repeated CampaignHistoryItem items = 1;
}

message CampaignHistoryItem {
  string previousStatus = 1;
  string newStatus = 2;
  string reason = 3;
  string changedAt = 4;
}
```

Sintaksis və semantika

Element	Mənası
`syntax = "proto3"`	Protobuf 3 sintaksisi: sahələr standart olaraq opsionaldır, `required` yoxdur
`java_multiple_files = true`	Hər mesaj ayrıca Java faylı/sinfi olur (bir böyük xarici sinif yerinə)
`java_package`	Generasiya olunan paket
`service` + `rpc`	Uzaqdan çağırıla bilən metodlar; hər biri bir sorğu və bir cavab mesajı (unary)
`repeated`	Siyahı
`= 1`, `= 2`	**Sahə nömrəsi** — wire formatında sahənin identifikasiyası; adı deyil
Boş mesaj `CampaignStatisticsRequest {}`	Parametrsiz sorğu; gələcəkdə sahə (məs. filtr) əlavə etmək üçün yer saxlayır — boş mesaj yerinə `google.protobuf.Empty` istifadə etməkdən daha genişləndirilə biləndir

Wire formatı (qısa): hər sahə `tag = (sahə_nömrəsi << 3) | wire_type` ilə kodlanır, ardınca dəyər (varint, uzunluq prefiksli bayt və s.). Sahə adları wire-a yazılmır. Buna görə:

- sahənin **adını** dəyişmək wire baxımından təhlükəsizdir (JSON transkodlamada isə yox);
- **nömrəni** dəyişmək və ya köhnə nömrəni yeni məna ilə təkrar istifadə etmək köhnə klientləri sındırır — silinən sahələrin nömrəsi `reserved` elan edilməlidir;
- yeni sahə əlavə etmək geriyə və irəliyə uyğundur: köhnə klient tanımadığı sahəni ötürür (unknown fields), yeni klient çatışmayan sahə üçün standart dəyər görür;
- tipi dəyişmək (`int64` → `string`) uyğun deyil.

proto3-də "null" yoxdur

Skalyar sahələrin standart dəyəri var: `string` → `""`, `int64` → `0`. "Dəyər yoxdur" ilə "boş sətir" fərqlənmir (`optional` açar sözü və ya wrapper tipləri istifadə edilmədikcə). Layihədə məhz bu problem həll olunur **FAKT**:

- server `previousStatus == null` olduqda `""` yazır (`CampaignGrpcEndpoint.java:39`);
- klient boş sətri yenidən `null`-a çevirir (`CampaignGrpcClient.java:60`, `95-97`).

Bu, **sərhəddə konvensiya**dır və hər iki tərəfdə yazılmalıdır. `optional string previousStatus = 1;` (proto3 `optional`) `hasPreviousStatus()` metodunu verərdi və konvensiyaya ehtiyac qalmazdı.

Kontrakt dizayn qeydləri **NƏTİCƏ**

- **Vaxt sətir kimi** (`changedAt` — ISO-8601 `Instant.toString()`). Protobuf-un standart `google.protobuf.Timestamp` tipi daha tipli və dildən asılı olmayan seçimdir.
- **Status sətir kimi** — `enum CampaignStatus { ... }` kompilyasiya zamanı yoxlanılan dəyərlər verərdi; lakin enum dəyişiklikləri (yeni status) köhnə klientlərdə tanınmayan dəyər kimi gələr. Sətir daha çevikdir, daha az tiplidir.
- **Sahə adları camelCase** (`campaignId`) — Protobuf üslub bələdçisi `snake_case` (`campaign_id`) tövsiyə edir; generasiya olunan Java getter-i hər iki halda `getCampaignId()` olur. Üslub məsələsidir.
- **Versiya yoxdur** — paket (`package campaign.v1;`) təyin olunmayıb. Böyük dəyişiklik lazım olduqda `v2` paketi ilə paralel servis təqdim etmək praktikadır.

13.3 Generasiya olunan kod

`protobuf-maven-plugin` (fəsil 3.8) `com.casestudy.insurancecampaign.grpc` paketində yaradır:

Generasiya olunan	Rolu
`CampaignStatisticsRequest`, `CampaignHistoryItem`, ...	Dəyişməz mesaj sinifləri + `Builder` (`newBuilder().setX(...).build()`)
`CampaignProto`	Fayl deskriptoru (reflection üçün)
`CampaignGrpcServiceGrpc.CampaignGrpcServiceImplBase`	Server tərəfi üçün abstrakt baza; metodlar override edilir
`CampaignGrpcServiceGrpc.CampaignGrpcServiceBlockingStub`	Sinxron klient stub-u
`CampaignGrpcServiceGrpc.CampaignGrpcServiceStub`	Asinxron (callback / `StreamObserver`) stub
`CampaignGrpcServiceGrpc.CampaignGrpcServiceFutureStub`	`ListenableFuture` qaytaran stub

13.4 gRPC runtime anlayışları

Anlayış	İzah
Channel (`ManagedChannel`)	Hədəfə məntiqi bağlantı; ad həlli, yük balansı, yenidən qoşulma; ağırdır — paylaşılmalıdır
Stub	Kanal üzərində yüngül klient obyekti; deadline/metadata konfiqurasiyası stub-a bağlanır
RPC növləri	**Unary** (1 sorğu → 1 cavab), **server streaming**, **client streaming**, **bidirectional streaming**
Deadline	Çağırışın mütləq son vaxtı; zəncir boyu ötürülür; keçərsə `DEADLINE_EXCEEDED`
Cancellation	Klient ləğv edərsə server `Context.isCancelled()` görür
Status kodu	`OK`, `INVALID_ARGUMENT`, `NOT_FOUND`, `ALREADY_EXISTS`, `PERMISSION_DENIED`, `UNAUTHENTICATED`, `UNAVAILABLE`, `DEADLINE_EXCEEDED`, `INTERNAL`, `UNKNOWN` və s.
Metadata	Açar-dəyər başlıqları (HTTP/2 headers): autentifikasiya token-i, trace konteksti
Interceptor	Server/klient tərəfli kəsişən narahatlıqlar (auth, logging, metrics, tracing)
Ad həlli	`static://host:port`, `dns:///host:port`, xüsusi resolver-lər (məs. `discovery:///service`)

Status kodlarının seçilməsi — klientin reaksiyası koddan asılıdır: `UNAVAILABLE` təkrar edilə bilər, `INVALID_ARGUMENT` və `NOT_FOUND` təkrar edilməməlidir, `UNKNOWN` heç bir məlumat vermir.

13.5 net.devh grpc-spring-boot-starter

Layihə **net.devh** `grpc-server-spring-boot-starter` / `grpc-client-spring-boot-starter` **3.1.0.RELEASE** istifadə edir (`pom.xml:37`). Bu, icma (community) layihəsidir. Rəsmi alternativ **Spring gRPC** (`spring-grpc`) layihəsidir. NAMƏLUM net.devh 3.1.0-ın Spring Boot 4 ilə rəsmi uyğunluğu repozitoriyadan müəyyən edilə bilmir; layihə README-si işlədiyini göstərir (gRPC versiya uyğunsuzluğu bölməsi), lakin mən bunu runtime-da yoxlamamışam.

Starter-in etdikləri [FAKT, kitabxana mənbəsi + layihə konfiqurasiyası]:

Xüsusiyyət	Nə edir	Layihədə
`@GrpcService`	Bean-i Spring komponenti kimi qeydiyyatdan keçirir və gRPC server-ə servis kimi əlavə edir	`CampaignGrpcEndpoint`
`grpc.server.port`	Ayrı Netty (`grpc-netty-shaded`) server portu	9090
Health servisi	`grpc.health.v1.Health` — `grpc.server.health-service-enabled` (standart true)	Aktiv
Reflection servisi	Klientlər proto-nu serverdən soruşa bilir (`grpcurl list`) — `reflection-service-enabled` (standart true)	Aktiv
Tracing	`GrpcServerMicrometerTraceAutoConfiguration` — `ObservationRegistry` bean-i varsa	**NƏTİCƏ** Aktiv olmalıdır
Eureka metadata	`GrpcMetadataEurekaConfiguration` gRPC portunu Eureka metadata-sına yazır	Klient `discovery:///` istifadə etmir
Security	`AuthenticationManager` bean-i varsa	Aktiv deyil (F04)
`@GrpcAdvice`	Exception → `Status` xəritələnməsi	İstifadə olunmur
`@GrpcClient("name")`	`grpc.client.<name>.*` konfiqurasiyasından kanal + stub	`CampaignGrpcClient`

13.6 Server: `CampaignGrpcEndpoint`

Mənbə: `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/grpc/CampaignGrpcEndpoint.java:16-48`

```
@GrpcService
public class CampaignGrpcEndpoint extends CampaignGrpcServiceGrpc.CampaignGrpcServiceImplBase {
    private final CampaignService campaignService;

    public CampaignGrpcEndpoint(CampaignService campaignService) {
        this.campaignService = campaignService;
    }

    @Override
    public void getCampaignStatistics(CampaignStatisticsRequest request, StreamObserver<CampaignStatistic
sResponse> responseObserver) {
        CampaignStatisticsResponse.Builder response = CampaignStatisticsResponse.newBuilder();
        campaignService.getStatistics().forEach((status, count) -> response.addCounts(
                CampaignStatusCount.newBuilder().setStatus(status).setCount(count).build()
        ));
        responseObserver.onNext(response.build());
        responseObserver.onCompleted();
    }

    @Override
    public void getCampaignHistory(CampaignHistoryRequest request, StreamObserver<CampaignHistoryResponse
> responseObserver) {
        CampaignHistoryResponse.Builder response = CampaignHistoryResponse.newBuilder();
        campaignService.getHistory(UUID.fromString(request.getCampaignId())).forEach(history -> response.
addItems(
                CampaignHistoryItem.newBuilder()
                        .setPreviousStatus(history.getPreviousStatus() == null ? "" : history.getPrevious
Status().name())
                        .setNewStatus(history.getNewStatus().name())
                        .setReason(history.getReason())
                        .setChangedAt(history.getChangedAt().toString())
                        .build()
        ));
```

```
        responseObserver.onNext(response.build());
        responseObserver.onCompleted();
    }
}
```

Mexanika:

- Server metodu `StreamObserver` ilə işləyir: unary üçün bir `onNext` + `onCompleted`; xəta üçün `onError(Status...asRuntimeException())`.
- Endpoint REST controller-in etdiyini edir — protokol tərcüməsi — və biznes məntiqini təkrarlamır: eyni `CampaignService` (`@Transactional`, `@Cacheable` daxil olmaqla) çağırılır. Statistika çağırışı Redis cache-dən qayıda bilər.
- Statistika xəritəsi `HashMap`-dır — `counts` siyahısının sırası qeyri-müəyyəndir; dashboard sıranı özü təyin edir.
- **Avtorizasiya yoxdur**: REST-də `/statistics` və `/history` yalnız ADMIN üçündür, gRPC-də isə hər kəs üçün. Eyni datanın iki fərqli giriş qaydası — **icazə modeli uyğunsuzluğu** (F04).

Xəta yolları (F19)

[NƏTİCƏ, grpc-java və net.devh davranışı] Server metodu istisna atarsa və `@GrpcAdvice` yoxdursa, grpc-java çağırışı `Status.UNKNOWN` ilə bağlayır. Layihədə:

- Mövcud olmayan kampaniya → `ResourceNotFoundException` → `UNKNOWN` (`NOT_FOUND` olmalı idi).
- Səhv UUID sətri → `IllegalArgumentException` (`UUID.fromString`) → `UNKNOWN` (`INVALID_ARGUMENT` olmalı idi). Dashboard path dəyişənini artıq `UUID` kimi qəbul etdiyi üçün bu yol praktikada yalnız birbaşa gRPC klientlərindən gələ bilər.
- DB xətası → `UNKNOWN`.

13.7 Klient: `CampaignGrpcClient`

Mənbə: dashboard-service/src/main/java/com/casestudy/insurancecampaign/dashboard/client/CampaignGrpcClient.java:22-68

```
@Component
public class CampaignGrpcClient {

    private static final List<String> DASHBOARD_VISIBLE_STATUSES = List.of(
            "PENDING_APPROVAL",
            "ACTIVE",
            "DEACTIVE"
    );

    @GrpcClient("campaign-service")
    private CampaignGrpcServiceGrpc.CampaignGrpcServiceBlockingStub campaignStub;

    // ... (buraxılıb: Javadoc)
    public DashboardStatisticsResponse getStatistics() {
        var response = campaignStub.getCampaignStatistics(CampaignStatisticsRequest.newBuilder().build
());
        return toDashboardStatisticsResponse(response);
    }

    // ... (buraxılıb: Javadoc)
    public DashboardCampaignHistoryResponse getCampaignHistory(UUID campaignId) {
        var request = CampaignHistoryRequest.newBuilder()
                .setCampaignId(campaignId.toString())
                .build();

        var response = campaignStub.getCampaignHistory(request);

        List<DashboardCampaignHistoryItemResponse> items = response.getItemsList()
```

```
                .stream()
                .map(item -> new DashboardCampaignHistoryItemResponse(
                        blankToNull(item.getPreviousStatus()),
                        item.getNewStatus(),
                        item.getReason(),
                        Instant.parse(item.getChangedAt())
                ))
                .toList();

        return new DashboardCampaignHistoryResponse(items);
    }
```

- **Blocking stub** — çağıran thread cavabı gözləyir. Tomcat sorğusu virtual thread-də işlədiyi üçün bloklama ucuzdur.
- **Statusların süzgəcdən keçirilməsi** (`toDashboardStatisticsResponse` , `:76-87`) — yalnız 3 "real" status, sabit sırada (`LinkedHashMap`), çatışmayanlar 0. `DUPLICATE` gizlədilir. Bu, **anti-corruption layer** nümunəsidir: dashboard öz modelini campaign-in daxili enum-undan qoruyur.
- **Deadline yoxdur** (F19). NƏTİCƏ gRPC-də deadline standart olaraq sonsuzdur. campaign-service cavab vermədən "asılı" qalsa (DB kilidi, uzun GC), dashboard thread-i qeyri-müəyyən müddət gözləyir. Tövsiyə: `campaignStub.withDeadlineAfter(2, TimeUnit.SECONDS).getCampaignStatistics(...)` — hər çağırışda (stub-un özündə deadline mütləq vaxtdır, ona görə hər çağırış üçün yeni stub törədilməlidir).
- **Xətaların tərcüməsi yoxdur** — `StatusRuntimeException` birbaşa `GlobalExceptionHandler` -in catch-all-ına düşür → 500 və mesaj `"UNKNOWN"` / `"UNAVAILABLE: io exception"` . Kampaniya tapılmadıqda klient 404 əvəzinə 500 alır.

Konfiqurasiya

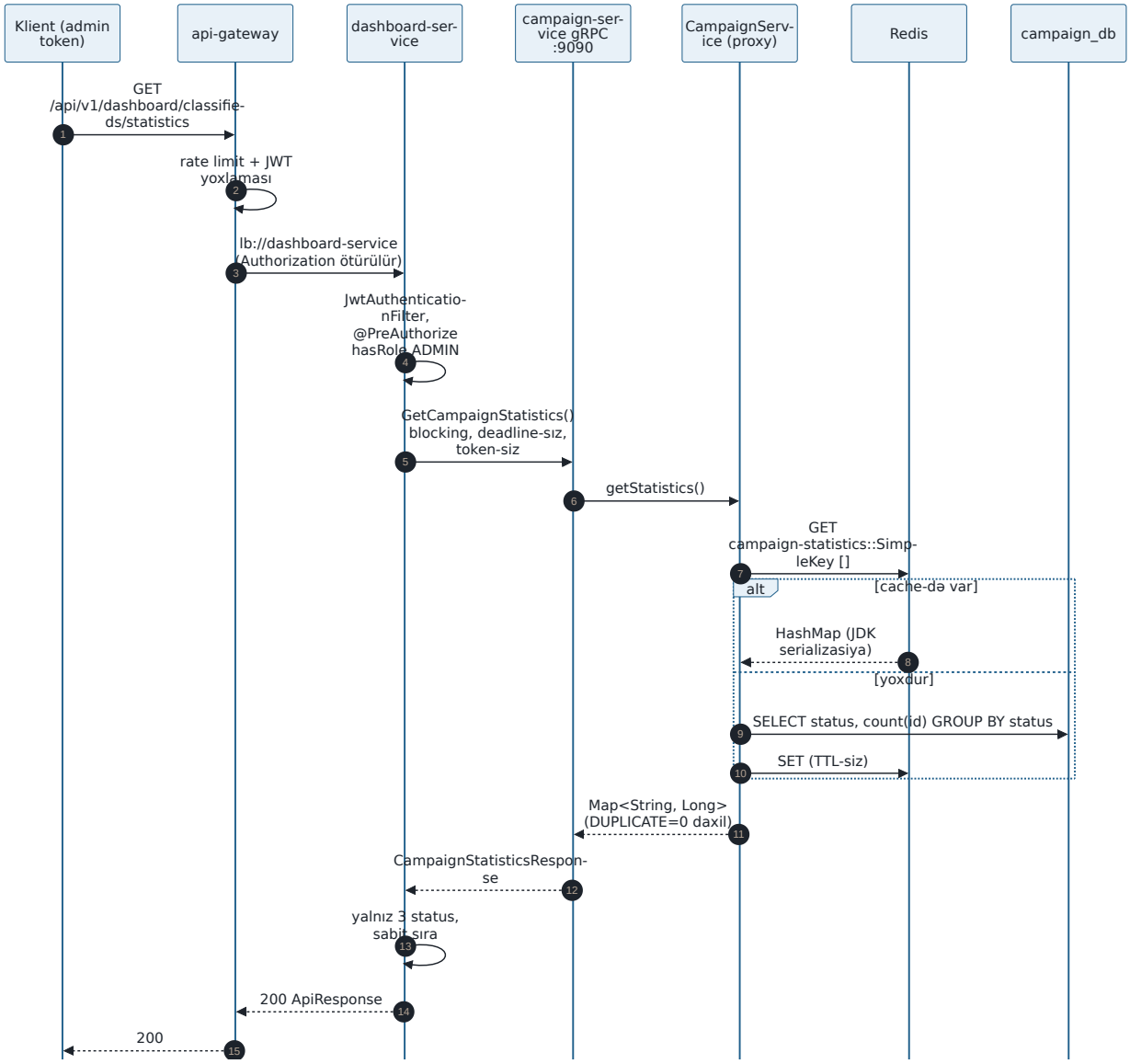
Mənbə: `dashboard-service/src/main/resources/application.yml:11-15`

```
grpc:
  client:
    campaign-service:
      address: ${CAMPAIGN_GRPC_ADDRESS:static://localhost:9090}
      negotiation-type: plaintext
```

- `campaign-service` — `@GrpcClient("campaign-service")` adı ilə uyğun gəlir.
- `static://` — sabit ünvan; Compose-da `static://campaign-service:9090` (`docker-compose.yml:225`). NƏTİCƏ campaign-service-in bir neçə instansı olsa, yük balansı olmaz. net.devh `discovery:///campaign-service` ünvanı ilə Eureka-dakı instansları və onların gRPC portunu (Eureka metadata) istifadə edə bilərdi; bunun üçün klient tərəfdə load-balancing siyasəti də (`round_robin`) seçilməlidir — gRPC standart olaraq `pick_first` istifadə edir.
- `negotiation-type: plaintext` — TLS yoxdur, HTTP/2 cleartext (h2c). Daxili şəbəkədə belə məlumat şifrəsiz gedir.

Server tərəfi: `grpc.server.port: ${CAMPAIGN_GRPC_PORT:9090}` (`campaign-service/src/main/resources/application.yml:29-31`); testlərdə `grpc.server.port = -1` — net.devh-də bu, server-in başladılmamasıdır (`campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractTestContainerConfiguration.java:41`).

13.8 İcra axını: dashboard statistikası



Dashboard statistikası — REST → gRPC → cache/DB

Uğursuzluq yolları: campaign-service düşüb → `UNAVAILABLE` → 500; campaign-service yavaşdır → dashboard sonsuz gözləyir; Redis düşüb → `@Cacheable` xətası → `UNKNOWN` → 500.

13.9 Test

`CampaignGrpcClientTest` stub-u Mockito ilə mock edir və `ReflectionTestUtils` ilə sahəyə qoyur (`CampaignGrpcClientTest.java:33-43`). Bu, mapping məntiqini (süzgəc, sıfır dəyərlər, `""` → `null`) yoxlayır, lakin real gRPC serializasiyasını, status kodlarını, deadline-ı yoxlamır. Server endpoint-i (`CampaignGrpcEndpoint`) üçün test yoxdur.

İllüstrativ nümunə (repozitoriyada yoxdur) — in-process server ilə həqiqi kontrakt testi (`grpc-inprocess` starter-lə classpath-dədir):

```
String name = InProcessServerBuilder.generateName();
Server server = InProcessServerBuilder.forName(name).directExecutor()
        .addService(new CampaignGrpcEndpoint(campaignService)).build().start();
ManagedChannel channel = InProcessChannelBuilder.forName(name).directExecutor().build();
var stub = CampaignGrpcServiceGrpc.newBlockingStub(channel);
```

```
when(campaignService.getHistory(any())).thenThrow(new ResourceNotFoundException("x"));
assertThatThrownBy(() → stub.getCampaignHistory(CampaignHistoryRequest.newBuilder()
        .setCampaignId(UUID.randomUUID().toString()).build()))
    .isInstanceOf(StatusRuntimeException.class)
    .extracting(e → ((StatusRuntimeException) e).getStatus().getCode())
    .isEqualTo(Status.Code.UNKNOWN);   // @GrpcAdvice əlavə edildikdən sonra NOT_FOUND gözlənilməlidir
```

13.10 Yeni layihədə tətbiq

İllüstrativ nümunə (repozitoriyada yoxdur):

```
syntax = "proto3";
package campaign.v1;
import "google/protobuf/timestamp.proto";

service CampaignQueryService {
  rpc GetStatistics(GetStatisticsRequest) returns (GetStatisticsResponse);
  rpc GetHistory(GetHistoryRequest) returns (GetHistoryResponse);
}
message GetHistoryRequest { string campaign_id = 1; }
message HistoryItem {
  optional string previous_status = 1;          // "yoxdur" ≠ ""
  string new_status = 2;
  string reason = 3;
  google.protobuf.Timestamp changed_at = 4;
  reserved 5; reserved "legacy_field";          // silinmiş sahə
}
```

```
@GrpcAdvice
class CampaignGrpcAdvice {
    @GrpcExceptionHandler(ResourceNotFoundException.class)
    Status notFound(ResourceNotFoundException e) { return Status.NOT_FOUND.withDescription(e.getMessage
()); }

    @GrpcExceptionHandler(IllegalArgumentException.class)
    Status invalid(IllegalArgumentException e) { return Status.INVALID_ARGUMENT.withDescription(e.getMess
age()); }
}

// Klient: hər çağırışda deadline, status → domen istisnası
try {
    return stub.withDeadlineAfter(2, TimeUnit.SECONDS).getHistory(request);
} catch (StatusRuntimeException e) {
    throw switch (e.getStatus().getCode()) {
        case NOT_FOUND → new ResourceNotFoundException("Campaign not found: " + id);
        case UNAVAILABLE, DEADLINE_EXCEEDED → new DownstreamUnavailableException("campaign-service", e);
// 503
        default → e;
    };
}
```

Əlavə: autentifikasiya üçün klient interceptor-u `authorization` metadata-sını ötürür, server interceptor-u yoxlayır; TLS/mTLS; `discovery:///` + `round_robin`; Resilience4j CircuitBreaker gRPC klientini də əhatə edə bilər.

13.11 gRPC və REST müqayisəsi

Meyar	gRPC	REST/JSON
Kontrakt	`.proto`, kod generasiyası, sərt tiplər	OpenAPI (opsional), çox vaxt sonradan
Format	Binary Protobuf	Mətn JSON

Meyar	gRPC	REST/JSON
Nəqliyyat	HTTP/2 (məcburi)	HTTP/1.1 və ya HTTP/2
Streaming	Doğma, 4 növ	SSE, WebSocket ilə ayrıca
Brauzer	gRPC-Web proxy tələb olunur	Doğma
Alətlər	`grpcurl`, Postman gRPC	curl, hər yerdə
Xəta modeli	Status kodları + detallar	HTTP statusları + gövdə
Uyğun ssenari	Daxili, yüksək həcmli, tipli servislərarası API	İctimai API, brauzer klientləri, sadə inteqrasiya

NƏTİCƏ Bu layihədə yük kiçikdir; gRPC seçimi performansdan çox texnologiya nümayişi və kontrakt-öncəlikli dizayn üçün məntiqlidir. Daxili API-nin brauzerdən istifadə edilməməsi və tipli kontrakt gRPC-ni uyğun seçim edir.

13.12 Müsahibə sualları

S1. Protobuf-da sahə nömrələri niyə vacibdir və sahəni necə təhlükəsiz silmək olar? Cavab: Wire formatında sahə adı deyil, nömrəsi yazılır. Nömrəni dəyişmək və ya təkrar istifadə etmək köhnə klientləri səhv deserializasiyaya aparır. Silinən sahənin nömrəsi və adı `reserved` elan edilməlidir.

S2. proto3-də `null` necə ifadə olunur? Layihədə necə həll olunub? Cavab: Skalyarların standart dəyəri var və `null` yoxdur. `optional` (presence tracking) və ya wrapper tiplər (`google.protobuf.StringValue`) istifadə olunur. Layihədə konvensiya: server `null`-u `""`, klient `""`-ni `null` edir.

S3. gRPC deadline nədir və niyə həmişə təyin etməlisiniz? Cavab: Çağırışın mütləq son vaxtı; keçərsə klient `DEADLINE_EXCEEDED` alır, server ləğvdən xəbərdar olur və deadline zəncir boyu ötürülür. Təyin edilməsə, asılı qalan server klientin thread-lərini və resurslarını qeyri-müəyyən müddət tutur — kaskad uğursuzluq.

S4. gRPC server metodu istisna atarsa klient nə görür? Cavab: Exception-ı statusa çevirən mexanizm (net.devh `@GrpcAdvice`, interceptor) yoxdursa, `Status.UNKNOWN`. Klient səbəbi ayırd edə bilmir, təkrar etmək lazım olub-olmadığını bilmir.

S5. Eyni datanı REST və gRPC ilə açmaqda hansı risk var? Cavab: İki giriş nöqtəsinin icazə, validasiya və xəta modellərinin ayrılması. Layihədə REST ADMIN tələb edir, gRPC isə autentifikasiyasızdır. Qaydalar servis qatında və ya ortaq interceptor-larda olmalıdır.

S6. gRPC-də yük balansı necə işləyir? Cavab: Uzunömürlü HTTP/2 bağlantısı səbəbindən L4 balanslayıcı bağlantıları balanslayır, sorğuları yox. Buna görə ya klient tərəfli balans (ad həlli bütün ünvanları qaytarır + `round_robin` siyasəti), ya da L7 proxy (Envoy) istifadə olunur. Layihədə static tək ünvan var — balans yoxdur.

13.13 Xülasə

- `campaign.proto` iki unary RPC təyin edir; kod Maven build-də generasiya olunur.
- Server endpoint REST ilə eyni servis qatını çağırır; klient blocking stub-dır və nəticəni dashboard modelinə tərcümə edir (süzgəc, `""` ↔ `null`).
- Boşluqlar: deadline yoxdur, exception → status xəritəsi yoxdur, autentifikasiya yoxdur, plaintext, static ünvan.
- Növbəti fəsil digər sinxron kanalı — auth → log HTTP Interface klientini və Resilience4j-ni öyrədir.

14. Servislərarası HTTP: HTTP Interface klientləri, Feign müqayisəsi və Resilience4j

14.1 Motivasiya: uzaq çağırış heç vaxt lokal çağırış deyil

Lokal metod çağırışı ya uğurlu olur, ya da istisna atır. Uzaq çağırışın daha çox nəticəsi var: uğur, xəta cavabı, timeout (server işi edib-etmədiyi bilinmir), bağlantının rədd edilməsi, yavaş cavab, qismən uğursuzluq. "Paylanmış sistemlərin yanlış fərziyyələri" (fallacies of distributed computing) — şəbəkə etibarlıdır, gecikmə sıfırdır və s. — məhz bu fərqi unutmaqdan doğur.

Layihədə servislərarası sinxron HTTP çağırışı bir yerdə var: **auth-service → log-service** audit hadisəsi. Bu, ikinci dərəcəli yan təsirdir: log-service düşsə belə login işləməlidir. Bu fəsil həmin çağırışın necə qurulduğunu və onu qoruyan **Resilience4j** nümunələrini (circuit breaker, retry, bulkhead) öyrədir.

14.2 Spring-də HTTP klient seçimləri

Klient	Model	Vəziyyət (Spring 7 / Boot 4)	Qeyd
`RestTemplate`	Sinxron, şablon	Saxlanılır, yeni inkişaf yoxdur	Köhnə kod
`RestClient`	Sinxron, fluent	Tövsiyə olunan sinxron klient (Spring 6.1+)	Virtual thread-lərlə ideal
`WebClient`	Reaktiv	WebFlux tətbiqləri üçün	Gateway `GatewayOpenApiController` istifadə edir
HTTP Interface (`@HttpExchange`)	Deklarativ interfeys, proxy	Spring Framework 6+, Spring 7-də **HTTP Service registry** (`@ImportHttpServices`)	**Bu layihədə auth → log**
OpenFeign (Spring Cloud OpenFeign)	Deklarativ interfeys	Spring Cloud layihəsi; "feature complete" elan edilib, yeni layihələr üçün HTTP Interface tövsiyə olunur	**Repozitoriyada yoxdur**

FAKT Repozitoriyada Feign istifadə olunmur (heç bir `@FeignClient`, `spring-cloud-starter-openfeign` yoxdur). Feign haqqında bölmə müqayisə üçündür.

REST və Feign/HTTP Interface fərqi — REST arxitektura üslubudur (resurslar, metodlar, statuslar); Feign və HTTP Interface isə REST (və ya istənilən HTTP) API-ni **Java interfeysi kimi çağırmaq** üçün klient tərəfli abstraksiyadır. Biri API dizaynı, digəri klient implementasiyası haqqındadır.

14.3 HTTP Interface + HTTP Service registry (Spring Framework 7)

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/client/LogServiceHttpClient.java:14-33

```
@HttpExchange("/api/v1/logs")
public interface LogServiceHttpClient {

    // ... (buraxılıb: Javadoc)
    @PostExchange("/auth-audit")
    @CircuitBreaker(name = "logServiceHttpClient")
    @Retry(name = "logServiceHttpClient")
    @Bulkhead(name = "logServiceHttpClient", type = Bulkhead.Type.SEMAPHORE)
```

```
    void sendAuthAuditLog(@RequestBody AuthAuditLog request);

}
```

İnterfeysin bean-ə çevrilməsi **FAKT** (`AuthServiceApplication.java:19`):

```
@ImportHttpServices(group = "log-service", types = LogServiceHttpClient.class)
```

Mexanizm [NƏTİCƏ, Spring Framework 7 / Boot 4 HTTP Service Client dəstəyi]:

1. `@ImportHttpServices` interfeysi `log-service` adlı **qrupa** qeydiyyatdan keçirir.
2. Boot (`spring-boot-starter-restclient`) hər qrup üçün bir `RestClient` qurur və onu `spring.http.serviceclient.<qrup>.*` xassələri ilə konfiqurasiya edir — burada `base-url` (`auth-service/src/main/resources/application.yml:21-24`).
3. `HttpServiceProxyFactory` interfeys üçün JDK dynamic proxy yaradır; metod çağırışı annotasiyalara görə HTTP sorğusuna çevrilir: `POST {base-url}/api/v1/logs/auth-audit` , gövdə = `AuthAuditLog` JSON.
4. `RestClientHttpServiceGroupConfigurer` bean-ləri qrupun `RestClient.Builder` -ini fərdiləşdirir — layihədə hər sorğuya daxili JWT əlavə olunur (`LogServiceHttpClientConfiguration.java:32-48` , fəsil 10.7).

Kontrakt uyğunluğu: auth-service `AuthAuditLog` record-unu göndərir (`AuthAuditLog.java:15-22`), log-service `AuthAuditLogRequest` record-unu qəbul edir (`AuthAuditLogRequest.java:15-22`) — sahə adları və sıraları eynidir, lakin **ayrı siniflərdir**. **NƏTİCƏ** Bu, servislərin kod səviyyəsində bir-birindən asılı olmamasını təmin edir (yaxşı), lakin kontrakt sinxronluğu yalnız insan nəzarətinə qalır — sahə adı dəyişsə, səhv compile vaxtı deyil, runtime-da (`null` sahə) üzə çıxar. Kafka hadisələri isə `common` modulundakı paylaşılan record-lardan istifadə edir. İki fərqli yanaşmanın müqayisəsi müsahibə üçün yaxşı mövzudur.

Timeout-lar konfiqurasiya olunmayıb. **NƏTİCƏ** Nə `spring.http.clients.*` / `spring.http.serviceclient.log-service.*` timeout xassələri, nə də Resilience4j `TimeLimiter` var. Klient kitabxanasının standart timeout-ları tətbiq olunur; bəzi klientlərdə oxu timeout-u standart olaraq yoxdur. Asılı qalan log-service virtual thread-ləri və bulkhead icazələrini qeyri-müəyyən müddət tuta bilər. **Hər uzaq çağırışın açıq connect və read timeout-u olmalıdır.**

Kəşf istifadə olunmur — `base-url` static-dir (`http://log-service:8084` Compose-da). log-service-in bir neçə instansı olsa, yük balansı olmaz (F31).

14.4 Asinxron icra: `@Async` + virtual thread-lər

`AuthService` audit metodlarını tranzaksiya daxilində çağırır, lakin metodlar `@Async` -dir (`AuthAuditService.java:51-135`) — çağırış dərhal qayıdır, iş `applicationTaskExecutor` -da (hər tapşırığa yeni virtual thread, fəsil 4.5) icra olunur. Nəticələr:

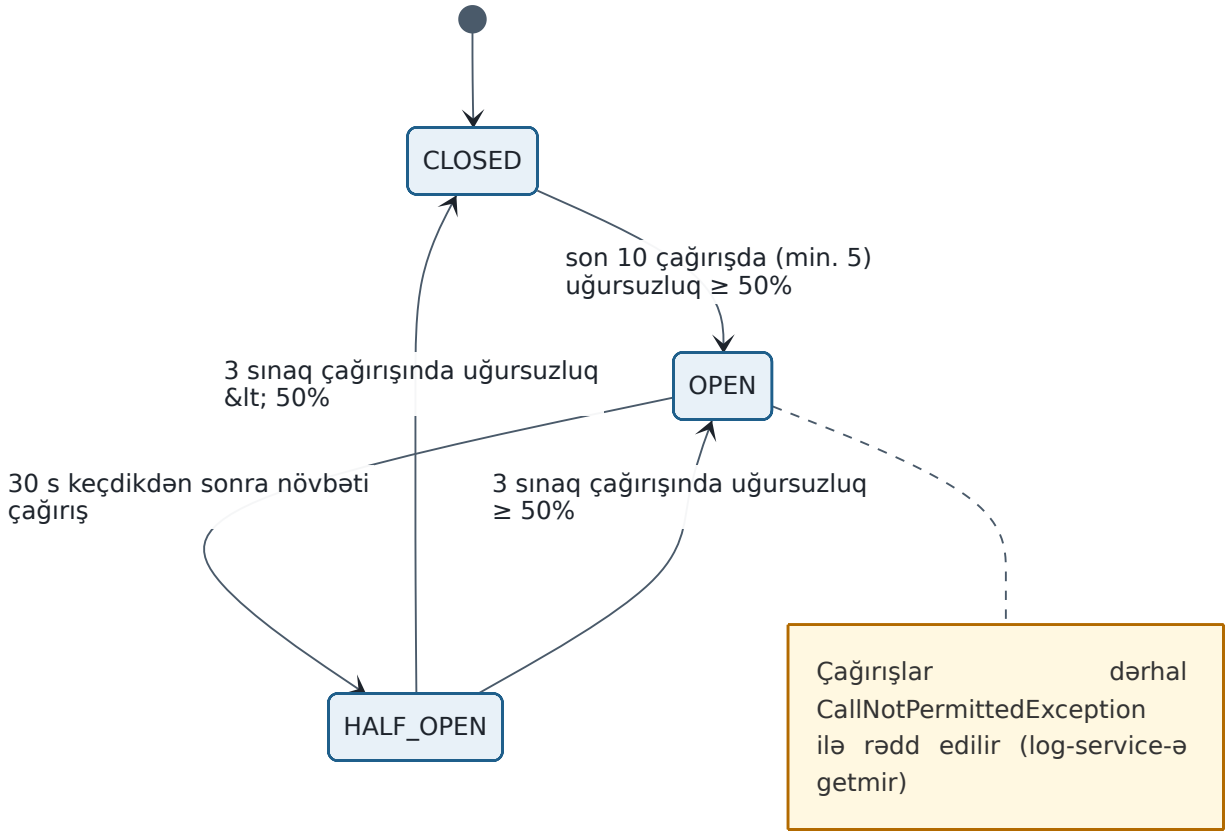
- Login cavabı log-service-in cavabını gözləmir.
- Retry gözləmələri (3 × 500 ms) istifadəçiyə təsir etmir.
- Audit **at-most-once**-dır: JVM bu anda dayansa, yaddaşdakı tapşırıqlar itir; log-service əlçatmazdırsa, audit itir (yalnız WARN log). Davamlılıq (durability) yoxdur.
- Audit commit-dən əvvəl göndərilə bilər (fəsil 7.8).

NƏTİCƏ Dizayn uyğunsuzluğu: yavaş sorğu jurnalları **Kafka** ilə (davamlı, retry + DLQ), audit hadisələri isə **HTTP** ilə (davamsız) gedir, halbuki audit, adətən, yavaş sorğu jurnalından daha vacib (təhlükəsizlik, uyğunluq) datadır. Case study-də hər iki texnikanın nümayişi üçün başa düşüləndir; istehsalda audit üçün Kafka + outbox daha uyğun olardı.

14.5 Resilience nümunələri: nəzəriyyə

Nümunə	Problem	Mexanizm
Timeout	Yavaş/asılı asılılıq resursları tutur	Müəyyən vaxtdan sonra imtina
Retry	Müvəqqəti xətalar (şəbəkə titrəməsi, restart)	Backoff (və jitter) ilə təkrar; yalnız idempotent və ya təhlükəsiz əməliyyatlar üçün
Circuit Breaker	Düşmüş asılılığa davamlı çağırış həm onu, həm çağıranı yorur	Uğursuzluq nisbəti həddi aşanda "açılır" və çağırışları dərhal rədd edir; müəyyən vaxtdan sonra sınaq çağırışları ilə yoxlayır
Bulkhead	Bir asılılığın yavaşlığı bütün thread/resursları tutur	Eyni vaxtlı çağırışların sayını məhdudlaşdırmaq (semafor və ya ayrıca thread pool)
Rate limiter	Asılılığı həddindən çox yükləmək	Vaxt vahidində icazə sayı
Fallback	Uğursuzluqda alternativ cavab	Cache-dən, standart dəyər, sonra emal üçün növbə

Circuit breaker vəziyyət maşını:



Circuit breaker vəziyyətləri və layihədəki parametrlər

Retry-ın qaranlıq tərəfi: idempotent olmayan əməliyyatın təkrarı dublikat yaradır (audit təkrar göndərilsə, ES-də iki qeyd — random id səbəbindən); çoxlu klientin eyni anda təkrarlaması düşən servisi "retry fırtınası" ilə boğa bilər — eksponensial backoff + jitter və circuit breaker bunu yumşaldır.

14.6 Layihədəki Resilience4j konfiqurasiyası

Mənbə: auth-service/src/main/resources/application.yml:84-114

```yaml
resilience4j:
  circuitbreaker:
    instances:
      logServiceHttpClient:
        failure-rate-threshold: ${LOG_SERVICE_CB_FAILURE_RATE_THRESHOLD:50}
        sliding-window-type: COUNT_BASED
        sliding-window-size: ${LOG_SERVICE_CB_SLIDING_WINDOW_SIZE:10}
        minimum-number-of-calls: ${LOG_SERVICE_CB_MINIMUM_NUMBER_OF_CALLS:5}
        permitted-number-of-calls-in-half-open-state: ${LOG_SERVICE_CB_PERMITTED_CALLS_IN_HALF_OPEN_STAT
E:3}
        wait-duration-in-open-state: ${LOG_SERVICE_CB_WAIT_DURATION_IN_OPEN_STATE:30s}
        record-exceptions:
          - java.lang.RuntimeException
        ignore-exceptions:
          - io.github.resilience4j.bulkhead.BulkheadFullException

  retry:
    instances:
      logServiceHttpClient:
        max-attempts: ${LOG_SERVICE_RETRY_MAX_ATTEMPTS:3}
        wait-duration: ${LOG_SERVICE_RETRY_WAIT_DURATION:500ms}
        retry-exceptions:
          - java.lang.RuntimeException
        ignore-exceptions:
          - io.github.resilience4j.circuitbreaker.CallNotPermittedException
          - io.github.resilience4j.bulkhead.BulkheadFullException

  bulkhead:
    instances:
      logServiceHttpClient:
        max-concurrent-calls: ${LOG_SERVICE_BULKHEAD_MAX_CONCURRENT_CALLS:10}
        max-wait-duration: ${LOG_SERVICE_BULKHEAD_MAX_WAIT_DURATION:0ms}
```

Parametr	Dəyər	Mənası
`sliding-window-type/size`	COUNT_BASED / 10	Son 10 çağırışın nəticəsi əsasında qiymətləndirmə
`minimum-number-of-calls`	5	5 çağırışdan az olduqda CB qərar vermir
`failure-rate-threshold`	50	Uğursuzluq ≥ 50% → OPEN
`wait-duration-in-open-state`	30 s	OPEN-də qalma müddəti
`permitted-number-of-calls-in-half-open-state`	3	Sınaq çağırışları
`record-exceptions`	`RuntimeException`	Bütün runtime xətaları (HTTP 4xx/5xx `RestClientResponseException` da daxil) uğursuzluq sayılır
`ignore-exceptions` (CB)	`BulkheadFullException`	Bulkhead rəddi log-service-in sağlamlığı haqqında deyil — sayılmır
`max-attempts`	3	**Cəmi** 3 cəhd (ilkin daxil)
`wait-duration`	500 ms	Sabit gözləmə (backoff/jitter yoxdur)
`ignore-exceptions` (Retry)	`CallNotPermittedException`, `BulkheadFullException`	CB açıqdırsa və ya bulkhead doludursa təkrar etmə
`max-concurrent-calls`	10	Eyni vaxtda log-service-ə ən çox 10 audit
`max-wait-duration`	0 ms	Yer yoxdursa dərhal rədd (`BulkheadFullException`)

Diqqət ediləcək nüans: Resilience4j-də `max-attempts` cəmi cəhd sayıdır (3 = ilkin + 2 təkrar); spring-kafka-nın `FixedBackOff(1000, 3)`-də isə 3 təkrar sayıdır (cəmi 4). Eyni "3" fərqli mənalar daşıyır.

Aspektlərin sırası

[FAKT, resilience4j-spring6 2.3.0 mənbəsi] Standart aspekt sıraları: `Retry` = `LOWEST_PRECEDENCE - 5`, `CircuitBreaker` = `LOWEST_PRECEDENCE - 4`, `RateLimiter` = `-3`, `TimeLimiter` = `-2`, `Bulkhead` = ən aşağı prioritet. Daha kiçik dəyər daha xaricdə deməkdir, nəticədə:

```
Retry( CircuitBreaker( Bulkhead( HTTP çağırışı ) ) )
```

Bunun nəticələri:

- Hər retry cəhdi **ayrıca CB çağırışı** kimi sayılır — bir audit 3 uğursuz cəhd ilə CB pəncərəsinə 3 uğursuzluq yazır.
- CB OPEN olduqda `CallNotPermittedException` Retry-a çatır; Retry onu ignore edir və təkrar etmir — düzgün.
- Bulkhead ən daxildədir: hər cəhd ayrıca icazə alır.

Aspektlər interfeys proxy-sinə tətbiq olunurmu?

`LogServiceHttpClient` bean-i özü JDK proxy-sidir. Resilience4j aspektləri (`@annotation(circuitBreaker)` pointcut-ı) Spring AOP auto-proxy mexanizmi ilə ona ikinci proxy kimi əlavə olunmalıdır. Asılılıq ağacında `aspectjweaver` var (`auth-service`: `spring-boot-starter-data-jpa` → `spring-data-jpa` → `spring-aspects` → `aspectjweaver 1.9.25.1`; Spring Boot-un AOP auto-configuration-u bu sinif classpath-də olduqda AspectJ auto-proxy-ni aktivləşdirir), bu da AspectJ annotasiya üslublu aspektlər üçün tələbdir. **NƏTİCƏ** Spring AOP pointcut uyğunluğunda JDK proxy hədəfləri üçün interfeys metodundakı annotasiyaya baxmaq imkanı var, ona görə aspektlərin işləməsi gözlənilir. Lakin **FAKT** repozitoriyada bunu təsdiqləyən test yoxdur: `AuthAuditServiceTest` servisi `new` ilə yaradır və istisnaları mock ilə **simulyasiya edir** (`AuthAuditServiceTest.java:229-264`) — real CB/Retry/Bulkhead davranışı yoxlanmır. Bu, "annotasiya var, deməli işləyir" fərziyyəsinin sınanmamış qaldığı tipik haldır.

Ssenari: log-service tamamilə düşüb

NƏTİCƏ Hər audit üçün:

Audit #	Cəhdlər	CB pəncərəsi (uğursuz/cəmi)	CB vəziyyəti	Nəticə
1	3 × uğursuz (500 ms ara)	3/3	CLOSED (min. 5 çatmayıb)	WARN "could not be sent"
2	1-ci cəhd uğursuz → 4/4; 2-ci → 5/5 → hədd aşıldı	5/5 = 100%	**OPEN**	3-cü cəhd `CallNotPermitted` → Retry dayanır → WARN "circuit breaker is open"
3…N (30 s ərzində)	0 real çağırış	—	OPEN	Dərhal WARN, log-service-ə yük yoxdur
30 s sonra	3 sınaq çağırışı	—	HALF_OPEN	Uğursuzdursa yenə OPEN

Login gecikməsi: heç biri — hamısı virtual thread-dədir.

14.7 `AuthAuditService.publishAudit`: xətaların yerli emalı

Mənbə: auth-service/src/main/java/com/casestudy/insurancecampaign/auth/service/AuthAuditService.java:137-180

```
private void publishAudit(String eventType, String usernameOrEmail, String message, String status) {
    AuthAuditLog auditLog = new AuthAuditLog(
            SERVICE_NAME,
            eventType,
            usernameOrEmail,
            message,
            status,
            Instant.now()
    );

    try {
        logServiceHttpClient.sendAuthAuditLog(auditLog);
        log.info(
                "Auth audit sent to log-service. eventType={}, usernameOrEmail={}, status={}",
                eventType,
                usernameOrEmail,
                status
        );
    } catch (CallNotPermittedException exception) {
        log.warn(
                "Auth audit skipped because log-service circuit breaker is open. eventType={}, usernameOr
Email={}, status={}, reason={}",
                eventType,
                usernameOrEmail,
                status,
                exception.getMessage()
        );
    } catch (BulkheadFullException exception) {
        log.warn(
                "Auth audit skipped because log-service bulkhead is full. eventType={}, usernameOrEmail=
{}, status={}, reason={}",
                eventType,
                usernameOrEmail,
                status,
                exception.getMessage()
        );
    } catch (Exception exception) {
        log.warn(
                "Auth audit could not be sent to log-service. eventType={}, usernameOrEmail={}, status=
{}, reason={}",
                eventType,
                usernameOrEmail,
                status,
                exception.getMessage()
        );
    }
}
```

- `catch` blokları spesifikdən ümumiyə doğru düzülüb — Java bunu tələb edir.
- Hər hal ayrıca mesajla loglanır — əməliyyat zamanı "niyə audit yoxdur?" sualına log cavab verir.
- **NƏTİCƏ** Ümumi `Exception` hissəsində stack trace loglanmır (yalnız `getMessage()`) — xətanın səbəbini tapmaq çətinləşə bilər.
- `@Async` metodundan istisna qaçmır — `AsyncUncaughtExceptionHandler`-ə ehtiyac yoxdur.
- Resilience4j-nin **fallback** mexanizmi (`fallbackMethod`) əvəzinə try/catch seçilib: davranış eynidir, lakin fallback interfeys proxy-sində tətbiq oluna bilməzdi (fallback metodu eyni sinifdə olmalıdır), try/catch isə şəffafdır.

14.8 Uğursuzluq ssenariləri

Ssenari	Mərhələ	Nəticə	Təkrar təhlükəsizdirmi	Dublikat?
log-service düşüb	HTTP bağlantısı	3 cəhd, sonra CB OPEN; audit itir	Bəli (yazma)	Ola bilər: timeout-dan sonra server yazmış ola bilər
log-service 403 (token problemi)	HTTP 4xx	`RestClientResponseException` (RuntimeException) → 3 cəhd + CB sayır	Mənasız — daimi xəta	—
Elasticsearch düşüb	log-service 500	Eyni kimi	—	—
Yavaş log-service	HTTP	Timeout yoxdur → virtual thread-lər və bulkhead icazələri tutulur → 11-ci audit `BulkheadFullException`	—	—
Audit sıçrayışı (>10 eyni vaxtda)	Bulkhead	Artıq auditlər dərhal atılır	—	—
auth-service restart	JVM	Gözləyən async tapşırıqlar itir	—	—

NƏTİCƏ 4xx (məs. 400, 403) daimi xətalardır, retry mənasızdır. `retry-exceptions: RuntimeException` hamısını təkrarlayır; düzgün variant — yalnız `ResourceAccessException` (I/O), `HttpServerErrorException` (5xx) və timeout-ları təkrarlamaq.

14.9 Yeni layihədə tətbiq

İllüstrativ nümunə (repozitoriyada yoxdur) — HTTP Interface + açıq timeout-lar + seçici retry:

```yaml
spring:
  http:
    serviceclient:
      log-service:
        base-url: http://log-service:8084
        connect-timeout: 1s
        read-timeout: 2s
resilience4j:
  retry:
    instances:
      logServiceHttpClient:
        max-attempts: 3
        wait-duration: 200ms
        enable-exponential-backoff: true
        exponential-backoff-multiplier: 2
        enable-randomized-wait: true
        retry-exceptions:
          - org.springframework.web.client.ResourceAccessException
          - org.springframework.web.client.HttpServerErrorException
```

Feign ekvivalenti (müqayisə üçün):

```java
@FeignClient(name = "log-service", path = "/api/v1/logs", configuration = InternalAuthFeignConfig.class)
public interface LogServiceFeignClient {
    @PostMapping("/auth-audit")
    void sendAuthAuditLog(@RequestBody AuthAuditLog request);
}
```

```
// name = "log-service" Spring Cloud LoadBalancer ilə Eureka-dan həll olunur;
// RequestInterceptor daxili JWT-ni əlavə edir; ErrorDecoder statusları domen istisnalarına çevirir.
```

	HTTP Interface (`@HttpExchange`)	OpenFeign
Mənbə	Spring Framework (core)	Spring Cloud
Əsas klient	`RestClient` / `WebClient`	Feign öz klienti (və ya Apache/OkHttp)
Reaktiv dəstək	Bəli (`WebClient` ilə)	Xeyr
Kəşf və balans	`@LoadBalanced` builder ilə	Doğma (`name` ilə)
Xəta dekodlaşdırma	`RestClient` status handler-ləri	`ErrorDecoder`
Gələcək	Spring-in tövsiyə etdiyi yol	Saxlanılır, yeni xüsusiyyət azdır

Test strategiyası: **WireMock** (və ya MockWebServer) ilə real HTTP serverini simulyasiya edib Spring konteksti ilə test — 503 cavablarında 3 cəhd edildiyini, 5 uğursuzluqdan sonra CB-nin açıldığını (`CircuitBreakerRegistry` ilə vəziyyəti yoxlamaq), timeout-ları və başlıqda `Authorization` -un olduğunu yoxlamaq.

14.10 Müsahibə sualları

S1. Retry və circuit breaker birlikdə istifadə olunduqda sıra niyə vacibdir? Cavab: Retry xaricdədirsə, hər cəhd CB-dən keçir və sayılır; CB açılanda retry dayanmalıdır (CallNotPermitted ignore). CB xaricdə olsaydı, bütün retry seriyası bir CB çağırışı sayılardı və CB gec açılardı. Resilience4j standartı: Retry(CircuitBreaker(RateLimiter(TimeLimiter(Bulkhead(fn))))).

S2. Semaphore bulkhead və thread-pool bulkhead fərqi? Cavab: Semaphore çağıranın thread-ində eyni vaxtlı çağırış sayını məhdudlaşdırır (ucuz, virtual thread-lərlə uyğundur); thread-pool bulkhead çağırışı ayrı, məhdud pool-da icra edir (çağıranı tamamilə təcrid edir, lakin thread keçidi və `CompletableFuture` tələb edir). Layihə semaphore istifadə edir; çağırış onsuz da `@Async` virtual thread-dədir.

S3. Hansı xətaları təkrar etmək olmaz? Cavab: Daimi xətaları (400, 401, 403, 404, 409, validasiya) və idempotent olmayan əməliyyatları (dublikat yarada bilənləri) — idempotency key olmadıqda. Müvəqqəti xətalar (bağlantı, 503, timeout) təkrarlana bilər.

S4. Niyə hər uzaq çağırışın timeout-u olmalıdır? Cavab: Timeout-suz çağırış asılılığın yavaşlığını çağırana ötürür: thread-lər, bağlantılar, bulkhead icazələri tutulur və kaskad uğursuzluq yaranır. CB yalnız uğursuzluqları sayır — heç vaxt bitməyən çağırış onu açmaz (slow-call konfiqurasiyası olmadıqda).

S5. HTTP Interface ilə Feign arasında seçim edərkən nəyə baxarsınız? Cavab: Spring versiyası (Spring 6+ → HTTP Interface doğmadır), reaktiv ehtiyac, mövcud Spring Cloud infrastrukturu (LoadBalancer/Eureka inteqrasiyası Feign-də hazırdır, HTTP Interface-də `@LoadBalanced` builder ilə qurulur), komanda təcrübəsi, uzunmüddətli dəstək.

S6. Audit üçün HTTP + `@Async` ilə Kafka + outbox arasında fərq nədir? Cavab: HTTP + async at-most-once-dır: alıcı düşəndə və ya proses dayananda audit itir. Outbox + Kafka davamlıdır və at-least-once-dır: audit DB tranzaksiyası ilə atomik yazılır, sonra çatdırılır; consumer idempotent olmalıdır.

14.11 Xülasə

- auth → log çağırışı Spring 7-nin HTTP Interface + HTTP Service registry mexanizmi ilə deklarativ yazılıb; daxili JWT qrup konfiqurasiyası ilə əlavə olunur.
- Resilience4j: Retry(3) → CircuitBreaker(10/5/50%/30 s) → Bulkhead(10, 0 ms); xətalar `publishAudit` -də udulur; `@Async` istifadəçini gecikmədən qoruyur.
- Boşluqlar: timeout yoxdur, 4xx də təkrarlanır, aspekt davranışı test edilmir, audit at-most-once-dır.

- Feign repozitoriyada yoxdur; HTTP Interface müasir alternativdir.
- Növbəti fəsil Redis-in iki istifadəsini — cache və rate limiting — öyrədir.

15. Redis: paylanmış cache və rate limiting

Redis layihədə iki tamamilə fərqli məqsədlə istifadə olunur: **campaign-service-də statistika cache-i** (Spring Cache abstraksiyası) və **api-gateway-də sorğu limitləməsi** (Spring Cloud Gateway `RedisRateLimiter`). Hər iki istifadə eyni Redis instansındadır (`docker-compose.yml:285-299`).

15.1 Redis əsasları

Redis yaddaşda işləyən açar-dəyər verilənlər strukturları serveridir:

Struktur	Nümunə istifadə
String	Cache dəyəri, sayğac (`INCR`)
Hash	Obyekt sahələri
List	Növbə
Set / Sorted set	Unikal üzvlər; reytinq, vaxta görə sıralama (sliding window rate limit)
Stream	Hadisə jurnalı

Əsas xüsusiyyətlər:

- **Əmrlər tək thread-də ardıcıl icra olunur** — hər əmr atomikdir; çox addımlı atomik əməliyyatlar üçün **Lua skriptləri** (`EVAL`) və ya `MULTI/EXEC` istifadə olunur.
- **TTL** — açarlara bitmə vaxtı (`EXPIRE`, `SETEX`).
- **Davamlılıq** — RDB snapshot və/və ya AOF jurnalı; lakin Redis əsas mənbə (source of truth) kimi deyil, adətən cache/köməkçi saxlama kimi istifadə olunur.
- **Yaddaş siyasəti** — `maxmemory` dolduqda `maxmemory-policy` (`allkeys-lru`, `volatile-ttl`, `noeviction`).
- **Yüksək əlçatanlıq** — replikasiya, Sentinel, Redis Cluster (açarlar 16384 hash slot üzrə bölünür).

Spring Boot standart olaraq **Lettuce** klientindən istifadə edir (asılılıq ağacı: `lettuce-core 7.5.2`): Netty əsaslı, thread-safe, bir bağlantı üzərindən çoxlu paralel əmr (pipelining).

15.2 Caching nəzəriyyəsi

Cache-aside (layihədəki model): oxuyanda əvvəl cache-ə bax; yoxdursa mənbədən oxu və cache-ə yaz. Yazanda mənbəni yenilə və cache-i **sil** (invalidate).

Əsas problemlər:

Problem	İzah	Tipik həll
Köhnə data (staleness)	Mənbə dəyişib, cache yox	Yazıda eviction + TTL ehtiyat kimi
Eviction yarışı	Oxuyan köhnə dəyəri DB-dən oxuyub cache-ə yazır, arada yazan evict edib	Qısa TTL, versiyalı açarlar, evict-i commit-dən sonra etmək
Stampede (thundering herd)	Populyar açar silinəndə çoxlu sorğu eyni anda DB-yə gedir	`@Cacheable(sync = true)`, kilid, erkən yeniləmə
Serializasiya	Dəyərin formatı versiyalar arasında uyğun olmalıdır	JSON, sxem dəyişikliyində açar prefiksini dəyişmək
Cache-in düşməsi	Cache xətası əsas əməliyyatı dayandırmamalıdır	Fail-open `CacheErrorHandler`

Lokal və paylanmış cache. Lokal (Caffeine, `ConcurrentMap`) ən sürətlidir, lakin hər instansın öz nüsxəsi olur və bir instansdakı eviction digərlərinə çatmır. Paylanmış (Redis) cache şəbəkə gedişi tələb edir, lakin bütün instanslar eyni dəyəri görür və eviction qlobaldır. campaign-service üçün Redis seçimi bu səbəbdən məntiqlidir.

15.3 Spring Cache abstraksiyası

Element	Mənası
`@EnableCaching`	`CacheInterceptor` proxy-sini aktivləşdirir (`CampaignServiceApplication.java:10`)
`CacheManager`	`spring.cache.type: redis` → `RedisCacheManager` (`campaign-service/src/main/resources/application.yml:22-23`)
`@Cacheable("x")`	Açar üçün dəyər varsa metod çağırılmır; yoxdursa çağırılır və nəticə saxlanılır
`@CacheEvict(value="x", allEntries=true)`	Metoddan **sonra** (standart `beforeInvocation=false`) bütün `x` cache-i silinir; metod istisna atarsa eviction olmur
Açar	`SimpleKeyGenerator`: parametrsiz metod → `SimpleKey.EMPTY`
`CacheErrorHandler`	Standart `SimpleCacheErrorHandler` cache xətasını **yenidən atır**

Boot-un Redis cache standartları

[FAKT, spring-boot-cache 4.1.0 `RedisCacheConfiguration`] `spring.cache.redis.*` xassələri verilmədikdə:

- dəyər serializatoru `JdkSerializationRedisSerializer`-dir (Java serializasiyası);
- açar `<cacheName>::<key>` formasındadır, məs. `campaign-statistics::SimpleKey []`;
- **TTL yoxdur** — `entryTtl` yalnız `spring.cache.redis.time-to-live` təyin olunduqda tətbiq olunur;
- `RedisCacheManager` transaction-aware deyil.

15.4 Layihədə cache

Mənbə: `campaign-service/src/main/java/com/casestudy/insurancecampaign/campaign/service/CampaignService.java:121-123`

```
@Transactional(readOnly = true)
@Cacheable("campaign-statistics")
public Map<String, Long> getStatistics() {
```

Eviction üç yazı metodunda: `create`, `approve`, `deactivate` — hamısında `@CacheEvict(value = "campaign-statistics", allEntries = true)` (`CampaignService.java:54-55`, `91-92`, `102-103`).

Qiymətləndirmə:

- **Doğru:** statistikanı dəyişən hər əməliyyat cache-i silir; cache paylanmışdır, ona görə bütün campaign instanslarında ardıcıldır; həm REST, həm gRPC eyni cache-dən yararlanır.
- **Dəyər tipi** `HashMap<String, Long>` — `Serializable`-dır, JDK serializasiyası işləyir. NƏTİCƏ JDK serializasiyası dildən asılıdır, Redis-də oxunmazdır və sinif dəyişikliklərinə həssasdır; `GenericJacksonJsonRedisSerializer` (JSON) daha şəffafdır.
- **Testlərdə Redis yoxdur** — `spring.cache.type=simple` (`campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractTestContainerConfiguration.java:35`): serializasiya, TTL, Redis xətası davranışı test edilmir.

Riskli nöqtələr (F13)

1. Evict və commit sırası. `@Transactional` və `@CacheEvict` eyni metodda olduqda iki interceptor işləyir. **NƏTİCƏ** Layihə onların sırasını açıq təyin etmir (`@EnableCaching(order=...)` yoxdur). Cache interceptor tranzaksiya interceptor-undan **daxildə** olarsa, eviction commit-dən **əvvəl** baş verir və belə yarış mümkündür:

Zaman	Yazan (approve)	Oxuyan (getStatistics)
t1	UPDATE (commit olunmayıb)	
t2	`@CacheEvict` — cache boşdur	
t3		cache yoxdur → DB-dən oxu (köhnə, commit olunmamış dəyişiklik görünmür)
t4		köhnə statistikanı cache-ə yaz
t5	COMMIT	

Nəticə: cache-də köhnə dəyər qalır və **TTL olmadığı üçün** növbəti yazı əməliyyatına qədər (saatlar, günlər) qalır. Həllər: `spring.cache.redis.time-to-live` (məs. 60 s) ehtiyat tor kimi; `RedisCacheManager.builder(...).transactionAware()` (evict commit-dən sonra icra olunur); və ya eviction-u `@TransactionalEventListener(AFTER_COMMIT)`-ə köçürmək.

2. Redis xətası. Standart `CacheErrorHandler` xətanı yenidən atır: Redis düşdükdə `getStatistics()` (REST və gRPC) 500/`UNKNOWN` verir; `create` isə metod uğurla bitdikdən sonra eviction xətası səbəbindən istisna ata bilər — DB-də kampaniya yaranıb (commit-in eviction-dan əvvəl/sonra olmasından asılı olaraq), klient isə 500 görür və təkrar edərsə 409 alır. Cache ikinci dərəcəli komponentdir; onun düşməsi əsas funksiyanı dayandırmamalıdır. İllüstrativ həll (repozitoriyada yoxdur):

```
@Configuration
class CacheConfig implements CachingConfigurer {
    @Override
    public CacheErrorHandler errorHandler() {
        return new SimpleCacheErrorHandler() {
            @Override public void handleCacheGetError(RuntimeException e, Cache c, Object k) { log.warn
("cache get failed", e); }
            @Override public void handleCachePutError(RuntimeException e, Cache c, Object k, Object v) {
log.warn("cache put failed", e); }
            @Override public void handleCacheEvictError(RuntimeException e, Cache c, Object k) { log.warn
("cache evict failed", e); }
            @Override public void handleCacheClearError(RuntimeException e, Cache c) { log.warn("cache cl
ear failed", e); }
        };
    }
}
```

3. Stampede — statistika cache-i boşaldıqda paralel sorğular eyni anda `GROUP BY` icra edir. Kiçik cədvəldə əhəmiyyətsizdir; `@Cacheable(value = "campaign-statistics", sync = true)` eyni instans daxilində bunu bir sorğuya endirir.

15.5 Rate limiting nəzəriyyəsi

Alqoritm	İdeya	Üstünlük	Çatışmazlıq
Fixed window	Hər dəqiqə üçün sayğac	Sadə	Pəncərə sərhədində ikiqat sıçrayış
Sliding window log	Hər sorğunun vaxtı (sorted set)	Dəqiq	Yaddaş

Alqoritm	İdeya	Üstünlük	Çatışmazlıq
Sliding window counter	İki pəncərənin çəkili cəmi	Yaxşı balans	Təqribi
Token bucket	Vedrə sabit sürətlə dolur, hər sorğu token xərcləyir	Burst-a icazə, ortalama sürət nəzarəti	Parametrləri başa düşmək
Leaky bucket	Sorğular sabit sürətlə "sızır"	Hamar çıxış	Burst yoxdur

Paylanmış sistemdə limit vəziyyəti bütün gateway instansları arasında paylaşılmalıdır — Redis buna görə istifadə olunur; atomiklik üçün Lua skripti.

15.6 Spring Cloud Gateway `RedisRateLimiter`

[FAKT, SCG 5.0.2 mənbəsi `META-INF/scripts/request_rate_limiter.lua`] Token bucket Lua skripti ilə həyata keçirilir:

```
local fill_time = capacity / rate
local ttl = math.floor(fill_time * 2)
local last_tokens = tonumber(redis.call("get", tokens_key)) or capacity
local last_refreshed = tonumber(redis.call("get", timestamp_key)) or 0
local delta = math.max(0, now-last_refreshed)
local filled_tokens = math.min(capacity, last_tokens+(delta*rate))
local allowed = filled_tokens ⩾ requested
local new_tokens = allowed and filled_tokens - requested or filled_tokens
```

Açarlar: `request_rate_limiter.{<routeId>.<key>}.tokens` və `.timestamp` (fiqurlu mötərizə Redis Cluster hash tag-idir — hər iki açar eyni slot-a düşür ki, skript atomik işləsin). Açarlar `2 × capacity/rate` saniyə TTL ilə saxlanılır.

Vacib nəticə FAKT: açarda **marşrut id-si** var. Deməli limit `(marşrut, istifadəçi/IP)` cütü üzrədir. README "eyni istifadəçi 5 saniyədə 1 sorğu" deyir — bu, **hər marşrut üçün ayrıca** doğrudur: admin eyni anda `campaign-service-api` və `dashboard-service-api` marşrutlarına bir sorğu göndərə bilər.

Layihənin parametrləri

Mənbə: `api-gateway/src/main/resources/application.yml:20-26`

```
default-filters:
- name: RequestRateLimiter
  args:
    key-resolver: "#{@requestRateLimiterKeyResolver}"
    redis-rate-limiter.replenishRate: ${RATE_LIMITER_REPLENISH_RATE:1} # → Redis adds 1 token per second
    redis-rate-limiter.burstCapacity: ${RATE_LIMITER_BURST_CAPACITY:5} # → Bucket can hold max 5 tokens
    redis-rate-limiter.requestedTokens: ${RATE_LIMITER_REQUESTED_TOKENS:5} # → Each request costs 5 toke
ns
```

- `replenishRate = 1` token/s, `burstCapacity = 5`, `requestedTokens = 5` → hər sorğu bütün vedrəni xərcləyir; vedrə 5 saniyədə dolur → **5 saniyədə 1 sorğu, burst yoxdur**. Bu, demo üçün çox sərt limitdir (Swagger UI-dan ardıcıl iki klik 429 verir); istehsalda adətən `requestedTokens = 1` və daha böyük `burstCapacity` istifadə olunur.
- `default-filters` — bütün marşrutlara (auth, campaign, dashboard, logs, OpenAPI proxy marşrutları) eyni limit. NƏTİCƏ Fərqli endpoint-lərin fərqli riski var: login brute-force-a qarşı sərt, oxumalar üçün yumşaq limit daha uyğundur — marşrut səviyyəli `filters` ilə.
- `/openapi/**` (gateway-in öz controller-i) marşrut deyil — rate limit tətbiq olunmur.
- Cavab başlıqları: `X-RateLimit-Remaining`, `X-RateLimit-Replenish-Rate`, `X-RateLimit-Burst-Capacity`, `X-RateLimit-Requested-Tokens`.

Açar resolver-i

Mənbə: api-gateway/src/main/java/com/casestudy/insurancecampaign/gateway/config/GatewayRateLimiterConfiguration.java:36-62

```
@Bean
public KeyResolver requestRateLimiterKeyResolver(GatewayJwtTokenService jwtTokenService) {
    return exchange → Mono.just(resolveKey(exchange, jwtTokenService));
}

private String resolveKey(ServerWebExchange exchange, GatewayJwtTokenService jwtTokenService) {
    String authorization = exchange.getRequest()
            .getHeaders()
            .getFirst(HttpHeaders.AUTHORIZATION);

    if (authorization ≠ null && authorization.startsWith(BEARER_PREFIX)) {
        String token = authorization.substring(BEARER_PREFIX.length());

        try {
            Claims claims = jwtTokenService.parseClaims(token);
            String subject = claims.getSubject();

            if (subject ≠ null && !subject.isBlank()) {
                return "user:" + subject;
            }
        } catch (RuntimeException exception) {
            return "ip:" + resolveClientIp(exchange);
        }
    }

    return "ip:" + resolveClientIp(exchange);
}
```

- Autentifikasiyalı sorğu → `user:<subject>`; token-siz və ya etibarsız → `ip:<client ip>`. Etibarsız token-lə saxta `user:` açarı yaratmaq mümkün deyil, çünki imza yoxlanılır — düzgün dizayn.
- `resolveClientIp` əvvəlcə `X-Forwarded-For`-un ilk dəyərinə, sonra `X-Real-IP`-yə, sonra TCP ünvanına baxır (`GatewayRateLimiterConfiguration.java:64-92`).

RİSK (F05) Gateway kənar qatdır və başlıqları klient özü göndərir. Hücumçu hər sorğuda fərqli `X-Forwarded-For` göndərərək hər dəfə yeni IP vedrəsi alır — anonim endpoint-lərdə (login, forgot-password) limit faktiki olaraq söndürülür. Düzgün yanaşma: XFF-ə yalnız etibarlı proxy-lərdən gəldikdə inanmaq. SCG bunun üçün `XForwardedRemoteAddressResolver.maxTrustedIndex(n)` təqdim edir — siyahının sağından etibarlı proxy sayı qədər keçir. Həmçinin F01 səbəbindən hücumçu yeni hesablar yaradıb yeni `user:` vedrələri əldə edə bilər.

429 cavabının formalaşdırılması

`RequestRateLimiter` filtri limit aşıldıqda statusu 429 edib cavabı boş gövdə ilə tamamlayır. Layihə bunu JSON-a çevirmək üçün cavabı "dekorasiya edən" qlobal filtr yazıb **FAKT** (`GatewayRateLimitErrorResponseFilter.java:54-68`):

Mənbə: api-gateway/src/main/java/com/casestudy/insurancecampaign/gateway/filter/GatewayRateLimitErrorResponseFilter.java:54-68

```
@Override
public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
    ServerHttpResponseDecorator decoratedResponse = new ServerHttpResponseDecorator(exchange.getResponse
()) {
        @Override
        public Mono<Void> setComplete() {
            if (HttpStatus.TOO_MANY_REQUESTS.equals(getStatusCode()) && !isCommitted()) {
                return writeRateLimitError(exchange, this);
            }

            return super.setComplete();
        }
```

```
    };

    return chain.filter(exchange.mutate().response(decoratedResponse).build());
}
```

- Filtr `HIGHEST_PRECEDENCE` ilə ən əvvəl işləyir və zəncirin qalanına **dekorasiya olunmuş cavab** ötürür. Rate limiter `setComplete()` çağırdıqda dekorator onu tutur və gövdəli cavab yazır. Bu, reaktiv proqramlaşdırmada "cavabı sonradan dəyişdirmə" texnikasının yaxşı nümunəsidir.
- `Retry-After: 5` və mesaj "Please wait 5 seconds" **sabit kodlanıb** (`:32-34`). **NƏTİCƏ** Limit parametrləri mühit dəyişənləri ilə dəyişdirilsə, mesaj yanlış olacaq — dəyər `burstCapacity` / `replenishRate`-dən hesablanmalı idi.

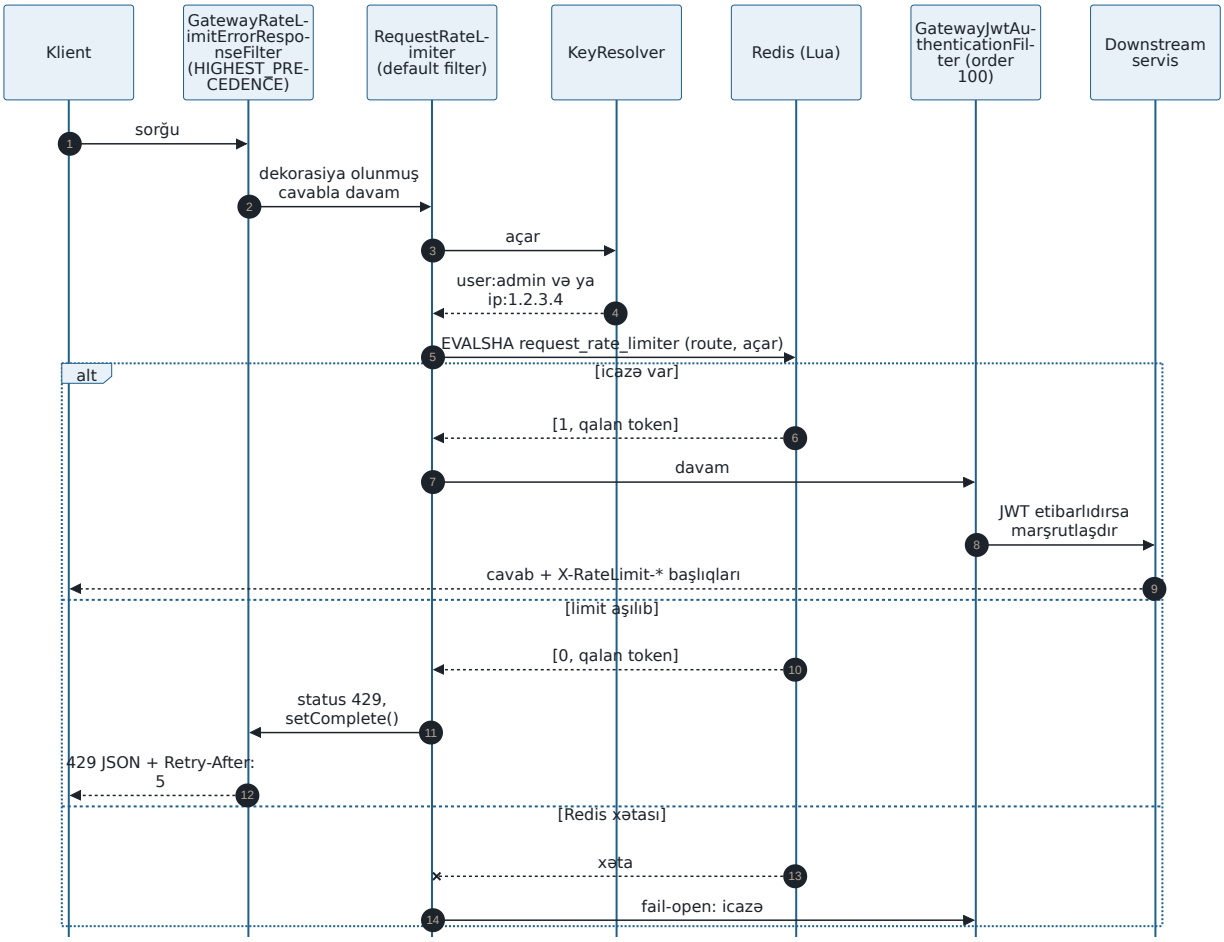
Fail-open (F06)

[FAKT, SCG mənbəsi] Redis əlçatmaz olduqda `RedisRateLimiter.isAllowed` xətanı loglayır və sorğuya **icazə verir**:

```
return flux.onErrorResume(throwable -> {
    log.error("Error calling rate limiter lua", throwable);
    return Flux.just(Arrays.asList(1L, -1L));
})
```

Mənbədəki şərh bunu əsaslandırır: "We don't want a hard dependency on Redis to allow traffic." Bu, əlçatanlığı təhlükəsizlikdən üstün tutan **şüurlu kompromisdir**: Redis düşsə API işləyir, lakin brute-force qorunması itir. Redis xətalarına alert qurulmalıdır. Gateway-də `spring.data.redis.timeout: 2s` (`api-gateway/src/main/resources/application.yml:8-12`) — Redis "donarsa", hər sorğu 2 s-yə qədər gözləyə bilər.

15.7 İcra axını: rate limit



Gateway-də sorğunun filtr sırası və rate limit qərarı

15.8 Uğursuzluq ssenariləri

Ssenari	Cache (campaign)	Rate limit (gateway)
Redis düşüb	Statistika 500 / gRPC `UNKNOWN`; yazılarda eviction xətası	Fail-open, ERROR log
Redis yavaş	Statistika sorğuları yavaşlayır	Hər sorğu Redis timeout-u (2 s) qədər gözləyə bilər
Redis yaddaşı dolur	`maxmemory-policy`-dən asılı: eviction və ya yazı xətası	Eyni
Köhnə cache dəyəri	TTL olmadığı üçün növbəti yazıya qədər qalır	—
Saxta XFF	—	Limit yan keçilir

15.9 Yeni layihədə tətbiq və alternativlər

Cache üçün yoxlama siyahısı: TTL (həmişə, ehtiyat tor kimi); JSON serializator; fail-open `CacheErrorHandler`; transaction-aware eviction; `sync = true` populyar açarlar üçün; açar adlarında versiya prefiksi; metrik (hit/miss — `spring.cache.redis.enable-statistics`).

```
spring:
  cache:
```

```
    type: redis
    redis:
      time-to-live: 60s
      enable-statistics: true
```

Rate limiting alternativləri: **Bucket4j** (JVM, Redis/Hazelcast backend), Resilience4j `RateLimiter` (lokal, instans daxilində), NGINX/Envoy/API Management səviyyəsində limitlər, WAF. Çoxqatlı yanaşma tövsiyə olunur: kənar qatda IP limitləri (etibarlı XFF ilə), gateway-də istifadəçi/marşrut limitləri, servisdə biznes limitləri (məs. "5 uğursuz login → hesab müvəqqəti kilidi").

15.10 Müsahibə sualları

S1. Token bucket alqoritmini izah edin; `replenishRate=1, burstCapacity=5, requestedTokens=5` nə deməkdir? Cavab: Vedrədə ən çox `burstCapacity` token olur, saniyədə `replenishRate` token əlavə olunur, hər sorğu `requestedTokens` xərcləyir. Burada hər sorğu bütün 5 tokeni xərcləyir və vedrə 5 s-də dolur — 5 saniyədə 1 sorğu.

S2. Rate limiter niyə Lua skripti istifadə edir? Cavab: "Oxu → hesabla → yaz" addımları atomik olmalıdır; ayrı əmrlərlə paralel sorğular eyni tokeni xərcləyə bilərdi. Redis Lua skriptini atomik icra edir.

S3. Redis düşəndə gateway nə edir və bu düzgündürmü? Cavab: Fail-open — sorğuları buraxır. Əlçatanlıq baxımından düzgündür (Redis-ə sərt asılılıq yoxdur), təhlükəsizlik baxımından risklidir (brute-force qorunması itir). Bu, biznes qərarıdır; monitorinq və alert vacibdir.

S4. `@CacheEvict` və `@Transactional` birlikdə olduqda hansı yarış yarana bilər? Cavab: Eviction commit-dən əvvəl baş verərsə, paralel oxuyan köhnə DB vəziyyətini cache-ə yaza bilər; TTL yoxdursa köhnə dəyər uzun müddət qalır. Transaction-aware cache manager, AFTER_COMMIT eviction və TTL bunu həll edir.

S5. X-Forwarded-For başlığına niyə ehtiyatla yanaşmaq lazımdır? Cavab: Klient onu istədiyi kimi göndərə bilər. Yalnız etibarlı proxy-lərin əlavə etdiyi hissəyə (siyahının sağından) inanmaq lazımdır; əks halda IP əsaslı limitlər və audit qeydləri saxtalaşdırılır.

S6. Lokal cache (Caffeine) əvəzinə Redis nə vaxt seçilir? Cavab: Bir neçə instans eyni dəyəri görməli və eviction qlobal olmalıdırsa; dəyər hesablanması bahalıdırsa və instanslar arasında paylaşılması faydalıdırsa. Lokal cache daha sürətlidir, lakin instanslar arası ardıcıllıq problemi var. İki səviyyəli (L1 lokal + L2 Redis) hibrid də mümkündür.

15.11 Xülasə

- campaign-service statistika üçün cache-aside modelini Spring Cache + Redis ilə həyata keçirir; eviction hər yazıda qlobaldır, lakin TTL yoxdur, sıra və xəta emalı riskləri var.
- Gateway SCG `RedisRateLimiter` (token bucket, Lua) istifadə edir; limit `(marşrut, istifadəçi/IP)` üzrədir, Redis düşəndə fail-open, XFF saxtalaşdırılmasına həssasdır.
- 429 cavabı reaktiv response decorator ilə JSON-a çevrilir.
- Növbəti fəsil gateway-in qalan hissəsini, Eureka-nı, Config Server-i və OpenAPI aqreqasiyasını izah edir.

16. Spring Cloud platforması: Gateway, Eureka, LoadBalancer, Config Server və OpenAPI aqreqasiyası

16.1 Motivasiya

Bir neçə müstəqil servis olduqda dörd platforma problemi yaranır:

1. **Vahid giriş nöqtəsi** — klient 4 servisin ünvanını bilməməli; kəsişən narahatlıqlar (autentifikasiya, rate limiting, CORS, loglama) bir yerdə olmalıdır → **API Gateway**.
2. **Servis kəşfi** — instansların IP/portları dinamikdir; çağıran onları adla tapmalıdır → **Eureka**.
3. **Klient tərəfli yük balansı** — eyni servisin bir neçə instansı arasında sorğuları paylamaq → **Spring Cloud LoadBalancer**.
4. **Mərkəzi konfiqurasiya** — ortaq parametrləri bir yerdə saxlamaq → **Config Server**.

Bu fəsil hər birini nəzəri olaraq və layihədəki konfiqurasiya ilə izah edir; sonra gateway-in OpenAPI aqreqasiyasına baxır.

16.2 Spring Cloud Gateway (WebFlux)

Arxitektura

Layihə `spring-cloud-starter-gateway-server-webflux` (SCG 5.0.2) istifadə edir (`api-gateway/pom.xml:34-37`): reaktiv, Netty üzərində işləyən gateway. (SCG-nin servlet əsaslı "Server MVC" variantı da mövcuddur.)

Anlayış	İzah
Route	`id`, hədəf `uri`, `predicates` (nə vaxt uyğun gəlir), `filters` (sorğu/cavabı necə dəyişir)
Predicate	`Path`, `Method`, `Header`, `Host`, `Query`, …
GatewayFilter	Marşruta bağlı filtr: `StripPrefix`, `RequestRateLimiter`, `RewritePath`, `Retry`, `CircuitBreaker` …
GlobalFilter	Bütün marşrutlara tətbiq olunan filtr (layihədə JWT və 429 filtrləri)
default-filters	Konfiqurasiyada bütün marşrutlara əlavə olunan GatewayFilter-lər

Sorğunun yolu: `RoutePredicateHandlerMapping` uyğun marşrutu tapır → `FilteringWebHandler` marşrut filtrlərini və qlobal filtrləri **bir siyahıda `Ordered` dəyərinə görə** sıralayır → "pre" hissələr icra olunur → `ReactiveLoadBalancerClientFilter` `lb://` URI-ni konkret instansa çevirir → `NettyRoutingFilter` downstream-ə HTTP sorğusu göndərir → "post" hissələr əks sırada icra olunur.

Konfiqurasiya ad fəzası. SCG 5-də xassələr `spring.cloud.gateway.server.webflux.*` altındadır (köhnə `spring.cloud.gateway.*` əvəzinə) — layihə yeni formadan istifadə edir (`api-gateway/src/main/resources/application.yml:17-19`). Köhnə məqalələrdən götürülmüş konfiqurasiya bu versiyada işləməyə bilər.

Layihənin marşrutları

FAKT (`api-gateway/src/main/resources/application.yml:27-67`):

Route id	Predicate	Hədəf	Filtrlər
`auth-service-api`	`Path=/api/v1/auth/**`	`${AUTH_SERVICE_URI:lb://auth-service}`	default (rate limit)
`campaign-service-api`	`Path=/api/v1/campaigns/**`	`lb://campaign-service`	default
`dashboard-service-api`	`Path=/api/v1/dashboard/**`	`lb://dashboard-service`	default
`log-service-api`	`Path=/api/v1/logs/**`	`lb://log-service`	default
`auth-service-openapi`	`Path=/auth/v3/api-docs,/auth/v3/api-docs/**`	`lb://auth-service`	`StripPrefix=1` + default
`campaign-/dashboard-/log-service-openapi`	analoji	analoji	`StripPrefix=1` + default

- `StripPrefix=1` birinci yol seqmentini atır: `/auth/v3/api-docs` → `/v3/api-docs`.
- `${AUTH_SERVICE_URI:lb://auth-service}` — README-yə görə lokal IDE-də `http://localhost:8081` kimi birbaşa URI verilə bilər (Eureka-sız); Compose-da `lb://` (`docker-compose.yml:62-65`).
- `/api/v1/logs/auth-audit` də gateway-dən keçir — lakin yalnız `ROLE_INTERNAL_SERVICE` token-i ilə işləyir; auth-service isə log-service-ə birbaşa müraciət edir.

Filtrlərin tam sırası

Sıra	Filtr	Növ
`HIGHEST_PRECEDENCE`	`GatewayRateLimitErrorResponseFilter`	Global (layihə)
1	`RequestRateLimiter`	Default filter (marşrut)
2	`StripPrefix` (yalnız OpenAPI marşrutları)	Marşrut
100	`GatewayJwtAuthenticationFilter`	Global (layihə)
SCG daxili	`ReactiveLoadBalancerClientFilter` (10150), `NettyRoutingFilter` (ən sonda) və s.	Global (SCG)

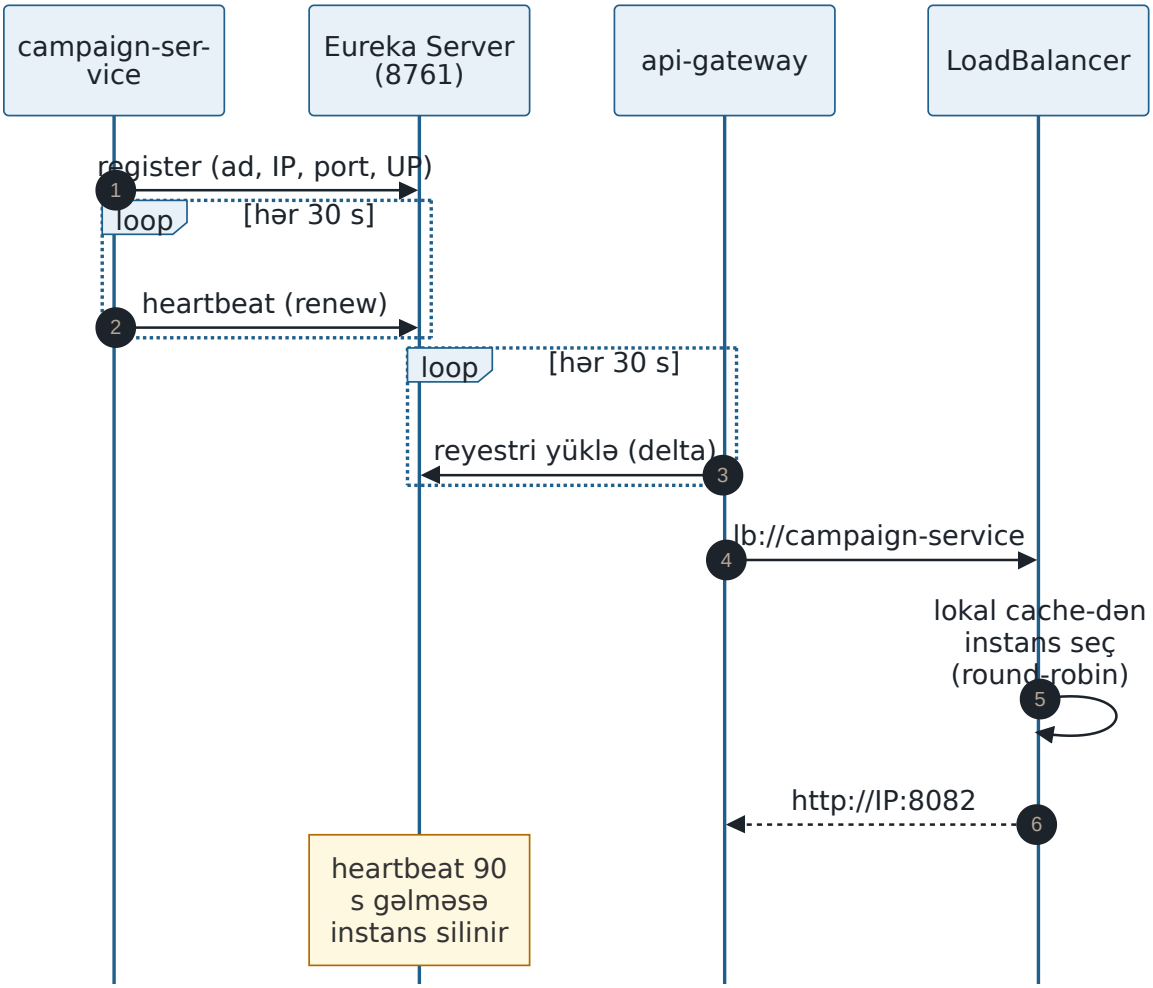
NƏTİCƏ Konfiqurasiyadakı default və marşrut filtrləri SCG tərəfindən yazılış sırasına görə 1, 2, ... dəyərləri alır; layihənin JWT filtrinin `getOrder()` şərhi (`GatewayJwtAuthenticationFilter.java:97-111`) "rate limiter-dən sonra" işləmə niyyətini açıq ifadə edir.

Başlıqlar və timeout-lar

- SCG standart olaraq `X-Forwarded-For`, `X-Forwarded-Proto`, `X-Forwarded-Host` başlıqlarını əlavə edir. **NƏTİCƏ** Downstream-dəki `SlowRequestLoggingFilter` `x-forwarded-for`-u Elasticsearch-ə yazır — klientin IP-si buradan gəlir.
- `Authorization` başlığı dəyişmədən ötürülür.
- **HTTP klient timeout-ları konfiqurasiya olunmayıb** (`spring.cloud.gateway.server.webflux.httpclient.connect-timeout`, `response-timeout`). **NƏTİCƏ** Standart cavab timeout-u yoxdur — asılı qalan servis gateway bağlantılarını tutur. Hər marşrut üçün `response-timeout` metadata-sı və ya qlobal dəyər tövsiyə olunur.
- `spring.cloud.loadbalancer.retry.enabled: true` (`api-gateway/src/main/resources/application.yml:13-16`) — **NƏTİCƏ** bu xassə Spring Cloud LoadBalancer-in `@LoadBalanced` **klientləri** (burada OpenAPI `WebClient`-i) üçün retry-ı aktivləşdirir; gateway marşrutları üçün təkrar `Retry` GatewayFilter-i ilə konfiqurasiya olunur və burada yoxdur.
- Marşrut səviyyəli `CircuitBreaker` filtri yoxdur.

16.3 Eureka: servis kəşfi

Model: hər servis başlanğıcda öz instans məlumatını (ad, IP, port, status, metadata) Eureka Server-ə **qeydiyyatdan keçirir** və periodik **heartbeat** (standart 30 s) göndərir; klientlər reyestri periodik **yükləyir** (standart 30 s) və lokal **cache**-dən istifadə edir. Heartbeat gəlməsə, instans lease müddətindən (standart 90 s) sonra silinir.



Eureka qeydiyyatı və gateway-də lb:// həlli (standart intervallar)

Server FAKT (`discovery-server/src/main/resources/application.yml:9-14`):

```yaml
eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
  server:
    enable-self-preservation: false
```

- Tək node-lu server özünü qeydiyyatdan keçirmir və reyestri başqa yerdən almır.
- **Self-preservation** — şəbəkə problemi zamanı heartbeat-lərin kütləvi itməsini "instanslar ölüb" kimi deyil, "şəbəkə problemi" kimi qəbul edib instansları silməməkdir. Dev-də söndürülməsi ölü instansların tez silinməsi üçündür; istehsalda adətən aktiv saxlanılır.

Klientlər FAKT (məs. `campaign-service/src/main/resources/application.yml:32-40`):

Xassə	Dəyər	Niyə
`service-url.defaultZone`	`${EUREKA_SERVER_URL:http://localhost:8761/eureka}`	Server ünvanı
`instance.prefer-ip-address`	`true`	Reyestrdə hostname əvəzinə IP
`instance.ip-address`	`${EUREKA_INSTANCE_IP_ADDRESS:127.0.0.1}`	Compose-da konteyner adı (`campaign-service`) — "IP" əvəzinə DNS adı yazılır
`instance.instance-id`	`${spring.application.name}:${server.port}`	Oxunaqlı instans id-si

NƏTİCƏ `ip-address`-ə konteyner adını vermək Docker şəbəkəsində işləyən praktik hiylədir: gateway reyestrdən `campaign-service:8082` alır və Docker DNS onu həll edir. Eyni servisin iki instansı eyni `instance-id`-ni alardı (`campaign-service:8082`) — miqyaslanma üçün id unikal olmalıdır (məs. `${random.value}` əlavə etmək).

`eureka.client.healthcheck.enabled` təyin edilməyib: **NƏTİCƏ** instansın Eureka statusu actuator health-dən deyil, yalnız heartbeat-dən asılıdır — DB-si düşmüş servis də `UP` görünür və trafik alır.

Kim istifadə edir? **FAKT** Yalnız api-gateway (marşrut `lb://` URI-ləri və OpenAPI `WebClient`-i). Qalan servislər qeydiyyatdan keçir, lakin heç kimi kəşf etmir (F31).

16.4 Spring Cloud LoadBalancer

`lb://campaign-service` URI-si `ReactiveLoadBalancerClientFilter` tərəfindən həll olunur: `ServiceInstanceListSupplier` (Eureka + caching) instans siyahısını verir, standart `RoundRobinLoadBalancer` birini seçir.

Gateway-in öz HTTP klienti üçün **FAKT** (`GatewayLoadBalancerConfiguration.java:26-30`):

```
@Bean
@LoadBalanced
public WebClient.Builder loadBalancedWebClientBuilder() {
    return WebClient.builder();
}
```

`@LoadBalanced` qualifier-i bu builder-ə load-balancer exchange filter-i əlavə edir; nəticədə `webClient.get().uri("lb://auth-service/v3/api-docs")` işləyir. **NƏTİCƏ** Diqqət: bu bean kontekstdə yeganə `WebClient.Builder` olarsa, onu inject edən **hər** komponent load-balanced builder alır; xarici URL-lərə gedən klient üçün ayrıca, `@LoadBalanced`-siz builder lazımdır.

16.5 Config Server

Server **FAKT** (`config-server/src/main/resources/application.yml:6-12`): `native` profil, `search-locations: classpath:/config` — konfiqurasiya faylları Git-dən deyil, config-server jar-ının öz classpath-indən oxunur. `config/application.yml` adı xüsusidir: **bütün** klient tətbiqlərinə paylanır (`/{application}/{profile}` sorğusunda `application.yml` həmişə daxil edilir).

Paylaşılan məzmun **FAKT** (`config-server/src/main/resources/config/application.yml`): virtual thread-lər, Kafka bootstrap və consumer parametrləri (`auto-offset-reset: earliest` , trusted packages), Eureka URL, `app.*` (yavaş sorğu həddi, mövzular, JWT issuer/public key), management/tracing.

Klientlər: `spring.config.import: optional:configserver:${CONFIG_SERVER_URL:http://localhost:8888}` (məs. `campaign-service/src/main/resources/application.yml:6-7`). Klient başlanğıcda `GET /campaign-service/default` sorğusu ilə konfiqurasiyanı alır.

Qiymətləndirmə **NƏTİCƏ**:

- **Dəyər yaratmır, risk gətirir**: paylanan xassələrin demək olar ki, hamısı servislərin lokal `application.yml`-ində də var. Fərqli olan yeganə mühüm dəyər `auto-offset-reset: earliest`-dir və o da yalnız Config Server əlçatan olduqda tətbiq olunur (F32).
- `native` + **classpath** — konfiqurasiyanı dəyişmək üçün config-server-i yenidən build və deploy etmək lazımdır; Git backend (versiyalama, audit, PR baxışı) Config Server-in əsas üstünlüyüdür.
- **Refresh yoxdur** — `@RefreshScope`, `/actuator/refresh` və ya Spring Cloud Bus istifadə olunmur; dəyişiklik yalnız restart ilə tətbiq olunur.
- **Sirrlərin idarəsi yoxdur** — `{cipher}` şifrələmə və ya Vault inteqrasiyası yoxdur (sirrlər onsuz da Compose env-dədir).
- **Discovery-first deyil** — klientlər Config Server-i Eureka ilə deyil, sabit URL ilə tapır; config-server-in Eureka-da qeydiyyatı istifadə olunmur.
- README-də göstərilən `config-repository/` qovluğu repozitoriyada yoxdur.

16.6 OpenAPI aqreqasiyası

Servislərdə: `springdoc-openapi-starter-webmvc-ui` hər servisdə `/v3/api-docs` və `/swagger-ui.html` verir (`campaign-service/pom.xml:133-137`); controller-lərdəki `@Operation` annotasiyaları sənədə düşür.

Gateway-də: `springdoc-openapi-starter-webflux-ui` Swagger UI-ı təqdim edir və onu dörd sənədə yönləndirir **FAKT** (`api-gateway/src/main/resources/application.yml:68-83`): `/openapi/auth`, `/openapi/campaign`, `/openapi/dashboard`, `/openapi/logs`. Bu URL-ləri gateway-in öz controller-i təqdim edir:

Mənbə: api-gateway/src/main/java/com/casestudy/insurancecampaign/gateway/openapi/GatewayOpenApiController.java:87-106

```
@GetMapping(value = "/{service}", produces = MediaType.APPLICATION_JSON_VALUE)
public Mono<ResponseEntity<String>> openApi(@PathVariable String service) {
    OpenApiTarget target = targets.get(service);
    if (target == null) {
        return Mono.just(ResponseEntity.status(HttpStatus.NOT_FOUND)
                .contentType(MediaType.APPLICATION_JSON)
                .body(json(Map.of(
                        "code", "OPENAPI_TARGET_NOT_FOUND",
                        "message", "Unknown OpenAPI target: " + service
                ))));
    }

    return webClient.get()
            .uri(target.upstreamUrl())
            .accept(MediaType.APPLICATION_JSON)
            .retrieve()
            .bodyToMono(String.class)
            .map(this::jsonResponse)
            .onErrorResume(exception → Mono.just(jsonResponse(fallbackDocument(target, exception))));
}
```

- Reaktiv zəncir: `WebClient` → `Mono<String>` → `ResponseEntity`; heç bir thread bloklanmır.
- `onErrorResume` — servis əlçatmaz olduqda Swagger UI-ın boş qalmaması üçün **fallback sənəd** qaytarılır. Bu, kiçik, lakin aydın **graceful degradation** nümunəsidir.
- Fallback yolları əl ilə yazılıb (`GatewayOpenApiController.java:172-219`) və kodun təkrarıdır. **FAKT** Onlar natamamdır: dashboard fallback-ında tarixçə endpoint-i, log fallback-ında `auth-audit` yoxdur. Endpoint dəyişdikcə fallback köhnələcək.
- `WebClient` çağırışında timeout yoxdur — servis asılı qalsa, Swagger UI da gözləyir (`onErrorResume` yalnız xəta siqnalında işləyir).

- [H] Canlı sənədin `servers` sahəsi springdoc tərəfindən **daxil olan sorğunun** URL-indən qurulur; sorğunu gateway-in WebClient-i servisin daxili ünvanına göndərdiyi üçün `servers` daxili host-u (`http://campaign-service:8082`) göstərə bilər və Swagger UI-ın "Try it out" funksiyası brauzerdən həmin ünvana getməyə çalışar. Bu, runtime-da yoxlanmayıb; standart həll — springdoc-da `servers`-i açıq təyin etmək və ya `X-Forwarded-*` başlıqlarını ötürüb `server.forward-headers-strategy=framework` istifadə etmək. Fallback sənəd isə `servers: http://localhost:8080`-ı açıq yazır.
- [H] (F30) springdoc 2.8.x xətti Spring Boot 3 üçün hazırlanıb; Spring Boot 4 / Framework 7 ilə rəsmi dəstəklənən xətt springdoc 3.x-dir. Uyğunsuzluq runtime xətalarına səbəb olarsa, gateway-in fallback mexanizmi bunu gizlədə bilər.

16.7 Konfiqurasiyanın izlənməsi (gateway)

Mühit dəyişəni (Compose)	Xassə	İstehlakçı
`AUTH_SERVICE_URI=lb://auth-service`	marşrut `uri`	SCG
`APP_OPENAPI_AUTH_SERVICE_URL`	`app.openapi.auth-service-url` (relaxed binding)	`GatewayOpenApiController` `@Value`
`JWT_PUBLIC_KEY_LOCATION`	`app.jwt.public-key-location`	`GatewayJwtProperties` → `GatewayJwtTokenService`
`REDIS_HOST`, `REDIS_PORT`	`spring.data.redis.*`	Reaktiv Redis → `RedisRateLimiter`
`RATE_LIMITER_*`	`redis-rate-limiter.*`	`RequestRateLimiter`
`EUREKA_SERVER_URL`	`eureka.client.service-url.defaultZone`	Eureka klienti

FAKT `application.yml`-də OpenAPI URL-lər `${AUTH_SERVICE_OPENAPI_URL:...}` yer tutanı ilə yazılıb (`api-gateway/src/main/resources/application.yml:121-126`), Compose isə `APP_OPENAPI_AUTH_SERVICE_URL` verir. **NƏTİCƏ** Spring Boot-un mühit dəyişəni relaxed binding-i `APP_OPENAPI_AUTH_SERVICE_URL`-i birbaşa `app.openapi.auth-service-url` xassəsinə bağlayır (yer tutanın özünü üstələyir), ona görə hər iki ad işləyir; dəyərlər onsuz da standart dəyərlərlə eynidir.

16.8 Uğursuzluq ssenariləri

Ssenari	Nəticə
Eureka düşüb (gateway işləyir)	Gateway-in lokal reyestr cache-i ilə köhnə instanslar istifadə olunur; yeni instanslar görünmür
Servis qeydiyyatdan keçməyib / hamısı düşüb	`lb://` həll olunmur → 503
Servis instansı ölüb, reyestrdə hələ var	Bağlantı xətası → 500/503, lease bitənə qədər (90 s + cache)
Servis asılı qalıb	Gateway timeout yoxdur → sorğu gözləyir
Config Server düşüb	`optional:` → servislər lokal konfiqurasiya ilə başlayır
Redis düşüb	Rate limit fail-open

16.9 Alternativlər və yeni layihədə tətbiq

Platforma ehtiyacı	Spring Cloud həlli (layihədə)	Kubernetes-də tipik həll
Servis kəşfi	Eureka	Kubernetes Service + DNS
Yük balansı	Spring Cloud LoadBalancer	kube-proxy / service mesh
Konfiqurasiya	Config Server	ConfigMap, Secret, external secrets

Platforma ehtiyacı	Spring Cloud həlli (layihədə)	Kubernetes-də tipik həll
Gateway	Spring Cloud Gateway	Ingress / Gateway API (Envoy, NGINX), və ya yenə SCG
Resilience	Resilience4j	Service mesh (Istio) retry/timeout/CB + Resilience4j

Kubernetes-ə köçürülən sistemdə Eureka və Config Server çox vaxt lazımsız olur; Spring Cloud Kubernetes adapterləri eyni proqramlaşdırma modelini saxlamağa imkan verir.

Yeni gateway üçün yoxlama siyahısı: marşrut başına timeout; retry yalnız idempotent metodlar üçün (GET); circuit breaker + fallback; vahid xəta formatı (`ErrorWebExceptionHandler`); CORS; etibarlı XFF; marşrut başına rate limit; trace yayılması; açıq yolların dəqiq siyahısı.

16.10 Müsahibə sualları

S1. Gateway-də GlobalFilter və GatewayFilter fərqi nədir? Cavab: GatewayFilter konkret marşruta (və ya `default-filters` ilə hamısına) konfiqurasiya ilə bağlanır; GlobalFilter bean kimi bütün marşrutlara tətbiq olunur. İcra zamanı hamısı bir siyahıda `Ordered` dəyərinə görə sıralanır.

S2. `lb://` URI-si necə həll olunur? Cavab: `ReactiveLoadBalancerClientFilter` servis adını `ReactorLoadBalancer`-ə ötürür, o, `ServiceInstanceListSupplier` (Eureka reyestri + cache) siyahısından RoundRobin ilə instans seçir və URI-ni `http://host:port`-a çevirir.

S3. Eureka self-preservation nədir? Cavab: Heartbeat-lərin gözləniləndən çox azalması zamanı Eureka instansları silməyi dayandırır, çünki bunu instansların deyil, şəbəkənin problemi kimi qiymətləndirir. Səhv-müsbət silinmələrin qarşısını alır, lakin ölü instansların reyestrdə qalmasına səbəb ola bilər.

S4. Config Server-dən gələn xassə ilə lokal xassə toqquşduqda hansı qalib gəlir? Config Server optional olduqda risk nədir? Cavab: Standart olaraq remote (Config Server) xassələri. Optional olduqda server əlçatmaz olsa tətbiq başlayır, lakin yalnız Config Server-də olan dəyərlər itir və davranış sakitcə dəyişir (layihədə `auto-offset-reset`).

S5. Gateway-də niyə timeout təyin etmək vacibdir? Cavab: Gateway bütün trafikin keçdiyi yerdir; bir yavaş servis gateway-in bağlantılarını və yaddaşını tutub digər marşrutlara da təsir edə bilər. Timeout uğursuzluğu lokallaşdırır.

S6. Eureka olmadan bu sistemi necə qurardınız? Cavab: Kubernetes-də Service DNS (`http://campaign-service:8082`), ConfigMap/Secret, Ingress və ya Gateway API; və ya sadə Compose-da static DNS adları (layihədə dashboard və auth onsuz da belə edir).

16.11 Xülasə

- SCG WebFlux: 8 marşrut, default rate limit, iki layihə qlobal filtri; timeout, retry və CB marşrutlarda yoxdur.
- Eureka bütün servislərin reyestridir, lakin yalnız gateway kəşfdən istifadə edir; health check inteqrasiyası yoxdur.
- Config Server `native` classpath rejimindədir; paylaşılan fayl əsasən təkrardır, `auto-offset-reset` istisnası risk yaradır.
- Gateway OpenAPI controller-i canlı sənədləri `lb://` WebClient ilə toplayır, xətada fallback verir.
- Növbəti fəsil metriklər, loglar və trace-lərdən ibarət observability stekini öyrədir.

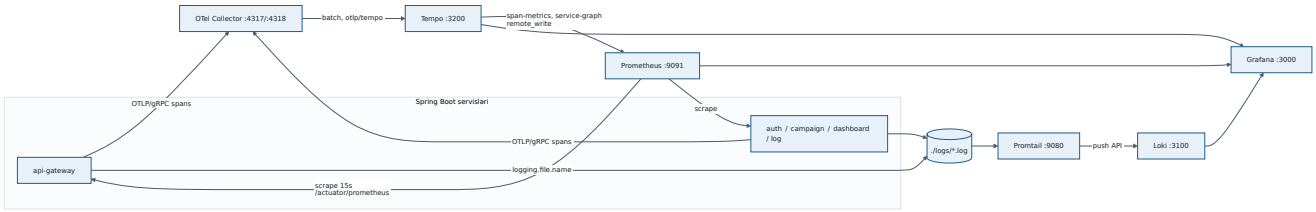
17. Observability: metriklər, trace-lər və loglar

17.1 Motivasiya: "nə baş verir?" sualına cavab vermək

Monolitdə bir log faylı və bir stack trace çox vaxt kifayətdir. Mikroservislərdə bir sorğu gateway, iki servis, Redis, Kafka və Elasticsearch-dən keçir. Problemi tapmaq üçün üç növ siqnal lazımdır:

Siqnal	Sual	Layihədə
Metriklər	Nə qədər? Nə qədər tez? Nə qədər xəta? (aqreqat, ucuz)	Micrometer → `/actuator/prometheus` → Prometheus
Trace-lər	Bu konkret sorğu hansı komponentlərdən keçdi, vaxt harada itdi?	Micrometer Tracing → OpenTelemetry → Collector → Tempo
Loglar	Hansı hadisə baş verdi, hansı detal ilə?	Log faylları → Promtail → Loki
Hamısının birləşməsi	Bir siqnaldan digərinə keçid	Grafana (traceId ilə korrelyasiya)

Əlavə olaraq layihədə tətbiq səviyyəli iki "jurnal" var: Elasticsearch-dəki **yavaş sorğular** və **auth auditləri**.



Observability axınları (docker-compose.yml və observability/ konfiqurasiyasına görə)

17.2 Metriklər: Actuator, Micrometer, Prometheus

Micrometer metrik fasadıdır (SLF4J-in metrik analoqu): kod `Timer`, `Counter`, `Gauge` yaradır, registry onları konkret formata (Prometheus, OTLP, ...) ixrac edir. Spring Boot avtomatik olaraq JVM (yaddaş, GC, thread-lər), sistem, HTTP server (`http.server.requests`), HikariCP, Tomcat, Kafka consumer (Boot factory-si olduqda), cache və s. metriklərini qeydiyyatdan keçirir.

Layihə konfiqurasiyası **FAKT** (məs. `campaign-service/src/main/resources/application.yml:50-59`):

```yaml
management:
  otlp:
    metrics:
      export:
        enabled: false
  endpoints:
    web:
      exposure:
        include: health,info,prometheus,metrics
```

- `micrometer-registry-prometheus` → `/actuator/prometheus` endpoint-i (mətn formatı).
- OTLP metrik ixracı söndürülüb — metriklər yalnız Prometheus **pull** modeli ilə toplanır; OTLP yalnız trace-lər üçündür.
- `health,info,prometheus,metrics` açıqdır və security-də `permitAll`-dur (F40).

Prometheus scrape **FAKT** (`observability/prometheus/prometheus.yml:1-24`): 15 s interval, beş iş (job): `api-gateway`, `auth-service`, `campaign-service`, `dashboard-service`, `log-service`. **FAKT** Docker variantında config-server və discovery-server scrape olunmur; lokal variant (`prometheus-local.yml`) isə onları da

`host.docker.internal` ilə scrape edir — iki fayl arasında uyğunsuzluq.

Prometheus Compose-da `--web.enable-remote-write-receiver` bayrağı ilə işləyir (`docker-compose.yml:378-382`) — bu, Tempo-nun generasiya etdiyi span metriklərini qəbul etmək üçündür (aşağıda).

Faydalı sorğular (PromQL):

```
# Servis üzrə sorğu sürəti (Grafana dashboard-undakı ilə eyni)
sum by (job) (rate(http_server_requests_seconds_count{uri!~"/actuator.*"}[5m]))
# 5xx nisbəti
sum by (job) (rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
  / sum by (job) (rate(http_server_requests_seconds_count[5m]))
# Orta gecikmə (histogram aktiv olmadıqda yalnız orta və max mövcuddur)
sum by (job) (rate(http_server_requests_seconds_sum[5m])) / sum by (job) (rate(http_server_requests_secon
ds_count[5m]))
# Bağlantı hovuzu gözləmələri
hikaricp_connections_pending
```

NƏTİCƏ `management.metrics.distribution.percentiles-histogram.http.server.requests` təyin edilməyib — Prometheus-da p95/p99 hesablamaq üçün histogram bucket-ləri yoxdur, yalnız `count`, `sum` və `max`. Gecikmə SLO-ları üçün histogram aktivləşdirilməlidir.

17.3 Distributed tracing

Nəzəriyyə

- **Trace** — bir sorğunun bütün sistem üzrə yolu; **span** — bir işin vahidi (HTTP server sorğusu, HTTP klient çağırışı, DB sorğusu); span-ların valideyn-uşaq ağacı trace-i təşkil edir.
- **Kontekst yayılması (propagation)** — trace id və valideyn span id prosesdən prosesə başlıqla ötürülür. **W3C Trace Context** standartı: `traceparent: 00-<32 hex trace-id>-<16 hex span-id>-<flags>`.
- **Sampling** — hansı trace-lərin saxlanacağı. Layihədə `probability: 1.0` (hamısı) — dev üçün uyğun, yüksək trafikdə bahalıdır (tipik dəyər 0.01–0.1 və ya tail sampling Collector-da).

Spring Boot 4 tracing steki

`spring-boot-starter-opentelemetry` (biznes servisləri) Micrometer Observation API-ni Micrometer Tracing → OpenTelemetry bridge → OTLP exporter zəncirinə bağlayır. config-server və discovery-server isə eyni nəticəni `micrometer-tracing-bridge-otel` + `opentelemetry-exporter-otlp` asılılıqları ilə alır (`config-server/pom.xml:29-36`).

Konfiqurasiya FAKT (`campaign-service/src/main/resources/application.yml:60-76`):

Mənbə: campaign-service/src/main/resources/application.yml:60-76

```
tracing:
  sampling:
    probability: ${MANAGEMENT_TRACING_SAMPLING_PROBABILITY:1.0}
  propagation:
    type: w3c
  export:
    otlp:
      enabled: ${MANAGEMENT_TRACING_EXPORT_OTLP_ENABLED:true}
opentelemetry:
  resource-attributes:
    service.name: ${OTEL_SERVICE_NAME:${spring.application.name}}
  enabled: true
  tracing:
    export:
      otlp:
```

```
        endpoint: ${OTEL_EXPORTER_OTLP_TRACES_ENDPOINT:http://localhost:4317}
        transport: ${OTEL_EXPORTER_OTLP_TRACES_PROTOCOL:grpc}
```

- `transport: grpc` → OTLP/gRPC, port 4317. OTLP/HTTP (4318) üçün endpoint `/v1/traces` ilə bitməlidir; README bu səhvin simptomunu (`404` export xətası) təsvir edir.
- `service.name` resurs atributu Tempo-da servisi müəyyən edir (`{ resource.service.name = "campaign-service" }`).
- **NƏTİCƏ** Compose-dakı `OTEL_TRACES_EXPORTER` və `OTEL_RESOURCE_ATTRIBUTES` dəyişənləri OpenTelemetry SDK autoconfigure konvensiyalarıdır; Spring Boot-un öz OpenTelemetry auto-configuration-u əsasən `management.*` xassələrindən istifadə edir. Yer tutanlarla xassələrə bağlanan dəyişənlər (`OTEL_EXPORTER_OTLP_TRACES_ENDPOINT`, `OTEL_SERVICE_NAME`) mütləq təsir edir; qalanlarının təsirini bu kitabda yoxlamamışam.

Observation fərdiləşdirməsi

`common` modulundakı konfiqurasiya iki bean təqdim edir **FAKT** (`HttpServerObservationConfiguration.java:37-83`):

Mənbə: common/src/main/java/com/casestudy/insurancecampaign/common/observability/HttpServerObservationConfiguration.java:37-47

```
@Bean
public ObservationPredicate ignoreActuatorHttpServerObservations() {
    return (name, context) -> {
        if (!HTTP_SERVER_REQUESTS.equals(name)) {
            return true;
        }

        String path = requestPath(context);
        return path == null || !path.startsWith("/actuator");
    };
}
```

- **`ObservationPredicate`** observasiyanın ümumiyyətlə yaradılıb-yaradılmayacağını həll edir. Actuator sorğuları üçün `false` qaytarır. **NƏTİCƏ** Nəticə: Prometheus scrape-ləri Tempo-nu doldurmur — **həm də** actuator sorğuları `http_server_requests` metriklərində görünmür, çünki eyni observasiya həm metrik, həm span yaradır.
- **`ObservationFilter`** (`:58-83`) span-ın kontekstual adını `http <method> <route>` edir (məs. `http get /api/v1/campaigns/{id}`) — route şablonu (aşağı kardinallıq) istifadə olunur, konkret id yox; bu, Tempo-da axtarış və span-metric qruplaşdırması üçün vacibdir.
- `requestPath` (`:111-132`) refleksiya ilə həm Servlet (`getRequestURI`), həm WebFlux (`getURI`) carrier-lərini dəstəkləyir — eyni kodun gateway-də də işləməsi üçün. Lakin gateway `common`-dan asılı olmadığı üçün kod üç dəfə kopyalanıb (F23).

Kontekst sistem üzrə necə yayılır

Sərhəd	Yayılırmı	Mexanizm / qeyd
Klient → gateway → servis (HTTP)	Bəli **NƏTİCƏ**	SCG Netty klienti instrumentasiya olunur, `traceparent` əlavə edilir
auth → log (`RestClient` HTTP Interface)	Ehtimal ki, bəli **NƏTİCƏ**	Boot-un `RestClient.Builder`-i observation ilə konfiqurasiya olunur
`@Async` sərhədi (audit)	**Xeyr** **NƏTİCƏ**	`TaskExecutorAdapter(newVirtualThreadPerTaskExecutor())` kontekst ötürmür (F34) → audit çağırışı ayrı trace kimi görünə bilər

Sərhəd	Yayılırmı	Mexanizm / qeyd
dashboard → campaign (gRPC)	Ehtimal ki, bəli **NƏTİCƏ**	net.devh `Grpc*MicrometerTraceAutoConfiguration` (`ObservationRegistry` bean-i olduqda)
campaign/log → Kafka → log-service	**Xeyr**	`KafkaTemplate` əl ilə yaradılıb, observation aktiv deyil; əvəzinə `traceId` payload-da ötürülür
Servis → PostgreSQL/Redis	Qismən	JDBC üçün xüsusi instrumentasiya (datasource-micrometer) yoxdur; Lettuce Micrometer inteqrasiyası Boot-da var

Collector və Tempo

OpenTelemetry Collector **FAKT** (`observability/otel-collector/otel-collector.yml:1-25`): `otlp` receiver (gRPC 4317, HTTP 4318) → `batch` processor → `otlp/tempo` (TLS-siz) və `debug` exporter-ləri. Collector-un faydası: tətbiqləri backend-dən ayırır (backend dəyişsə tətbiq dəyişmir), batch, retry, sampling, atribut emalı bir yerdə.

Tempo **FAKT** (`observability/tempo/tempo.yml`): lokal disk saxlaması; `metrics_generator` `span-metrics`, `service-graphs` və `service-graphs-connection-info` prosessorları ilə trace-lərdən RED metrikləri və servis qrafını çıxarıb Prometheus-a `remote_write` edir (`tempo.yml:16-28`, `40-44`). Grafana-nın Tempo datasource-u servis xəritəsi üçün məhz bu metriklərdən istifadə edir.

TraceQL nümunələri (README-dən):

```
{ resource.service.name = "campaign-service" && duration > 500ms }
{ status = error }
```

17.4 Loglar: fayllar, Promtail, Loki

Hər servis log-u fayla yazır və səviyyə nümunəsinə trace kontekstini əlavə edir **FAKT** (`campaign-service/src/main/resources/application.yml:87-91`):

```
logging:
  file:
    name: ${LOGGING_FILE_NAME:logs/campaign-service.log}
  pattern:
    level: '%5p [campaign-service,%X{traceId:-},%X{spanId:-}]'
```

`logging.pattern.level` Boot-un standart sətir nümunəsindəki səviyyə hissəsini əvəzləyir: `INFO [campaign-service, 4bf92f3577b34da6a3ce929d0e0e4736,00f067aa0ba902b7]`. `%X{traceId:-}` MDC-dən oxuyur — Micrometer Tracing aktiv span-ın id-lərini MDC-yə yazır.

Compose bütün servislərin `/app/logs`-unu host-un `./logs` qovluğuna bağlayır, Promtail həmin qovluğu oxuyur.

Promtail pipeline-ı **FAKT** (`observability/promtail/promtail-config.yml:39-49`):

Mənbə: observability/promtail/promtail-config.yml:39-49

```
pipeline_stages:
  - regex:
      expression: '^(?P<timestamp>\d{4}-\d{2}-\d{2}T\S+)\s+(?P<level>TRACE|DEBUG|INFO|WARN|ERROR)\s+\
[[^,\]]+,(?P<trace_id>[^,]*),(?P<span_id>[^\]]*)\].*$'
  - timestamp:
      source: timestamp
```

```
      format: RFC3339Nano
      action_on_failure: fudge
  - labels:
      level:
      trace_id:
      span_id:
```

- Regex Boot-un ISO vaxt damğası + yuxarıdakı səviyyə nümunəsi ilə uyğunlaşdırılıb.
- **RİSK** (F24) `trace_id` və `span_id` **label**-a çevrilir. Loki-də hər unikal label dəyəri kombinasiyası ayrı **stream** yaradır; hər sorğunun yeni trace id-si olduğu üçün stream sayı sorğu sayı qədər artır — indeks şişir, performans düşür. Loki-nin tövsiyəsi: yüksək kardinallıqlı dəyərləri label etməmək, sorğuda filtr (`⊨ "<traceId>"`) və ya **structured metadata** istifadə etmək.
- Çoxsətirli stack trace-lər üçün `multiline` mərhələsi yoxdur — hər stack trace sətri ayrı log qeydi olur.
- Yalnız beş servisin faylı toplanır; config-server və discovery-server log-ları (fayla yazılsa da) Loki-yə getmir.
- **NƏTİCƏ** Grafana Promtail-i köhnəlmiş (deprecated) elan edib və Grafana Alloy-a keçidi tövsiyə edir; yeni layihədə Alloy və ya OpenTelemetry Collector-un `filelog` receiver-i istifadə olunmalıdır.

Loki **FAKT** (`observability/loki/loki-config.yml`): auth söndürülüb, fayl sistemi saxlaması, TSDB sxemi v13, `inmemory` ring, replikasiya 1 — tək node-lu dev konfiqurasiyası; saxlanma (retention) siyasəti yoxdur.

17.5 Grafana: korrelyasiya

Datasource provisioning **FAKT** (`observability/grafana/provisioning/datasources/datasources.yml`):

Datasource	UID	Korrelyasiya
Prometheus	`prometheus`	Standart datasource
Loki	`loki`	**Derived field** `TraceID` : regex `\[[^,]+,([a-fA-F0-9]{16,32}),[a-fA-F0-9]{16}\]` (`datasources.yml:24-29`) log sətrindən trace id-ni çıxarır və Tempo-ya link edir
Tempo	`tempo`	**tracesToLogsV2** — trace-dən Loki-yə: `{job=~".*-logs", trace_id="${__trace.traceId}"}` (`:39-49`); `serviceMap` → Prometheus

Beləliklə üç istiqamətdə keçid mümkündür: log → trace (derived field), trace → loglar (tracesToLogs), metrik → trace (exemplar-lar aktiv olsaydı; Tempo `send_exemplars: true` göndərir).

Dashboard **FAKT** (`observability/grafana/dashboards/campaign-logs-traces.json`): "Campaign - Logs and Traces", servis seçici dəyişəni və üç panel — Loki log siyahısı, Prometheus HTTP sorğu sürəti, səviyyə üzrə log sayı.

17.6 Sağlamlıq yoxlamaları

- `/actuator/health` hər servisdə açıqdır.
- Compose-da healthcheck yalnız infrastruktur üçündür (PostgreSQL, Redis, Kafka, Elasticsearch) — Spring servisləri üçün `healthcheck` yoxdur, ona görə `depends_on: condition: service_healthy` yalnız infrastruktura tətbiq oluna bilir.
- Eureka statusu health-ə bağlı deyil (fəsil 16.3).
- Kubernetes-də Boot liveness/readiness qruplarını (`/actuator/health/liveness` , `/readiness`) avtomatik aktivləşdirir; Compose-da bunlar istifadə olunmur.

17.7 Tətbiq səviyyəli jurnallar və metriklərin üst-üstə düşməsi

Yavaş sorğu jurnalı (Kafka → ES) ilə `http_server_requests` metrikləri və trace-lər eyni sualın fərqli cavablarıdır: metriklər aqreqat gecikməni, trace-lər konkret sorğunun strukturunu, ES qeydi isə konkret sorğunun metadata-sını (yol, status, müddət, user-agent, XFF, traceId) verir. **NƏTİCƏ** Tempo + histogram metrikləri olduqda ES yavaş sorğu jurnalı əsasən təkrardır; onun dəyəri case study-də Kafka → ES axınını nümayiş etdirməkdir. İstehsalda bu məlumat adətən trace backend-dən (TraceQL `duration > X`) alınır.

17.8 Real insident araşdırması: "Dashboard statistikası 500 qaytarır"

1. **Simptom**: admin `GET /api/v1/dashboard/classifieds/statistics` → 500, `{"code":"INTERNAL_SERVER_ERROR","message":"UNAVAILABLE: io exception"}`.
2. **Metrik**: Grafana → `http_server_requests_seconds_count{job="dashboard-service",status="500"}` artıb; `campaign-service` üçün sorğu sürəti sıfıra düşüb → campaign-service problemi ehtimalı.
3. **Trace**: Tempo → `{ resource.service.name = "dashboard-service" && status = error }`. Trace-də gateway span-ı → dashboard server span-ı → gRPC klient span-ı xəta ilə; campaign-service span-ı yoxdur → çağırış ora çatmayıb.
4. **Log**: Loki → `{job="dashboard-service-logs", level="ERROR"}`. **RİSK** (F38) `GlobalExceptionHandler` loglamadığı üçün burada stack trace **tapılmayacaq** — bu, araşdırmanı çətinləşdirən real boşluqdur. `{job="campaign-service-logs"}` — son log-lar startup xətası və ya OOM göstərə bilər.
5. **Platforma**: `docker compose ps campaign-service`, Eureka dashboard-da instans statusu.
6. **Kök səbəb** (nümunə): campaign-service `campaign-db` hazır olmadan başlayıb, Liquibase bağlantı xətası ilə çıxıb, `restart` siyasəti olmadığı üçün yenidən başlamayıb (F29).
7. **Düzəliş və qarşısının alınması**: `depends_on: condition: service_healthy`, `restart: on-failure`, gRPC klientində status → 503 xəritəsi, exception handler-də loglama.

17.9 Yeni layihədə tətbiq

- RED metrikləri (Rate, Errors, Duration) hər servis üçün; histogram + SLO alert-ləri (məs. "5 dəq ərzində 5xx > 1%").
- Strukturlaşdırılmış JSON loglar: Spring Boot 3.4+ `logging.structured.format.console=ecs|logstash|gelf` — trace id-lər ayrıca sahə kimi; regex parsing lazım olmur.
- Kafka observation: `KafkaTemplate.setObservationEnabled(true)`, konteynerdə `setObservationEnabled(true)` (və ya Boot xassələri `spring.kafka.template.observation-enabled`, `spring.kafka.listener.observation-enabled`).
- `@Async` üçün `ContextPropagatingTaskDecorator`.
- İstehsalda sampling < 100%, Collector-da tail sampling (xətalı və yavaş trace-lər həmişə saxlanılır).
- Loki label-ları: yalnız aşağı kardinallıqlı (`service`, `level`, `env`).
- Actuator-u ayrı port və şəbəkə ilə qorumaq.

17.10 Müsahibə sualları

S1. Metrik, log və trace fərqi və hər birinin xərci? Cavab: Metrik — aqreqat ədədlər, ucuz, uzunmüddətli saxlanılır, "nə qədər" sualına cavab. Log — diskret hadisələr, detallıdır, həcmi böyükdür. Trace — bir sorğunun komponentlər üzrə strukturu, "harada" sualına cavab; sampling tələb edir. Yaxşı sistem onları id-lərlə əlaqələndirir.

S2. W3C `traceparent` başlığında nə var? Cavab: Versiya, 16 baytlıq trace-id, 8 baytlıq valideyn span-id, flags (sampled). Hər hop yeni span yaradır, trace-id eyni qalır.

S3. Loki-də trace_id-ni label etmək niyə pisdir? Cavab: Hər label kombinasiyası ayrı stream-dir; trace_id unikal olduğu üçün stream sayı partlayır, indeks böyüyür, sorğular yavaşlayır. Trace id log mətnində və ya structured metadata-da qalmalı, sorğuda filtr ilə tapılmalıdır.

S4. `ObservationPredicate` actuator-u süzəndə metriklərə nə olur? Cavab: Observasiya ümumiyyətlə yaradılmır — həm span, həm `http.server.requests` timer-i yox olur. Yalnız trace-ləri süzmək üçün span səviyyəli filtr (məs. sampler və ya `SpanExportingPredicate`) istifadə olunmalıdır.

S5. Kafka sərhədində trace kontekstini necə ötürmək olar? Cavab: Producer observation `traceparent`-i Kafka başlığına yazır, consumer observation onu oxuyub yeni span-ı valideynə bağlayır. Spring Kafka-da `observationEnabled`. Layihədə bu yoxdur — traceId payload-da daşınır, Tempo-da əlaqə qurulmur.

S6. Collector-un tətbiqdən birbaşa Tempo-ya göndərməkdən üstünlüyü? Cavab: Backend müstəqilliyi, batch/retry, sampling (xüsusilə tail sampling), atributların zənginləşdirilməsi və ya təmizlənməsi (PII), bir neçə backend-ə eyni anda göndərmə, tətbiq konfiqurasiyasının sadələşməsi.

17.11 Xülasə

- Metriklər Prometheus ilə pull, trace-lər OTLP/gRPC → Collector → Tempo, loglar fayl → Promtail → Loki; Grafana hamısını traceId ilə əlaqələndirir.
- Span adları route şablonu ilə, actuator süzülür (həm metrikdən, həm trace-dən).
- Boşluqlar: Kafka və `@Async` sərhədlərində trace yayılmır; Loki-də yüksək kardinallıqlı label-lar; histogram yoxdur; exception handler loglamır; 100% sampling.
- Növbəti fəsil bütün bu komponentlərin konteynerlərdə necə işə salındığını və CI/CD-ni izah edir.

18. Konteynerlər, Docker Compose və CI/CD

18.1 Motivasiya

Yeddi Spring Boot tətbiqi və on iki infrastruktur komponenti olan sistemi əl ilə işə salmaq mümkün deyil. Konteynerlər hər komponenti eyni qaydada paketləyir; Compose onları bir şəbəkədə, düzgün parametrlərlə birlikdə başladır; CI/CD isə hər dəyişikliyi avtomatik yoxlayıb image-ləri nəşr edir. Bu fəsil layihənin bu üç qatını izah edir və istehsal səviyyəsinə çatmaq üçün nəyin çatışmadığını göstərir.

18.2 Dockerfile-lar

Bütün tətbiq Dockerfile-ları eyni şablondadır (fəsil 3.11-də tam çıxarış: `campaign-service/Dockerfile:1-26`):

Addım	Nə edir	Qiymətləndirmə
`FROM maven:3.9.11-amazoncorretto-25 AS build`	Build mərhələsi	Versiya bərkidilib — yaxşı
`COPY . .`	Bütün repozitoriya (build konteksti = kök)	Hər fayl dəyişikliyi asılılıqların yenidən endirilməsinə səbəb olur
`RUN mvn -pl <svc> -am clean package -DskipTests`	Servis və asılı modullar	Testlər CI-da ayrıca işləyir — məqbul
`FROM amazoncorretto:25`	Runtime	Tam JDK — image böyükdür; JRE və ya distroless kifayətdir
`COPY --from=build .../target/<svc>-*.jar <svc>.jar`	Fat jar	Layered jar istifadə olunmur
`EXPOSE`	Sənədləşmə	—
`ENTRYPOINT ["java", "--enable-preview", "-jar", ...]`	İşə salma	Exec forması (PID 1 = java, SIGTERM düzgün çatır) — yaxşı

Çatışmayanlar **NƏTİCƏ**:

- **Root istifadəçi** — `USER` direktivi yoxdur; konteyner qaçışı (escape) zamanı riski artırır.
- **`HEALTHCHECK` yoxdur** — orkestrator tətbiqin hazır olduğunu bilmir.
- **JVM yaddaş parametrləri yoxdur** — müasir JVM konteyner limitlərini tanıyır, lakin standart heap konteyner yaddaşının 25%-idir; Spring tətbiqləri üçün adətən `-XX:MaxRAMPercentage=75` təyin edilir.
- **Layer cache** — asılılıqlar mənbə ilə eyni layer-dadır.

İllüstrativ təkmilləşdirilmiş Dockerfile (repozitoriyada yoxdur):

```
FROM maven:3.9.11-amazoncorretto-25 AS build
WORKDIR /workspace
COPY pom.xml .
COPY common/pom.xml common/ ; COPY security/pom.xml security/ ; COPY grpc-contract/pom.xml grpc-contract/
COPY campaign-service/pom.xml campaign-service/
RUN --mount=type=cache,target=/root/.m2 mvn -q -pl campaign-service -am dependency:go-offline
COPY . .
RUN --mount=type=cache,target=/root/.m2 mvn -q -pl campaign-service -am package -DskipTests \
 && java -Djarmode=tools -jar campaign-service/target/campaign-service-*.jar extract --layers --destinati
on /extracted

FROM amazoncorretto:25-al2023-headless
RUN useradd --system --uid 10001 app
USER app
WORKDIR /app
COPY --from=build /extracted/dependencies/ ./
```

```
COPY --from=build /extracted/spring-boot-loader/ ./
COPY --from=build /extracted/snapshot-dependencies/ ./
COPY --from=build /extracted/application/ ./
ENV JAVA_TOOL_OPTIONS="-XX:MaxRAMPercentage=75 --enable-preview"
ENTRYPOINT ["java", "-jar", "campaign-service-1.0.0.jar"]
```

Alternativ: `mvn spring-boot:build-image` (Cloud Native Buildpacks) — Dockerfile yazmadan qatlı, non-root, optimallaşdırılmış image.

18.3 `docker-compose.yml`: tam stek

FAKT Bir `insurance-network` bridge şəbəkəsi (`docker-compose.yml:497-499`), on bir adlı volume (`:485-496`) və `./logs` bind mount-u. Konteynerlər bir-birini servis adı ilə tapır (Docker daxili DNS).

Başlanğıc asılılıqları

Servis	`depends_on`	Şərt	`restart`
discovery-server	otel-collector	started	—
config-server	discovery-server, otel-collector	started	—
api-gateway	config, discovery, otel, redis	started	—
auth-service	auth-db	**service_healthy**	`unless-stopped`
campaign-service	campaign-db, redis, kafka, config, otel	**started (sağlamlıq yoxlanmır)**	**yoxdur**
dashboard-service	campaign-service, config, otel	started	`unless-stopped`
log-service	kafka, elasticsearch	**service_healthy**	`on-failure:5`

RİSK (F29) NƏTİCƏ:

- campaign-service PostgreSQL-in sağlamlığını gözləmir; DB qəbul etməyə hazır olmadan başlayarsa Liquibase bağlantı xətası ilə çıxar və `restart` siyasəti olmadığı üçün **dayanmış qalar**. auth-service-də isə həm sağlamlıq şərti, həm restart var — iki servis arasında ardıcıllıq yoxdur.
- Servislər Config Server-i yalnız `service_started` ilə gözləyir. Config Server HTTP-ni hələ açmayıbsa, `optional:configserver:` sakitcə uğursuz olur və servis paylaşılan konfiqurasiyasız işləyir (F32) — bu ssenari Compose-da real olaraq yarana bilər.
- Spring servisləri üçün healthcheck olmadığı üçün başqa servislər onların hazır olmasını gözləyə bilmir.

Digər Compose müşahidələri

- **`latest` teqləri və `pull_policy: always`** (PostgreSQL, Redis, Kafka, Prometheus, Grafana, Tempo, Loki, Collector, ...) — hər işə salmada yeni versiya gələ bilər; README bunu "local case study" üçün qəbul edir. Nəticə: build-lər təkrar oluna bilmir, qəfil uyğunsuzluq yarana bilər. Elasticsearch istisnadır (`9.4.2`). Testcontainers isə fərqli image-lər istifadə edir (F28).
- **PostgreSQL volume yolu** `/var/lib/postgresql` (`docker-compose.yml:109`, `128`) — README-yə görə yeni PostgreSQL image-lərinin data qovluğu düzülüşünə uyğunlaşmaq üçündür.
- `container_name` sabitdir — `docker compose up --scale campaign-service=2` mümkün deyil.
- **Portlar host-a açıqdır** — bütün servislər və DB-lər (fəsil 10.8).
- **Sirrlər açıq mətndə** — DB şifrələri, `GF_SECURITY_ADMIN_PASSWORD` (dəyər redaktə olunub; `docker-compose.yml:396-398`); JWT private key image daxilində (F22).
- **Kafka KRaft** dinləyiciləri fəsil 11.10-da izah olunub.

Mühit dəyişənləri → xassələr

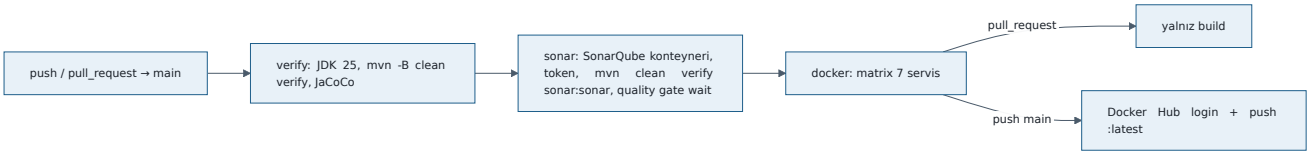
Dəyişən	Servis(lər)	Xassə
`CONFIG_SERVER_URL`	gateway, biznes servisləri	`spring.config.import`
`EUREKA_SERVER_URL`, `EUREKA_INSTANCE_IP_ADDRESS`	hamısı	`eureka.client.service-url.defaultZone`, `eureka.instance.ip-address`
`AUTH_DB_URL/USERNAME/PASSWORD`	auth	`spring.datasource.*`
`CAMPAIGN_DB_URL/...`	campaign	`spring.datasource.*`
`REDIS_HOST/PORT`	gateway, campaign	`spring.data.redis.*`
`KAFKA_BOOTSTRAP_SERVERS`	campaign, log	`spring.kafka.bootstrap-servers`
`ELASTICSEARCH_URIS`, `SPRING_ELASTICSEARCH_*`	log	`spring.elasticsearch.*`
`LOG_SERVICE_URL`	auth	`spring.http.serviceclient.log-service.base-url`
`CAMPAIGN_GRPC_ADDRESS`	dashboard	`grpc.client.campaign-service.address`
`JWT_PUBLIC_KEY_LOCATION`, `JWT_PRIVATE_KEY_LOCATION`	hamısı / auth	`app.jwt.*-location`
`LOG_SERVICE_CB_*`, `LOG_SERVICE_RETRY_*`, `LOG_SERVICE_BULKHEAD_*`	auth	`resilience4j.*`
`RATE_LIMITER_*`	gateway	`redis-rate-limiter.*`
`OTEL_EXPORTER_OTLP_TRACES_ENDPOINT/PROTOCOL`, `OTEL_SERVICE_NAME`	hamısı	`management.opentelemetry.*`
`MANAGEMENT_TRACING_*`	hamısı	`management.tracing.*`
`LOGGING_FILE_NAME`	hamısı	`logging.file.name`

18.4 `docker-compose-local.yml`: yalnız infrastruktur

IDE-dən işə salınan servislər üçün eyni infrastrukturu verir, fərqlər **FAKT**:

- ayrı layihə adı (`insurance-campaign-local`), şəbəkə və volume adları;
- Prometheus `prometheus-local.yml` ilə host-dakı servisləri `host.docker.internal` vasitəsilə scrape edir və `extra_hosts: host.docker.internal:host-gateway` (Linux-da bu ad standart olaraq mövcud deyil) (`docker-compose-local.yml:124-134`);
- **FAKT** lokal Prometheus-da `--web.enable-remote-write-receiver` bayrağı yoxdur — **NƏTİCƏ** Tempo-nun span metriklərini `remote_write` etməsi lokal rejimdə uğursuz olar və Grafana-da servis xəritəsi boş qalar.

18.5 CI/CD: `.github/workflows/ci-cd.yml`



GitHub Actions pipeline (ci-cd.yml)

verify (`ci-cd.yml:21-49`): Temurin JDK 25, Maven cache, `mvn -B clean verify` — unit və Testcontainers testləri (GitHub-un ubuntu runner-ində Docker var), JaCoCo hesabatları.

sonar (`ci-cd.yml:51-257`) — maraqlı həll: SonarQube serveri **pipeline daxilində konteyner kimi** qaldırılır (`docker compose -f docker-compose-local.yml up -d sonarqube`), API ilə hazır olması gözlənilir, standart admin şifrəsi dəyişdirilir, token yaradılır, analiz `-Dsonar.qualitygate.wait=true` ilə işləyir (quality gate uğursuzdursa job düşür), nəticələr artefakt və step summary kimi nəşr olunur. **NƏTİCƏ** Qiymətləndirmə:

- Üstünlük: xarici Sonar serveri tələb olunmur, hər işə salma təcrid olunub.
- Çatışmazlıq: Sonar tarixçəsi saxlanılmır (konteyner hər dəfə silinir) — "yeni kod" müqayisəsi və trendlər işləmir; testlər iki dəfə icra olunur (verify və sonar job-larında `clean verify`); pipeline uzanır.
- Workflow faylında ephemeral konteynerin yeni admin şifrəsi açıq yazılıb (`ci-cd.yml:96-115`, maskalanır) — ephemeral olduğu üçün risk aşağıdır, lakin sirrləri koda yazmaq vərdişi tövsiyə olunmur.

docker (`ci-cd.yml:259-303`): `verify` və `sonar`-dan sonra; matrix ilə 7 servis paralel; PR-də yalnız build (image-lərin qurula bildiyini yoxlayır), `main`-ə push-da `secrets.DOCKERHUB_USERNAME` / `DOCKER_PASSWORD` ilə login və `<user>/campaign-<service>:latest` push. **NƏTİCƏ** Yalnız `latest` teqi var — konkret commit-ə bağlı (məs. `:sha-1500818`) dəyişməz teq olmadığı üçün geri qayıtma (rollback) və "hansı versiya işləyir?" sualı çətinləşir. Docker layer cache (`cache-from: type=gha`) istifadə olunmur.

mark-done-on-merge.yml — runtime ilə əlaqəsi yoxdur: birləşdirilmiş PR-də və ona bağlı issue-larda label-ları `done` ilə əvəzləyən `actions/github-script` avtomatlaşdırmasıdır (ayrı token secret-i `INSURANCE_GITHUB_TOKEN` ilə).

Deploy — repozitoriyada Kubernetes manifestləri, Helm chart-ları və ya deploy job-u yoxdur. **NAMƏLUM** Image-lərin Docker Hub-dan sonra harada və necə işlədildiyi repozitoriyadan müəyyən edilmir.

18.6 Troubleshooting (README + təhlil)

Problem	Səbəb	Həll
Kafka/Tempo/Loki `Exited`	Köhnə volume vəziyyəti, `latest` image dəyişikliyi	`docker compose down -v --remove-orphans && pull && up -d`
campaign-service dayanıb qalıb	DB hazır olmadan başlayıb, restart yoxdur	`docker compose up -d campaign-service`; Compose-a healthcheck şərti və restart əlavə etmək
Servis paylaşılan konfiqurasiyanı almayıb	Config Server gec hazır olub (`optional:`)	Servisi restart; Config Server-ə healthcheck + `service_healthy`
Tempo-da trace yoxdur	Səhv OTLP endpoint/protokol	gRPC: `http://otel-collector:4317`, `/v1/traces` yalnız HTTP üçün
Loki-də log yoxdur	`./logs` boşdur, Promtail target-ləri	`http://localhost:9080/targets`, `ls -la ./logs`
gRPC sinifləri tapılmır (IDE)	Generasiya edilməyib	`mvn -pl grpc-contract -am clean install`
Testcontainers xətası	Docker işləmir	Docker Desktop/daemon

18.7 Yeni layihədə tətbiq: yoxlama siyahısı

1. Image-lər: non-root, JRE/distroless, layered jar və ya Buildpacks, `HEALTHCHECK`, JVM yaddaş faizi.
2. Teqlər: dəyişməz (`sha`, semver) + `latest` yalnız rahatlıq üçün; infrastruktur image-lərini versiyalarla bərkitmək.
3. Compose: hər servis üçün healthcheck (actuator `/actuator/health/readiness`), `depends_on: condition: service_healthy`, `restart: on-failure`; sirrləri `.env` (gitignore-dadır) və ya Docker secrets ilə.

4. CI: testlər bir dəfə; Sonar davamlı serverə; Docker build cache; SBOM və image skanı (Trivy/Grype); imzalama (cosign).
5. CD: mühit başına konfiqurasiya, miqrasiyaların ayrıca addımı, mavi-yaşıl və ya canary deploy, avtomatik rollback.

18.8 Müsahibə sualları

S1. Çoxmərhələli Docker build nə qazandırır? Cavab: Build alətləri (Maven, JDK, mənbə kodu) son image-ə düşmür — image kiçik və təhlükəsiz olur; build mühiti təkrarlanandır.

S2. `depends_on` niyə tətbiqin hazır olmasına zəmanət vermir? Cavab: Standart `service_started` yalnız konteynerin başladığını gözləyir, prosesin port açmasını və ya hazır olmasını yox. `service_healthy` healthcheck tələb edir. Üstəlik tətbiq özü də asılılıqların gecikməsinə dözümlü olmalıdır (retry, restart), çünki istehsal orkestratorları başlanğıc sırasına zəmanət vermir.

S3. `latest` teqinin problemləri? Cavab: Dəyişkəndir: eyni teq fərqli vaxtlarda fərqli image-ə işarə edir; təkrar oluna bilməyən build-lər, səssiz uyğunsuzluqlar, rollback çətinliyi, hansı versiyanın işlədiyini bilməmək.

S4. Bu pipeline-da Sonar quality gate deploy-u necə bloklayır? Cavab: `docker` job-u `needs: [verify, sonar]`-dır; `sonar` job-u `-Dsonar.qualitygate.wait=true` ilə quality gate uğursuz olduqda düşür, ona görə image push edilmir.

S5. Konteynerdə JVM yaddaşını necə konfiqurasiya edərdiniz? Cavab: Konteyner limitini təyin edib JVM-ə faizlə heap vermək (`-XX:MaxRAMPercentage=70-75`), qalan hissəni metaspace, thread stack-ları, direct buffer-lər (Netty, Kafka) üçün saxlamaq; OOMKilled hallarını metriklərlə izləmək.

18.9 Xülasə

- Hər tətbiqin çoxmərhələli Dockerfile-ı var; non-root, healthcheck, layer cache və JVM parametrləri çatışmır.
- Compose iki variantdadır: tam stek və yalnız infrastruktur; başlanğıc asılılıqlarında sağlamlıq şərtləri və restart siyasətləri qeyri-ardıcıldır, `latest` teqləri təkrarlanmanı zəiflədir.
- CI: verify → ephemeral SonarQube ilə quality gate → 7 servisin image build/push; yalnız `latest` teqi.
- Növbəti fəsil test strategiyasını və test infrastrukturunu (MockMvc, Mockito, Testcontainers) izah edir.

19. Test strategiyası və test infrastrukturu

19.1 Məqsəd: testlər nəyi sübut etməlidir?

Testlər üç sualı cavablandırır: (1) biznes qaydaları düzgündürmü (unit), (2) komponentlər bir-biri ilə və framework ilə düzgün işləyirmi (inteqrasiya: Spring konteksti, security, DB, broker), (3) servislər arası kontraktlar uyğundurmu (kontrakt/E2E). **Test piramidası**: çoxlu sürətli unit test, orta sayda inteqrasiya testi, az sayda E2E test.

19.2 Repozitoriyadakı testlərin inventarı

FAKT 24 test faylı, 93 `@Test` metodu; `common`, `security`, `api-gateway`, `config-server`, `discovery-server`, `grpc-contract` modullarında test yoxdur.

Modul	Test sinfi	Sayı	Növ	Nəyi yoxlayır
auth-service	`AuthServiceTest`	22	Unit (Mockito)	Qeydiyyat, login, refresh, forgot/reset, logout — uğur və xəta halları
	`AuthMapperTest`	14	Unit (generasiya olunmuş `AuthMapperImpl`)	MapStruct xəritələri, null halları
	`AuthAuditServiceTest`	12	Unit + `OutputCaptureExtension`	Audit payload-ı, CB/bulkhead/ümumi xətanın loglanması
	`AuthControllerTest`	4	`@SpringBootTest` + MockMvc (**filtrlər söndürülüb**) + PostgreSQL konteyneri	HTTP bağlama və cavab formatı
campaign-service	`CampaignControllerTest`	14	`@SpringBootTest` + MockMvc (**filtrlər aktiv**, real RS256 token) + PostgreSQL + Kafka konteynerləri	Rol icazələri (201/200/403), cavab formatı, 409 xəritəsi
	`CampaignServiceTest`	3	Unit	`create`: LIFE → ACTIVE, TSS → PENDING, dublikat rəddi
dashboard-service	`CampaignGrpcClientTest`	6	Unit (mock stub)	Status süzgəci, sıfır dəyərlər, `""` → `null`
	`DashboardControllerTest`	6	`@SpringBootTest` + MockMvc (filtrlər aktiv)	ADMIN/USER/anonim → 200/403/401
log-service	`AuthAuditLogControllerTest`	4	`@SpringBootTest` + MockMvc + Kafka + **Elasticsearch** konteynerləri	`INTERNAL_SERVICE` rolu; real ES-ə yazı
	`SlowRequestLogControllerTest`	4	Eyni (repository mock)	Səhifələmə, rol

Modul	Test sinfi	Sayı	Növ	Nəyi yoxlayır
	`AuthAuditLogServiceTest`, `SlowRequestLogConsumerTest`	2 + 2	Unit	Sənəd xəritələnməsi

19.3 Test infrastrukturu: baza siniflər

Hər modulda üç baza sinif var:

1. **`AbstractBaseServiceTest`** — Mockito unit testləri üçün **FAKT** (`campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractBaseServiceTest.java:8-12`):

```
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
public class AbstractBaseServiceTest {
}
```

NƏTİCƏ `Strictness.LENIENT` istifadə olunmayan stub-lar haqqında xəbərdarlıqları söndürür. Mockito-nun standart `STRICT_STUBS` rejimi lazımsız `when(...)` sətirlərini aşkarlayıb testləri təmiz saxlayır; LENIENT bunu gizlədir. Maraqlıdır ki, `CampaignServiceTest` bu baza sinfi istifadə etmir — standart sərt rejimlə işləyir.

2. **`AbstractTestContainerConfiguration`** — real infrastruktur **FAKT** (`campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractTestContainerConfiguration.java:13-43`):

Mənbə: `campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractTestContainerConfiguration.java:13-44`

```
@Testcontainers
public abstract class AbstractTestContainerConfiguration {

    private static final Logger log = LoggerFactory.getLogger(AbstractTestContainerConfiguration.class);

    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("postgres:17-alpine");

    @Container
    static KafkaContainer kafka = new KafkaContainer(
            DockerImageName.parse("confluentinc/cp-kafka:7.6.1")
    );

    @DynamicPropertySource
    static void properties(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", postgres::getJdbcUrl);
        registry.add("spring.datasource.username", postgres::getUsername);
        registry.add("spring.datasource.password", postgres::getPassword);
        registry.add("spring.kafka.bootstrap-servers", kafka::getBootstrapServers);
        registry.add("spring.jpa.hibernate.ddl-auto", () -> "create-drop");
        registry.add("spring.liquibase.enabled", () -> false);
        registry.add("spring.data.redis.host", () -> "localhost");
        registry.add("spring.cache.type", () -> "simple");
        registry.add("eureka.client.enabled", () -> false);
        registry.add("spring.cloud.config.enabled", () -> false);
        registry.add("app.jwt.issuer", () -> "allianz-campaign-api");
        registry.add("app.jwt.expiration-minutes", () -> 120);
        registry.add("app.jwt.public-key-location", () -> "classpath:keys/public.pem");
        registry.add("grpc.server.port", () -> -1);
    }

}
```

Sətir	Mənası
`@Testcontainers` + `@Container static`	JUnit 5 extension test sinfi başlamazdan əvvəl konteyneri qaldırır, sinif bitəndə dayandırır
`@DynamicPropertySource`	Konteynerin təsadüfi portunu Spring xassələrinə ötürür (kontekst yaradılmazdan əvvəl)
`ddl-auto=create-drop` , `liquibase.enabled=false`	Sxemi Hibernate yaradır — miqrasiyalar test edilmir (F27)
`spring.cache.type=simple`	Redis yerinə `ConcurrentMapCacheManager`
`eureka/config disabled`	Platforma servislərindən təcrid
`grpc.server.port=-1`	gRPC server-i başlatmamaq

NƏTİCƏ Performans qeydləri:

- `@Container static` abstrakt baza sinifdə olduqda konteynerlər **hər test sinfi üçün** yenidən qaldırılır. "Singleton container" nümunəsi (statik initsializatorda bir dəfə `start()`) və ya Spring Boot 3.1+ `@ServiceConnection` + `@TestConfiguration` ilə konteyner bean-ləri bunu sürətləndirir.
- Hər sinifdə konteyner portları fərqli olduğu üçün Spring test kontekst cache-i siniflər arasında paylaşılmır.
- `KafkaContainer` (`org.testcontainers.containers`) Testcontainers 1.21-də köhnəlmiş API-dir; yeni modul `org.testcontainers.kafka.ConfluentKafkaContainer` / `KafkaContainer` (apache image) təklif edir.
- campaign controller testində servis mock olduğu halda Kafka konteyneri qaldırılır; dashboard testində isə **heç bir DB istifadə olunmadığı halda** PostgreSQL konteyneri qaldırılır və auth-a aid xassələr təyin edilir (`dashboard-service/src/test/java/com/casestudy/insurancecampaign/dashboard/base/AbstractTestContainerConfiguration.java:16-36` — dəyişən adı da `AUTH_POSTGRES` -dir). Bu, kopyala-yapışdır qalığıdır və CI vaxtını artırır.
- Image-lər istehsaldakılardan fərqlidir: `postgres:17-alpine` ↔ `postgres:latest` , `cp-kafka:7.6.1` ↔ `apache/kafka:latest` (F28).

3. **`AbstractRestControllerTest`** — `@SpringBootTest` + MockMvc + real JWT **FAKT** (`campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractRestControllerTest.java:23-77`):

Mənbə: campaign-service/src/test/java/com/casestudy/insurancecampaign/campaign/base/AbstractRestControllerTest.java:62-77

```
private String generateAccessToken(String username, List<String> roles) {
    Instant now = Instant.now();
    Instant expiresAt = now.plusSeconds(120 * 60);

    return Jwts.builder()
            .header()
            .add("typ", TOKEN_TYPE)
            .and()
            .issuer("allianz-campaign-api")
            .subject(username)
            .claims(Map.of("roles", roles))
            .issuedAt(Date.from(now))
            .expiration(Date.from(expiresAt))
            .signWith(readPrivateKey("classpath:keys/private.pem"), Jwts.SIG.RS256)
            .compact();
}
```

Bu, layihənin ən güclü test texnikalarından biridir: test öz private key-i (`src/test/resources/keys/private.pem`) ilə **həqiqi RS256 token** imzalayır və `@AutoConfigureMockMvc(addFilters = true)` sayəsində sorğu bütün Spring Security filtr zəncirindən və `@PreAuthorize` -dan keçir. Nəticədə avtorizasiya qaydaları (ADMIN/USER/anonim) həqiqətən test edilir. `@WithMockUser` kimi qısayollar JWT filtrini yan keçərdi.

NƏTİCƏ auth-service-in controller testi isə `@AutoConfigureMockMvc(addFilters = false)` istifadə edir (`auth-service/src/test/java/com/casestudy/insurancecampaign/auth/base/AbstractRestControllerTest.java:10-19`): auth endpoint-ləri onsuz da açıqdır, lakin security konfiqurasiyasının auth-service-də düzgün tətbiq olunduğu yoxlanmır.

19.4 Test texnikaları

`@MockitoBean` — Spring Framework 6.2+-da `@MockBean`-in əvəzi; kontekstdəki bean-i Mockito mock-u ilə əvəzləyir (`CampaignControllerTest.java:43-47`). Hər fərqli mock dəsti ayrı kontekst deməkdir.

Mockito arqument yoxlamaları — `argThat(...)` ilə servisə ötürülən obyektin sahələri yoxlanılır (`CampaignControllerTest.java:88-92`); `verifyNoInteractions(campaignService)` 403 hallarında servisin heç çağırılmadığını sübut edir (`:117-118`) — avtorizasiyanın **servisdən əvvəl** işlədiyinin dəqiq testidir.

Log çıxışının yoxlanması — `@ExtendWith(OutputCaptureExtension.class)` və `CapturedOutput` ilə (`AuthAuditServiceTest.java:23-24`, `192-206`): log mesajları əməliyyat kontraktının bir hissəsidirsə (alert-lər onlara bağlıdırsa), test edilməlidir.

Xəta simulyasiyası — Resilience4j istisnaları mock-dan atılır (`AuthAuditServiceTest.java:229-301`): `CallNotPermittedException.createCallNotPermittedException(CircuitBreaker.ofDefaults(...))`. Bu, `publishAudit`-in xəta emalını yoxlayır, lakin **aspektlərin özünü** yox.

Struktur — testlər `// Given`, `// When`, `// Then`, `// Verify` şərhləri ilə bölünüb və adlar `shouldXWhenY` formasındadır — oxunaqlı və ardıcıl.

19.5 Nə test edilir, nə edilmir

Komponent / davranış	Test edilib?	Qeyd
Campaign rol icazələri	✓	Real JWT ilə
`CampaignService.create` qaydaları	✓ (3 test)	Mock repository
`approve` / `deactivate` / `getHistory` / `getStatistics`	×	F33; F36 bu boşluqda qalıb
Dublikat üçün DB partial unique indeksi	×	Liquibase testdə söndürülüb (F27)
`countByStatus` JPQL proyeksiyası	×	Repository testi yoxdur
`GlobalExceptionHandler` xəritəsi (səhv JSON, səhv UUID)	×	F07 aşkarlanmayıb
Etibarsız/vaxtı keçmiş JWT	×	F08
`SlowRequestLoggingFilter`	×	—
Kafka produce → consume → ES	×	Listener testdə işləmir; Kafka-ya mesaj göndərilmir
DLQ və deserializasiya xətası	×	F11 aşkarlanmayıb
Redis cache davranışı	×	`simple` cache
Resilience4j aspektləri	×	Simulyasiya
gRPC endpoint (server)	×	—
Gateway (marşrutlar, JWT filtri, rate limit, 429 formatı)	×	Modulda test yoxdur
Servislər arası kontraktlar (audit DTO-ları, proto, Kafka JSON)	×	Kontrakt testi yoxdur
Auth servis məntiqi	✓ (22 test)	Ən yaxşı örtülmüş hissə

19.6 Təkmilləşdirmə: tövsiyə olunan testlər (illüstrativ nümunələr, repozitoriyada yoxdur)

Repository + miqrasiya testi — Liquibase aktiv, real PostgreSQL:

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = Replace.NONE)
@Testcontainers
class CampaignRepositoryIT {
    @Container @ServiceConnection
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:18");   // istehsalla eyni ver
siya

    @Autowired CampaignRepository repo;

    @Test
    void uniqueIndexRejectsDuplicate() {
        repo.saveAndFlush(new CampaignEntity(UUID.randomUUID(), "Title 123456", "Description 1234567890",
TSS, PENDING_APPROVAL, null));
        assertThatThrownBy(() -> repo.saveAndFlush(
                new CampaignEntity(UUID.randomUUID(), "Title 123456", "Description 1234567890", TSS, PEND
ING_APPROVAL, null)))
            .isInstanceOf(DataIntegrityViolationException.class);
    }

    @Test
    void countByStatusProjectionWorks() { /* bir neçə status yaz, countByStatus nəticəsini yoxla */ }
}
```

Kafka uçdan-uca testi — Awaitility ilə:

```
kafkaTemplate.send("log-events", "/x", new SlowRequestLogEvent("svc", "GET", "/x", 200, 12, "t", Map.of
(), Instant.now()));
await().atMost(Duration.ofSeconds(10)).untilAsserted(() ->
        assertThat(slowRequestLogRepository.findAll()).anyMatch(d -> d.getPath().equals("/x")));
```

Resilience4j testi — WireMock 503 qaytarır, `CircuitBreakerRegistry` ilə vəziyyət yoxlanılır:

```
stubFor(post("/api/v1/logs/auth-audit").willReturn(serviceUnavailable()));
for (int i = 0; i < 2; i++) authAuditService.publishLoginSuccessAuditAsync("demo");
await().untilAsserted(() -> assertThat(registry.circuitBreaker("logServiceHttpClient").getState()).isEqua
lTo(OPEN));
verify(5, postRequestedFor(urlEqualTo("/api/v1/logs/auth-audit")));   // 3 + 2 cəhd, sonra CB açıq
```

Exception mapping testi — `POST /api/v1/campaigns` gövdəsində `"category":"XYZ"` → 400 gözlənilməlidir (hazırda 500).

Kontrakt testləri — Spring Cloud Contract və ya Pact: auth-service (consumer) ilə log-service (provider) arasında audit kontraktı; Kafka hadisələri üçün JSON Schema və ya Avro + Schema Registry uyğunluq yoxlaması; proto üçün `buf breaking` ilə geriyə uyğunluq yoxlaması.

Arxitektura testləri — ArchUnit: "controller paketləri repository-yə birbaşa müraciət etməsin", "entity-lər web qatına çıxmasın".

Gateway testləri — `@SpringBootTest(webEnvironment = RANDOM_PORT)` + `WebTestClient` + WireMock downstream + Redis Testcontainers: 401 (token yox), 429 (iki ardıcıl sorğu), marşrut StripPrefix.

19.7 Müsahibə sualları

S1. `@WithMockUser` əvəzinə real JWT ilə test etməyin üstünlüyü? Cavab: `@WithMockUser` SecurityContext-i birbaşa qurur və JWT filtrini, imza yoxlamasını, claim → authority çevrilməsini yan keçir. Real token bütün zənciri (filtr, issuer, rollar, `@PreAuthorize`) yoxlayır — istehsal davranışına daha yaxındır.

S2. Testlərdə Liquibase-i söndürüb `create-drop` istifadə etməyin riski? Cavab: Testlərdəki sxem istehsal sxemi deyil: partial indekslər, default-lar, seed-lər, constraint-lərin adları, xüsusi SQL yoxdur. DB səviyyəli qaydalara əsaslanan davranış test edilmir, miqrasiya xətaları gec aşkarlanır.

S3. Testcontainers testlərini necə sürətləndirmək olar? Cavab: Singleton konteynerlər (bütün test sinifləri üçün bir dəfə), `@ServiceConnection` ilə eyni konteksti paylaşmaq, lazımsız konteynerləri qaldırmamaq, reusable containers (lokal), paralel icra, test kontekst cache-ni sındıran fərqli `@MockitoBean` dəstlərindən qaçmaq.

S4. Mockito-da LENIENT strictness nə vaxt əsaslıdır? Cavab: Paylaşılan `@BeforeEach` stub-larının bəzi testlərdə istifadə olunmadığı hallarda; lakin standart olaraq STRICT_STUBS lazımsız stub-ları və səhv arqumentli stub-ları aşkarlayır. Baza sinifdə hamı üçün LENIENT etmək problemləri gizlədir.

S5. Mikroservislər arası kontraktı necə test edərdiniz? Cavab: Consumer-driven contract testing (Pact/Spring Cloud Contract): consumer gözləntisini yazır, provider build-i onu yoxlayır. Asinxron hadisələr üçün sxem reyestri və uyğunluq qaydaları; gRPC üçün proto breaking-change yoxlaması.

19.8 Xülasə

- 93 test: auth məntiqi və campaign/dashboard/log avtorizasiyası yaxşı örtülüb; real RS256 token ilə MockMvc testləri nümunəvidir.
- Infrastruktur: Mockito, MockMvc, `@MockitoBean`, OutputCapture, Testcontainers (PostgreSQL, Kafka, Elasticsearch).
- Əsas boşluqlar: status keçidləri, DB constraint-ləri və miqrasiyalar, exception mapping, JWT xəta yolu, Kafka/DLQ, cache, Resilience4j, gRPC server, gateway, kontraktlar.
- IV hissə burada bitir. V hissə bütün bu komponentləri real uçdan-uca axınlarda birləşdirir.

V hissə. Uçdan-uca biznes axınları

Real icra yolları: giriş nöqtəsindən müşahidə olunan nəticəyə qədər

20. Uçdan-uca iş axınları

Əvvəlki fəsillər texnologiyaları tək-tək izah etdi. Bu fəsil onları real sorğuların yolu boyunca birləşdirir. Hər axın eyni şablonla izlənir:

Sahə	Sual
Giriş nöqtəsi	Sorğu hansı URL/metod ilə, hansı komponentə daxil olur?
Zəncir	Hansı filtr, controller, servis, repository, klient çağırılır?
Təhlükəsizlik	Harada, hansı qayda ilə yoxlanılır?
Tranzaksiya	Hansı sərhəd, nə commit olunur?
Sinxron / asinxron	Klient nəyi gözləyir, nə arxa planda baş verir?
Uğursuzluq yolları	Nə sına bilər və klient nə görür?
Müşahidə olunan nəticə	Cavab, DB, Kafka, Elasticsearch, log, metrik, trace-də nə qalır?

FAKT Kod statik oxunub; axınlar işlədilməyib (fəsil 1.6). Ardıcıllıqdakı framework addımları (filtr sırası, proxy-lər, commit anı) Spring-in sənədləşdirilmiş davranışına əsaslanır və NƏTİCƏ kimi işarələnib.

20.1 Bütün axınlar üçün ümumi gateway hissəsi

Xarici klientin hər sorğusu `api-gateway`-dən (8080) keçir. Gateway-dəki addımlar bütün axınlarda eynidir, ona görə bir dəfə izah olunur.

1. **`GatewayRateLimitErrorResponseFilter`** (`Ordered.HIGHEST_PRECEDENCE`, `GatewayRateLimitErrorResponseFilter.java:118-119`) cavab obyektini dekorasiya edir ki, sonradan 429 yaranarsa JSON gövdə yazsın.
2. **`RequestRateLimiter`** — bütün marşrutlara tətbiq olunan default filtr (`api-gateway/src/main/resources/application.yml:20-26`). NƏTİCƏ SCG marşrut filtrlərinə siyahıdakı mövqeyə görə kiçik `order` verir, ona görə o, `order = 100` olan JWT filtrindən **əvvəl** işləyir. Açar `requestRateLimiterKeyResolver` tərəfindən hesablanır (`GatewayRateLimiterConfiguration.java:36-62`): etibarlı Bearer token varsa `user:<sub>`, yoxdursa `ip:<X-Forwarded-For-un birinci dəyəri | X-Real-IP | remote address>`. Redis Lua skripti tokeni çıxır; limit aşılıbsa 429 qaytarılır.
3. **`GatewayJwtAuthenticationFilter`** (`GatewayJwtAuthenticationFilter.java:76-95`) — `OPEN_PATHS` siyahısındakı prefikslər (`/api/v1/auth`, `/actuator`, Swagger/OpenAPI yolları; `GatewayJwtAuthenticationFilter.java:39-50`) yoxlanmadan keçir. Qalanlarında `Authorization: Bearer ...` olmalı və RS256 imzası ilə `iss` yoxlanmalıdır; əks halda **401, boş gövdə**. Rollar burada yoxlanmır — gateway yalnız autentifikasiya edir.
4. **Marşrutlaşdırma** — `Path` predikatı marşrutu seçir (`api-gateway/src/main/resources/application.yml:27-43`), `lb://<servis>` Eureka reyestrindən instansı LoadBalancer vasitəsilə seçir (compose-da `docker-compose.yml:62-65`). `Authorization` başlığı dəyişmədən ötürülür.

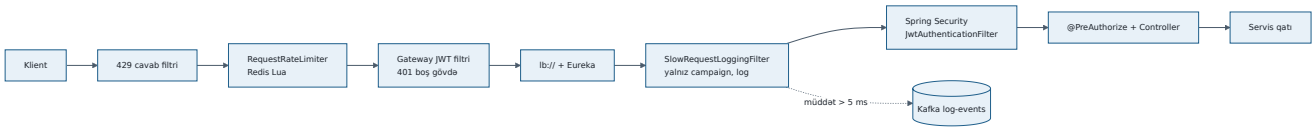
NƏTİCƏ Nəticə: autentifikasiya olunmuş hər sorğuda JWT **iki dəfə** parse olunur (açar resolver-də və JWT filtrində), downstream servisdə isə üçüncü dəfə. RS256 yoxlaması ucuzdur, lakin bu, təkrarlanan işdir; parse nəticəsini exchange atributunda saxlamaq olardı.

Downstream servisdə (`common` + `security` modulları ilə) eyni addımlar təkrarlanır:

- `SlowRequestLoggingFilter` (`SlowRequestLoggingFilter.java:88-120`) bütün zənciri ölçür — **yalnız campaign-service və log-service-də**, çünki `common`-da `spring-kafka` `optional`-dır (`common/pom.xml:27-`

`31`) və auth/dashboard-un classpath-ında yoxdur (fəsil 2.5);

- Spring Security zənciri: `JwtAuthenticationFilter` (`JwtAuthenticationFilter.java:68-83`) token-dən `SecurityContext` qurur; URL qaydaları: `/api/v1/auth/**`, `/actuator/**`, OpenAPI yolları `permitAll`, qalanı `authenticated()` (`ServiceSecurityConfiguration.java:96-107`);
- `DispatcherServlet` → controller; `@PreAuthorize` metod səviyyəsində rolu yoxlayır.



Bütün xarici sorğular üçün ümumi qatlar (sadələşdirilmiş)

20.2 Axın A: qeydiyyat və login (audit ilə)

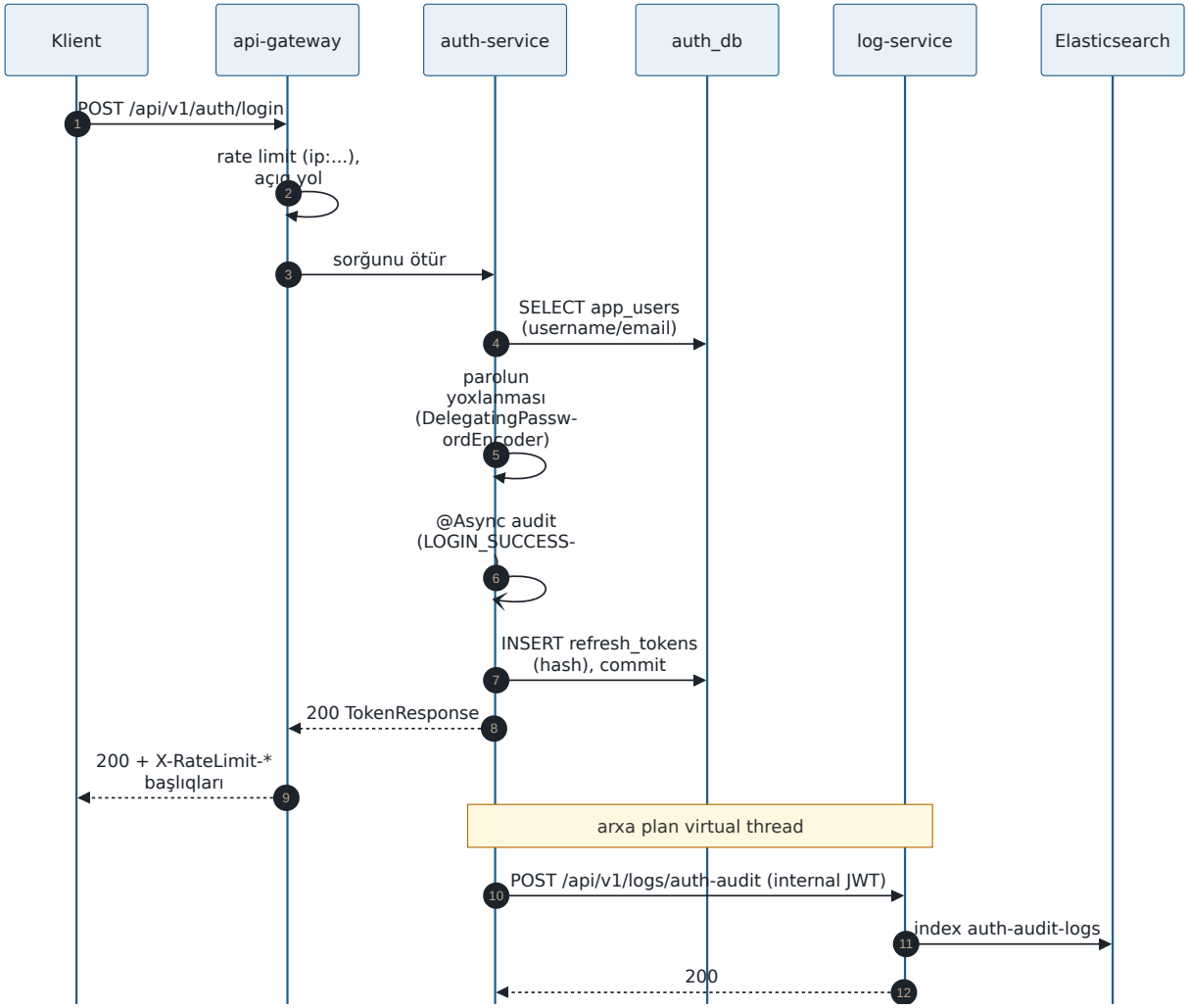
Giriş nöqtəsi: `POST /api/v1/auth/register`, `POST /api/v1/auth/login` → gateway → auth-service (8081) (`AuthController.java:47-64`).

Zəncir (login):

1. Gateway: açar `ip:...` (token yoxdur), JWT filtri yolu açıq sayır.
2. auth-service: `/api/v1/auth/**` `permitAll`; `@Valid LoginRequest` yoxlanır (səhvdirsə 400 `VALIDATION_ERROR`).
3. `AuthService.login` (`AuthService.java:140-161`), `@Transactional`:
   - `findByUsernameOrEmail` — `@` varsa e-poçt, yoxdursa istifadəçi adı ilə axtarış (`AuthService.java:328-334`);
   - istifadəçi yoxdur / aktiv deyil / parol uyğun gəlmir → `BusinessException(UNAUTHORIZED)` → 401; hər halda `publishLoginFailureAuditAsync` çağırılır;
   - uğur: `publishLoginSuccessAuditAsync`, sonra `createTokenResponse` (`AuthService.java:292-315`) — RS256 access token (120 dəq), təsadüfi refresh token; DB-yə yalnız refresh token-in SHA-256 hash-i yazılır.
4. Cavab: `ApiResponse<TokenResponse>` — access və refresh token, `Bearer`, vaxtlar, rollar.

Asinxron hissə — audit: `AuthAuditService`-in bütün `publish*Async` metodları `@Async`-dir (`AuthAuditService.java:51-135`); `@EnableAsync` `common`-dakı `VirtualThreadConfiguration`-dadır. Arxa plan thread-i `LogServiceHttpClient.sendAuthAuditLog`-u çağırır (`LogServiceHttpClient.java:27-31`):

- `RestClientHttpServiceGroupConfigurer` hər sorğuya `sub=auth-service`, `roles=[ROLE_INTERNAL_SERVICE]` olan təzə JWT əlavə edir (`LogServiceHttpClientConfiguration.java:32-48`);
- Resilience4j: Retry (3 cəhd, 500 ms) → CircuitBreaker (10 çağırışlıq pəncərə, 50%) → Bulkhead (10 eyni vaxtda, gözləmə
  0. (`auth-service/src/main/resources/application.yml:84-114`);
- log-service: `POST /api/v1/logs/auth-audit`, `@PreAuthorize("hasRole('INTERNAL_SERVICE')")` (`AuthAuditLogController.java:29-35`) → `AuthAuditLogService.save` → Elasticsearch `auth-audit-logs` indeksi (`AuthAuditLogService.java:25-36`);
- istənilən xəta `publishAudit`-də tutulur və WARN kimi loglanır (`AuthAuditService.java:147-179`) — login-ə təsir etmir.



Login axını: sinxron token cavabı və asinxron audit

Uğursuzluq yolları:

Hadisə	Klient	Yan təsir
Yanlış parol / istifadəçi yoxdur	401 `UNAUTHORIZED`, eyni mesaj	Audit `LOGIN_FAILURE`, `FAILED`
İstifadəçi aktiv deyil	401, mesaj **"User is not active"**	Fərqli mesaj hesabın mövcudluğunu açır (F03)
log-service düşüb	Login uğurlu	3 cəhd, sonra CB açılır; audit itir, WARN log
auth_db düşüb	500 (catch-all)	Hikari bağlantı gözləməsi (standart 30 s)
Eyni IP-dən sürətli təkrar	429	5 saniyədə 1 sorğu (fəsil 15.6)

Qeydiyyat fərqi (`AuthService.java:103-126`): `existsByUsername` / `existsByEmail` yoxlamaları, parolun hash-lənməsi, `save`, `publishRegisterAuditAsync`. **NƏTİCƏ** Audit tapşırığı metod daxilində, **commit-dən əvvəl** executor-a verilir. Paralel qeydiyyatda unikal constraint commit zamanı pozularsa (F18) klient 500 alır, lakin audit artıq "User registered successfully" yazmış ola bilər. Audit yalnız uğurlu commit-dən sonra göndərilməlidir (`@TransactionalEventListener(phase = AFTER_COMMIT)`). Qeydiyyatda `role` sahəsi klientdən gəlir — istənilən şəxs `ROLE_ADMIN` hesab yarada bilər (F01).

Müşahidə olunan nəticə: `app_users` / `refresh_tokens` sətirləri; ES `auth-audit-logs` sənədi; auth-service logunda `Auth audit sent to log-service` INFO; Tempo-da gateway → auth span-ları **NƏTİCƏ** (RestClient observation ilə auth → log span-ı da ola bilər). auth-service-də yavaş sorğu filtri yoxdur, ona görə login nə

qədər çəksə də `log-events`-ə qeyd düşmür.

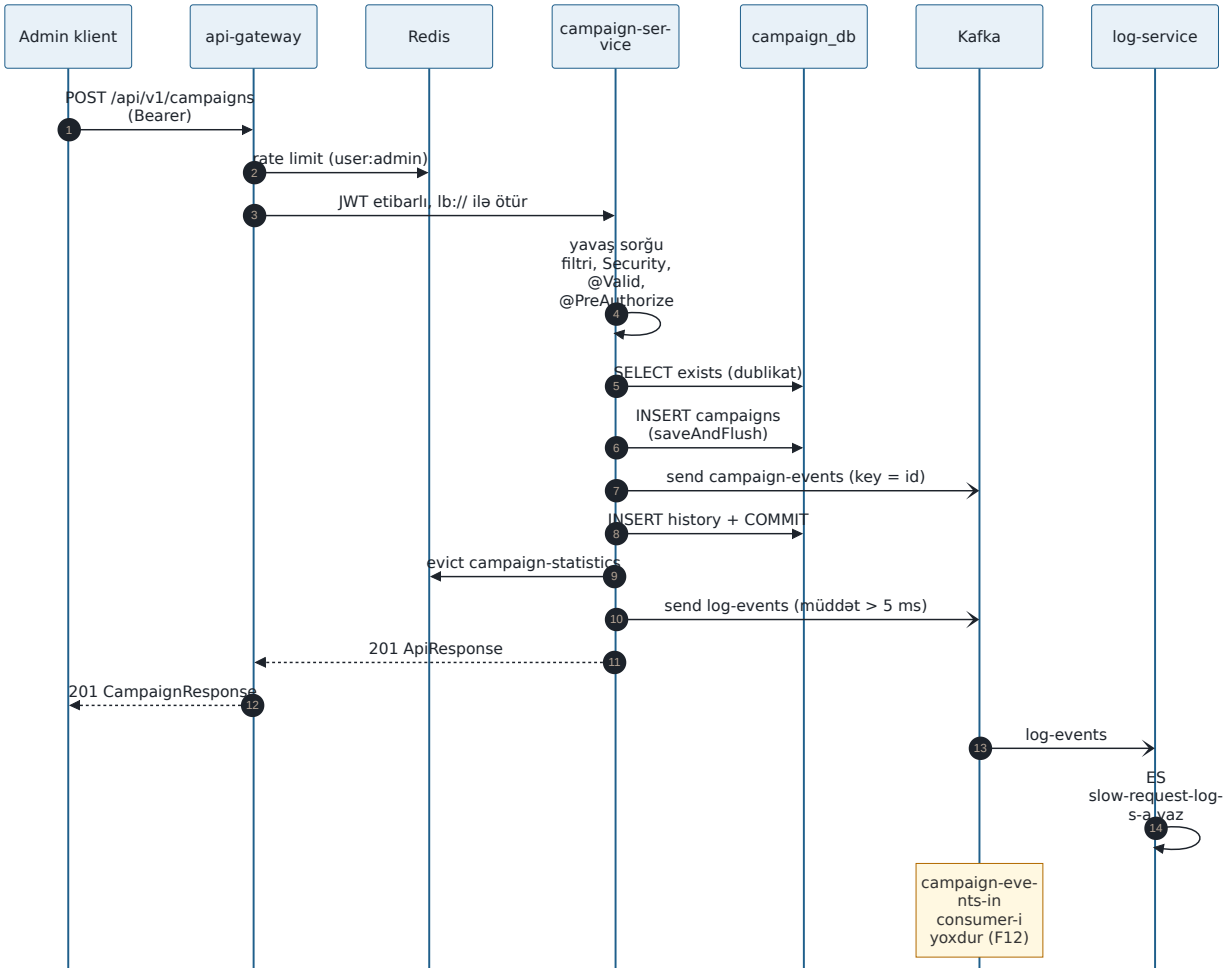
20.3 Axın B: kampaniyanın yaradılması (tam zəncir)

Bu, layihənin ən zəngin axınıdır: rate limit, iki səviyyəli JWT, Bean Validation, metod təhlükəsizliyi, tranzaksiya, dublikat qaydası (həm tətbiq, həm DB səviyyəsində), status tarixçəsi, cache eviction, Kafka hadisəsi və yavaş sorğu jurnalı.

Giriş nöqtəsi: `POST /api/v1/campaigns`, gövdə `CreateCampaignRequest` (title 10–50 simvol, hərf və ya rəqəmlə başlamalı; description 20–200; category məcburi — `CreateCampaignRequest.java:10-19`), `CampaignController.create` (`CampaignController.java:42-48`), cavab `201 Created`.

Zəncir:

1. Gateway (20.1): açar `user:admin`, JWT etibarlıdır → `lb://campaign-service`.
2. campaign-service: `SlowRequestLoggingFilter` saatı işə salır.
3. `JwtAuthenticationFilter` → `SecurityContext`: `admin`, `ROLE_ADMIN`.
4. `anyRequest().authenticated()` keçir.
5. `@Valid` — arqumentlər həll olunarkən (controller metodu çağırılmazdan **əvvəl**) yoxlanır → səhv sahələr → 400 `VALIDATION_ERROR` + sahə xətaları. Nəticə: səhv gövdə göndərən `ROLE_USER` 403 deyil, 400 alır (fəsil 8.9).
6. `@PreAuthorize("hasRole('ADMIN')")` — metod proxy-də, çağırış anında → `ROLE_USER` → 403 `FORBIDDEN` (`GlobalExceptionHandler` vasitəsilə).
7. `CampaignService.create` (`CampaignService.java:54-77`) — `@Transactional` və `@CacheEvict("campaign-statistics", allEntries = true)` proxy-ləri:
   1. `rejectIfDuplicateExists` — `existsByCategoryAndTitleAndDescriptionAndStatusNot(..., DUPLICATE)` (`CampaignService.java:140-150`, `CampaignRepository.java:13-18`) → varsa 409 `DUPLICATE_CAMPAIGN`;
   2. `resolveInitialStatus` — `LIFE` → `ACTIVE`, digərləri → `PENDING_APPROVAL` (`CampaignService.java:133-138`);
   3. `saveAndFlush` — INSERT dərhal icra olunur; paralel dublikat partial unique indeksə dəyir (`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:101-113`) → `DataIntegrityViolationException` → 409;
   4. `saveHistory` — `campaign_status_histories` sətri (`previous_status = NULL`, `"Campaign created"`);
   5. `publishStatusChange` — `kafkaTemplate.send("campaign-events", campaignId, event)` (`CampaignService.java:181-191`);
   6. domain obyektinə çevrilir, controller onu `CampaignResponse`-a xəritələyir.
8. Proxy-lər: **NƏTİCƏ** commit və `@CacheEvict` (Redis-dən `campaign-statistics` açarları silinir). Bu iki interceptor-un sırası açıq təyin olunmayıb (fəsil 15, F13); diaqram variantlardan birini göstərir.
9. Filtr: müddət > 5 ms → `SlowRequestLogEvent` `log-events` mövzusuna (açar — URI).
10. Gateway cavabı klientə qaytarır.



Kampaniya yaradılması — sinxron yol və asinxron yan təsirlər

Sinxron / asinxron: Klient DB commit-ini və cache eviction-u gözləyir. `kafkaTemplate.send` asinxrondur — nəticə (`CompletableFuture`) yoxlanmır; lakin metadata yoxdursa və ya buffer doludursa `send()` `max.block.ms` (60 s) qədər çağıran thread-i bloklaya bilər (F10). Bu, tranzaksiya və DB bağlantısı açıq ikən baş verir.

Uğursuzluq yolları:

Hadisə	Klient	Məlumat vəziyyəti
Dublikat (ardıcıl)	409 `DUPLICATE_CAMPAIGN`	Heç nə yazılmır
Dublikat (paralel, yarış)	409 (indeks + `saveAndFlush`)	Rollback
Kafka əlçatmazdır	Gecikmə ≤ 60 s, sonra **NƏTİCƏ** `send()` istisnası → catch-all 500 → rollback	DB-də sətir yoxdur; ya da göndərmə asinxron uğursuz olur, DB-də sətir var, hadisə yoxdur
Hadisə göndərildi, commit uğursuz oldu	500	Kafka-da **mövcud olmayan** kampaniya haqqında hadisə (F09)
Redis düşüb	**NƏTİCƏ** Eviction xətası → 500	Interceptor sırasından asılıdır: evict commit-dən sonradırsa kampaniya yaradılıb və təkrar cəhd 409 alır; əvvəldirsə rollback
campaign-service instansı yoxdur	503 (gateway)	—

Müşahidə olunan nəticə: `campaigns` və `campaign_status_histories`-də bir sətir; `campaign-events`-də bir qeyd (heç kim oxumur); Redis-də `campaign-statistics` boşalır; ES `slow-request-logs`-da ehtimalən bir sənəd (5 ms həddi demək olar ki, hər yazı sorğusunu tutur — F25); Prometheus-da

`http_server_requests_seconds{uri="/api/v1/campaigns",status="201"}`; Tempo-da gateway → campaign trace-i (Kafka hissəsi trace-ə bağlanmır).

20.4 Axın C: təsdiq və deaktivasiya

Giriş nöqtəsi: `PATCH /api/v1/campaigns/{id}/approve` və `/deactivate`, yalnız `ADMIN` (`CampaignController.java:66-78`).

Zəncir: `findEntity` (yoxdursa 404) → `ensureNotDuplicate` (409) → status yoxlaması (səhv keçid → 409 `INVALID_STATUS_TRANSITION`) → `changeStatus` (`CampaignService.java:162-169`): entity statusu dəyişir, `save` (idarə olunan entity üçün əlavə SELECT ilə merge — F14), tarixçə sətri, Kafka hadisəsi; commit zamanı dirty checking UPDATE yaradır və `@PreUpdate` `updatedAt`-i təyin edir.

Keçid qaydaları (`CampaignService.java:91-111`):

Əvvəlki status	approve	deactivate
`PENDING_APPROVAL`	→ `ACTIVE`	→ `DEACTIVE`
`ACTIVE`	409	→ `DEACTIVE`
`DEACTIVE`	409	409
`DUPLICATE`	409 (dublikat)	409 (dublikat)

Diqqət ediləcək detallar:

- Cavabdakı `updatedAt` köhnədir (F36): domain obyekti commit-dən əvvəl yaradılır, `@PreUpdate` isə flush zamanı işləyir.
- `@Version` yoxdur (F37): iki admin eyni anda approve və deactivate etsə, hər ikisi `PENDING_APPROVAL` oxuya bilər; son commit qalib gəlir və tarixçədə bir-birinə zidd iki sətir olur.
- Hər uğurlu keçid `campaign-statistics` cache-ni təmizləyir.

20.5 Axın D: oxuma sorğuları (kampaniya, siyahı, tarixçə)

Endpoint	Rol	Servis metodu	Qeyd
`GET /api/v1/campaigns/{id}`	USER, ADMIN	`getById` (`CampaignService.java:79-84`)	404 yoxdursa; səhv UUID → `MethodArgumentTypeMismatchException` → catch-all → **500** (F07, fəsil 8)
`GET /api/v1/campaigns?page&size&sort`	USER, ADMIN	`getAll` (`CampaignService.java:86-89`)	Standart `size=10`, `createdAt DESC`; `CampaignPageResponse`
`GET /api/v1/campaigns/{id}/history`	ADMIN	`getHistory` (`CampaignService.java:113-119`)	`existsById` + `findByCampaignIdOrderByChangedAtAsc` (indekssiz FK — F15)
`GET /api/v1/campaigns/statistics`	ADMIN	`getStatistics` (cache)	Axın E ilə eyni metod

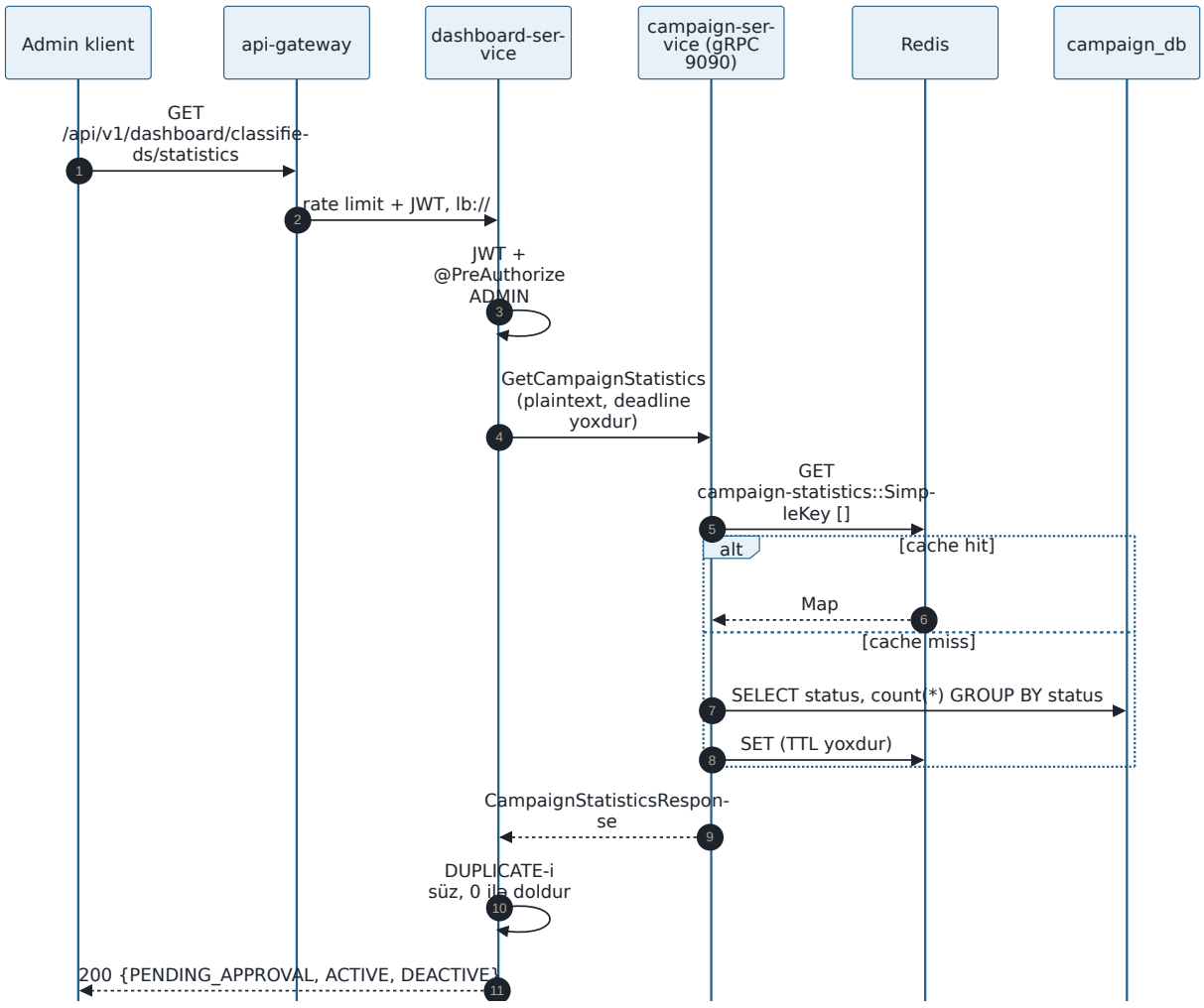
`@Transactional(readOnly = true)` Hibernate-də flush-u söndürür və **NƏTİCƏ** JDBC sürücüsünə read-only işarəsi ötürür. `open-in-view: false` olduğu üçün (`campaign-service/src/main/resources/application.yml:15`) lazy əlaqələr controller-də yüklənə bilməz — bu layihədə lazy əlaqə olmadığı üçün problem yoxdur.

20.6 Axın E: dashboard statistikası (REST → gRPC → cache → DB)

Giriş nöqtəsi: `GET /api/v1/dashboard/classifieds/statistics`, yalnız `ADMIN` (`DashboardController.java:39-44`).

Zəncir:

1. Gateway → `lb://dashboard-service` (8083).
2. dashboard-service: JWT, `@PreAuthorize` (yavaş sorğu filtri bu serviste yoxdur).
3. `CampaignGrpcClient.getStatistics` (`CampaignGrpcClient.java:39-42`) — bloklayan stub, ünvan `static://campaign-service:9090`, plaintext (`dashboard-service/src/main/resources/application.yml:11-15`, `docker-compose.yml:225`). **Deadline yoxdur** (F19), **JWT ötürülmür** — gRPC portu autentifikasiyasızdır (F04).
4. campaign-service gRPC server (net.devh, port 9090): `CampaignGrpcEndpoint.getCampaignStatistics` (`CampaignGrpcEndpoint.java:24-32`) → `campaignService.getStatistics()`.
5. `@Cacheable("campaign-statistics")` (`CampaignService.java:121-131`): Redis-də dəyər varsa DB-yə getmir; yoxdursa `countByStatus` JPQL `GROUP BY` (`CampaignRepository.java:20-21`), bütün statuslar 0 ilə doldurulur, nəticə Redis-ə yazılır (TTL yoxdur).
6. dashboard: `DASHBOARD_VISIBLE_STATUSES` — yalnız `PENDING_APPROVAL`, `ACTIVE`, `DEACTIVE`; `DUPLICATE` süzülür (`CampaignGrpcClient.java:25-29`, `76-87`).



Dashboard statistikası: REST → gRPC → Redis cache → PostgreSQL

Uğursuzluq yolları:

Hadisə	Klient	Səbəb
campaign-service düşüb	500, mesaj məs. `UNAVAILABLE: io exception`	`StatusRuntimeException` catch-all-a düşür (F07, F19)
campaign-service asılıb	Dashboard thread-i qeyri-müəyyən gözləyir	Deadline yoxdur
Redis düşüb	500 (gRPC `UNKNOWN`)	Cache oxuma xətası, `CacheErrorHandler` yoxdur
Köhnə statistika	200, köhnə rəqəmlər	Eviction commit-dən əvvəl olubsa və arada oxuma baş veribsə (F13)

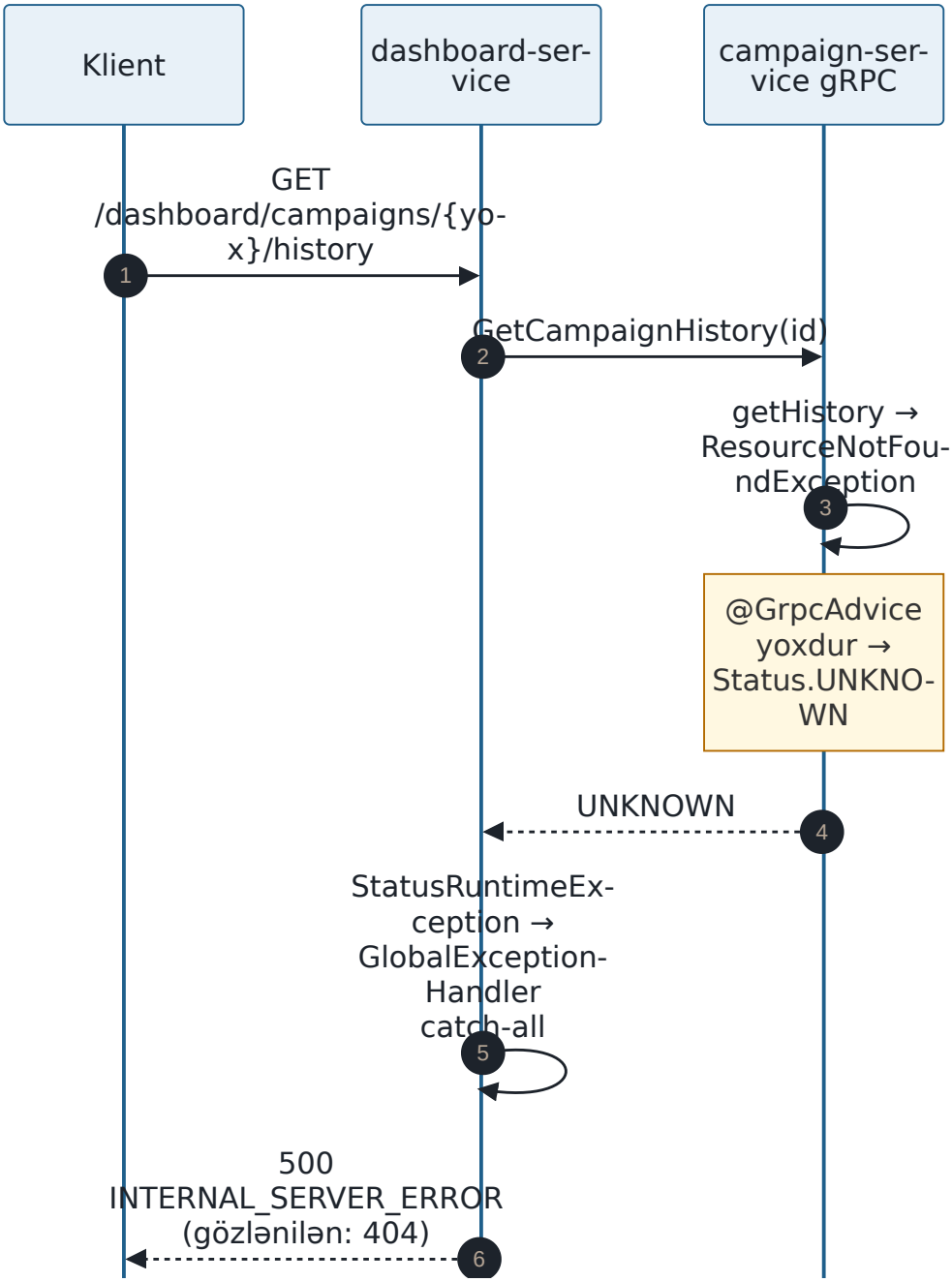
Arxitektura qeydi: eyni məlumat `GET /api/v1/campaigns/statistics` ilə birbaşa da əlçatandır. Dashboard-un dəyəri — süzgəc və gələcək aqreqasiya üçün ayrıca BFF qatı; qiyməti — əlavə şəbəkə hop-u və iki servis arasında sinxron asılılıq.

20.7 Axın F: dashboard tarixçəsi və xəta yayılması

Giriş nöqtəsi: `GET /api/v1/dashboard/campaigns/{campaignId}/history` (`DashboardController.java:52-57`).

`CampaignGrpcClient.getCampaignHistory` (`CampaignGrpcClient.java:50-68`) UUID-i sətir kimi göndərir; server `UUID.fromString` və `getHistory` çağırır (`CampaignGrpcEndpoint.java:34-47`). Boş `previousStatus` `""` kimi gəlir və klientdə `null`-a çevrilir; `changedAt` ISO sətirdir və `Instant.parse` ilə oxunur.

Bu axın xəta yayılmasının ən yaxşı nümunəsidir:



Mövcud olmayan kampaniya üçün tarixçə — 404 əvəzinə 500

Eyni sorğu REST ilə (`GET /api/v1/campaigns/{id}/history`) düzgün 404 qaytarır. Düzəliş iki tərəflidir: serverdə `@GrpcAdvice` ilə `ResourceNotFoundException → Status.NOT_FOUND`, klientdə `StatusRuntimeException`-ı status koduna görə domen istisnasına çevirmək (fəsil 13.6 və 13.10).

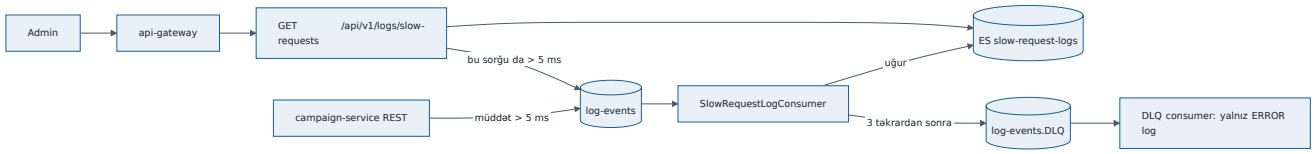
20.8 Axın G: yavaş sorğu jurnalı (Kafka → Elasticsearch) və onun oxunması

Yazma tərəfi: `common` modulundakı `SlowRequestLoggingFilter` yalnız classpath-ında `KafkaTemplate` olan servislərdə aktivdir (`@ConditionalOnClass`, `SlowRequestLoggingFilter.java:31-33`): **campaign-service və log-service**. auth-service və dashboard-service `common`-dan asılıdır, lakin `spring-kafka` orada `optional` elan olunduğu üçün (`common/pom.xml:27-31`) onlara keçmir [FAKT, Maven asılılıq ağacı]. gRPC çağırışları servlet

filtrindən keçmir. `/actuator`, Swagger və OpenAPI yolları istisnadır (`SlowRequestLoggingFilter.java:67-74`). Müddət həddən (`5 ms`) böyükdürsə: `SlowRequestLogEvent(service, method, path, status, durationMs, traceId (MDC), {user-agent, x-forwarded-for}, occurredAt)` → `log-events`, açar = URI.

Oxuma (consumer) tərəfi: `SlowRequestLogConsumer.consume` (`SlowRequestLogConsumer.java:35-47`), qrup `log-service` → `SlowRequestDocument` → ES `slow-request-logs` indeksi. Xəta → `DefaultErrorHandler` 1 s fasilə ilə 3 təkrar, sonra `log-events.DLQ`-a eyni partisiyaya (`KafkaDlqConfiguration.java:26-33`); DLQ consumer yalnız ERROR loglayır (`SlowRequestLogConsumer.java:54-65`).

Sorğu tərəfi: `GET /api/v1/logs/slow-requests?page&size`, yalnız `ADMIN` (`SlowRequestLogController.java:35-43`) — `findAll(PageRequest.of(page, size))`, sıralama yoxdur (F42), Spring Data `Page` birbaşa qaytarılır (F26).



Yavaş sorğu jurnalının həyat dövrü və öz-özünü qidalandıran dövrə

NƏTİCƏ **Öz-özünü qidalandıran dövrə:** log-service də `common`-u istifadə edir, ona görə yavaş sorğular siyahısını oxuyan sorğunun özü (ES-ə şəbəkə çağırışı demək olar ki, həmişə 5 ms-dən uzundur) yeni yavaş sorğu hadisəsi yaradır. Hər səhifə baxışı jurnala bir qeyd əlavə edir. Bu, sonsuz dövrə deyil (bir oxuma → bir yazı), lakin həddin real dəyərə (məs. 500 ms) qaldırılmalı olduğunu göstərir.

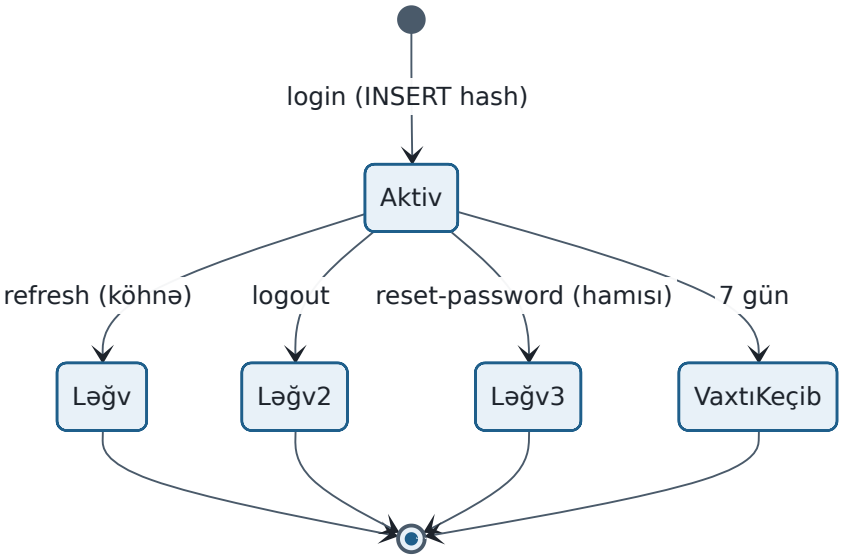
Poison pill (F11): `ErrorHandlingDeserializer` olmadığı üçün səhv JSON `poll()` daxilində `RecordDeserializationException` yaradır; `DefaultErrorHandler` onu emal edə bilmir, növbəti `poll()` eyni offsetdən başlayır və partisiya sonsuz döngüdə bloklanır — DLQ boş qalır (fəsil 11-də spring-kafka mənbəsi ilə izah olunub).

20.9 Axın H: refresh, logout, forgot və reset

Addım	Kod	DB dəyişikliyi	Audit	Xəta
Refresh	`AuthService.java:175-190`	Köhnə token `revoked_at`, yeni token INSERT	`REFRESH_TOKEN`	Yoxdur/ləğv/vaxtı keçib → 401
Logout	`AuthService.java:269-279`	`revoked_at` (idempotent)	`LOGOUT`	Naməlum token → 401
Forgot	`AuthService.java:205-230`	`password_reset_tokens` INSERT (hash)	`FORGOT_PASSWORD`	İstifadəçi yoxdur → **404** (F03)
Reset	`AuthService.java:244-260`	Parol UPDATE, token `used`, bütün aktiv refresh token-lər ləğv	`RESET_PASSWORD`	Etibarsız/istifadə olunub → 401

Kritik müşahidələr:

1. **Access token ləğv olunmur** (F35). Logout və reset yalnız refresh token-ləri ləğv edir; artıq verilmiş access token 120 dəqiqə etibarlı qalır, çünki gateway və servislər yalnız imzanı yoxlayır.
2. **Reset token cavabda qaytarılır** (F02) — istifadəçi adını bilən istənilən şəxs parolu dəyişə bilər. Kod bunu mesajda etiraf edir (`AuthService.java:225-229`).
3. **Refresh yarışı** (F17): iki paralel refresh eyni token-i oxuyur, hər ikisi `isRevoked() == false` görür və iki yeni token cütü yaranır. Pessimistik kilid (`SELECT ... FOR UPDATE`) və ya `UPDATE ... WHERE revoked_at IS NULL` ilə şərti yeniləmə lazımdır.



Token həyat dövrü: login → refresh → logout / reset

20.10 Axın I: Swagger UI və OpenAPI aqreqasiyası

Giriş nöqtəsi: `GET /swagger-ui.html` (gateway). springdoc UI dörd URL göstərir: `/openapi/auth`, `/openapi/campaign`, `/openapi/dashboard`, `/openapi/logs` (`api-gateway/src/main/resources/application.yml:68-83`).

`GatewayOpenApiController` (`GatewayOpenApiController.java:87-105`) servisin `lb://<servis>/v3/api-docs` sənədini `WebClient` ilə alır; xəta olarsa `onErrorResume` əl ilə yığılmış **fallback** sənəd qaytarır (`GatewayOpenApiController.java:128-140`) — yollar siyahısı və xəta mesajı ilə. Beləliklə bir servis düşəndə də UI açılır. Swagger-dən göndərilən "Try it out" sorğuları adi API axınlarıdır və rate limit-ə tabedir (5 saniyədə 1).

Paralel olaraq `/auth/v3/api-docs` və s. marşrutları `StripPrefix=1` ilə birbaşa servisə ötürülür (`api-gateway/src/main/resources/application.yml:44-67`) — fallback olmadan.

20.11 Axınlar × komponentlər matrisi

Axın	Gateway	Redis	PostgreSQL	Kafka	ES	gRPC	HTTP (auth→log)
A. Login/qeydiyyat	✓	RL	auth_db		audit		✓
B. Yaratma	✓	RL + evict	campaign_db	campaign + yavaş	yavaş		
C. Approve/deactivate	✓	RL + evict	campaign_db	campaign + yavaş	yavaş		
D. Oxuma	✓	RL	campaign_db	yavaş	yavaş		
E. Dashboard statistika	✓	RL + cache	campaign_db (miss)			✓	
F. Dashboard tarixçə	✓	RL	campaign_db			✓	
G. Yavaş sorğu oxuma	✓	RL		yavaş	✓		
H. Refresh/logout/reset	✓	RL	auth_db		audit		✓

Axın	Gateway	Redis	PostgreSQL	Kafka	ES	gRPC	HTTP (auth→log)
I. OpenAPI	✓	RL					

RL — rate limiter. "Yavaş" — `log-events` / `slow-request-logs`.

Asılılıq nəticəsi: Redis gateway-də fail-open olduğu üçün (F06) onun düşməsi yalnız campaign cache-li axınları (B, C,

E. sındırır. Kafka-nın düşməsi isə `max.block.ms` səbəbindən campaign-service və log-service-in **bütün** REST axınlarını (B, C, D, G) yavaşladır, çünki yavaş sorğu filtri hər yavaş cavabdan sonra eyni thread-də `send()` çağırır (F10); auth və dashboard axınları (dashboard-un gRPC çağırışı daxil) təsirlənmir.

20.12 Müsahibə sualları

S1. Kampaniya yaratma sorğusunu gateway-dən DB-yə və Kafka-ya qədər izah edin. Cavab: Rate limiter (Redis token bucket, `user:<sub>` açarı) → gateway JWT yoxlaması → `lb://` ilə instans seçimi → servisdə yavaş sorğu filtri → JWT filtri ilə `SecurityContext` → `@Valid` → `@PreAuthorize ADMIN` → tranzaksiya: dublikat yoxlaması, `saveAndFlush`, tarixçə, Kafka `send` → commit → cache evict → 201. Asinxron: `campaign-events` (consumer-siz) və `log-events` → log-service → ES.

S2. Bu axında atomiklik hansı nöqtədə pozulur? Cavab: Kafka göndərişi DB tranzaksiyasının daxilindədir, amma onun hissəsi deyil. Commit uğursuz olarsa hadisə artıq gedib; göndəriş uğursuz olarsa və nəticə yoxlanmırsa, DB-də sətir var, hadisə yoxdur. Həll — transactional outbox.

S3. Niyə dashboard tarixçəsi mövcud olmayan id üçün 500 qaytarır? Cavab: gRPC server istisnanı `Status.UNKNOWN`-a çevirir, klient `StatusRuntimeException`-ı tanımır və catch-all 500 qaytarır. Status xəritəsi (`NOT_FOUND`) və klient tərəfində tərcümə lazımdır.

S4. Logout-dan sonra access token niyə işləməyə davam edir? Cavab: Access token stateless JWT-dir, yalnız imza və `exp` yoxlanır. Həllər: qısa ömür (5–15 dəq), `jti` deny-list (Redis), token versiyası istifadəçidə saxlanıb claim ilə müqayisə edilməsi.

S5. Kafka düşəndə hansı axınlar təsirlənir və niyə? Cavab: campaign-service və log-service-in REST axınları, çünki yavaş sorğu filtri (yalnız bu iki serviste aktivdir) sinxron `send()` çağırır və metadata yoxdursa 60 s bloklanır; kampaniya yazıları isə əlavə olaraq tranzaksiya daxilində bloklanır. Həll: `max.block.ms`-i azaltmaq, göndərişi ayrı executor-a çıxarmaq, outbox.

S6. Audit niyə commit-dən sonra göndərilməlidir? Cavab: Əks halda rollback olunan əməliyyat üçün "uğurlu" audit qeydi yaranır. `@TransactionalEventListener(AFTER_COMMIT)` və ya outbox audit-i yalnız real nəticəyə bağlayır.

20.13 Xülasə

- Hər xarici sorğu: 429 filtri → rate limiter → gateway JWT → `lb://` → (campaign/log-da yavaş sorğu filtri) → Spring Security → `@PreAuthorize` → controller → servis.
- Yazı axınları (B, C) DB + tarixçə + Kafka + cache eviction-u bir metodda birləşdirir; atomiklik yalnız DB üçündür.
- Dashboard axınları sinxron gRPC asılılığını və xəta statuslarının itməsini göstərir.
- Yavaş sorğu jurnalı campaign-service və log-service-in sorğu yolunu Kafka-ya bağlayır; 5 ms həddi demək olar ki, onların hər sorğusunu qeyd edir.
- Auth axınlarında əsas risklər: admin self-register, reset token-in cavabda qaytarılması, enumerasiya, ləğv olunmayan access token, refresh yarışı.

- Növbəti fəsil bütün bu müşahidələri kateqoriyalara ayırır və prioritetləşdirir.

VI hissə. Qiymətləndirmə və dizayn

Mühəndislik keyfiyyəti, kompromislər və oxşar sistemin layihələndirilməsi

21. Mühəndislik qiymətləndirməsi

Bu fəsil əvvəlki fəsillərdə ayrı-ayrılıqda qeyd olunan bütün müşahidələri bir reyestrdə toplayır, kateqoriyalara ayırır, prioritet verir və düzəliş yol xəritəsi təklif edir. Məqsəd layihəni "pisləmək" deyil: bu, case-study kimi yazılmış, müasir stack üzərində qurulmuş və bir çox düzgün qərar verilmiş sistemdir. Məqsəd — istehsala hazırlıq baxımından nəyin çatışmadığını dəqiq və sübutla göstərməkdir.

21.1 Metodologiya

Kateqoriyalar (fəsil 1.3-dəki etiketlərlə uyğun):

Kod	Kateqoriya	Meyar
D	Təsdiqlənmiş defekt	Kod oxunuşundan birmənalı görünən səhv davranış; reproduksiya ssenarisi göstərilə bilir
R	Risk	Müəyyən şəraitdə (yük, uğursuzluq, hücum) problem yaradır; hazırda görünməyə bilər
T	Trade-off	Şüurlu və ya məqbul seçim; qiyməti var, lakin kontekstdə əsaslı ola bilər
M	Saxlanıla bilənlik	Davranışa təsir etmir, lakin dəyişikliyi çətinləşdirir, səhv ehtimalını artırır
H	Hipotez	Runtime yoxlaması olmadan təsdiq edilə bilməyən ehtimal

Ciddilik (müəllifin qiymətləndirməsidir, NƏTİCƏ): **Kritik** — dərhal istismar oluna bilən təhlükəsizlik boşluğu; **Yüksək** — əlçatanlığı və ya məlumat bütövlüyünü geniş miqyasda poza bilər; **Orta** — məhdud təsir və ya xüsusi şərait tələb edir; **Aşağı** — kosmetik və ya nadir.

Sübut qaydası: hər tapıntı ya repozitoriya faylına və sətir aralığına, ya da kitabxana mənbəyinə (versiyası ilə) istinad edir. Runtime-da yoxlanmayan hər şey NƏTİCƏ və ya H kimi işarələnib. Tam reyestr: `analysis/01-architecture-map.md`.

21.2 Güclü tərəflər

Qiymətləndirmə yalnız problemlərdən ibarət olsa, qərəzli olar. Repozitoriyada aşağıdakı qərarlar düzgündür və yeni layihədə saxlanmağa dəyər FAKT:

Sahə	Qərar	Niyə yaxşıdır
Data sahibliyi	Hər servisin öz DB-si (auth_db, campaign_db), başqa servisin cədvəlinə giriş yoxdur	Servislər müstəqil dəyişə bilir
Sxem idarəsi	Liquibase + `ddl-auto: validate`, `open-in-view: false` (`campaign-service/src/main/resources/application.yml:12-15`)	Sxem kodla versiyalanır; lazy-loading sızması yoxdur
Dublikat qaydası	Tətbiq yoxlaması **və** partial unique indeks (`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:101-113`)	Yarış vəziyyətində DB son müdafiə xəttidir

Sahə	Qərar	Niyə yaxşıdır
Token təhlükəsizliyi	RS256 (asimmetrik), servislərdə yalnız public key; refresh/reset token-lər SHA-256 hash kimi saxlanır	Servis kompromisi token buraxmağa imkan vermir; DB sızması token-ləri açmır
Refresh rotasiyası	Hər refresh köhnə token-i ləğv edir; reset bütün refresh token-ləri ləğv edir	Oğurlanmış refresh token-in ömrü qısalır
Servislərarası auth	auth → log çağırışı `ROLE_INTERNAL_SERVICE` JWT ilə (`LogServiceHttpClientConfiguration.java:32-48`)	Daxili endpoint istifadəçi token-i ilə çağırıla bilmir
Dayanıqlıq	Audit çağırışı Retry + CircuitBreaker + Bulkhead ilə qorunur, xəta login-i sındırmır	Qeyri-kritik asılılıq kritik axını aşağı salmır
Asinxron log	Yavaş sorğular Kafka ilə ayrılır; log-service-də DLQ (`KafkaDlqConfiguration.java:26-33`)	Log yazısı sorğu yolundan ayrılıb
Xəta formatı	Vahid `ApiResponse` / `ApiError` + `ErrorCode` enum, security handler-ləri eyni formatda	Klient üçün proqnozlaşdırıla bilən kontrakt
Kontrakt	`grpc-contract` ayrıca modul, proto-dan generasiya	Server və klient eyni kontraktdan qurulur
Müasir platforma	Java 25, Boot 4, virtual thread-lər, `@ImportHttpServices` , Testcontainers	Aktual texnikaların praktiki nümunəsi
Test keyfiyyəti	campaign/dashboard controller testləri real RS256 token və aktiv filtrlərlə	Təhlükəsizlik konfiqurasiyası həqiqətən test olunur
Müşahidə	Prometheus + OTel/Tempo + Loki + Grafana, log-larda `traceId`	Metrik, trace və log birləşdirilə bilir
Exhaustive switch	`resolveInitialStatus` enum üzərində `switch` ifadəsi (`CampaignService.java:133-138`)	Yeni kateqoriya əlavə olunanda kompilyator xəbərdar edir

21.3 Tapıntılar: təsdiqlənmiş defektlər (D)

ID	Ciddilik	Tapıntı	Sübut	Düzəliş
F01	**Kritik**	Anonim istifadəçi `role: ROLE_ADMIN` ilə qeydiyyatdan keçir və bütün admin əməliyyatlarını edir	`RegisterRequest.java:18-24` , `AuthService.java:103-126` , `ServiceSecurityConfiguration.java:96-104`	Qeydiyyat həmişə `ROLE_USER` ; admin yalnız admin endpoint-i və ya seed ilə
F02	**Kritik**	`forgot-password` xam reset token-i cavabda qaytarır → hesabın ələ keçirilməsi	`AuthService.java:205-230`	Token yalnız e-poçt/SMS kanalı ilə; cavab həmişə eyni (`202`)
F07	Orta	Catch-all handler 4xx xətalarını (səhv JSON, UUID, enum, 405, 404) 500-ə çevirir və `getMessage()` -i qaytarır	`GlobalExceptionHandler.java:130-139`	`ResponseEntityExceptionHandler` / `ProblemDetail` ; catch-all yalnız ümumi mesaj + log

ID	Ciddilik	Tapıntı	Sübut	Düzəliş
F18	Aşağı	Paralel qeydiyyatda unique pozuntusu 500 + SQL mesajı	`AuthService.java:103-126`	`DataIntegrityViolationException` → 409; `saveAndFlush`
F36	Aşağı	`approve` / `deactivate` cavabında köhnə `updatedAt`	`CampaignService.java:162-169`, `CampaignEntity.java:65-68`	`saveAndFlush` və ya vaxtı servisdə təyin etmək

21.4 Tapıntılar: risklər (R)

ID	Ciddilik	Tapıntı	Tetikleyici şərt	Düzəliş
F04	Yüksək	gRPC 9090 autentifikasiyasız, Compose host-a açır (`docker-compose.yml:182-184`)	Şəbəkəyə çıxışı olan istənilən şəxs	Portu açmamaq; mTLS və ya JWT interceptor
F22	Yüksək (istehsalda)	Demo private key və şifrələr repozitoriyada	Repozitoriyaya çıxış	Secret manager; açarlar build-dən kənarda; rotasiya
F10	Yüksək	Kafka əlçatmaz olduqda `send()` 60 s-ə qədər bloklayır — filtrdə və tranzaksiyada	Broker/metadata problemi	`max.block.ms` 1–2 s, göndərişi ayrı executor-a, outbox
F11	Yüksək	`ErrorHandlingDeserializer` yoxdur → poison pill partisiyanı bloklayır	Bir səhv JSON mesajı	`ErrorHandlingDeserializer` + DLQ
F05	Orta	Rate limit açarı `X-Forwarded-For`-a etibar edir	Anonim sorğularda saxta başlıq	Yalnız etibarlı proxy-dən gələn XFF; `ForwardedHeaderTransformer` / trusted proxies
F03	Orta	İstifadəçi enumerasiyası (fərqli mesajlar və 404)	Hücumçu istifadəçi adlarını yoxlayır	Eyni cavab və eyni gecikmə
F08	Orta	JWT filtri `JwtException`-ı tutmur	Etibarsız/vaxtı keçmiş token	Spring Security Resource Server və ya açıq handler
F09	Orta	Dual write (DB + Kafka)	Commit və ya göndəriş uğursuzluğu	Transactional outbox / CDC
F17	Orta	Refresh token yarışı	Paralel iki refresh	Şərti UPDATE və ya `@Version`
F19	Orta	gRPC deadline yoxdur, statuslar `UNKNOWN`	campaign-service asılır və ya 404	`withDeadlineAfter`, `@GrpcAdvice`, klientdə tərcümə
F37	Orta	`@Version` yoxdur → lost update	Paralel admin əməliyyatları	`@Version` + 409
F38	Orta	`GlobalExceptionHandler` loglamır	Hər 500	5xx üçün ERROR log + traceId
F40	Orta	Actuator `permitAll` və portlar açıq	Şəbəkə çıxışı	Ayrı management portu, şəbəkə izolyasiyası, auth

ID	Ciddilik	Tapıntı	Tetikleyici şərt	Düzəliş
F24	Orta	Loki-də `trace_id` label-ı	Yük artdıqca	traceId log sətrində qalsın, label olmasın
F43	Orta	ES indekslərində ILM/retention yoxdur	Zamanla disk dolur	ILM, data stream, rollover
F27	Orta	Testlərdə miqrasiya işləmir	Sxem dəyişikliyi	Testlərdə Liquibase + `validate`
F13	Aşağı	Evict/commit sırası, TTL yoxdur	Paralel oxu-yazı	Transaction-aware cache, TTL
F06	Aşağı	Rate limiter fail-open	Redis düşür	Şüurlu seçim; alert ilə müşayiət
F32	Aşağı	Config Server əlçatmazdırsa paylanan xassələr itir	Config Server düşür	Kritik xassələri lokal da saxlamaq, `fail-fast`
F34	Aşağı (H)	`@Async` -də trace konteksti ötürülmür	Audit çağırışları	`ContextPropagatingTaskDecorator`
F46	Aşağı	`@Async` audit commit-dən əvvəl göndərilir → rollback olunan əməliyyat üçün "uğurlu" audit	Commit xətası (məs. F18)	`@TransactionalEventListener(AFTER_COMMIT)`

21.5 Tapıntılar: trade-off-lar (T)

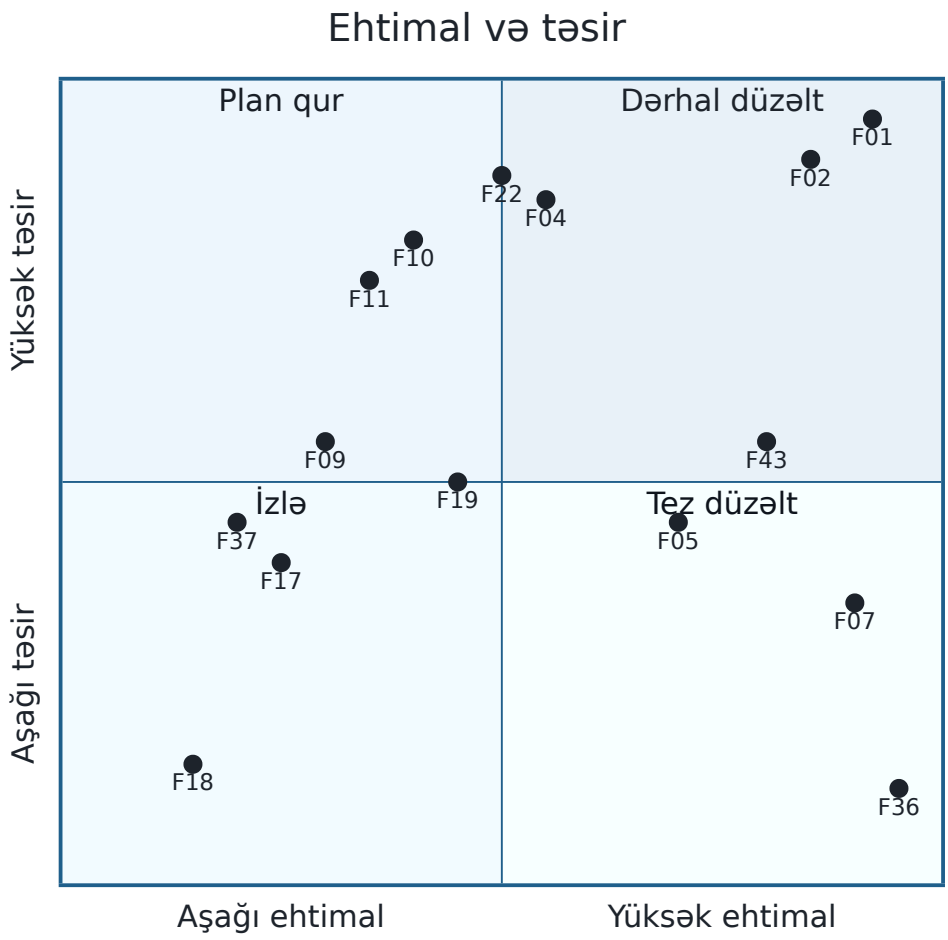
ID	Seçim	Qazanc	Qiymət	Qiymətləndirmə
F12	`campaign-events` consumer-siz dərc olunur	Gələcək inteqrasiya üçün hazır hadisə axını	Dual-write riski heç bir faydasız daşınır; sxem sənədləşməyib	Ya consumer əlavə edin, ya da outbox ilə birlikdə saxlayın
F25	Yavaş sorğu həddi 5 ms	Demo-da pipeline-ın işlədiyini görmək asandır	campaign-service və log-service-də praktiki olaraq hər sorğu Kafka-ya gedir; log-service öz oxumasını da qeyd edir	Mühitə görə konfiqurasiya; istehsalda yüzlərlə ms
F31	Daxili çağırışlar static ünvanla, Eureka yalnız gateway-də	Sadəlik, Compose DNS kifayətdir	İki kəşf mexanizmi; ölçəkləmədə qarışıqlıq	Kubernetes-də Eureka-nı tamamilə çıxarmaq olar
F35	120 dəqiqəlik access token, ləğv mexanizmi yoxdur	Stateless yoxlama, DB-yə müraciət yoxdur	Logout/reset-dən sonra 2 saat giriş	5–15 dəq ömür + refresh
F06	Rate limiter fail-open	Redis problemi API-ni dayandırmır	Redis düşəndə limit yoxdur	Əksər API-lər üçün düzgün seçim
F44	Gateway JWT/PEM kodu `security` modulundan ayrıca yazılıb	Gateway servlet asılılıqlarını (`HttpSecurity`) çəkmir	İki yerdə eyni məntiq (`GatewayPemKeyUtils.java:59` , `PemKeyUtils.java:59`)	Servlet-dən asılı olmayan `jwt-core` modulu ilə birləşdirmək olar

21.6 Tapıntılar: saxlanıla bilənlik (M) və hipotezlər (H)

ID	Tapıntı	Təsir	Tövsiyə
F14	Təyin olunmuş UUID + `@Version` yoxdur → hər `save` əvvəl SELECT	Əlavə sorğular	`Persistable.isNew()` və ya `@Version`
F15	Tarixçə FK-sı indekssiz, `idx_campaign_duplicate_lookup` təkrarlanır	Tarixçə sorğusu böyüdükcə yavaşlayır	`campaign_id` indeksi, təkrar indeksi silmək
F16	auth-da `unique` + ayrıca indeks	Lazımsız yazı xərci	Yalnız unique constraint
F20	Jackson 2 `ObjectMapper` yalnız Eureka tranzitivi ilə	Eureka çıxarılsa kontekst qalxmır	`JsonMapper` (Jackson 3) istifadə etmək
F21	Deprecated `JsonSerializer` / `JsonDeserializer`	Gələcək versiyada silinəcək	`JacksonJsonSerializer` / `Deserializer`
F23	Observation konfiqurasiyası 4 yerdə	Dəyişiklik 4 yerdə	Ortaq modul
F26	log-service `Page` birbaşa qaytarır	Daxili model API-yə sızır	DTO, `PagedModel`
F28	Test və Compose image-ləri fərqlidir; Compose-da ES-dən başqa demək olar ki, bütün image-lər `latest` teqlidir (`docker-compose.yml:94` , `286` , `301`)	"Testdə işləyir, prodda yox"; build təkrarlana bilmir	Eyni, sabitlənmiş versiyalar (və ya digest)
F29	Compose `depends_on` sağlamlıqsız, `restart` yoxdur	Başlanğıc yarışları	`condition: service_healthy`
F33	Test boşluqları (keçidlər, gRPC, gateway, filtrlər)	Reqressiya riski	Fəsil 19.6
F39	Audit request validasiyasız, `page/size` yoxlanmır	Səhv məlumat ES-ə düşür; mənfi page → 500	`@Valid` + constraint-lər
F41	CORS yoxdur	Brauzer klientləri başqa origin-dən işləmir	Gateway-də CORS
F42	`slow-requests` sıralamasız, dinamik mapping	Nəticələr ixtiyari sırada	`Sort.by(occurredAt).descending()` , açıq mapping
F45	`DUPLICATE` statusu heç bir axında təyin olunmur, lakin 4 yerdə nəzərə alınır	Ölü vəziyyət, oxucunu çaşdırır	Ya axını reallaşdırmaq, ya statusu silmək
F30 (H)	springdoc 2.8.13 Boot 4 ilə rəsmi uyğunluq xəttindən kənar	Runtime-da yoxlanmayıb	springdoc 3.x xəttinə keçid, smoke test

F45 sübutu: `CampaignStatus.java:3-8` , `CampaignService.java:140-150` (`StatusNot(DUPLICATE)`), `CampaignService.java:171-175` (`ensureNotDuplicate`), `CampaignGrpcClient.java:25-29` (süzgəc) və partial indeksin `WHERE status <> 'DUPLICATE'` şərti.

21.7 Risk matrisi



Seçilmiş tapıntılar: ehtimal × təsir (müəllifin qiymətləndirməsi)

21.8 Prioritetləşdirilmiş düzəliş yol xəritəsi

Səy təxminləri (S — saatlar, M — 1–2 gün, L — həftə) NƏTİCƏ müəllifin qiymətləndirməsidir.

Mərhələ	Tapıntılar	Hədəf	Səy
P0 — təhlükəsizlik	F01, F02, F04, F22, F40	İstismar oluna bilən boşluqları bağlamaq	S–M
P1 — düzgünlük	F07, F08, F18, F19, F36, F37, F17, F38	Düzgün HTTP statusları, ziddiyyətsiz məlumat, görünən xətalar	M
P2 — dayanıqlıq	F10, F11, F09, F13, F46, F05, F03	Kafka/Redis uğursuzluğunda proqnozlaşdırıla bilən davranış	M–L
P3 — əməliyyat	F24, F25, F43, F42, F32, F29, F34	İstehsalda müşahidə və saxlama	M
P4 — gigiyena	F14, F15, F16, F20, F21, F23, F26, F27, F28, F30, F33, F39, F41, F44, F45	Saxlanıla bilənlik və test etibarlılığı	M–L

Asılılıqlar: F27 (testlərdə miqrasiya) F15 və F37 düzəlişlərindən **əvvəl** edilməlidir — əks halda yeni indeks və `version` sütunu test olunmur. F09 (outbox) F10-un tranzaksiya daxilindəki hissəsini də aradan qaldırır. F38 (loglama) hər şeydən əvvəl edilə bilər və digər düzəlişlərin diaqnostikasını asanlaşdırır.

21.9 SOLID və təbəqələnmə təhlili

SRP (tək məsuliyyət). `CampaignService` persistensiya, tarixçə, hadisə dərci və cache invalidasiyasını koordinasiya edir. 192 sətirlik sinif üçün bu məqbuldur — o, *tətbiq servisi* (orkestrator) rolundadır. Problem məsuliyyətlərin sayında deyil, hadisə dərcinin infrastrukturla birbaşa bağlı olmasındadır (bax: DIP). `SlowRequestLoggingFilter` həm ölçür, həm Kafka-ya yazır — ölçməni `ObservationRegistry`-yə, dərci isə ayrıca komponentə vermək testi asanlaşdırar.

OCP (açıq/qapalı). Başlanğıc status exhaustive `switch` ilə həll olunur — yaxşı. Keçid qaydaları isə `approve` və `deactivate` metodlarında `if` şərtləri kimi səpələnib (`CampaignService.java:91-111`). Yeni status (məs. `EXPIRED`) əlavə etmək bir neçə metodu dəyişməyi tələb edir. Alternativ — keçid cədvəlini enum-a köçürmək:

```
// İllüstrativ (repozitoriyada yoxdur)
public enum CampaignStatus {
    PENDING_APPROVAL, ACTIVE, DEACTIVE, DUPLICATE;

    private static final Map<CampaignStatus, Set<CampaignStatus>> ALLOWED = Map.of(
            PENDING_APPROVAL, EnumSet.of(ACTIVE, DEACTIVE),
            ACTIVE, EnumSet.of(DEACTIVE),
            DEACTIVE, EnumSet.noneOf(CampaignStatus.class),
            DUPLICATE, EnumSet.noneOf(CampaignStatus.class));

    public boolean canTransitionTo(CampaignStatus target) {
        return ALLOWED.get(this).contains(target);
    }
}
```

LSP. `BusinessException` iyerarxiyası (`DuplicateCampaignException`, `InvalidStatusTransitionException`, `ResourceNotFoundException`) ardıcıldır: hər alt sinif `ErrorCode` daşıyır və handler onları eyni şəkildə emal edir. Pozuntu müşahidə olunmur.

ISP. `LogServiceHttpClient` tək metodlu interfeysdir; `CampaignRepository` yalnız lazım olan sorğuları əlavə edir; proyeksiya interfeysi (`CampaignStatusCountProjection`) minimaldır. Yaxşı.

DIP. `CampaignService` `KafkaTemplate<String, Object>`-dən birbaşa asılıdır (`CampaignService.java:37`) — domen məntiqi infrastruktur detalına bağlıdır. `CampaignEventPublisher` portu (interfeys) və Kafka/outbox adapteri həm testi, həm outbox-a keçidi asanlaşdırar. `DashboardController` konkret `CampaignGrpcClient` sinfindən asılıdır; testlərdə `@MockitoBean` ilə əvəz olunduğu üçün praktiki problem yoxdur.

Təbəqələnmə. Controller → servis → repository istiqaməti pozulmur; controller-lər repository çağırmır — **istisna** `SlowRequestLogController`-dir, o, servis qatı olmadan birbaşa repository-yə müraciət edir (`SlowRequestLogController.java:22-43`). Sadə oxu üçün məqbuldur, lakin layihədaxili konvensiyanı pozur.

Modullar arası ardıcıllıq (M). auth-service Lombok + MapStruct + ayrıca domen record-ları istifadə edir; campaign-service əl ilə yazılmış mapper və getter-lərlə işləyir; log-service DTO-suz `Page` qaytarır. Bir komandada bu, "hansı üslub standartdır?" sualını doğurur. Kod-review qaydası və ya arxitektura qərar qeydi (ADR) ilə bir üsul seçilməlidir.

Paylaşılan modullar. `common` həm xəta handler-i, həm Kafka filtri, həm virtual thread və observation konfiqurasiyası daşıyır; komponent skanı ilə bütün servislərə avtomatik daxil olur (fəsil 4.4). Bu, "shared kernel" yaradır: `common`-dakı dəyişiklik bütün servislərin yenidən yayımlanmasını tələb edir və davranış (məs. Kafka filtri) classpath-dan asılı olaraq gizli aktivləşir. Kiçik komanda üçün məqbuldur; böyüdükcə auto-configuration modulu (`@AutoConfiguration` + `AutoConfiguration.imports`) və açıq `@ConditionalOnProperty` keçidləri daha idarəolunandır.

21.10 Arxitektura səviyyəsində qiymətləndirmə

Sual	Cavab	Etiket
Servis sərhədləri düzgündürmü?	auth və campaign aydın domenlərdir; log-service texniki servisdir (müşahidə məlumatı); dashboard nazik BFF-dir və hazırda yalnız süzgəc əlavə edir	NƏTİCƏ
Sinxron asılılıq zənciri	gateway → dashboard → campaign (gRPC) → Redis/PostgreSQL: dörd hop, deadline-sız	FAKT
Asinxron inteqrasiya	Yalnız log məlumatı üçün; biznes hadisəsi (`campaign-events`) heç kim tərəfindən istehlak olunmur	FAKT
Data tutarlılığı	Servis daxilində ACID; servislərarası yalnız log/audit (itirilə bilən)	FAKT
Miqyaslana bilənlik	Servislər stateless (JWT), horizontal miqyaslana bilər; Kafka açarları (URI, kampaniya id-si) partisiyalanmaya uyğundur	NƏTİCƏ
Tək uğursuzluq nöqtələri	Compose-da hər infrastruktur komponenti tək instansdır, replika yoxdur (iki ayrı PostgreSQL, bir Redis, bir Kafka broker, bir ES node)	FAKT
Konfiqurasiya	12-factor-a yaxın: env dəyişənləri + Config Server (optional)	FAKT
Monolit alternativ	Bu həcmdə (≈ 6 200 sətir) modul-monolit daha sadə olardı; layihə mikroservis texnikalarını nümayiş etdirmək üçün yazılıb	NƏTİCƏ

21.11 Komanda üçün açıq suallar (naməlumlar)

1. `campaign-events` hadisəsini kim istehlak etməlidir və hansı sxem uyğunluğu tələb olunur? NAMƏLUM
2. `DUPLICATE` statusu planlaşdırılan funksiyadır, yoxsa qalıqdır? NAMƏLUM
3. Admin hesabları necə yaradılmalıdır (seed, daxili panel, IdP)? NAMƏLUM
4. 5 ms həddi və 1 sorğu / 5 s rate limit demo üçündür, yoxsa tələbdir? NAMƏLUM
5. Audit qeydlərinin itirilməsi məqbuldurmu (hüquqi tələb)? Cavab "xeyr"-dirsə, HTTP + CB əvəzinə davamlı növbə lazımdır. NAMƏLUM
6. Hədəf yerləşdirmə mühiti (Compose, Kubernetes, bulud)? Eureka və Config Server qərarları bundan asılıdır. NAMƏLUM

21.12 Müsahibə sualları

S1. Kod-review zamanı tapıntıları necə prioritetləşdirirsiniz? Cavab: Təsir × ehtimal; təhlükəsizlik və məlumat bütövlüyü əvvəl, sonra əlçatanlıq, sonra saxlanıla bilənlik. Hər tapıntı üçün reproduksiya ssenarisi və ya sübut; hipotezləri faktlardan ayırmaq.

S2. Bu layihədə ən kritik iki problem hansıdır? Cavab: Admin rolunun özünü qeydiyyatı (F01) və reset token-in cavabda qaytarılması (F02) — hər ikisi anonim hücumçuya tam nəzarət verir və bir neçə sətirlə düzəlir.

S3. "Catch-all exception handler" niyə təhlükəlidir? Cavab: Framework-un düzgün 4xx xəritələrini udur, klientə daxili mesajlar sızdırır, loglanmırsa xətalar görünməz olur. Düzgün: `ResponseEntityExceptionHandler`-i genişləndirmək, catch-all-da ümumi mesaj və ERROR log.

S4. Layihədə DIP-in pozulduğu yeri və nəticəsini göstərin. Cavab: `CampaignService` `KafkaTemplate`-dən birbaşa asılıdır; outbox-a keçid və ya hadisə dərcinin test edilməsi servis kodunu dəyişməyi tələb edir. Port/adapter ayrılığı bunu lokallaşdırır.

S5. Trade-off ilə defekti necə ayırırsınız? Cavab: Trade-off — alternativlər bilinərək seçilmiş və qiyməti kontekstdə məqbul olan qərardır (fail-open rate limiter). Defekt — niyyət edilən davranışla real davranış arasında fərqdir (köhnə `updatedAt`). Fərq niyyətdədir, ona görə sənədləşmə (ADR) vacibdir.

S6. Modul-monolit bu layihə üçün daha yaxşı olardımı? Cavab: Həcm və komanda ölçüsü baxımından bəli — tək yerləşdirmə, tək DB tranzaksiyası, daha az şəbəkə xətası. Mikroservis müstəqil yerləşdirmə, fərqli miqyaslama və komanda muxtariyyəti lazım olanda özünü doğruldur. Layihənin məqsədi texnika nümayişidir, ona görə seçim kontekstdə başa düşüləndir.

21.13 Xülasə

- 46 tapıntı: 5 təsdiqlənmiş defekt, 21 risk, 6 trade-off (F06 həm risk, həm trade-off kimi baxılır), 14 saxlanıla bilənlik, 1 hipotez (F30); F34 risk + hipotezdir.
- İki kritik təhlükəsizlik defekti (F01, F02) bir neçə sətirlə düzəlir və birinci prioritetdir.
- Ən geniş əlçatanlıq riski Kafka-dır (F10, F11): log pipeline-ı campaign-service və log-service-in sorğu yoluna bağlıdır.
- Güclü tərəflər real və çoxdur: DB-per-service, Liquibase, RS256, hash-lənmiş token-lər, dayanıqlıq pattern-ləri, DLQ, real token ilə inteqrasiya testləri, tam müşahidə stack-i.
- Növbəti fəsil bu dərsləri oxşar sistemin sıfırdan dizaynına çevirir.

22. Oxşar sistemi sıfırdan dizayn etmək

Bu fəsil "repozitoriyanı başa düşdüm, indi özüm necə qurardım?" sualına cavab verir. Burada təqdim olunan kod və konfiqurasiyalar **illüstrativdir, repozitoriyada yoxdur**; onlar əvvəlki fəsillərdə müəyyən edilən boşluqları (fəsil 21) nəzərə alan hədəf dizaynı göstərir. Versiya və API adları layihənin istifadə etdiyi Spring Boot 4 / Spring Framework 7 xəttinə uyğun seçilib; yeni layihədə onları rəsmi sənədlə yoxlayın.

22.1 Tələblərdən başlamaq

Repozitoriyanın README və kodundan bərpa olunan tələblər **NƏTİCƏ**:

Növ	Tələb	Dizayna təsiri
Funksional	Admin kampaniya yaradır; LIFE dərhal aktiv, digərləri təsdiq gözləyir	Status maşını, rol modeli
Funksional	Eyni kateqoriya + başlıq + təsvir ilə dublikat qadağandır	Unikal constraint
Funksional	Status dəyişikliklərinin tarixçəsi	Append-only cədvəl və ya hadisə jurnalı
Funksional	Dashboard: status üzrə statistika və tarixçə	Oxu modeli, cache
Funksional	Qeydiyyat, login, refresh, logout, parol bərpası	IdP və ya auth servisi
Funksional	Autentifikasiya auditi və yavaş sorğu jurnalı	Asinxron log pipeline, axtarış
Qeyri-funksional	Rol əsaslı giriş, rate limit	Gateway + resource server
Qeyri-funksional	Müşahidə (metrik, trace, log)	OTel, Prometheus
Qeyri-funksional	Konteynerləşdirmə, CI	Docker, pipeline

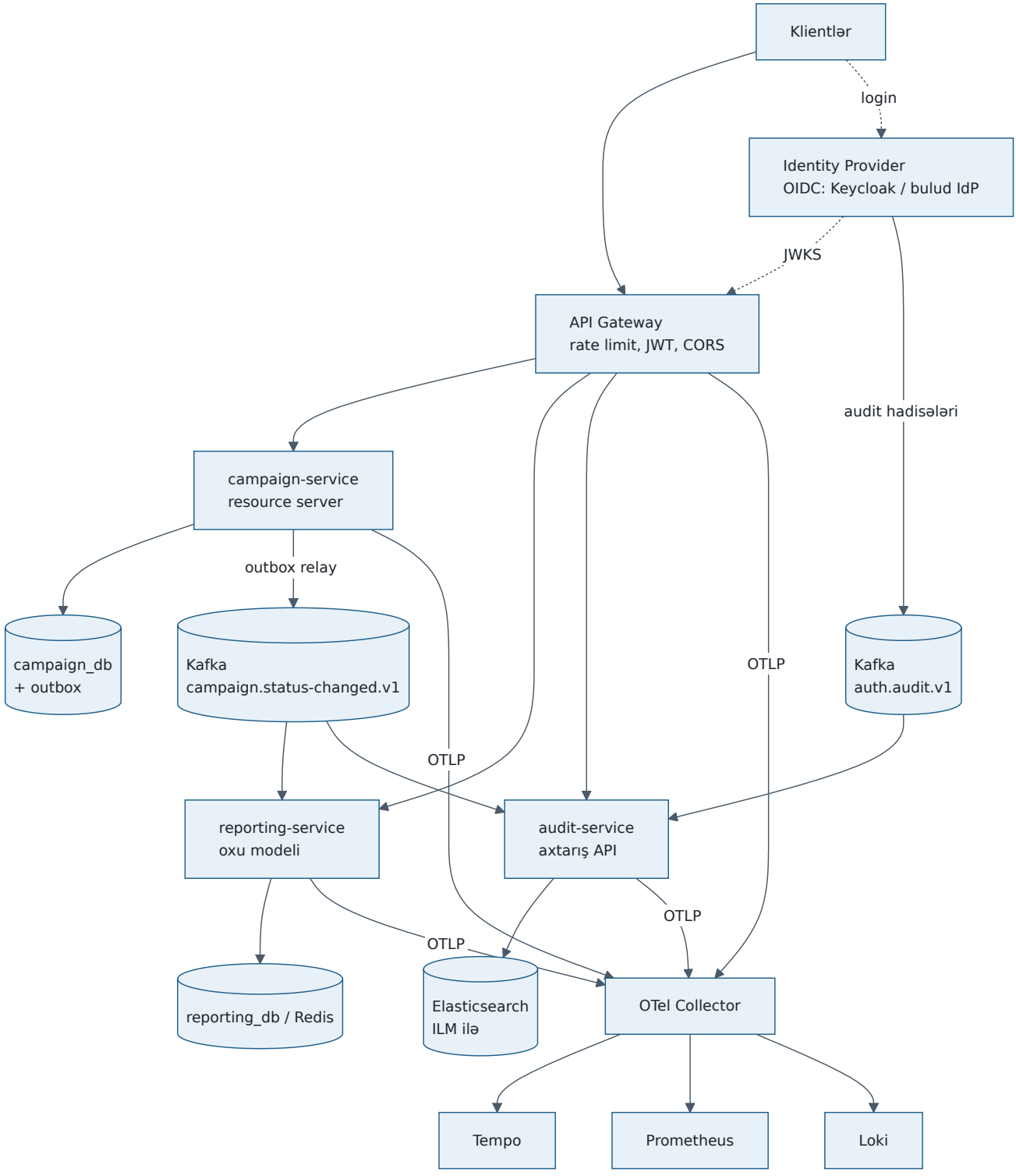
Dizayndan əvvəl cavablandırılmalı suallar (fəsil 21.11): audit itə bilərmi? `campaign-events`-i kim oxuyur? Gözlənilən yük (sorğu/s, kampaniya sayı)? Yerləşdirmə mühiti? Bu cavablar aşağıdakı qərarların hər birini dəyişə bilər.

22.2 Birinci qərar: modul-monolit, yoxsa mikroservislər?

Meyar	Modul-monolit	Mikroservislər
Komanda	1–2 komanda	Hər servisə sahib komanda
Tranzaksiya	Bir DB tranzaksiyası, outbox lazım deyil (daxili hadisələr üçün)	Servislərarası eventual consistency
Yerləşdirmə	Bir artefakt	Müstəqil, lakin koordinasiya tələb edir
Uğursuzluq	Proses daxili çağırışlar	Şəbəkə xətaları, timeout, retry
Miqyaslama	Bütün tətbiq birlikdə	Servis üzrə
Əməliyyat xərci	Aşağı	Yüksək (kəşf, konfiqurasiya, trace, çoxlu DB)

NƏTİCƏ Bu domen üçün tövsiyə: **modul-monolitlə başlamaq** (Spring Modulith və ya Maven modulları ilə ciddi sərhədlər), sərhədləri hadisələrlə qurmaq və yalnız real səbəb (fərqli miqyas, fərqli komanda, fərqli yerləşdirmə tezliyi) yarandıqda servisi ayırmaq. Repozitoriya isə mikroservis texnikalarını öyrətmək üçün qəsdən paylanmış qurulub; aşağıdakı dizayn **paylanmış variantı** təsvir edir, çünki öyrənmə məqsədi budur.

22.3 Hədəf arxitektura



Təklif olunan hədəf arxitektura (illüstrativ)

Repozitoriyadan əsas fərqlər və səbəbləri:

Repozitoriya	Hədəf	Səbəb
Öz auth-service-i, öz JWT kodu	OIDC IdP + Spring Security Resource Server	Parol, MFA, bərpa, enumerasiya qorunması hazırdır; F01–F03, F08, F35 sinfi problemləri azalır

Repozitoriya	Hədəf	Səbəb
Dashboard sinxron gRPC ilə campaign-dən oxuyur	Reporting servisi hadisələrdən öz oxu modelini qurur (CQRS)	Sinxron asılılıq, deadline və status problemləri (F19) yox olur; `campaign-events` real istehlakçı qazanır (F12)
`kafkaTemplate.send` tranzaksiya daxilində	Transactional outbox + relay	Dual write (F09) və bloklanma (F10) aradan qalxır
Audit HTTP + Circuit Breaker, itə bilər	Audit hadisəsi Kafka-ya (outbox ilə)	Davamlılıq; log-service düşəndə audit itmir
Xüsusi yavaş sorğu filtri → Kafka → ES	OTel metrik və trace-lər, tail sampling	Standart alət; ayrıca pipeline lazım deyil (F25, F43)
Eureka + Config Server	Platformanın kəşfi (Kubernetes Service DNS) və ConfigMap/Secret	Daha az hərəkətli hissə (F31, F32)

Yerləşdirmə Compose-dursa və IdP istifadə etmək mümkün deyilsə, öz auth servisi qala bilər — lakin fəsil 22.6-dakı təhlükəsizlik qaydaları ilə.

22.4 Addım-addım qurma planı

İterasiya	Nəticə	Yoxlama meyarı
0. Skelet	Maven BOM (Boot + Cloud), modullar, Dockerfile, CI (build + test)	`mvn verify` yaşıl, image qurulur
1. Kontrakt	OpenAPI spesifikasiyası, hadisə sxemləri (JSON Schema / Avro), xəta formatı (`ProblemDetail`)	Review edilmiş kontraktlar
2. Domen	Status maşını enum-da, keçid cədvəli, unit testlər	Bütün keçidlər test olunur
3. Persistensiya	Liquibase: cədvəllər, `version` sütunu, FK indeksləri, partial unique; `ddl-auto: validate`	Testcontainers-də miqrasiya + validate
4. API	Controller, validasiya, `ProblemDetail` handler, səhifələmə DTO-su	MockMvc testləri: 400/401/403/404/409
5. Təhlükəsizlik	Resource server, rol xəritəsi, metod təhlükəsizliyi, actuator ayrı port	Real token ilə inteqrasiya testi
6. Hadisələr	Outbox cədvəli, relay, hadisə versiyası	Commit olmayan əməliyyat hadisə yaratmır (test)
7. İstehlakçılar	İdempotent consumer, `ErrorHandlingDeserializer` , DLQ, retry topologiyası	Poison pill testi DLQ-ya düşür
8. Oxu modeli	Reporting proyeksiyası, cache TTL ilə	Hadisələrdən sonra statistika düzgündür
9. Gateway	Marşrutlar, rate limit (etibarlı IP), CORS, timeout-lar	429, CORS preflight testləri
10. Müşahidə	OTel, Kafka observation, log korrelyasiyası, dashboard və alert-lər	Bir trace gateway → consumer-ə qədər görünür
11. Dayanıqlıq	Timeout-lar, deadline-lar, retry yalnız idempotent əməliyyatlara	Chaos testi: broker/Redis/DB düşür
12. Yerləşdirmə	Helm/Kustomize və ya Compose profilləri, secret manager, sabit image versiyaları	Staging-də smoke test

22.5 Əsas dizayn qərarları (ADR formatında)

ADR-1: Kampaniya statusu dəyişikliyi necə dərc olunur?

- *Kontekst:* DB yazısı və hadisə atomik olmalıdır.
- *Variantlar:* (a) tranzaksiya daxilində `send` (repozitoriya), (b) Kafka tranzaksiyaları + DB (iki resursun sinxronizasiyası, tam atomik deyil), (c) transactional outbox + polling relay, (d) outbox + CDC (Debezium).
- *Qərar:* (c), yük artanda (d).
- *Nəticə:* At-least-once çatdırılma; istehlakçılar idempotent olmalıdır (hadisə id-si ilə dedup).

ADR-2: Dashboard məlumatı haradan gəlir?

- *Variantlar:* sinxron sorğu (gRPC/REST), hadisələrdən qurulan oxu modeli.
- *Qərar:* oxu modeli. *Qiymət:* eventual consistency (saniyələr), əlavə saxlama. *Qazanc:* campaign-service düşəndə dashboard işləyir, sorğu yükü ayrılır.

ADR-3: Token strategiyası.

- *Qərar:* OIDC IdP; access token 5–15 dəq, refresh token rotasiyası IdP-də; servislərdə yalnız JWKS ilə yoxlama.
- *Ləğv:* qısa ömür + kritik hallarda `jti` deny-list.

ADR-4: Audit davamlılığı.

- *Qərar:* audit hadisəsi eyni outbox mexanizmi ilə; audit-service idempotent yazır; ES-də ILM ilə saxlama müddəti (məs. 400 gün — hüquqi tələbə görə).

ADR-5: Servis kəşfi və konfiqurasiya.

- *Qərar:* Kubernetes-də platformanın DNS-i və ConfigMap/Secret; Eureka və Config Server yalnız platforma bunu təmin etmirsə.

22.6 Kritik komponentlərin nümunə implementasiyası

Resource server və rol xəritəsi

Repozitoriyadakı əl ilə yazılmış JWT filtri əvəzinə Spring Security-nin standart mexanizmi xəta emalını (F08), `aud` / `exp` yoxlamasını və açar rotasiyasını həll edir:

```
// İllüstrativ (repozitoriyada yoxdur)
@Configuration
@EnableMethodSecurity
class SecurityConfig {

    @Bean
    SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/v3/api-docs/**", "/swagger-ui/**").permitAll()
                .anyRequest().authenticated())
            .oauth2ResourceServer(rs -> rs.jwt(jwt ->
                jwt.jwtAuthenticationConverter(rolesConverter())));
        return http.build();
    }

    private JwtAuthenticationConverter rolesConverter() {
        var authorities = new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix("");          // claim artıq ROLE_ prefiksi ilə gəlir
```

```
        var converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }
}
```

```
# İllüstrativ
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://idp.example.com/realms/campaigns   # JWKS avtomatik tapılır
management:
  server:
    port: 9091            # actuator ayrı portda, xaricə açılmır (F40)
```

Status maşını və optimistik kilid

```
// İllüstrativ
@Entity
class CampaignEntity {
    @Id private UUID id;
    @Version private long version;                    // F37, F14
    @Enumerated(EnumType.STRING) private CampaignStatus status;
    private Instant updatedAt;

    void transitionTo(CampaignStatus target, Clock clock) {
        if (!status.canTransitionTo(target)) {
            throw new InvalidStatusTransitionException(status, target);
        }
        this.status = target;
        this.updatedAt = clock.instant();             // F36: cavabda düzgün vaxt
    }
}
```

`Clock` inyeksiyası vaxtı testlərdə idarə etməyə imkan verir. `@Version` paralel yeniləmədə `ObjectOptimisticLockingFailureException` atır; handler onu 409-a çevirir.

Transactional outbox

```
// İllüstrativ
@Transactional
public Campaign approve(UUID id) {
    CampaignEntity campaign = repository.findById(id).orElseThrow(() → new ResourceNotFoundException(i
d));
    CampaignStatus previous = campaign.getStatus();
    campaign.transitionTo(CampaignStatus.ACTIVE, clock);
    history.save(CampaignStatusHistoryEntity.of(campaign, previous, "Campaign approved", clock));
    outbox.save(OutboxEvent.of("campaign.status-changed.v1", campaign.getId(),
            new CampaignStatusChanged(UUID.randomUUID(), campaign.getId(), previous, campaign.getStatus
(), clock.instant())));
    return mapper.toDomain(campaign);                 // Kafka çağırışı yoxdur
}

@Component
class OutboxRelay {
    @Scheduled(fixedDelay = 500)
    @Transactional
    void publish() {
        // SELECT ... FROM outbox WHERE published_at IS NULL ORDER BY id LIMIT 100 FOR UPDATE SKIP LOCKED
        for (OutboxEvent e : outbox.lockNextBatch(100)) {
            kafka.send(e.topic(), e.aggregateId().toString(), e.payload()).join();   // ack gözlənilir
            e.markPublished(clock.instant());
        }
```

```
    }
}
```

Relay bir neçə instansda işləsə də `SKIP LOCKED` eyni sətri iki dəfə götürməyə imkan vermir. Göndərildikdən sonra commit uğursuz olarsa hadisə təkrar göndərilir — buna görə istehlakçıda dedup lazımdır.

İdempotent və poison-pill-ə davamlı consumer

```yaml
# İllüstrativ
spring:
  kafka:
    producer:
      properties:
        max.block.ms: 2000                # F10
    consumer:
      group-id: reporting-service
      auto-offset-reset: earliest
      value-deserializer: org.springframework.kafka.support.serializer.ErrorHandlingDeserializer   # F11
      properties:
        spring.deserializer.value.delegate.class: org.springframework.kafka.support.serializer.JacksonJso
nDeserializer
    listener:
      observation-enabled: true           # trace konteksti Kafka başlıqlarında
    template:
      observation-enabled: true
```

```java
// İllüstrativ
@KafkaListener(topics = "campaign.status-changed.v1")
@Transactional
void on(CampaignStatusChanged event) {
    if (!processedEvents.markProcessed(event.eventId())) {   // INSERT ... ON CONFLICT DO NOTHING
        return;                                               // dublikat çatdırılma
    }
    statistics.apply(event.previousStatus(), event.newStatus());
}
```

Xəta formatı: `ProblemDetail`

```java
// İllüstrativ
@RestControllerAdvice
class ApiExceptionHandler extends ResponseEntityExceptionHandler {   // 400/404/405/415 standart xəritələ
r (F07)

    private static final Logger log = LoggerFactory.getLogger(ApiExceptionHandler.class);

    @ExceptionHandler(BusinessException.class)
    ProblemDetail business(BusinessException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(ex.getErrorCode().status(), ex.getMessage());
        pd.setProperty("code", ex.getErrorCode().name());
        return pd;
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex) {
        log.error("Unhandled exception", ex);                          // F38
        return ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR, "Unexpected error");
    }
}
```

Sinxron çağırışlar üçün timeout və deadline

```yaml
# İllüstrativ — HTTP Interface klientləri üçün. Xassə adları Boot 4.1 HttpClientProperties mənbəsindən yo
xlanılıb
spring:
```

```yaml
  http:
    serviceclient:
      audit:
        base-url: http://audit-service:8080
        connect-timeout: 1s
        read-timeout: 2s
```

```java
// İllüstrativ — gRPC: hər çağırışa deadline
var response = stub.withDeadlineAfter(500, TimeUnit.MILLISECONDS).getCampaignStatistics(request);
```

22.7 Paket quruluşu

Repozitoriyanın texniki qat üzrə paketləri (`controller` , `service` , `repository`) kiçik servisdə işləyir. Böyüyən servis üçün funksional (feature) paketləmə tövsiyə olunur:

```
campaign-service/src/main/java/.../campaign/
├── CampaignServiceApplication.java
├── campaign/                  # aqreqat: API + tətbiq servisi + domen + persistensiya
│   ├── api/                   # controller, request/response DTO, mapper
│   ├── application/           # CampaignService (use-case-lər), portlar (CampaignEventPublisher)
│   ├── domain/                # Campaign, CampaignStatus (keçid cədvəli), domen istisnaları
│   └── infrastructure/        # JPA entity, repository, outbox adapteri
├── history/
├── outbox/                    # OutboxEvent, OutboxRelay
└── config/                    # security, kafka, cache, observability
```

Paket-private görünürlük (`infrastructure` sinifləri `public` deyil) sərhədləri kompilyator səviyyəsində qoruyur; Spring Modulith və ya ArchUnit testləri qaydaları avtomatik yoxlayır.

22.8 İstehsala hazırlıq yoxlama siyahısı

Sahə	Yoxlama
Təhlükəsizlik	Rol klientdən gəlmir; bərpa token-i yalnız xarici kanalla; eyni cavab enumerasiyanı bağlayır; daxili portlar (gRPC, actuator) xaricə açılmır; secret-lər repozitoriyada yoxdur; etibarlı proxy siyahısı; CORS
Məlumat	`@Version` ; FK indeksləri; unikal constraint-lər; miqrasiyalar testdə işləyir; geri qaytarma planı
Hadisələr	Outbox; versiyalı mövzu adları; sxem reyestri/uyğunluq; idempotent consumer; DLQ və onun emalı (alert + replay)
Dayanıqlıq	Hər uzaq çağırışda timeout; retry yalnız idempotent əməliyyatlarda; circuit breaker; bulkhead; `max.block.ms`
Cache	TTL; transaction-aware eviction; `CacheErrorHandler` (fail-open)
Müşahidə	RED metrikləri; Kafka lag; DLQ ölçüsü; trace-lər Kafka sərhədindən keçir; 5xx loglanır; aşağı kardinallı label-lar
Əməliyyat	Sabit image versiyaları; health/readiness ayrımı; graceful shutdown; resurs limitləri; ES ILM; backup
Test	Unit (domen), slice (web/data), Testcontainers (miqrasiya ilə), kontrakt testləri, gateway testləri

22.9 Bu repozitoriyadan çıxarılan dərslər

Nümunə	Dərs
`role` qeydiyyat sorğusunda	Səlahiyyət heç vaxt klientin özünü təyin etdiyi sahədən gəlməməlidir
Catch-all 500	Framework-un düzgün davranışını udmayın; gözlənilməz xətanı loglayın
`send()` tranzaksiyada	Xarici sistemə yazı tranzaksiyanın hissəsi deyil — outbox
Consumer-siz mövzu	Hadisə yalnız istehlakçı və kontrakt olduqda dəyər yaradır
5 ms həddi	Müşahidə məlumatının da qiyməti var: həcm, saxlama, kardinallıq
Deadline-sız gRPC	Sinxron zəncirdə hər hop gözləmə müddətini məhdudlaşdırmalıdır
Testdə `create-drop`	Testlər istehsal sxemi ilə işləməlidir
İki üslub (Lombok/MapStruct vs əl ilə)	Konvensiyanı ADR ilə sabitləyin

22.10 Müsahibə: "Kampaniya idarəetmə sistemi dizayn edin"

Sistem dizaynı müsahibəsində 45 dəqiqəlik cavabın strukturu:

1. **Tələblər (5 dəq):** aktorlar, status maşını, dublikat qaydası, audit, dashboard; yük təxmini (məs. gündə min kampaniya, saniyədə yüzlərlə oxu) — yazı az, oxu çox.
2. **API (5 dəq):** `POST /campaigns`, `PATCH /campaigns/{id}/status` (və ya `.../approve`), `GET` səhifələmə ilə; idempotency açarı yaratma üçün.
3. **Data modeli (5 dəq):** `campaigns` (+ `version`), `campaign_status_history`, `outbox`; partial unique indeks.
4. **Yüksək səviyyəli arxitektura (10 dəq):** gateway, campaign-service, outbox → Kafka, reporting oxu modeli, audit, IdP.
5. **Dərinləşmə (15 dəq):** atomik dərc (outbox), paralel yeniləmə (`@Version`), cache invalidasiyası, rate limit, təhlükəsizlik (rol, token ömrü), uğursuzluq ssenariləri (broker düşür → outbox yığılır, sonra boşalır).
6. **Müşahidə və əməliyyat (5 dəq):** RED metrikləri, consumer lag, DLQ alert-i, trace-lər.

Bu repozitoriyanı nümunə kimi gətirmək güclü arqumentdir: "Mən bunu qurulmuş bir sistemdə gördüm, burada dual write və deadline-sız gRPC problemləri var idi; dizaynımda onları belə həll edirəm."

22.11 Xülasə

- Başlanğıc nöqtəsi tələblər və açıq suallardır; arxitektura onlardan çıxır.
- Bu ölçüdə modul-monolit daha sadədir; paylanmış variant seçilərsə — outbox, oxu modeli, IdP, platformanın kəşfi.
- Fəsil 21-dəki hər tapıntının hədəf dizaynda konkret qarşılığı var: `@Version`, `ProblemDetail`, resource server, `ErrorHandlingDeserializer`, timeout/deadline, TTL, ILM.
- Plan iterativdir və hər iterasiyanın yoxlanıla bilən meyarı var.

VII hissə. Production və müsahibəyə hazırlıq

Runbook, troubleshooting, sistemin tam izahı, sual-cavablar və əlavələr

23. İstehsal mühəndisliyi: əməliyyat və troubleshooting runbook-u

Bu fəsil sistemi işlədən mühəndis üçündür: "nə baş verir və nə etməliyəm?" Repozitoriyada yalnız Docker Compose yerləşdirməsi var **FAKT**, ona görə əmrlər Compose-a uyğundur. İstehsal mühiti (Kubernetes, bulud) üçün eyni məntiq `kubectl logs/exec`-ə çevrilir. Əmrlərdə şifrə və token **göstərilmir** — `<...>` yer tutucuları mühitinizin secret-ləri ilə əvəz olunmalıdır.

23.1 Sistem xəritəsi: portlar və sağlamlıq yoxlamaları

Komponent	Konteyner	Host portu	Sağlamlıq yoxlaması	Compose healthcheck
api-gateway	`api-gateway`	8080	`GET /actuator/health`	Yoxdur
auth-service	`auth-service`	8081	`GET /actuator/health`	Yoxdur
campaign-service	`campaign-service`	8082, gRPC 9090	`GET /actuator/health`	Yoxdur
dashboard-service	`dashboard-service`	8083	`GET /actuator/health`	Yoxdur
log-service	`log-service`	8084	`GET /actuator/health`	Yoxdur
discovery-server	`discovery-server`	8761	Eureka UI	Yoxdur
config-server	`config-server`	8888	`GET /actuator/health`	Yoxdur
PostgreSQL (auth / campaign)	`auth-db` / `campaign-db`	5433 / 5434	`pg_isready`	Var (`docker-compose.yml:103-107` , `122-126`)
Redis	`redis`	6379	`redis-cli ping`	Var (`docker-compose.yml:291-295`)
Kafka	`kafka`	9092	`kafka-topics.sh --list`	Var (`docker-compose.yml:326-331`)
Elasticsearch	`elasticsearch`	9200	`_cluster/health`	Var (`docker-compose.yml:363-368`)
Prometheus / Grafana	`prometheus` / `grafana`	9091 / 3000	UI	—
Tempo / Loki / OTel Collector	`tempo` / `loki` / `otel-collector`	3200 / 3100 / 4317, 4318	—	—
Kafka UI	`kafka-ui`	8090	UI	—

FAKT Tətbiq servislərinin healthcheck-i yoxdur, ona görə `depends_on: condition: service_healthy` onlar üçün işləmir (F29). Actuator `health,info,prometheus,metrics` açıqdır və `permitAll`-dur (F40) — istehsalda ayrı management portu və şəbəkə izolyasiyası lazımdır.

23.2 Nəyi izləmək lazımdır: siqnallar

Prometheus bütün beş tətbiq servisini `/actuator/prometheus` üzərindən toplayır (`observability/prometheus/prometheus.yml:5-24`). discovery-server və config-server scrape olunmur **FAKT**.

Siqnal	Metrik (Micrometer adı → Prometheus)	Niyə vacibdir
Sorğu sürəti, xəta, gecikmə (RED)	`http_server_requests_seconds_count/sum/bucket` (`uri`, `status`, `method`)	Əsas SLI
Gateway 429 payı	`http_server_requests_seconds_count{status="429"}` (gateway)	Rate limit həddinin düzgünlüyü
DB bağlantı hovuzu	`hikaricp_connections_active`, `_pending`, `_timeout_total`	Hovuz tükənməsi gecikmə kimi görünür
JVM	`jvm_memory_used_bytes`, `jvm_gc_pause_seconds`, `jvm_threads_live_threads`	Yaddaş sızması, GC təzyiqi
Circuit breaker	`resilience4j_circuitbreaker_state` (auth-service; `resilience4j-micrometer` classpath-dadır) **NƏTİCƏ**	log-service problemlərinin erkən siqnalı
Kafka consumer	`kafka_consumer_fetch_manager_records_lag_max` [NƏTİCƏ: Boot-un consumer factory-si Micrometer listener əlavə edir]	log-service geri qalırmı?
Kafka producer	**Yoxdur** — `ProducerFactory` əl ilə yaradılıb (`KafkaJsonConfiguration.java:9-17`) **NƏTİCƏ**	Producer xətaları görünmür
DLQ	Ayrıca metrik yoxdur; yalnız ERROR log `DLQ message received`	Alert log əsasında qurulmalıdır
Elasticsearch	Scrape olunmur	`_cluster/health`, disk istifadəsi əl ilə

```yaml
# İllüstrativ Prometheus alert qaydaları (repozitoriyada yoxdur)
groups:
- name: campaign-platform
  rules:
  - alert: High5xxRate
    expr: sum by (job) (rate(http_server_requests_seconds_count{status=~"5.."}[5m]))
          / sum by (job) (rate(http_server_requests_seconds_count[5m])) > 0.02
    for: 5m
  - alert: SlowP95
    expr: histogram_quantile(0.95, sum by (job, le) (rate(http_server_requests_seconds_bucket[5m]))) > 1
    for: 10m
  - alert: HikariPending
    expr: max by (job) (hikaricp_connections_pending) > 0
    for: 2m
  - alert: AuditCircuitOpen
    expr: max(resilience4j_circuitbreaker_state{name="logServiceHttpClient",state="open"}) == 1
    for: 1m
  - alert: LogConsumerLag
    expr: max(kafka_consumer_fetch_manager_records_lag_max{client_id=~".*log-service.*"}) > 1000
    for: 10m
```

`histogram_quantile` üçün bucket-lər lazımdır; Boot standart olaraq `http.server.requests` üçün histogram bucket-ləri dərc etmir — `management.metrics.distribution.percentiles-histogram.http.server.requests: true` əlavə edilməlidir **NƏTİCƏ**. Repozitoriyada bu xassə yoxdur.

23.3 Diaqnostika alətləri

```bash
# Konteyner vəziyyəti və loglar
docker compose ps
docker compose logs -f --since=10m campaign-service
docker compose logs campaign-service | grep -E "ERROR|WARN" | tail -50
```

```
# Servis sağlamlığı
curl -s localhost:8082/actuator/health
curl -s localhost:8082/actuator/metrics/hikaricp.connections.active

# Eureka reyestri (gateway-in görə bildiyi instanslar)
curl -s -H 'Accept: application/json' localhost:8761/eureka/apps | head

# Kafka: mövzular, consumer group lag, DLQ məzmunu
docker exec kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092 --list
docker exec kafka /opt/kafka/bin/kafka-consumer-groups.sh --bootstrap-server localhost:9092 \
  --describe --group log-service
docker exec kafka /opt/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 \
  --topic log-events.DLQ --from-beginning --max-messages 5 --property print.headers=true

# Redis: cache və rate limiter açarları (KEYS əvəzinə SCAN — istehsalda KEYS bloklayır)
docker exec redis redis-cli --scan --pattern 'campaign-statistics*'
docker exec redis redis-cli --scan --pattern 'request_rate_limiter*' | head
docker exec redis redis-cli info memory | grep used_memory_human

# Elasticsearch: indekslər, sağlamlıq, son sənədlər
curl -s 'localhost:9200/_cat/indices?v'
curl -s 'localhost:9200/_cluster/health?pretty'
curl -s 'localhost:9200/slow-request-logs/_search?size=3&sort=occurredAt:desc&pretty'

# PostgreSQL (şifrə mühitdən daxil edilir, burada göstərilmir)
docker exec -it campaign-db psql -U <campaign_db_user> -d campaign_db \
  -c "select status, count(*) from campaigns group by status;"
docker exec -it campaign-db psql -U <campaign_db_user> -d campaign_db \
  -c "select id, filename, exectype, dateexecuted from databasechangelog order by orderexecuted;"
```

Loki (LogQL): `{service="campaign-service", level="ERROR"}`; traceId ilə bütün servislər üzrə: `{trace_id="<traceId>"}`. Bu sorğular Promtail-in `service`, `level`, `trace_id`, `span_id` label-larına əsaslanır (`observability/promtail/promtail-config.yml:15-49`); `trace_id`-nin label olması rahatdır, lakin yüksək kardinallıq yaradır (F24) — düzəlişdən sonra `{service=~".+"} |= "<traceId>"` istifadə olunmalıdır. **Tempo:** Grafana Explore → Tempo → TraceQL `{ resource.service.name = "api-gateway" && status = error }`.

FAKT `occurredAt` açıq `Date` mapping-ə malikdir (`SlowRequestDocument.java:25-26`), ona görə yuxarıdakı sıralama işləyir; digər sahələr dinamik mapping ilə yaranır (F42) — `GET slow-request-logs/_mapping` ilə tipləri yoxlayın.

23.4 Hadisə playbook-ları

Hər playbook: **simptom → ehtimal olunan səbəb → yoxlama → hərəkət**.

P1. Bütün sorğular 429 qaytarır

- **Səbəb:** `requestedTokens = burstCapacity = 5`, `replenishRate = 1` → bir açar üçün 5 saniyədə 1 sorğu (fəsil 15.6). Bir NAT arxasındakı bütün anonim istifadəçilər eyni `ip:` açarını paylaşır.
- **Yoxlama:** cavabda `X-RateLimit-Remaining`, `Retry-After` başlıqları; `redis-cli --scan --pattern 'request_rate_limiter*'`.
- **Hərəkət:** `RATE_LIMITER_REQUESTED_TOKENS=1`, `RATE_LIMITER_BURST_CAPACITY` artırmaq (`api-gateway/src/main/resources/application.yml:24-26`) və gateway-i yenidən başlatmaq.

P2. Bütün qorunan sorğular 401 qaytarır

- **Səbəblər:** (a) auth-service-in private key-i ilə gateway/servislərin public key-i uyğun gəlmir (açar rotasiyası yarımçıq); (b) `app.jwt.issuer` fərqlidir; (c) token vaxtı keçib (120 dəq); (d) saat fərqi (clock skew).

- **Yoxlama:** gateway 401-i **boş gövdə** ilə qaytarır, servis isə `ApiError` JSON-u ilə — bu, rədd edən qatı göstərir. Token-in payload hissəsini base64 ilə açıb `iss`, `exp` dəyərlərinə baxın (imzanı paylaşmayın).
- **Hərəkət:** açar cütünü bütün komponentlərdə eyni vaxtda yeniləmək (23.7); `kid` dəstəyi olmadığı üçün rotasiya qısa fasilə ilə nəticələnir.

P3. Gateway 503 qaytarır

- **Səbəb:** `lb://` marşrutu üçün Eureka-da instans yoxdur (servis düşüb və ya qeydiyyatdan keçməyib).
- **Yoxlama:** `localhost:8761` UI; servis logunda `DiscoveryClient` xətaları; `EUREKA_INSTANCE_IP_ADDRESS`.
- **Hərəkət:** servisi yenidən başlatmaq; qeydiyyatın 30 saniyəyə qədər gecikə biləcəyini nəzərə almaq [NƏTİCƏ: Eureka klientinin standart yeniləmə intervalları].

P4. campaign/log sorğuları ~60 saniyə çəkir

- **Səbəb:** Kafka əlçatmazdır; `KafkaTemplate.send()` metadata üçün `max.block.ms` (60 s) gözləyir — yavaş sorğu filtrində (yalnız campaign-service və log-service) və kampaniya yazılarında (F10). auth və dashboard təsirlənmir — bu, diaqnozda faydalı əlamətdir. Klientin gördüyü status `NƏTİCƏ` cavabın artıq commit olunub-olunmamasından asılıdır; kampaniya yazısında istisna tranzaksiyanı geri qaytarır və 500 verir.
- **Yoxlama:** `docker compose ps kafka`; servis logunda `TimeoutException ... not present in metadata after 60000 ms` [NƏTİCƏ: kafka-clients mesaj formatı]; Tempo-da uzun span-lar.
- **Hərəkət:** Kafka-nı bərpa etmək. Müvəqqəti yumşaltma: `app.slow-request-threshold-ms`-i çox böyük dəyərə qaldırmaq (filtr göndərmir) — kampaniya yazıları isə Kafka olmadan yenə bloklanır. Daimi həll: fəsil 22.6.

P5. Dashboard 500 qaytarır (`UNAVAILABLE` / `UNKNOWN`)

- **Səbəb:** campaign-service gRPC (9090) əlçatmazdır və ya server istisna atıb.
- **Yoxlama:** dashboard logunda yox, **cavab gövdəsində** `StatusRuntimeException` mesajı (handler loglamır — F38); `docker compose logs campaign-service`; `CAMPAIGN_GRPC_ADDRESS`.
- **Hərəkət:** campaign-service-i bərpa etmək. `UNKNOWN` + mövcud olmayan id → bu, 404 ssenarisidir (fəsil 20.7), infrastruktur problemi deyil.

P6. Statistika köhnədir

- **Səbəb:** evict/commit yarışı və TTL olmaması (F13).
- **Yoxlama:** `GET /api/v1/campaigns/statistics` ilə DB-dəki `group by` nəticəsini müqayisə etmək.
- **Hərəkət:** `redis-cli DEL` ilə `campaign-statistics::SimpleKey []` açarını silmək (və ya `--scan` ilə tapılan açarı); növbəti sorğu DB-dən doldurur. Daimi həll: TTL + transaction-aware cache.

P7. Yavaş sorğu jurnalında yeni qeyd görünmür

Ehtimal olunan səbəblər ardıcıllıqla:

1. **Poison pill** (F11) — log-service logunda dövri `Consumer exception` / `SerializationException`, consumer group lag artır, DLQ boşdur. Hərəkət: problemli offseti keçmək (`kafka-consumer-groups.sh --reset-offsets --to-offset <N+1> --topic log-events:0 --group log-service --execute`, consumer dayandırılmış halda), sonra `ErrorHandlingDeserializer` əlavə etmək.
2. **Elasticsearch düşüb / yalnız oxu rejimində** — `DefaultErrorHandler` 3 cəhddən sonra qeydləri `log-events.DLQ`-a göndərir; logda `DLQ message received`. ES disk "flood stage" həddinə çatanda indeksə `index.blocks.read_only_allow_delete` qoyulur [NƏTİCƏ: ES standart davranışı]. Hərəkət: disk boşaltmaq, bloku götürmək, DLQ-nu yenidən emal etmək (repozitoriyada replay mexanizmi yoxdur — əl ilə).
3. **Producer tərəfi** — servis Kafka-ya yaza bilmir (P4).

4. **Mövzu yoxdur** — avtomatik yaradılma söndürülübsə (`KAFKA_AUTO_CREATE_TOPICS_ENABLE` , `docker-compose.yml:322`).

P8. Auth audit qeydləri itir

- **Səbəb:** log-service düşüb → Retry (3×500 ms) → circuit breaker açıqdır (30 s) → çağırışlar atlanır; bulkhead dolub (10 eyni vaxtda).
- **Yoxlama:** auth-service logunda `Auth audit skipped because log-service circuit breaker is open` ; `resilience4j_circuitbreaker_state` .
- **Hərəkət:** log-service-i bərpa etmək. İtmiş audit bərpa olunmur — audit hüquqi tələbdirsə dizayn dəyişməlidir (fəsil 22.5, ADR-4).

P9. Servis başlamır

Log mesajı / əlamət	Səbəb	Hərəkət
`Schema-validation: missing column/table`	`ddl-auto: validate` sxemlə entity arasında fərq	Liquibase changeset-in tətbiq olunduğunu yoxlamaq
`Validation Failed: ... checksum` (Liquibase)	Tətbiq olunmuş changeset dəyişdirilib	Changeset-i dəyişməmək; yeni changeset; son çarə `validCheckSum`
`Connection refused` config-server-ə	Config Server əlçatmazdır	`optional:` sayəsində başlayır, lakin paylanan xassələr itir (F32)
`No qualifying bean of type ObjectMapper`	Jackson 2 bean-i yoxdur (Eureka asılılığı çıxarılıb)	F20 — `JsonMapper` -ə keçid
Private/public key oxuna bilmir	`JWT_*_KEY_LOCATION` səhv	Fayl yolunu və formatı (PEM) yoxlamaq
`UnsupportedClassVersionError` / preview xətası	JVM 25 deyil və ya `--enable-preview` yoxdur	Image və işə salma parametrləri

P10. DB bağlantı hovuzu tükənir

- **Simptom:** gecikmə artır, sonra `Connection is not available, request timed out after 30000ms` [NƏTİCƏ: Hikari standart `connectionTimeout`].
- **Səbəb:** virtual thread-lər çox sayda eyni vaxtlı sorğu qəbul edir, hovuz (standart 10) isə məhduddur; tranzaksiya daxilində yavaş xarici çağırış (Kafka `send` bloklanması — F10) bağlantını uzun müddət tutur.
- **Hərəkət:** `hikaricp_connections_pending` metrikinə baxmaq; xarici çağırışı tranzaksiyadan çıxarmaq; hovuz ölçüsünü DB-nin `max_connections` limitinə görə planlaşdırmaq (servis sayı × instans × hovuz ölçüsü).

P11. Gözlənilməz 409 `DUPLICATE_CAMPAIGN`

- **Səbəb:** eyni kateqoriya + başlıq + təsvir ilə aktiv/deaktiv kampaniya artıq var; yoxlama `DEACTIVE` statusunu da nəzərə alır (yalnız `DUPLICATE` istisnadır) — deaktiv kampaniyanı eyni mətnlə yenidən yaratmaq mümkün deyil **FAKT** (`CampaignService.java:140-150`).
- **Hərəkət:** biznes qaydasının niyyətini komanda ilə dəqiqləşdirmək (fəsil 21.11).

23.5 Yerləşdirmə və geri qaytarma

Dəyişiklik növü	Risk	Prosedur
Kod (sxemsiz)	Aşağı	Yeni image, rolling restart; köhnə image teqi geri qaytarmaq üçün saxlanır (`latest` istifadə etməyin)

Dəyişiklik növü	Risk	Prosedur
Liquibase changeset	Orta–yüksək	Geriyə uyğun dəyişiklik (expand → migrate → contract); rollback blokları var (`campaign-service/src/main/resources/db/changelog/db.changelog-master.yml:111-113`), lakin məlumat itkisini nəzərə alın
Hadisə sxemi	Yüksək	Yalnız əlavə sahələr; silmə/ad dəyişmə → yeni versiyalı mövzu
Proto (gRPC)	Yüksək	Sahə nömrələrini dəyişməmək, yalnız yeni sahə; əvvəl server, sonra klient
JWT açarı	Yüksək	23.7
Rate limit / hədd parametrləri	Aşağı	Env dəyişəni + restart

23.6 Tutum və tənzimləmə

- **Virtual thread-lər + Hikari:** servlet sorğuları qeyri-məhdud paralelliklə gəlir, hovuz isə 10 bağlantıdır; darboğaz DB hovuzuna keçir. Hovuz ölçüsünü artırmaq əvəzinə tranzaksiyaları qısa saxlamaq daha effektivdir.
- **Kafka:** bütün mövzular 1 partisiyadır (`KAFKA_NUM_PARTITIONS: 1`) — log-service yalnız bir consumer ilə paralelləşə bilər. Miqyas üçün partisiya sayını əvvəlcədən planlaşdırın (artırmaq açar → partisiya xəritəsini dəyişir).
- **Elasticsearch:** tək node, `-Xms512m -Xmx512m` (`docker-compose.yml:353-362`), ILM yoxdur (F43). 5 ms həddi ilə `slow-request-logs` sürətlə böyüyür; rollover + saxlama müddəti mütləqdir.
- **Redis:** `maxmemory` və eviction siyasəti təyin olunmayıb [FAKT: compose-da konfiqurasiya yoxdur]; cache açarlarının TTL-i yoxdur — yaddaş məhdudlaşdırılmalıdır.

23.7 Təhlükəsizlik əməliyyatları

JWT açar rotasiyası. Token-lərdə `kid` başlığı və servislərdə çoxlu açar dəstəyi yoxdur [NƏTİCƏ: `JwtTokenService` tək public key ilə yoxlayır]. Ona görə rotasiya ardıcıllığı: (1) yeni cüt yaratmaq; (2) bütün yoxlayıcılara (gateway, 4 servis) yeni public key; (3) auth-service-ə yeni private key; (4) köhnə token-lər 120 dəqiqə ərzində rədd olunur — istifadəçilər yenidən login olur. Fasiləsiz rotasiya üçün JWKS və `kid` lazımdır (fəsil 10, 22).

Secret-lər. Repozitoriyadakı demo açar və şifrələr (F22) istehsalda istifadə olunmamalıdır: secret manager (Vault, bulud KMS), konteynerə fayl və ya env kimi inyeksiya, rotasiya siyasəti. Repozitoriya tarixçəsindən secret-i silmək kifayət deyil — kompromis sayılmalı və dəyişdirilməlidir.

Şəbəkə. Xaricə yalnız gateway açılmalıdır; gRPC 9090 (F04), DB, Redis, Kafka, ES, actuator portları daxili şəbəkədə qalmalıdır.

23.8 Müsahibə sualları

S1. İstehsalda bütün sorğular birdən yavaşladı. Addımlarınız? Cavab: RED metriklərində hansı servis və endpoint; ortaq asılılıq (DB hovuzu, Kafka, Redis); trace-də ən uzun span; son dəyişiklik (deploy, konfiqurasiya). Bu sistemdə ilk şübhəli — Kafka (`max.block.ms`) və Hikari hovuzu.

S2. Consumer lag artır, DLQ boşdur. Nə düşünürsünüz? Cavab: Consumer irəliləmir, lakin qeydlər DLQ-ya da getmir — ya consumer dayanıb, ya da poison pill (`ErrorHandlingDeserializer` yoxdur). Logda təkrarlanan deserializasiya xətası axtarmaq, offseti keçmək, konfiqurasiyanı düzəltmək.

S3. Liquibase checksum xətası nə deməkdir? Cavab: Artıq tətbiq olunmuş changeset-in məzmunu dəyişdirilib. Düzgün həll — dəyişikliyi geri qaytarıb yeni changeset yazmaq; `validCheckSum` yalnız şüurlu istisna üçün.

S4. Circuit breaker açıq qalır. Necə araşdırırsınız? Cavab: `resilience4j_circuitbreaker_state` və çağırış nəticələri metrikləri; qorunan servisin sağlamlığı; CB-nin hansı istisnaları uğursuzluq sayması (`record-exceptions` — burada bütün `RuntimeException`-lar, 4xx də daxil).

S5. JWT açarını fasiləsiz necə dəyişərdiniz? Cavab: JWKS endpoint-i, `kid` başlığı, yoxlayıcılarda köhnə və yeni açarın paralel qəbulu, sonra köhnə açarın silinməsi.

23.9 Xülasə

- Compose-da tətbiq servislərinin healthcheck-i yoxdur; actuator açıqdır.
- Əsas siqnallar: RED metrikləri, Hikari, circuit breaker vəziyyəti, consumer lag; producer və ES metrikləri yoxdur.
- Ən çox ehtimal olunan insidentlər: rate limit (P1), Kafka bloklanması (P4), poison pill (P7), dashboard gRPC (P5).
- Açar rotasiyası, secret-lər və şəbəkə izolyasiyası istehsala keçidin ilkin şərtləridir.

24. Sistemi təqdim etmək və müsahibəyə hazırlıq

Hər fəslin sonunda mövzu üzrə suallar var. Bu fəsil onları üç formaya salır: (1) sistemi 1 və 5 dəqiqədə izah etmək,

2. mövzular üzrə sual bankı (qısa, layihəyə bağlı cavablarla), (3) bu repozitoriya haqqında verilə biləcək "çətin" suallar və onlara dürüst cavablar. Cavablar yadda saxlamaq üçün deyil — məntiqi və faktları göstərmək üçündür; öz sözlərinizlə danışın.

24.1 Yadda saxlanmalı faktlar

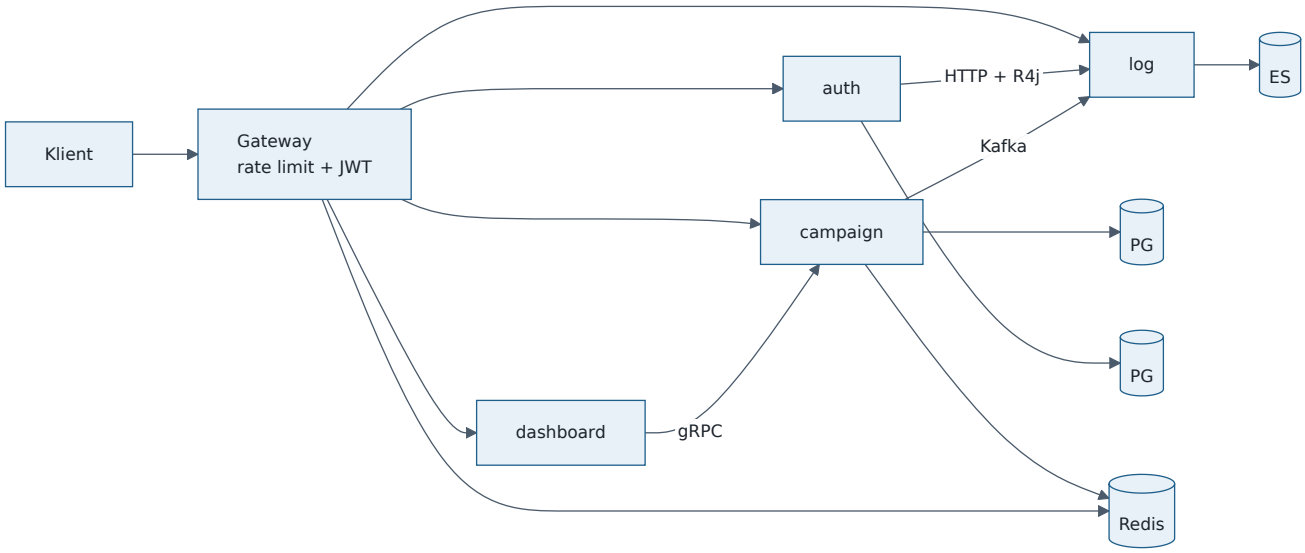
Sahə	Dəyər
Platforma	Java 25 (`--enable-preview`), Spring Boot 4.1.0, Spring Cloud 2025.1.2, Maven multi-module (10 modul)
Servislər və portlar	gateway 8080, auth 8081, campaign 8082 (+ gRPC 9090), dashboard 8083, log 8084, Eureka 8761, Config 8888
Verilənlər	PostgreSQL ×2 (auth_db, campaign_db), Redis, Kafka (KRaft, 1 broker), Elasticsearch 9.4.2
Mövzular	`log-events` , `log-events.DLQ` , `campaign-events` (consumer-siz)
ES indeksləri	`auth-audit-logs` , `slow-request-logs`
Rollar	`ROLE_USER` , `ROLE_ADMIN` , `ROLE_INTERNAL_SERVICE` (servislərarası)
Statuslar	`PENDING_APPROVAL` , `ACTIVE` , `DEACTIVE` , `DUPLICATE` (istifadə olunmur); `LIFE` → dərhal `ACTIVE`
Token-lər	RS256; access 120 dəq; refresh 7 gün (hash); reset 15 dəq (hash)
Rate limit	replenish 1/s, burst 5, hər sorğu 5 token → 5 saniyədə 1 sorğu (marşrut + açar üzrə)
Resilience4j (auth → log)	Retry 3 × 500 ms; CB 50% / 10 çağırış / 30 s; Bulkhead 10, gözləmə 0
DLQ	`DefaultErrorHandler` + `FixedBackOff(1000, 3)` → `<mövzu>.DLQ`
Yavaş sorğu həddi	5 ms
Cache	`campaign-statistics` , Redis, TTL yoxdur
Testlər	24 fayl, 93 `@Test` ; Testcontainers (PostgreSQL, Kafka, Elasticsearch)

24.2 Bir dəqiqəlik təqdimat

> "Bu, sığorta kampaniyalarını idarə edən mikroservis sistemidir. Admin kampaniya yaradır; həyat sığortası kampaniyaları dərhal aktiv olur, digərləri təsdiq gözləyir. Sistem beş tətbiq servisindən ibarətdir: Spring Cloud Gateway — rate limit və JWT yoxlaması ilə vahid giriş; auth-service — RS256 token-lər, refresh rotasiyası; campaign-service — PostgreSQL, Liquibase, Redis cache, status dəyişikliklərini Kafka-ya dərc edir və dashboard üçün gRPC server; dashboard-service — gRPC klienti; log-service — Kafka-dan yavaş sorğuları və HTTP ilə audit qeydlərini Elasticsearch-ə yazır. Java 25,

> Spring Boot 4, virtual thread-lər, Testcontainers, Prometheus/Tempo/Loki/Grafana. Kodu təhlil edərkən bir neçə ciddi problem tapdım — məsələn, admin rolunun özünü qeydiyyatı və tranzaksiya daxilində Kafka göndərişi — və onların həllini bilirəm."

24.3 Beş dəqiqəlik izah (struktur)



Lövhədə çəkiləcək sadə sxem

Dəqiqə	Mövzu	Nə demək
0:00–0:45	Biznes	Kampaniya, kateqoriya, status maşını, dublikat qaydası, rollar
0:45–2:00	Arxitektura	Sxemi çəkin; hər servisin məsuliyyəti; DB-per-service; sinxron (REST, gRPC, HTTP) və asinxron (Kafka) kanallar
2:00–3:15	Bir axın dərindən	Kampaniya yaratma: gateway filtrləri → JWT → `@Valid` → `@PreAuthorize` → tranzaksiya → partial unique indeks → tarixçə → Kafka → cache evict → yavaş sorğu jurnalı (fəsil 20.3)
3:15–4:15	Texniki qərarlar	RS256 və public key paylanması; refresh token hash-i; Resilience4j ilə audit; DLQ; virtual thread-lər; Testcontainers
4:15–5:00	Təkmilləşdirmələr	F01/F02 (təhlükəsizlik), outbox (F09/F10), `ErrorHandlingDeserializer` (F11), gRPC deadline və status xəritəsi (F19), `@Version` (F37)

Məsləhət: təkmilləşdirmələri "kod pisdir" kimi deyil, "istehsala keçid üçün növbəti addımlar" kimi təqdim edin və hər birinin səbəbini bir cümlə ilə izah edin.

24.4 Sual bankı

Java və Spring əsasları

1. Spring Boot auto-configuration necə işləyir və Boot 4-də nə dəyişdi? `AutoConfiguration.imports` faylındakı siniflər `@Conditional*` şərtləri ilə qiymətləndirilir. Boot 4-də auto-config texnologiya üzrə ayrıca modullara bölünüb (`spring-boot-kafka`, `spring-boot-jackson` və s.); starter-lər onları gətirir (fəsil 3.6, 4.5).

2. `@ConditionalOnClass(name = "...KafkaTemplate")` niyə sətir ilə yazılıb? Sinif classpath-da olmadıqda belə annotasiya oxuna bilsin deyə; `common` modulu Kafka-sız servislərdə də yüklənir.

3. Virtual thread-lər nəyi dəyişir? Bloklayan I/O ucuzlaşır: Tomcat hər sorğuya virtual thread verir (`spring.threads.virtual.enabled`). Darboğaz DB hovuzu kimi məhdud resurslara keçir; `synchronized` daxilində bloklanma (pinning) JDK 24+-da əsasən aradan qalxıb.

4. Spring proxy-ləri və self-invocation problemi? `@Transactional`, `@Cacheable`, `@Async`, `@PreAuthorize` proxy ilə işləyir; eyni sinif daxilində `this.method()` çağırışı proxy-dən keçmir və annotasiya işləmir.

5. `@Async` metodunun nəticəsi və istisnası haraya gedir? `void` metodda istisna `AsyncUncaughtExceptionHandler`-ə gedir; layihədə `publishAudit` bütün istisnaları özü tutur. Trace konteksti `TaskDecorator` olmadan ötürülmür (F34).

6. Constructor injection niyə üstündür? Asılılıqlar açıqdır, `final` sahələr, testdə `new` ilə yaratmaq, dövri asılılıq erkən aşkarlanır. Layihədə hamısı constructor injection-dır; istisna — `@GrpcClient` sahə inyeksiyası.

REST, validasiya, xətalar

7. 400, 401, 403, 404, 409, 422, 500 fərqləri və layihədəki istifadəsi? 400 validasiya; 401 autentifikasiya yoxdur/etibarsız; 403 rol çatmır; 404 resurs yoxdur; 409 dublikat və səhv status keçidi; 500 gözlənilməz xəta. Layihə catch-all səbəbindən bəzi 400/404/405 hallarını 500 qaytarır (F07).

8. `@Valid` və `@Validated` fərqi? `@Valid` (Jakarta) kaskad validasiya; `@Validated` (Spring) qrupları dəstəkləyir və sinif səviyyəsində metod parametrlərinin validasiyasını aktivləşdirir.

9. Niyə DTO, domen və entity ayrılır? API kontraktı, biznes modeli və persistensiya modeli fərqli səbəblərdən dəyişir. Entity-ni API-yə çıxarmaq lazy-loading, sızma və kontrakt kövrəkliyi yaradır (fəsil 5.6).

10. Səhifələmə cavabını necə qurmaq lazımdır? Öz DTO-nuz (`content`, `page`, `size`, `totalElements`) — `CampaignPageResponse` kimi; Spring Data `Page`-i birbaşa serializasiya etmək sabit olmayan formatdır (F26).

Persistensiya və tranzaksiyalar

11. `save()` və `saveAndFlush()` fərqi? Layihədə niyə `saveAndFlush`? `saveAndFlush` SQL-i dərhal icra edir; unique pozuntusu metod daxilində `DataIntegrityViolationException` kimi tutulub 409-a çevrilir. `save` ilə xəta commit zamanı, servisdən kənarda yaranardı (F18 məhz budur).

12. Təyin olunmuş UUID id ilə `save()` niyə SELECT edir? `isNew()` id `null` olmadığı üçün `false` qaytarır, Spring Data `merge` çağırır; Hibernate əvvəlcə SELECT edir (F14). Həll: `Persistable` və ya `@Version`.

13. Dirty checking nədir? Persistence context entity-nin ilkin vəziyyətini saxlayır; flush zamanı fərqləri UPDATE edir. `changeStatus`-dan sonra `save` əslində lazım deyil.

14. Optimistik və pessimistik kilid? Optimistik — `@Version`, konflikt commit-də aşkarlanır (409); pessimistik — `SELECT ... FOR UPDATE`. Layihədə heç biri yoxdur (F37, F17).

15. Partial unique indeks nəyi həll edir? `WHERE status <> 'DUPLICATE'` ilə yalnız "aktiv" sətirlər arasında unikallıq; tətbiq yoxlaması ilə birlikdə yarış vəziyyətində son müdafiə.

16. `open-in-view: false` niyə vacibdir? Sessiya controller/serializasiya zamanı açıq qalmır; lazy yükləmə xidmət qatında açıq edilməlidir, bağlantı tez qaytarılır.

17. Liquibase-də changeset dəyişdirilsə nə olur? Checksum uyğunsuzluğu, tətbiq başlamır. Qayda: tətbiq olunmuş changeset dəyişdirilmir, yenisi yazılır.

Təhlükəsizlik

18. JWT RS256 ilə HS256 fərqi və layihənin seçimi? RS256 asimmetrikdir: yalnız auth-service private key saxlayır, digərləri public key ilə yoxlayır. HS256-da hər yoxlayıcı token buraxa bilərdi.

19. Gateway JWT-ni yoxlayırsa, servislər niyə yenidən yoxlayır? Defense in depth: servis portları birbaşa əlçatan ola bilər (Compose host-a açır); rollar yalnız servisdə yoxlanır.

20. Refresh token niyə hash kimi saxlanır? DB sızması aktiv token-ləri açmasın; SHA-256 təsadüfi yüksək entropiyalı token üçün kifayətdir (parol kimi yavaş hash lazım deyil).

21. Access token-i necə ləğv etmək olar? Qısa ömür, `jti` deny-list, istifadəçi üzrə token versiyası. Layihədə yoxdur (F35).

22. Layihədəki ən ciddi təhlükəsizlik boşluqları? F01 (özünü admin kimi qeydiyyat), F02 (reset token cavabda), F04 (autentifikasiyasız gRPC), F05 (XFF ilə rate limit yan keçmə), F03 (enumerasiya).

23. Servislərarası autentifikasiya necə qurulub? auth-service hər audit sorğusuna `sub=auth-service`, `ROLE_INTERNAL_SERVICE` olan qısa ömürlü JWT əlavə edir; log-service `hasRole('INTERNAL_SERVICE')` tələb edir. Alternativlər: mTLS, OAuth2 client credentials.

24. CSRF niyə söndürülüb? Stateless API, cookie əsaslı autentifikasiya yoxdur; token `Authorization` başlığındadır, brauzer onu avtomatik göndərmir.

Kafka

25. At-least-once, at-most-once, exactly-once? Consumer offseti emaldan sonra commit edirsə — at-least-once (dublikat mümkün); əvvəl — at-most-once. Exactly-once Kafka daxilində tranzaksiyalarla; xarici sistemlə idempotent emal lazımdır.

26. Açar seçimi nəyə təsir edir? Partisiya və sıra. `campaign-events` açarı kampaniya id-sidir — bir kampaniyanın hadisələri sıralıdır; `log-events` açarı URI-dir.

27. DLQ necə işləyir? `DefaultErrorHandler` 1 s fasilə ilə 3 təkrar, sonra `DeadLetterPublishingRecoverer` qeydi `log-events.DLQ`-a eyni partisiya nömrəsinə, xəta başlıqları ilə göndərir.

28. Poison pill nədir və layihədə niyə təhlükəlidir? Emal oluna bilməyən mesaj. `ErrorHandlingDeserializer` olmadığı üçün deserializasiya xətası DLQ-ya getmir, eyni offset sonsuz təkrarlanır (F11).

29. Dual write problemi və həlli? DB commit və Kafka göndərişi atomik deyil. Transactional outbox və ya CDC (F09).

30. `max.block.ms` nədir? `send()`-in metadata və buffer üçün çağıran thread-i bloklaya biləcəyi maksimum vaxt (standart 60 s). Kafka düşəndə campaign-service və log-service-in HTTP sorğuları bu qədər gecikir (F10).

31. Consumer group rebalance nədir? Qrupa üzv qoşulanda/çıxanda partisiyaların yenidən paylanması; zamanı emal dayanır. Partisiya sayı consumer paralelliyinin yuxarı həddidir (layihədə 1).

Elasticsearch

32. Niyə audit və yavaş sorğular üçün ES? Tam mətn axtarışı, vaxt seriyası sorğuları, aqreqasiyalar; append-only məlumat. Əsas biznes məlumatı üçün deyil.

33. `text` və `keyword` fərqi? `text` analiz olunur (tokenlər), `keyword` dəqiq dəyər (filtr, sıralama, aqreqasiya). Dinamik mapping hər sətir üçün ikisini də yaradır.

34. ILM nədir və niyə lazımdır? Index lifecycle management: rollover, köhnə indekslərin silinməsi. Olmasa, log indeksləri limitsiz böyüyür (F43).

35. ES-də sıralamasız `findAll` nə qaytarır? Daxili sıra ilə ixtiyari nəticələr; ən son qeydlər üçün `occurredAt desc` sıralaması lazımdır (F42).

gRPC

36. gRPC-nin REST-dən üstünlükləri və çatışmazlıqları? Kontrakt (proto), binar format, HTTP/2, streaming, kod generasiyası. Çatışmazlıq: brauzer dəstəyi, debug çətinliyi, daha mürəkkəb infrastruktur. Layihədə bir sadə çağırış üçün istifadə olunur — nümayiş məqsədli.

37. Deadline niyə vacibdir? Sinxron zəncirdə gözləmə vaxtını məhdudlaşdırır və ləğvi serverə ötürür. Layihədə yoxdur (F19).

38. gRPC status kodlarını necə xəritələmək lazımdır? Serverdə `@GrpcAdvice` (domen istisnası → `NOT_FOUND`, `INVALID_ARGUMENT`), klientdə status → HTTP. Layihədə `UNKNOWN` → 500 olur.

39. Proto-nu necə təkamül etdirmək olar? Sahə nömrələrini dəyişməmək və təkrar istifadə etməmək, silinən sahələri `reserved` etmək, yalnız yeni sahələr əlavə etmək.

Redis, cache, rate limit

40. `@Cacheable` / `@CacheEvict` və tranzaksiya sırası? İki interceptor; sıra açıq deyilsə eviction commit-dən əvvəl ola bilər və köhnə dəyər yenidən cache-ə yazılar (F13). Həll: transaction-aware cache, TTL.

41. Token bucket alqoritmi? Vedrə sabit sürətlə dolur, hər sorğu token xərcləyir; burst vedrə tutumu qədərdir. SCG bunu Redis Lua skripti ilə atomik edir.

42. Rate limiter niyə fail-open-dur və bu düzgündürmü? Redis xətasında sorğunu buraxır — əlçatanlıq üçün düzgün standartdır; amma alert olmalıdır (F06).

43. Rate limit açarı üçün IP-yə etibar etmək təhlükəsizdirmi? Yalnız etibarlı proxy-nin yazdığı başlığa; klient `X-Forwarded-For`-u özü yaza bilər (F05).

Spring Cloud

44. Eureka və client-side load balancing? Servislər qeydiyyatdan keçir, gateway reyestri keşləyir və `lb://` üçün Spring Cloud LoadBalancer instans seçir. Layihədə daxili çağırışlar static ünvanla gedir (F31).

45. Config Server `optional:` prefiksi nə edir? Server əlçatmaz olsa da servis başlayır — əlçatanlıq üçün yaxşı, lakin paylanan xassələr səssizcə itir (F32).

46. Gateway-də GlobalFilter və GatewayFilter fərqi? GlobalFilter bütün marşrutlara tətbiq olunur; GatewayFilter marşruta (və ya default-filters ilə hamısına) bağlıdır. Hamısı birlikdə `order`-ə görə sıralanır.

Dayanıqlıq

47. Retry + Circuit Breaker birlikdə necə işləyir? Resilience4j aspekt sırasında Retry xaricdədir: hər cəhd CB-dən keçir və nəticəsi CB pəncərəsinə yazılır; CB açılanda `CallNotPermittedException` retry olunmur (konfiqurasiyada ignore).

48. Bulkhead nə üçündür? Bir asılılığın bütün thread-ləri/icazələri tutmasının qarşısını alır. Semafor bulkhead 10 icazə, gözləmə 0 — artıq çağırış dərhal rədd olunur.

49. Hansı əməliyyatları retry etmək təhlükəsizdir? İdempotent olanları. Audit POST idempotent deyil — timeout-dan sonra retry dublikat yarada bilər.

Müşahidə

50. Metrik, trace və log nə vaxt? Metrik — trend və alert (nə qədər?); trace — bir sorğunun yolu (harada?); log — detallar (nə?). traceId onları birləşdirir.

51. Kardinallıq problemi? Hər unikal label kombinasiyası ayrıca seriyadır; `trace_id` label-ı Loki-də indeksi partladır (F24).

52. Trace kontekstini Kafka sərhədindən necə keçirmək olar? Producer və consumer observation (`traceparent` başlığı). Layihədə yoxdur; traceId payload-dadır.

Test və CI/CD

53. Testcontainers niyə H2-dən yaxşıdır? Real PostgreSQL davranışı: partial indeks, tiplər, SQL dialekti. Amma layihədə testlər Liquibase-i söndürür (F27).

54. `@MockitoBean` nədir? Boot 3.4+ / Framework 6.2+ mexanizmi (köhnə `@MockBean`-in yerinə); kontekstdəki bean-i mock ilə əvəz edir. Fərqli mock dəstləri kontekst cache-ni parçalayır.

55. Surefire və Failsafe fərqi? Surefire `test` fazasında unit testlər, Failsafe `integration-test` / `verify` fazasında `*IT`. Layihədə inteqrasiya testləri də `*Test` adlanır.

56. CI pipeline-da nə olmalıdır? Build + test, coverage, statik analiz (Sonar), image build, zəiflik skanı, artefaktın versiyalanması, deploy mərhələləri.

Dizayn və davranış

57. Bu layihəni yenidən qursaydınız nəyi dəyişərdiniz? Fəsil 22: IdP + resource server, outbox, hadisələrdən oxu modeli, `ProblemDetail`, `@Version`, timeout/deadline, OTel-əsaslı müşahidə, testlərdə miqrasiya.

58. Mikroservis bu ölçüdə əsaslıdırmı? Texniki nümayiş üçün bəli; istehsal üçün modul-monolit daha sadə olardı (fəsil 22.2).

59. Kodda tapdığınız problemi komandaya necə çatdırardınız? Reproduksiya ssenarisi, təsir, təklif olunan düzəliş və test; ciddiliyə görə prioritet; şəxsi deyil, kod haqqında.

60. Yeni komandaya qoşulanda kod bazasını necə öyrənirsiniz? Build və modul qrafı → giriş nöqtələri (controller, listener) → bir axını uçdan-uca izləmək → testlər → konfiqurasiya və yerləşdirmə → müşahidə alətləri. Bu kitabın quruluşu da belədir.

24.5 Bu repozitoriya haqqında çətin suallar

Sual	Dürüst cavab
"Niyə bir çağırış üçün gRPC?"	Texnikanı nümayiş etdirmək üçün; real ehtiyac yoxdur, REST eyni işi görərdi. İstehsalda hadisələrdən qurulan oxu modeli daha uyğundur.
"`campaign-events`-i kim oxuyur?"	Heç kim. Hadisə gələcək inteqrasiya üçün dərc olunur, lakin dual-write riskini daşıyır.
"Testlər niyə Liquibase-i söndürür?"	Sürət və sadəlik üçün; nəticədə partial indeks və sxem validasiyası test olunmur. Düzəltmək lazımdır.
"5 ms həddi niyə?"	Demo-da pipeline görünsün deyə; istehsalda mənasızdır və log həcmini partladır.

Sual	Dürüst cavab
"Kafka düşsə nə olar?"	campaign-service və log-service-in REST sorğuları `max.block.ms` səbəbindən 60 s-ə qədər gecikə bilər (yavaş sorğu filtri yalnız bu iki serviste aktivdir) — ən böyük əlçatanlıq riski.
"Admin necə yaranır?"	Qeydiyyatda rol klientdən gəlir — bu, kritik boşluqdur; düzgün yol seed və ya admin endpoint-idir.
"Bu kodu işə salıb yoxlamısınız?"	Bu kitab statik təhlilə əsaslanır (mühitdə Java 25 yox idi); runtime iddiaları **NƏTİCƏ** kimi işarələnib. Müsahibədə də fakt və fərziyyəni ayırın.

24.6 Xülasə

- 1 dəqiqəlik təqdimat: məqsəd, servislər, texnologiyalar, bir-iki təkmilləşdirmə.
- 5 dəqiqəlik izah: biznes → sxem → bir axın dərindən → qərarlar → təkmilləşdirmələr.
- Sual bankı fəsillərə istinad edir; dərin cavab üçün həmin fəslə qayıdın.
- Çətin suallarda dürüstlük və trade-off-u izah etmək bacarığı faktların özündən daha çox dəyər qazanır.

25. Əlavələr

25.1 Terminologiya lüğəti və abbreviaturalar

Termin	İzah	Fəsil
ACID	Atomiklik, ardıcıllıq, izolyasiya, davamlılıq — relational tranzaksiya zəmanətləri	7
ADR	Architecture Decision Record — qərarın konteksti, variantları və nəticəsi	21, 22
Aggregate (aqreqat)	Birlikdə dəyişən və bir tranzaksiya sərhədi olan domen obyektləri qrupu	22
At-least-once	Mesaj ən azı bir dəfə çatdırılır; dublikat mümkündür	11
Auto-configuration	Classpath və şərtlərə görə Spring Boot-un avtomatik yaratdığı bean-lər	3, 4
BFF	Backend for Frontend — klientə uyğunlaşdırılmış nazik aqreqasiya servisi (dashboard)	13, 20
BOM	Bill of Materials — Maven-də versiyaları mərkəzləşdirən `dependencyManagement` idxalı	3
Bulkhead	Bir asılılığa eyni vaxtda gedən çağırışları məhdudlaşdıran izolyasiya	14
Burst capacity	Token bucket-in maksimum tutumu	15
Cardinality (kardinallıq)	Metrik/log label dəyərlərinin unikal kombinasiyalarının sayı	17
CDC	Change Data Capture — DB jurnalından hadisə çıxarmaq (Debezium)	22
Circuit breaker	Uğursuzluq nisbəti həddi aşanda çağırışları müvəqqəti dayandıran mexanizm	14
Consumer group	Partisiyaları öz aralarında bölüşən Kafka consumer-ləri	11
CQRS	Yazı və oxu modellərinin ayrılması	22
Deadline	gRPC çağırışının mütləq bitmə vaxtı; serverə ötürülür	13
Dirty checking	Hibernate-in idarə olunan entity dəyişikliklərini flush-da avtomatik UPDATE etməsi	7
DLQ	Dead Letter Queue/Topic — emal oluna bilməyən mesajlar üçün ayrıca mövzu	11
Dual write	İki sistemə (DB və broker) atomik olmayan yazı	11, 21
Eventual consistency	Məlumatın bir müddət sonra uyğunlaşması	22
Fail-open / fail-closed	Asılılıq xətasında sorğunu buraxmaq / rədd etmək	15

Termin	İzah	Fəsil
Idempotency	Təkrar icranın nəticəni dəyişməməsi	11, 14
ILM	Index Lifecycle Management (Elasticsearch)	12
ISR	In-Sync Replicas — leader ilə sinxron Kafka replikaları	11
JWKS	JSON Web Key Set — public açarların paylanması üçün endpoint	10, 22
JWT	JSON Web Token — imzalanmış claim-lər dəsti	10
KRaft	ZooKeeper-siz Kafka metadata rejimi	11, 18
Lost update	Paralel yeniləmədə birinin dəyişikliyinin itməsi	7, 21
Mapping (ES)	Elasticsearch indeksinin sahə tipləri sxemi	12
MDC	Mapped Diagnostic Context — log sətirlərinə `traceId` kimi kontekst əlavə edən mexanizm	17
mTLS	Hər iki tərəfin sertifikatla autentifikasiyası	10
Observation	Micrometer-in metrik və trace yaradan vahid API-si	17
Offset	Kafka partisiyasında qeydin ardıcıl nömrəsi	11
OIDC	OpenID Connect — OAuth2 üzərində identifikasiya protokolu	22
Outbox	Hadisəni DB tranzaksiyasında cədvələ yazıb sonra broker-ə ötürmə nümunəsi	11, 22
Partial index	`WHERE` şərtli indeks (PostgreSQL)	7
Persistence context	Hibernate sessiyasında idarə olunan entity-lərin birinci səviyyə cache-i	7
Poison pill	Consumer-i sonsuz xəta döngüsünə salan mesaj	11
ProblemDetail	RFC 9457 xəta formatı; Spring-in dəstəklədiyi standart	8, 22
Proxy (Spring AOP)	Annotasiya əsaslı davranışı (tranzaksiya, cache) həyata keçirən əhatəedici obyekt	4
RED metrikləri	Rate, Errors, Duration — servis səviyyəli əsas siqnallar	17, 23
Resource server	Token-i yoxlayan, lakin buraxmayan servis (OAuth2 rolu)	10, 22
RS256	RSA + SHA-256 asimmetrik JWT imzası	10
Shared kernel	Bir neçə servisin paylaşdığı ortaq kod modulu	21
Span / trace	Paylanmış izləmədə əməliyyat vahidi / onların ağacı	17
Testcontainers	Testlərdə real asılılıqları Docker konteynerində işə salan kitabxana	19

Termin	İzah	Fəsil
Token bucket	Sabit sürətlə dolan vedrə əsaslı rate limit alqoritmi	15
Virtual thread	JVM tərəfindən idarə olunan yüngül thread (Java 21+)	4

25.2 Endpoint kataloqu

REST (gateway `:8080` üzərindən)

Metod	Yol	Servis	Rol	Gövdə / parametrlər	Uğur	Əsas xətalar
POST	`/api/v1/auth/register`	auth	Açıq	`username` 3–100, `email`, `password` 8–100, `fullName` 2–150, `role`	200	400, 409 (mövcuddur), 500 (yarış, F18)
POST	`/api/v1/auth/login`	auth	Açıq	`usernameOrEmail`, `password`	200	400, 401
POST	`/api/v1/auth/refresh-token`	auth	Açıq	`refreshToken`	200	400, 401
POST	`/api/v1/auth/forgot-password`	auth	Açıq	`usernameOrEmail`	200 (token cavabda — F02)	404 (F03), 401
POST	`/api/v1/auth/reset-password`	auth	Açıq	`resetToken`, `newPassword` 8–100	200	400, 401
POST	`/api/v1/auth/logout`	auth	Açıq	`refreshToken`	200	400, 401
POST	`/api/v1/campaigns`	campaign	ADMIN	`title` 10–50, `description` 20–200, `category`	201	400, 403, 409
GET	`/api/v1/campaigns/{id}`	campaign	USER, ADMIN	—	200	404, 500 (səhv UUID — F07)
GET	`/api/v1/campaigns`	campaign	USER, ADMIN	`page`, `size` (10), `sort` (`createdAt,desc`)	200	500 (səhv `sort` — F07)
PATCH	`/api/v1/campaigns/{id}/approve`	campaign	ADMIN	—	200	404, 409
PATCH	`/api/v1/campaigns/{id}/deactivate`	campaign	ADMIN	—	200	404, 409
GET	`/api/v1/campaigns/{id}/history`	campaign	ADMIN	—	200	404
GET	`/api/v1/campaigns/statistics`	campaign	ADMIN	—	200	500 (Redis)

Metod	Yol	Servis	Rol	Gövdə / parametrlər	Uğur	Əsas xətalar
GET	`/api/v1/dashboard/classifieds/statistics`	dashboard	ADMIN	—	200	500 (gRPC — F19)
GET	`/api/v1/dashboard/campaigns/{campaignId}/history`	dashboard	ADMIN	—	200	500 (yoxdursa da — F19)
GET	`/api/v1/logs/slow-requests`	log	ADMIN	`page` (0), `size` (20)	200	500 (mənfi `page` — F39)
POST	`/api/v1/logs/auth-audit`	log	INTERNAL_SERVICE	`AuthAuditLogRequest` (validasiyasız — F39)	200	401, 403

Mənbələr: `AuthController.java:47-117`, `CampaignController.java:42-92`, `DashboardController.java:39-57`, `SlowRequestLogController.java:35-43`, `AuthAuditLogController.java:29-35`. Gateway səviyyəsində əlavə olaraq: 401 (boş gövdə, token yoxdur/etibarsız), 429 (rate limit), 503 (instans yoxdur).

Sənədləşmə və əməliyyat endpoint-ləri

Yol	Harada	Qeyd
`/swagger-ui.html`	gateway	Dörd servisin sənədi seçilə bilir
`/openapi/{auth,campaign,dashboard,logs}`	gateway	Aqreqasiya + fallback (`GatewayOpenApiController.java:87-105`)
`/{auth,campaign,dashboard,logs}/v3/api-docs`	gateway	`StripPrefix=1` ilə birbaşa ötürmə
`/v3/api-docs`, `/swagger-ui.html`	hər servis	springdoc
`/actuator/{health,info,prometheus,metrics}`	hər servis	Autentifikasiyasız (F40)

gRPC (`campaign-service:9090`, plaintext)

RPC	Sorğu	Cavab	İstifadəçi
`GetCampaignStatistics`	boş	`repeated CampaignStatusCount {status, count}`	dashboard
`GetCampaignHistory`	`campaign_id` (sətir UUID)	`repeated CampaignHistoryItem {previous_status, new_status, reason, changed_at}`	dashboard

Kontrakt: `grpc-contract/src/main/proto/campaign.proto:8-9`.

Kafka mövzuları və Elasticsearch indeksləri

Ad	Növ	Yazan	Oxuyan	Açar / qeyd
`log-events`	Kafka	campaign-service, log-service (yavaş sorğu filtri)	log-service (`log-service`)	URI
`log-events.DLQ`	Kafka	`DeadLetterPublishingRecoverer`	log-service (`log-service-dlq`) — yalnız log	Orijinal açar
`campaign-events`	Kafka	campaign-service	—	Kampaniya id-si
`slow-request-logs`	ES indeksi	log-service consumer	`GET /api/v1/logs/slow-requests`	Yalnız `occurredAt` açıq mapping
`auth-audit-logs`	ES indeksi	`POST /api/v1/logs/auth-audit`	Repozitoriyada oxuyan API yoxdur	`AuthAuditLogDocument`

25.3 Konfiqurasiya və mühit dəyişənləri arayışı

Aşağıdakı cədvəl `*/src/main/resources/**/*.yml` fayllarındakı `${DƏYİŞƏN:standart}` yer tutucularından avtomatik çıxarılıb. Şifrələrin standart dəyərləri **redaktə olunub** — onlar repozitoriyada mövcuddur (F22), lakin kitabda təkrar edilmir.

Dəyişən	Servislər	Standart dəyər
`CONFIG_SERVER_URL`	gateway, auth, campaign, dashboard, log	`http://localhost:8888`
`EUREKA_SERVER_URL`	gateway, auth, campaign, config, dashboard, log	`http://localhost:8761/eureka`
`EUREKA_INSTANCE_IP_ADDRESS`	eyni	`127.0.0.1`
`REDIS_HOST` / `REDIS_PORT`	gateway, campaign	`localhost` / `6379`
`REDIS_TIMEOUT`	gateway	`2s`
`RATE_LIMITER_REPLENISH_RATE` / `_BURST_CAPACITY` / `_REQUESTED_TOKENS`	gateway	`1` / `5` / `5`
`AUTH_SERVICE_URI` , `CAMPAIGN_SERVICE_URI` , `DASHBOARD_SERVICE_URI` , `LOG_SERVICE_URI`	gateway	`lb://<servis>`
`*_SERVICE_OPENAPI_URL` (4 ədəd)	gateway	`lb://<servis>/v3/api-docs`
`JWT_PUBLIC_KEY_LOCATION`	gateway, auth, campaign, dashboard, log (+ config-server-in payladığı konfiqurasiya)	`classpath:keys/public.pem`
`JWT_PRIVATE_KEY_LOCATION`	auth	`classpath:keys/private.pem`
`AUTH_DB_URL` / `AUTH_DB_USERNAME` / `AUTH_DB_PASSWORD`	auth	`jdbc:postgresql://localhost:5433/auth_db` / `auth_user` / *redaktə*
`CAMPAIGN_DB_URL` / `CAMPAIGN_DB_USERNAME` / `CAMPAIGN_DB_PASSWORD`	campaign	`jdbc:postgresql://localhost:5434/campaign_db` / `campaign_user` / *redaktə*
`LOG_SERVICE_URL`	auth	`http://localhost:8084`
`LOG_SERVICE_CB_*` (5 ədəd)	auth	50 / 10 / 5 / 3 / `30s`

Dəyişən	Servislər	Standart dəyər
`LOG_SERVICE_RETRY_MAX_ATTEMPTS` / `_WAIT_DURATION`	auth	`3` / `500ms`
`LOG_SERVICE_BULKHEAD_MAX_CONCURRENT_CALLS` / `_MAX_WAIT_DURATION`	auth	`10` / `0ms`
`KAFKA_BOOTSTRAP_SERVERS`	campaign, log (+ config-server)	`localhost:9092`
`CAMPAIGN_GRPC_PORT`	campaign	`9090`
`CAMPAIGN_GRPC_ADDRESS`	dashboard	`static://localhost:9090`
`ELASTICSEARCH_URIS`	log	`http://localhost:9200`
`SPRING_ELASTICSEARCH_CONNECTION_TIMEOUT` / `_SOCKET_TIMEOUT`	log	`10s` / `30s`
`MANAGEMENT_TRACING_SAMPLING_PROBABILITY`	bütün 7 tətbiq	`1.0`
`MANAGEMENT_TRACING_EXPORT_OTLP_ENABLED`	bütün 7 tətbiq	`true`
`OTEL_SERVICE_NAME`	bütün 7 tətbiq	`${spring.application.name}`
`OTEL_EXPORTER_OTLP_TRACES_ENDPOINT` / `_PROTOCOL`	bütün 7 tətbiq	`http://localhost:4317` / `grpc`
`LOGGING_FILE_NAME`	bütün 7 tətbiq	`logs/<servis>.log`

Yer tutucu olmayan, lakin vacib sabit xassələr: `app.slow-request-threshold-ms: 5`, `app.kafka.topics.*`, `app.jwt.expiration-minutes: 120`, `app.auth.refresh-token-expiration-days: 7`, `app.auth.password-reset-token-expiration-minutes: 15` (`auth-service/src/main/resources/application.yml:73-78`). Compose əlavə olaraq `APP_OPENAPI_*_URL` dəyişənlərini təyin edir (`docker-compose.yml:66-69`); Spring-in relaxed binding qaydası ilə onlar birbaşa `app.openapi.*-url` xassələrinə bağlanır və yer tutucudakı dəyişəndən üstündür **NƏTİCƏ**.

25.4 Fəsillərdə ayrıca müzakirə olunmayan fayllar

Əhatə matrisi (fəsil 26) hər faylın hansı fəsildə istinad olunduğunu göstərir. Aşağıdakı fayllar sadə və ya təkrarlanan olduğu üçün fəsillərdə ayrıca təhlil edilməyib; onların məzmunu burada qeyd olunur.

Fayl(lar)	Məzmun	Qeyd
`api-gateway/Dockerfile`, `auth-service/Dockerfile`, `config-server/Dockerfile`, `dashboard-service/Dockerfile`, `discovery-server/Dockerfile`, `log-service/Dockerfile`	Eyni iki mərhələli şablon: `maven:3.9.11-amazoncorretto-25` ilə `mvn -pl <modul> -am clean package -DskipTests`, sonra `amazoncorretto:25` üzərində `java --enable-preview -jar`. Fərq yalnız modul adı və `EXPOSE` portudur	Şablon fəsil 3.11 və 18-də campaign-service nümunəsində izah olunub; JRE əvəzinə tam JDK image, root istifadəçi, layer cache yoxdur
`api-gateway/src/main/java/com/casestudy/insurancecampaign/gateway/ApiGatewayApplication.java`, `dashboard-service/src/main/java/com/casestudy/insurancecampaign/dashboard/DashboardServiceApplication.java`, `log-service/src/main/java/com/casestudy/insurancecampaign/log/LogServiceApplication.java`	`@SpringBootApplication(scanBasePackages = "com.casestudy.insurancecampaign")` + `@EnableDiscoveryClient`; log-service-də əlavə `@EnableKafka`	Geniş skan `common` və `security` bean-lərini daxil edir (fəsil 4.4)

Fayl(lar)	Məzmun	Qeyd
`api-gateway/src/main/java/com/casestudy/insurancecampaign/gateway/config/GatewayHttpServerObservationConfiguration.java`, `config-server/src/main/java/com/casestudy/insurancecampaign/config/observability/ConfigServerHttpServerObservationConfiguration.java`, `discovery-server/src/main/java/com/casestudy/insurancecampaign/discovery/observability/DiscoveryServerHttpServerObservationConfiguration.java`	`ObservationPredicate` actuator sorğularını trace-dən çıxarır, `ObservationFilter` span adlarını `http get /yol` formatına salır	config və discovery versiyaları paket/sinif adından başqa eynidir; gateway versiyası reaktiv kontekstə uyğunlaşdırılıb (F23)
`auth-service/src/main/java/com/casestudy/insurancecampaign/auth/model/enums/UserStatus.java`	`ACTIVE`, `PASSIVE`, `DELETED`	Kod yalnız `ACTIVE` təyin edir; statusu dəyişən endpoint yoxdur — "User is not active" yolu yalnız DB-də əl ilə dəyişiklikdən sonra mümkündür
`auth-service/src/main/java/com/casestudy/insurancecampaign/auth/repository/AppUserRepository.java`	`findByUsername`, `findByEmail`, `existsByUsername`, `existsByEmail`	Törədilmiş sorğular (fəsil 7.9)
`auth-service/src/main/java/com/casestudy/insurancecampaign/auth/repository/PasswordResetTokenRepository.java`	`findByTokenHash`	Hash üzrə axtarış; unikal indeksə əsaslanır
`auth-service/src/main/resources/keys/public.pem`	auth-service-in öz public açarı (PEM)	Məzmunu kitabda təkrar edilmir
`campaign-service/src/test/resources/keys/private.pem`, `campaign-service/src/test/resources/keys/public.pem`, `dashboard-service/src/test/resources/keys/private.pem`, `dashboard-service/src/test/resources/keys/public.pem`, `log-service/src/test/resources/keys/private.pem`, `log-service/src/test/resources/keys/public.pem`	Testlərdə real RS256 token yaratmaq üçün açar cütləri	Yalnız test üçündür; məzmunu təkrar edilmir. İstehsal açarı ilə eyni olmamalıdır
`dashboard-service/pom.xml`	`common`, `security`, `grpc-contract`, gRPC kitabxanaları, `grpc-client-spring-boot-starter`, web, Eureka, actuator, OTel, springdoc; testdə `postgresql`	Kafka yoxdur → yavaş sorğu filtri aktiv deyil
`discovery-server/pom.xml`	`spring-cloud-starter-netflix-eureka-server`, actuator, Prometheus, OTel bridge + OTLP exporter	`common`-dan asılı deyil
`log-service/pom.xml`	`common`, `security`, web, `spring-kafka`, `spring-boot-kafka`, `spring-boot-starter-data-elasticsearch`, Eureka, actuator, OTel, springdoc; testdə Testcontainers `kafka`, `elasticsearch`	Fəsil 11, 12

Fayl(lar)	Məzmun	Qeyd
`postman/Insurance Campaign Microservices.postman_collection.json`	Postman v2.1 kolleksiyası: "Authentication" qovluğunda 10 sorğu, kökdə 16 sorğu (kampaniya, dashboard, slow-requests); dəyişənlər: `gatewayUrl`, servis URL-ləri, `token`, `adminToken`, `userToken`, `campaignId` və s.; bəzi sorğularda token-i dəyişənə yazan test skriptləri	Bəzi sorğular gateway-i keçərək birbaşa servis portlarına gedir (`authServiceUrl`, `logServiceUrl`) — bu, F40 və 10.8-dəki "gateway-dən yan keçmə" ssenarisinin praktiki nümunəsidir
`.gitignore`	`target/`, IDE faylları, `docker-data/`, `logs/`, `*.log`, `.env`	`keys/*.pem` istisna edilmir — açarlar commit olunub (F22)

25.5 Rəsmi sənədlər və əlavə oxu

Mövzu	Mənbə
Spring Boot	docs.spring.io/spring-boot (reference, "Upgrading to 4.0" qeydləri)
Spring Framework	docs.spring.io/spring-framework (HTTP Interface, transaction, AOP)
Spring Security	docs.spring.io/spring-security (servlet architecture, resource server)
Spring Cloud Gateway	docs.spring.io/spring-cloud-gateway (Server WebFlux, `RequestRateLimiter`)
Spring for Apache Kafka	docs.spring.io/spring-kafka (error handling, DLT, observation)
Spring Data Elasticsearch / JPA / Redis	docs.spring.io/spring-data/...
Apache Kafka	kafka.apache.org/documentation (producer/consumer configs)
Elasticsearch	elastic.co/guide (mapping, ILM)
gRPC Java	grpc.io/docs/languages/java; net.devh grpc-spring-boot-starter sənədləri
Resilience4j	resilience4j.readme.io
Liquibase	docs.liquibase.com
OpenTelemetry	opentelemetry.io/docs
Testcontainers	java.testcontainers.org
Transactional outbox, saga	microservices.io/patterns
Kitablar	Martin Kleppmann — *Designing Data-Intensive Applications*; Sam Newman — *Building Microservices*; Michael Nygard — *Release It!*

25.6 Audit xülasəsi

Kitabı necə yoxladığımı `audit/audit-report.md`-də ətraflı təsvir etmişəm. Qısaca:

- **Audit A (məzmun tamlığı):** tapşırıqda tələb olunan bütün bölmələrin (texnologiya fəsilləri, iş axınları, dizayn fəsli, istehsal mühəndisliyi, qiymətləndirmə, əhatə matrisi, diaqramlar, müsahibə hazırlığı) mövcudluğu yoxlanılıb.
- **Audit B (texniki dəqiqlik):** `build/verify_refs.py` bütün `fayl:sətir` istinadlarını (faylın mövcudluğu, sətir aralığı) və bütün kod çıxarışlarını (sətirlərin mənbədə ardıcıl olaraq mövcudluğu) avtomatik yoxlayır; nəticə `audit/verify-refs.json`-dadır. Semantik yoxlama (istinad olunan sətirlərin iddianı

həqiqətən dəstəkləməsi) əl ilə, seçmə üsulu ilə aparılıb; tapılan səhvlər düzəldilib və hesabatda sadalanıb.
- **Audit C (PDF bütövlüyü):** mündəricatın səhifə nömrələri, şriftlər, diaqramların render-i, kod bloklarının və cədvəllərin kəsilməməsi səhifələrin PNG render-ləri ilə vizual yoxlanılıb.

Məhdudiyyətlər: layihə kompilyasiya edilməyib və işə salınmayıb (mühitdə Java 25 yox idi); runtime davranışı haqqında bütün iddialar NƏTİCƏ kimi işarələnib.

26. Əhatə matrisi və tapıntı indeksi

> Bu fəsil `build/coverage.py` tərəfindən avtomatik yaradılır; əl ilə redaktə etməyin. Tam (9 sütunlu) matris: `analysis/coverage-matrix.md`.

26.1 Necə oxumalı

- **Fəsillər** — faylın tam yolu və ya sinif/fayl adı ilə istinad olunduğu fəsillər (bu fəsil hesaba daxil deyil).
- **Yoxlama** — `OK`: həmin fayla edilən bütün `fayl:sətir` istinadları və kod çıxarışları `build/verify_refs.py` ilə avtomatik doğrulanıb; `–`: faylın sətir səviyyəli istinadı yoxdur (yalnız adla xatırlanır).
- Yollar qısaldılıb: `main:` = `src/main/java/com/casestudy/insurancecampaign/`, `test:` = eyni test qovluğu.

Cəmi izlənən fayl: **178**; ən azı bir fəsildə istinad olunan: **178**; sətir səviyyəsində avtomatik doğrulanan: **101**.

26.2 Modullar üzrə xülasə

Modul	Fayl	İstinad olunan	Sətir səviyyəsində doğrulanan
`(kök)`	5	5	3
`.github`	2	2	1
`api-gateway`	14	14	8
`auth-service`	45	45	24
`campaign-service`	28	28	23
`common`	15	15	10
`config-server`	6	6	3
`dashboard-service`	16	16	6
`discovery-server`	5	5	1
`grpc-contract`	2	2	2
`log-service`	23	23	10
`observability`	9	9	5
`postman`	1	1	0
`security`	7	7	5

26.3 Fayl səviyyəsində matris

(kök)

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`.gitignore`	Git istisnaları	25	—
`README.md`	Layihə sənədi	01, 02, 03, 05, 06, 08, 09, 10, 13, 15, 16, 17, 18, 22	—
`docker-compose-local.yml`	Yalnız infrastruktur (servislər IDE-dən)	01, 18	OK

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`docker-compose.yml`	Tam stek (7 tətbiq + infrastruktur)	01, 02, 04, 10, 11, 12, 13, 15, 16, 17, 18, 20, 21, 23, 25	OK
`pom.xml`	Parent POM, modul siyahısı, BOM-lar, plugin-lər	01, 02, 03, 13, 18	OK

.github

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`workflows/ci-cd.yml`	CI/CD: verify → sonar → docker	18	OK
`workflows/mark-done-on-merge.yml`	Merge olunan PR və bağlı issue-larda `done` label	18	—

api-gateway

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Image	25	—
`pom.xml`	Gateway asılılıqları	10, 16	OK
`main: gateway/ApiGatewayApplication.java`	Gateway giriş nöqtəsi	25	—
`main: gateway/config/GatewayHttpServerObservationConfiguration.java`	Observation (kopya)	25	—
`main: gateway/config/GatewayLoadBalancerConfiguration.java`	@LoadBalanced WebClient.Builder	16	OK
`main: gateway/config/GatewayRateLimiterConfiguration.java`	Rate limit açar resolver-i	10, 15, 20	OK
`main: gateway/filter/GatewayJwtAuthenticationFilter.java`	Gateway JWT yoxlaması	08, 09, 10, 15, 16, 20	OK
`main: gateway/filter/GatewayRateLimitErrorResponseFilter.java`	429 üçün JSON gövdə	03, 04, 05, 15, 16, 20	OK
`main: gateway/openapi/GatewayOpenApiController.java`	OpenAPI aqreqasiyası + fallback	14, 16, 20, 25	OK
`main: gateway/security/GatewayJwtProperties.java`	Gateway app.jwt.*	16	—
`main: gateway/security/GatewayJwtTokenService.java`	Gateway JWT doğrulama	03, 15, 16	—
`main: gateway/security/GatewayPemKeyUtils.java`	Public key oxuma	03, 21	OK
`src/main/resources/application.yml`	Marşrutlar, rate limit, springdoc	15, 16, 20, 23	OK
`src/main/resources/keys/public.pem`	RSA public key	10	—

auth-service

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Çoxmərhələli image build	25	—
`pom.xml`	Modul asılılıqları	03	OK
`main: auth/AuthServiceApplication.java`	AuthServiceApplication	04, 09, 14	OK
`main: auth/client/LogServiceHttpClient.java`	LogServiceHttpClient	04, 09, 14, 20, 21	OK
`main: auth/config/AuthConfiguration.java`	AuthConfiguration	04, 09	OK
`main: auth/config/LogServiceHttpClientConfiguration.java`	LogServiceHttpClientConfiguration	02, 04, 10, 14, 20, 21	OK
`main: auth/controller/AuthController.java`	AuthController	02, 05, 08, 20, 25	OK
`main: auth/model/domain/AppUser.java`	AppUser	02, 05, 07	—
`main: auth/model/domain/AuthAuditLog.java`	AuthAuditLog	02, 14	OK
`main: auth/model/domain/PasswordResetToken.java`	PasswordResetToken	02, 05	—
`main: auth/model/domain/RefreshToken.java`	RefreshToken	02, 05	—
`main: auth/model/dto/request/ForgotPasswordRequest.java`	ForgotPasswordRequest	09	—
`main: auth/model/dto/request/LoginRequest.java`	LoginRequest	08, 09, 20	OK
`main: auth/model/dto/request/LogoutRequest.java`	LogoutRequest	09	—
`main: auth/model/dto/request/RefreshTokenRequest.java`	RefreshTokenRequest	09	—
`main: auth/model/dto/request/RegisterRequest.java`	RegisterRequest	02, 08, 09, 21	OK
`main: auth/model/dto/request/ResetPasswordRequest.java`	ResetPasswordRequest	07, 08	OK
`main: auth/model/dto/response/ForgotPasswordResponse.java`	ForgotPasswordResponse	09	—
`main: auth/model/dto/response/LogoutResponse.java`	LogoutResponse	07, 09	—
`main: auth/model/dto/response/RegisteredUserResponse.java`	RegisteredUserResponse	05, 09	—

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`main: auth/model/dto/response/TokenResponse.java`	TokenResponse	05, 09, 20	—
`main: auth/model/entity/AppUserEntity.java`	AppUserEntity	07, 09	OK
`main: auth/model/entity/PasswordResetTokenEntity.java`	PasswordResetTokenEntity	07, 09	—
`main: auth/model/entity/RefreshTokenEntity.java`	RefreshTokenEntity	07, 09	OK
`main: auth/model/enums/UserRole.java`	UserRole	02, 08, 09, 10	OK
`main: auth/model/enums/UserStatus.java`	UserStatus	25	—
`main: auth/model/mapper/AuthMapper.java`	AuthMapper	03, 04, 05	OK
`main: auth/repository/AppUserRepository.java`	AppUserRepository	25	—
`main: auth/repository/PasswordResetTokenRepository.java`	PasswordResetTokenRepository	25	—
`main: auth/repository/RefreshTokenRepository.java`	RefreshTokenRepository	07	OK
`main: auth/service/AuthAuditService.java`	AuthAuditService	02, 04, 08, 09, 14, 20	OK
`main: auth/service/AuthService.java`	AuthService	04, 05, 07, 08, 09, 14, 20, 21	OK
`main: auth/util/StringUtils.java`	StringUtils	08, 09	OK
`main: auth/util/TokenHashUtils.java`	TokenHashUtils	04, 07, 09	OK
`src/main/resources/application.yml`	Servis konfiqurasiyası	02, 04, 09, 14, 20, 25	OK
`src/main/resources/db/changelog/db.changelog-master.yml`	Liquibase changelog	01, 07	OK
`src/main/resources/keys/private.pem`	RSA açarı (demo)	09, 10	—
`src/main/resources/keys/public.pem`	RSA açarı (demo)	25	—
`test: auth/base/AbstractBaseServiceTest.java`	Test: AbstractBaseServiceTest	19	—
`test: auth/base/AbstractRestControllerTest.java`	Test: AbstractRestControllerTest	19	OK
`test: auth/base/AbstractTestContainerConfiguration.java`	Test: AbstractTestContainerConfiguration	07, 13, 15, 19	—
`test: auth/controller/AuthControllerTest.java`	Test: AuthControllerTest	19	—
`test: auth/model/mapper/AuthMapperTest.java`	Test: AuthMapperTest	19	—

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`test: auth/service/AuthAuditServiceTest.java`	Test: AuthAuditServiceTest	14, 19	OK
`test: auth/service/AuthServiceTest.java`	Test: AuthServiceTest	09, 19	OK

campaign-service

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Çoxmərhələli image build	03, 18	OK
`pom.xml`	Modul asılılıqları	03, 07, 16, 18	OK
`main: campaign/CampaignServiceApplication.java`	CampaignServiceApplication	03, 04, 06, 15, 22	OK
`main: campaign/controller/CampaignController.java`	CampaignController	02, 04, 05, 06, 08, 10, 20, 25	OK
`main: campaign/domain/Campaign.java`	Campaign	00, 01, 02, 03, 04, 05, 06, 11, 13, 17, 19, 20, 22, 25	OK
`main: campaign/domain/CampaignCategory.java`	CampaignCategory	02, 06, 07, 08, 11	OK
`main: campaign/domain/CampaignStatus.java`	CampaignStatus	02, 06, 07, 11, 13, 21, 22	OK
`main: campaign/dto/CampaignHistoryResponse.java`	CampaignHistoryResponse	06, 13	—
`main: campaign/dto/CampaignPageResponse.java`	CampaignPageResponse	05, 06, 20, 24	OK
`main: campaign/dto/CampaignResponse.java`	CampaignResponse	05, 06, 20	—
`main: campaign/dto/CampaignStatisticsResponse.java`	CampaignStatisticsResponse	06, 13, 20	—
`main: campaign/dto/CreateCampaignRequest.java`	CreateCampaignRequest	04, 05, 06, 08, 20	OK
`main: campaign/entity/CampaignEntity.java`	CampaignEntity	03, 05, 06, 07, 19, 21, 22	OK
`main: campaign/entity/CampaignStatusHistoryEntity.java`	CampaignStatusHistoryEntity	05, 06, 07, 22	OK
`main: campaign/grpc/CampaignGrpcEndpoint.java`	CampaignGrpcEndpoint	04, 05, 06, 13, 20	OK
`main: campaign/mapper/CampaignMapper.java`	CampaignMapper	04, 05, 06	OK
`main: campaign/repository/CampaignRepository.java`	CampaignRepository	04, 05, 06, 07, 19, 20, 21	OK

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`main: campaign/repository/CampaignStatusHistoryRepository.java`	CampaignStatusHistoryRepository	04, 06	OK
`main: campaign/service/CampaignService.java`	CampaignService	01, 02, 04, 05, 06, 07, 08, 11, 13, 15, 19, 20, 21, 22, 23	OK
`src/main/resources/application.yml`	Servis konfiqurasiyası	05, 06, 07, 11, 13, 15, 16, 17, 20, 21	OK
`src/main/resources/db/changelog/db.changelog-master.yml`	Liquibase changelog	06, 07, 20, 21, 23	OK
`test: campaign/base/AbstractBaseServiceTest.java`	Test: AbstractBaseServiceTest	19	OK
`test: campaign/base/AbstractRestControllerTest.java`	Test: AbstractRestControllerTest	19	OK
`test: campaign/base/AbstractTestContainerConfiguration.java`	Test: AbstractTestContainerConfiguration	07, 13, 15, 19	OK
`test: campaign/controller/CampaignControllerTest.java`	Test: CampaignControllerTest	08, 19	OK
`test: campaign/service/CampaignServiceTest.java`	Test: CampaignServiceTest	02, 04, 19	OK
`src/test/resources/keys/private.pem`	Test RSA açarı	25	—
`src/test/resources/keys/public.pem`	Test RSA açarı	25	—

common

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`pom.xml`	Ortaq modulun asılılıqları	02, 03, 11, 18, 20	OK
`main: common/api/ApiError.java`	Xəta cavab zərfi	02, 05, 08, 10, 21, 23	OK
`main: common/api/ApiResponse.java`	Uğurlu cavab zərfi	02, 04, 05, 08, 12, 13, 20, 21	OK
`main: common/config/KafkaJsonConfiguration.java`	Kafka producer (String açar, JSON dəyər)	03, 04, 06, 11, 23	OK
`main: common/config/VirtualThreadConfiguration.java`	@EnableAsync + virtual thread executor	04, 20	OK
`main: common/event/CampaignStatusChangedEvent.java`	Kampaniya status hadisəsi	02, 06, 11	—
`main: common/event/SlowRequestLogEvent.java`	Yavaş sorğu hadisəsi	02, 04, 11, 19, 20	—
`main: common/exception/BusinessException.java`	Biznes istisna bazası	05, 07, 08, 09, 20, 21, 22	OK
`main: common/exception/DuplicateCampaignException.java`	Təkrar kampaniya	06, 07, 08, 21	—

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`main: common/exception/ErrorCode.java`	Xəta kodları	05, 07, 08, 09, 10, 21	OK
`main: common/exception/GlobalExceptionHandler.java`	Mərkəzi istisna xəritələnməsi	02, 03, 04, 05, 08, 09, 10, 13, 17, 19, 20, 21	OK
`main: common/exception/InvalidStatusTransitionException.java`	Yanlış status keçidi	06, 08, 21, 22	—
`main: common/exception/ResourceNotFoundException.java`	Resurs tapılmadı	05, 06, 08, 09, 13, 20, 21, 22	—
`main: common/observability/HttpServerObservationConfiguration.java`	Span adı və actuator filtri	04, 17	OK
`main: common/web/SlowRequestLoggingFilter.java`	Yavaş sorğuları Kafka-ya göndərir	03, 04, 05, 11, 16, 19, 20, 21	OK

config-server

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Image	25	—
`pom.xml`	Config server asılılıqları	17	OK
`main: config/ConfigServerApplication.java`	Config Server	04	—
`main: config/observability/ConfigServerHttpServerObservationConfiguration.java`	Observation (kopya)	25	—
`src/main/resources/application.yml`	native profil, search-locations	04, 16	OK
`src/main/resources/config/application.yml`	Bütün klientlərə paylanan konfiqurasiya	04, 16	OK

dashboard-service

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Çoxmərhələli image build	25	—
`pom.xml`	Modul asılılıqları	25	—
`main: dashboard/DashboardServiceApplication.java`	DashboardServiceApplication	25	—
`main: dashboard/client/CampaignGrpcClient.java`	CampaignGrpcClient	04, 13, 20, 21	OK
`main: dashboard/controller/DashboardController.java`	DashboardController	02, 05, 10, 20, 21, 25	OK
`main: dashboard/dto/DashboardCampaignHistoryItemResponse.java`	DashboardCampaignHistoryItemResponse	13	—
`main: dashboard/dto/DashboardCampaignHistoryResponse.java`	DashboardCampaignHistoryResponse	13	—

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`main: dashboard/dto/DashboardStatisticsResponse.java`	DashboardStatisticsResponse	13	—
`src/main/resources/application.yml`	Servis konfiqurasiyası	02, 13, 20	OK
`test: dashboard/base/AbstractBaseServiceTest.java`	Test: AbstractBaseServiceTest	19	—
`test: dashboard/base/AbstractRestControllerTest.java`	Test: AbstractRestControllerTest	19	—
`test: dashboard/base/AbstractTestContainerConfiguration.java`	Test: AbstractTestContainerConfiguration	07, 13, 15, 19	OK
`test: dashboard/client/CampaignGrpcClientTest.java`	Test: CampaignGrpcClientTest	04, 13, 19	OK
`test: dashboard/controller/DashboardControllerTest.java`	Test: DashboardControllerTest	03, 19	OK
`src/test/resources/keys/private.pem`	Test RSA açarı	25	—
`src/test/resources/keys/public.pem`	Test RSA açarı	25	—

discovery-server

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Image	25	—
`pom.xml`	Eureka server asılılıqları	25	—
`main: discovery/DiscoveryServerApplication.java`	Eureka Server	04	—
`main: discovery/observability/DiscoveryServerHttpServerObservationConfiguration.java`	Observation (kopya)	25	—
`src/main/resources/application.yml`	Eureka konfiqurasiyası	16	OK

grpc-contract

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`pom.xml`	protoc + grpc-java generasiyası	03, 18	OK
`src/main/proto/campaign.proto`	gRPC kontraktı	00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25	OK

log-service

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Dockerfile`	Çoxmərhələli image build	25	—
`pom.xml`	Modul asılılıqları	25	—
`main: log/LogServiceApplication.java`	LogServiceApplication	25	—
`main: log/config/KafkaDlqConfiguration.java`	KafkaDlqConfiguration	04, 11, 20, 21	OK
`main: log/controller/AuthAuditLogController.java`	AuthAuditLogController	02, 08, 10, 12, 20, 25	OK
`main: log/controller/SlowRequestLogController.java`	SlowRequestLogController	02, 05, 10, 12, 20, 21, 25	OK
`main: log/model/document/AuthAuditLogDocument.java`	AuthAuditLogDocument	12, 25	OK
`main: log/model/document/SlowRequestDocument.java`	SlowRequestDocument	05, 11, 12, 20, 23	OK
`main: log/model/dto/request/AuthAuditLogRequest.java`	AuthAuditLogRequest	08, 14, 25	OK
`main: log/repository/AuthAuditLogRepository.java`	AuthAuditLogRepository	12	—
`main: log/repository/SlowRequestLogRepository.java`	SlowRequestLogRepository	12	—
`main: log/service/AuthAuditLogService.java`	AuthAuditLogService	12, 20	OK
`main: log/service/SlowRequestLogConsumer.java`	SlowRequestLogConsumer	04, 11, 12, 20	OK
`src/main/resources/application.yml`	Servis konfiqurasiyası	11, 12	OK
`test: log/base/AbstractBaseServiceTest.java`	Test: AbstractBaseServiceTest	19	—
`test: log/base/AbstractRestControllerTest.java`	Test: AbstractRestControllerTest	19	—
`test: log/base/AbstractTestContainerConfiguration.java`	Test: AbstractTestContainerConfiguration	07, 13, 15, 19	—
`test: log/controller/AuthAuditLogControllerTest.java`	Test: AuthAuditLogControllerTest	19	—
`test: log/controller/SlowRequestLogControllerTest.java`	Test: SlowRequestLogControllerTest	05, 19	OK
`test: log/service/AuthAuditLogServiceTest.java`	Test: AuthAuditLogServiceTest	19	—

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`test: log/service/SlowRequestLogConsumerTest.java`	Test: SlowRequestLogConsumerTest	19	—
`src/test/resources/keys/private.pem`	Test RSA açarı	25	—
`src/test/resources/keys/public.pem`	Test RSA açarı	25	—

observability

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`grafana/dashboards/campaign-logs-traces.json`	Observability konfiqurasiyası: campaign-logs-traces.json	17	—
`grafana/provisioning/dashboards/dashboards.yml`	Observability konfiqurasiyası: dashboards.yml	17	—
`grafana/provisioning/datasources/datasources.yml`	Observability konfiqurasiyası: datasources.yml	17	OK
`loki/loki-config.yml`	Observability konfiqurasiyası: loki-config.yml	17	—
`otel-collector/otel-collector.yml`	Observability konfiqurasiyası: otel-collector.yml	17, 18, 23	OK
`prometheus/prometheus-local.yml`	Observability konfiqurasiyası: prometheus-local.yml	17, 18	—
`prometheus/prometheus.yml`	Observability konfiqurasiyası: prometheus.yml	02, 10, 17, 18, 23, 25	OK
`promtail/promtail-config.yml`	Observability konfiqurasiyası: promtail-config.yml	17, 23	OK
`tempo/tempo.yml`	Observability konfiqurasiyası: tempo.yml	17, 23	OK

postman

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`Insurance Campaign Microservices.postman_collection.json`	Manual API test kolleksiyası	25	—

security

Fayl	Məsuliyyət	Fəsillər	Yoxlama
`pom.xml`	Security modul asılılıqları	18	—
`main: security/JwtAuthenticationFilter.java`	Bearer token → SecurityContext	04, 05, 08, 10, 13, 20	OK
`main: security/JwtProperties.java`	app.jwt.* xassələri	04, 10	OK
`main: security/JwtTokenService.java`	JWT imzalama/doğrulama (RS256/HMAC)	04, 08, 09, 10, 23	OK
`main: security/PemKeyUtils.java`	PEM açar oxuma	10, 21	OK
`main: security/ServiceSecurityConfiguration.java`	SecurityFilterChain + method security	02, 03, 04, 05, 08, 09, 10, 20, 21	OK
`src/main/resources/keys/public.pem`	RSA public key (demo)	10	—

26.4 Tapıntı indeksi

Kateqoriyalar fəsil 21.1-dədir. Tam mətn və sübut: `analysis/01-architecture-map.md` və fəsil 21.

ID	Kateqoriya	Tapıntı (qısa)	Fəsillər
F01	D (təhlükəsizlik)	İstənilən anonim istifadəçi `role: ROLE_ADMIN` ilə qeydiyyatdan keçə bilir	02, 08, 09, 10, 15, 20, 21, 22, 24
F02	D (təhlükəsizlik)	`forgot-password` xam reset token-i cavabda qaytarır → istifadəçi adını bilən hər kəs şifrəni dəyişə bilər	09, 10, 20, 21, 24, 25
F03	R	İstifadəçi enumerasiyası: login "User is not active", forgot-password `404 User not found`	08, 09, 10, 20, 21, 22, 24, 25
F04	R	gRPC server (9090) autentifikasiyasızdır, Compose host-a açır	10, 13, 20, 21, 23, 24
F05	R	Gateway rate limit açarı `X-Forwarded-For`-a etibar edir → anonim sorğular saxta başlıqla limiti keçə bilər	10, 15, 21, 24
F06	R (fakt kitabxanada)	Redis əlçatmaz olduqda `RedisRateLimiter` sorğunu buraxır (fail-open)	10, 15, 20, 21, 24
F07	D	`GlobalExceptionHandler` catch-all `Exception` handler-i Spring MVC-nin standart 4xx xətalarını (səhv JSON, yanlış enum, səhv UUID, 405, 404 …	05, 06, 08, 10, 19, 20, 21, 22, 24, 25
F08	R	`JwtAuthenticationFilter` etibarsız/vaxtı keçmiş token üçün `JwtException`-ı tutmur; nəticə servlet error-dispatch-dən asılıdır, test yoxdur	08, 10, 19, 21, 22
F09	R	Dual-write: DB tranzaksiyası daxilində `kafkaTemplate.send` — commit uğursuz olsa hadisə artıq göndərilmiş ola bilər; send uğursuz olsa hadisə itir …	06, 11, 20, 21, 22, 24
F10	R	Kafka əlçatmaz olduqda `KafkaTemplate.send` metadata üçün `max.block.ms` (standart 60 s) qədər bloklaya bilər: həm HTTP filtrində, həm tranzaksiya …	04, 06, 07, 11, 20, 21, 22, 23, 24
F11	R (fakt kitabxanada)	log-service `ErrorHandlingDeserializer` istifadə etmir: deserializasiya xətası DLQ-ya getmir, `DefaultErrorHandler.handleOtherException` …	08, 11, 19, 20, 21, 22, 23, 24
F12	T	`campaign-events` mövzusunun repozitoriyada consumer-i yoxdur	11, 20, 21, 22
F13	R	Cache eviction və tranzaksiya commit-inin sırası zəmanətli deyil; RedisCacheManager transaction-aware deyil → qısa müddətli köhnə statistika …	04, 15, 20, 21, 23, 24
F14	M	Təyin olunmuş UUID id + `@Version` yoxdur → `save()` / `saveAndFlush()` `merge` edir: hər insertdən əvvəl SELECT	06, 07, 09, 20, 21, 22, 24
F15	M (performans)	`campaign_status_histories.campaign_id` üzrə indeks yoxdur; `idx_campaign_duplicate_lookup` partial unique indekslə eyni sütunları təkrarlayır	06, 07, 20, 21
F16	M	`refresh_tokens.token_hash` və `password_reset_tokens.token_hash` üçün həm `unique` constraint, həm ayrıca indeks	07, 21
F17	R	Refresh token rotasiyasında optimistik kilid yoxdur → paralel iki refresh eyni token-lə iki yeni cüt ala bilər	09, 10, 20, 21, 24
F18	D	Paralel qeydiyyatda DB unique pozuntusu (`DataIntegrityViolationException`) tutulmur → 500 + SQL mesajı	07, 08, 09, 20, 21, 24, 25
F19	R	gRPC klientində deadline yoxdur; server tərəfində `@GrpcAdvice` yoxdur → `ResourceNotFoundException` gRPC `UNKNOWN`, dashboard-da 500 olur (404 …	08, 13, 20, 21, 22, 24, 25

ID	Kateqoriya	Tapıntı (qısa)	Fəsillər
F20	M	Jackson 2 `ObjectMapper` bean-i yalnız Eureka client-in tranzitiv gətirdiyi `spring-boot-jackson2` (deprecated) sayəsində mövcuddur …	03, 21, 23
F21	M	`JsonSerializer` / `JsonDeserializer` spring-kafka 4.0-dan `@Deprecated(forRemoval=true)`	21
F22	R	Repozitoriyada demo RSA private key və DB/Grafana şifrələri var	01, 09, 10, 18, 21, 23, 25
F23	M	Observation konfiqurasiyası 3 dəfə kopyalanıb (gateway, config, discovery)	03, 17, 21, 25
F24	R (performans)	Promtail `trace_id` / `span_id` -ni Loki label-ı edir → yüksək kardinallıq	17, 21, 23, 24
F25	T	Yavaş sorğu həddi 5 ms → filtrin aktiv olduğu servislərdə (campaign, log) praktiki olaraq hər sorğu Kafka-ya gedir	11, 20, 21, 22
F26	M	log-service `Page<SlowRequestDocument>` -u birbaşa serializə edir (DTO-suz); campaign-service isə `CampaignPageResponse` istifadə edir	05, 20, 21, 24
F27	R	Migrasiyalar testlərdə işləmir (`spring.liquibase.enabled=false` , `ddl-auto=create-drop`) → partial unique indeks və sxem validasiyası test edilmir	07, 19, 21, 24
F28	M	Testcontainers image-ləri istehsal image-lərindən fərqlidir (postgres:17-alpine vs postgres:latest; cp-kafka 7.6.1 vs apache/kafka:latest)	18, 19, 21
F29	M	Docker Compose-da bir çox `depends_on` sağlamlıq şərtisiz; campaign-service-də `restart` siyasəti yoxdur	07, 17, 18, 21, 23
F30	H	springdoc 2.8.13 (Boot 3 xətti) Boot 4 / Framework 7 ilə rəsmi uyğunluq xəttindən kənardır; runtime-da yoxlanmayıb	16, 21
F31	T	Dashboard → campaign gRPC və auth → log HTTP static ünvan istifadə edir; Eureka yalnız gateway marşrutları və gateway OpenAPI klienti üçün işlənir	14, 16, 21, 22, 24
F32	R	Config Server əlçatmaz olduqda `optional:configserver:` sayəsində servislər lokal `application.yml` ilə başlayır; paylanan …	04, 11, 16, 18, 21, 22, 23, 24
F33	M	Testlər: campaign approve/deactivate keçidləri, gRPC endpoint, gateway, filtrlər, exception mapping üçün test yoxdur	19, 21
F34	R	`@Async` audit icrası (virtual thread) üçün trace kontekst ötürülməsi konfiqurasiya olunmayıb → audit HTTP çağırışları ayrı trace kimi görünə bilər	04, 17, 21, 24
F35	T	Access token 120 dəq; logout yalnız refresh token-i ləğv edir, access token vaxtı bitənə qədər etibarlıdır	09, 10, 20, 21, 22, 24
F36	D	`approve` / `deactivate` cavabında `updatedAt` köhnə dəyərlə qayıdır: `@PreUpdate` flush zamanı (commit-də) işləyir, mapper isə ondan əvvəl çağırılır	06, 07, 19, 20, 21, 22
F37	R	`CampaignEntity` -də `@Version` yoxdur: paralel `approve` və `deactivate` hər ikisi uğurlu ola bilər (lost update), tarixçədə ziddiyyətli iki qeyd və …	06, 08, 20, 21, 22, 24
F38	R (observability)	`GlobalExceptionHandler` heç bir istisnanı loglamır; catch-all ilə emal olunan 500 xətalarının stack trace-i log-a düşmür	08, 17, 21, 22, 23
F39	M	`AuthAuditLogRequest` üçün heç bir validasiya yoxdur (`@Valid` və constraint-lər yoxdur); `slow-requests` `page/size` yoxlanmır (mənfi `page` → 500)	08, 21, 25
F40	R	`/actuator/**` bütün servislərdə `permitAll` ; gateway-in öz actuator-u da açıq yollardadır; Compose servis portlarını host-a açır → `prometheus` …	10, 17, 21, 22, 23, 25

ID	Kateqoriya	Tapıntı (qısa)	Fəsillər
F41	M	CORS konfiqurasiyası yoxdur; başqa origin-dən brauzer klientləri bloklanır (Swagger UI gateway ilə eyni origin-dədir)	21
F42	M	`GET /slow-requests` sıralamasız `findAll(PageRequest)` edir — nəticələr `occurredAt` üzrə deyil, ixtiyari sırada; `SlowRequestDocument` sahələrinin …	12, 20, 21, 23, 24
F43	R (əməliyyat)	Elasticsearch indekslərində saxlanma/ILM siyasəti yoxdur; 5 ms həddi ilə `slow-request-logs` məhdudiyyətsiz böyüyür	12, 21, 22, 23, 24
F44	T	Gateway JWT/PEM kodu `security` modulundan ayrıca yazılıb (reaktiv gateway servlet asılılıqlarını çəkməsin deyə) — eyni məntiq iki yerdə	21
F45	M	`DUPLICATE` statusu heç bir axında təyin olunmur, lakin sorğu, yoxlama, süzgəc və partial indeksdə nəzərə alınır (ölü vəziyyət)	21
F46	R	`@Async` audit tapşırığı tranzaksiya daxilində, commit-dən əvvəl verilir → rollback olunan əməliyyat üçün "uğurlu" audit qeydi yarana bilər	21